



המכללה הטכנולוגית
של חיל האוויר
חיפה

טל' - 04-8697961
ד.צ. 02229 - פקס. 04-8697389
דוא"ל - Techni21@hotmail.com

סמל ביה"ס - 770396  

פרויקט גמר במסגרת לימודי הנדסאים אלקטרוניקה ומחשבים

מגמה: אלקטרוניקה ומחשבים בהתמחות במערכות אלקטרוניות

נושא: SantiBot, רובוט שומר ("רובוט רואה יורה")

סמל השאלון: 711918

מאת: ריי ברוק

מספר ת.ז.: 215500703

העבודה בוצעה בהנחיית: מרק טסליצקי

הצהרת הסטודנט ואישור המנחה

הצהרת הסטודנט:

שם הסטודנט: **ריי ברוק** מספר ת"ז **215500703**

אני הח"מ, מצהיר בזאת כי פרויקט/עבודת הגמר וספר הפרויקט המצ"ב נעשו על ידי בלבד. הפרויקט מסכם ידע, מיומנויות והרגלים שלמדתי במסגרת לימודי ההתמחות במגמה ובאופן עצמאי. הפרויקט וספר תיעוד הפרויקט נעשו על בסיס ההנחיות שקיבלתי מהמנחה שלי. מקורות המידע בהם השתמשתי לביצוע פרויקט מצוינים ברשימת המקורות שבסוף הספר. אני מודע לאחריות שהנני מקבל על עצמי על ידי חתימתי על הצהרה זו שכל הנכתב בה אמת.

חתימת הסטודנט:  תאריך: 12.3.26

אישור מנחה הפרויקט/עבודת הגמר:

הריני מאשר שהפרויקט בוצע בהנחייתי, קראתי את ספר הפרויקט ומצאתי כי הוא ראוי להגשה.

שם המנחה: **מרק טסליצקי**

חתימה:

תאריך:

➡ לחצו כאן לחזרה לתוכן העניינים

תודות והערכה

"למידה היא תהליך שאינו נגמר, וההצלחה בו תלויה באנשים המלווים אותך לאורך הדרך".

עם סיומו של פרויקט הגמר, הנקודה המשמעותית ביותר במסע הלימודי שלי, אני מוצא לנכון לעצור ולהביע את הערכתי העמוקה לכל אלו שסייעו, כיוונו ותמכו בי עד להגשמת היעד.

ראשית, ברצוני להביע תודה עמוקה ואישית למנחה הפרויקט שלי, מר מרק טסליצקי. מרק, תודה על הליווי המקצועי הצמוד ועל האמונה בי לאורך כל הדרך. התמיכה המקצועית שלך, הירידה לפרטים והיכולת להעניק פתרונות יצירתיים לכל אתגר טכני היוו עבורי עמוד תוֹך משמעותי. אני מודה לך על כך שדלתך תמיד הייתה פתוחה בפניי, על הסבלנות האינסופית בכל פעם שהתייעצתי איתך, ועל הרעיונות המקוריים שהענקת לי – רעיונות ששדרגו את הפרויקט והפכו אותו מתוכנית תאורטית למוצר עובד ואיכותי. הניסיון העשיר שלך והנכונות לחלוק אותו הם שהפכו את תהליך העבודה לחוויה מלמדת ומעשירה.

תודה מיוחדת נתונה לצוות המנחים והמורים המקצועיים בבית הספר. תודה על הידע המקצועי הרב שהנחלתם לי במהלך שנות לימודיי. העצות המקצועיות שקיבלתי מכם לאורך שלבי הפיתוח השונים סייעו לי לגבש תפיסה הנדסית רחבה ולהתגבר על מכשולים שלא פעם נראו בלתי עבירים. ההכוונתכם והדרכתכם המקצועית ניכרות בכל שלב ושלב בפרויקט זה.

כמו כן, אני מבקש להודות להנהלת בית הספר ולצוות המעבדות, על העמדת הציוד והמשאבים הנדרשים לרשותי, ועל יצירת סביבת עבודה מעודדת ויצירתית שאפשרה לי להוציא את המיטב מעצמי.

לבסוף, תודה חמה למשפחתי היקרה ולחבריי לספסל הלימודים. תודה על העידוד ברגעים הקשים, על ההקשבה ועל התמיכה המורלית. האמונה שלכם בי העניקה לי את המוטיבציה להמשיך, לחקור ולהגיע לתוצאה הסופית עליה אני גאה היום.

תודה לכולכם על שהייתם חלק מהמסע הזה.

תוכן עניינים

לצורך ניווט מהיר בין פרקי העבודה, ניתן להשתמש בתוכן העניינים המופיע בעמוד הבא.

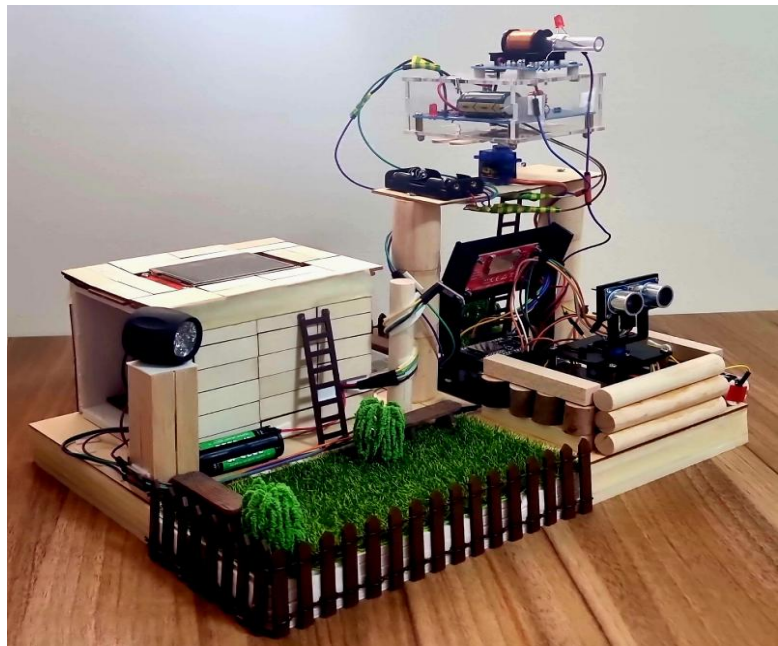
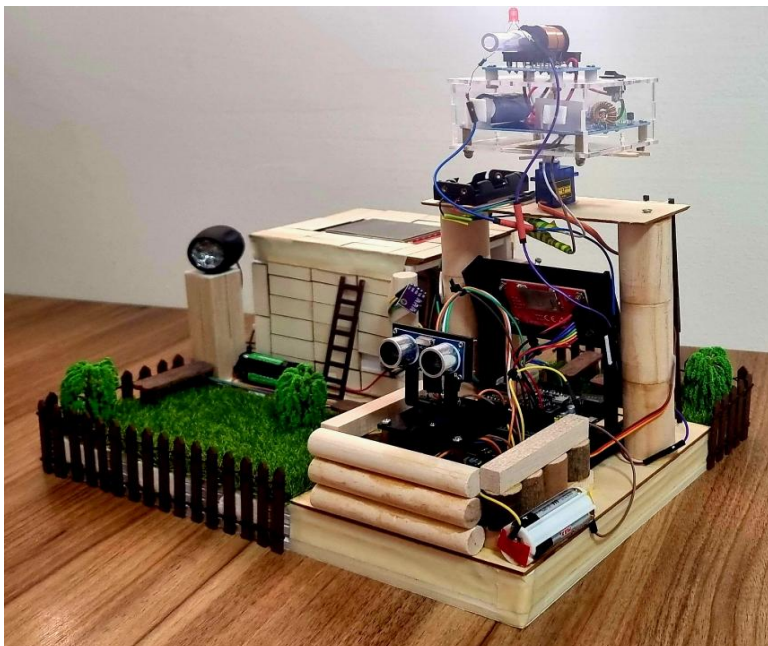
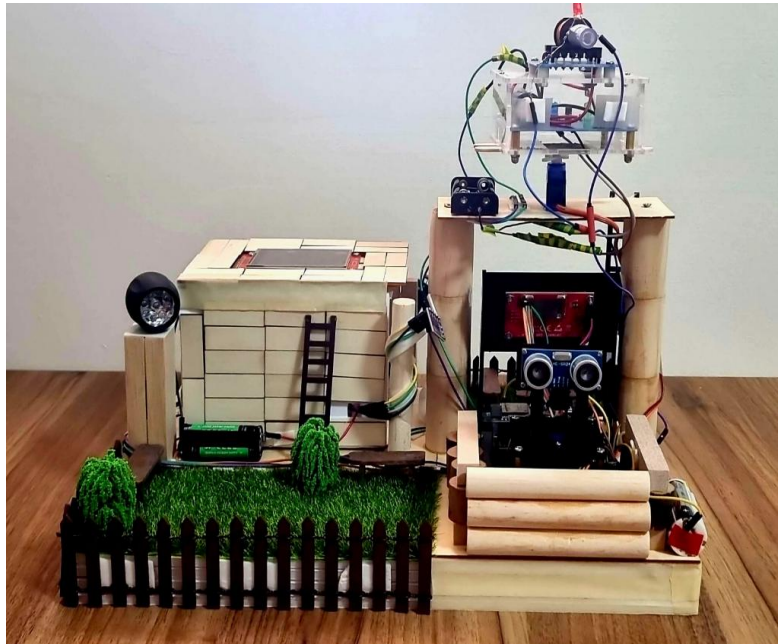
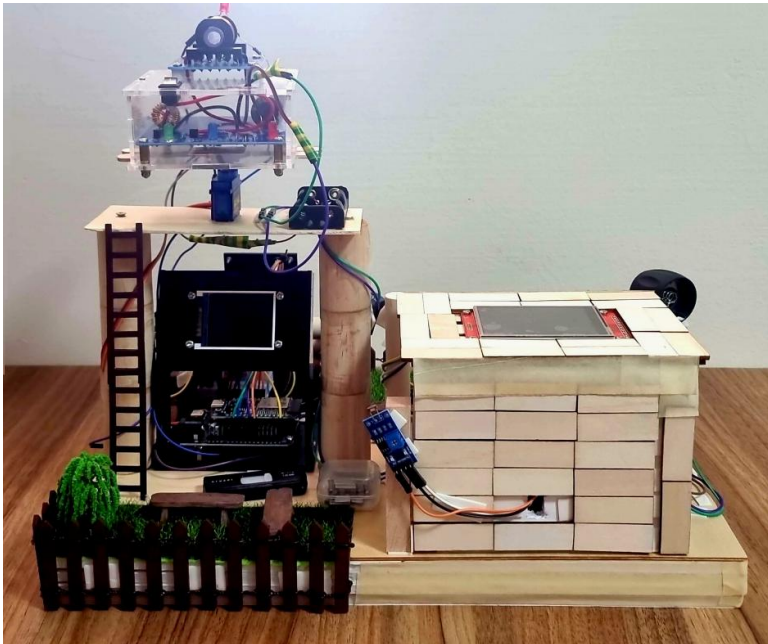
אם ברצונכם לעבור ישירות לנושא מסוים בספר פרויקט יש ללחוץ על הכותרת

2	הצהרת הסטודנט ואישור המנחה
3	תודות והערכה
6	פרויקט גמר
7	תקציר:
8	תיאור הבעיה והצורך המבצעי
9	תרשים זרימה של הפרוקיט מ- diagrams.net
10	תרשים מלבנים ראשוני diagrams.net
12	שרטוט חשמלי ראשוני בתוכנת EASYEDA
13	תרשים מלבני סופי לפרויקט diagrams.net
15	שרטוט חשמלי סופי בתוכנת EASYEDA
16	הסבר תרשים חשמלי מפורט – מערכת מכ"ם ובקרת קרקע
17	רשימת רכיבים – מפרט טכני של רכיבי המערכת
18	מיקרו בקר ESP32
25	ESP_NOW
29	תקשורת טורית UART
34	תקשורת SPI
40	תקשורת I2C
46	מסך 1.8 – TFT (ST7735)
49	מסך גרפי טאץ' 9341 TFT
57	חיישן המרחק – HC-SR04
61	חיישן טמפרטורה LM75
65	מודול חיישן האור KY-018
68	מנוע SERVO
73	מודול ממסר כפול 5 וולט
74	תיאור פשוט של אופן פעולת הרכיב
80	תיעודים, בדיקות והרכבות
80	רובה אלקטרומגנטי (Electromagnetic gun) + הרכבה
87	מיני רדאר (Mini Radar) + הרכבה
90	תיעודים- בדיקות רכיבי המערכת הראשונה
90	תיעוד של אופן פעולת החיישן האולטרסוניק ומדידת מתחים 5.9.25:
92	תיעוד של אופן הפעולה 20.9.25:
95	קוד סופי למערכת המכ"ם (MASTER):

101.....	תיעודים עבור רכיבים למערכת השנייה
101.....	תיעוד של אופן פעולת המסך טאץ' המסך השני בו אני משתמש בפרויקט 27.9.25
103.....	תיעוד של חיבור המסך השני למערכת כולה ואופן פעולת המערכת 28.9.25 :
112.....	תיעוד של תקשורת בין שני המיקרו בקרים ומציאת כתובת MAC של המקבל, 15.11.25
115.....	תיעוד של אופן התקשורת בין שתי המערכות, 22.11.25 :
118.....	בניית הנורה/מיני פרוז'קטור לפרויקט, 29.11.25 :
122.....	תיעוד ראשוני של בניית הדגם, 13.12.25 :
124.....	תיעוד בהתקדמות הפרויקט 21.12.25 :
125.....	תיעוד ההתקדמות 22.12.25 :
129.....	תיעוד 30.1.26
130.....	תיעוד 7.2.26 :
132.....	קוד סופי עבור המערכת בקרה וניטור סביבתי (SLAVE)
138.....	רפלקציה על התהליך ועל התוצר
140.....	ביבליוגרפיה :
141.....	קישור לאתר שלי

פרויקט גמר

SANTIBOT - רובוט רואה יורה רובוט שמירה לשטח



פרויקט "רובוט רואה יורה"- רובוט ששומר על שטח ומנטר לנו נתונים סביבתיים של 180 מעלות מהשטח. המנוע שעליו נמצא החיישן מסתובב באופן קבוע ומשרטט את הרדאר במסך הקטן הראשון שבמערכת הרדאר. לאחר שהתגלו אובייקטים וישנן נקודות אדומות על המסך המנוע השני שעליו נמצא התותח מסנכרן ישירות עם המנוע הראשון ונדלק הנורית לד האדומה שעל קנה הרובה שמסמנת יריות רצופות כל עוד ישנם אובייקטים קרובים (מכיוון שלסליל לוקח זמן להיטען). מערכת הבקרה/הניטור מנתר את הערכים של הטמפרטורה הסביבתית שמשפיעה על דיוק החיישן אולטרסוניק ואת עומת האור בסביבה וידעת האם להדליק את הפרויקטור או לא.

מערכת הגנה אקטיבית וניטור סביבתי (SantiBot)

תקציר:

הפרויקט המוצג משלב מספר תתי-מערכות הפועלות בסנכרון מלא על גבי דגם אחד. המערכת תוכננה כפתרון הנדסי מקצועי, המדמה מערכות הגנה צבאיות מתקדמות, תוך דגש על יעילות אנרגטית, יכולת זיהוי אובייקטים ותגובה בזמן אמת.

מערכת המכ"ם (Radar System)

לב המערכת הינו מכ"ם המבוסס על חיישן אולטרסאונד המותקן על גבי מנוע סרוו. המערכת מבצעת סריקה רציפה של השטח בזווית של 180 מעלות.

- **זיהוי אובייקטים:** החיישן משדר אותות קוליים ומנתח את ההד החוזר לקביעת מרחק האובייקט.
- **תצוגה ויזואלית:** הנתונים מוצגים על גבי מסך בגודל 1.8 אינץ'. חיווי המרחק מתבצע באמצעות צבעים: נקודות **צהובות** מייצגות אובייקטים בטווח שמעל 100 ס"מ, בעוד נקודות **אדומות** מתריעות על אובייקטים קרובים בטווח של 0–100 ס"מ.

מערכת התותח והנשק האקטיבי

אל מערכת המכ"ם מסונכרן תותח המותקן על מנוע סרוו נפרד. הפעלת התותח מתבצעת באופן מושכל לצורך חיסכון באנרגיה:

- **סנכרון תנועה:** התותח עוקב אחר תנועת הרדאר רק כאשר מזוהה אובייקט בטווח הקרוב ("נקודות אדומות"). כאשר האובייקטים רחוקים (מעל 100 ס"מ), התותח נותר במקומו כדי לחסוך בצריכת חשמל ובלאי מכני.
- **דימוי ירי:** התותח הינו תותח אלקטרומגנטי הדורש זמן טעינה ממושך. על מנת לדמות ירי רציף בזמן אמת, שולבה נורת LED אדומה בקנה התותח, הנדלקת בעת זיהוי מטרה קרובה ומדמה ירי הרתעת. הירי הפיזי של הקליע מתבצע עם סיום טעינת המערכת האלקטרומגנטית.

מערכת בקרה וניטור סביבתי (Environmental Monitoring)

המערכת השנייה בפרויקט משמשת כמרכז בקרה וניהול נתונים, המתקשרת עם מערכת המכ"ם בזמן אמת:

- **ממשק משתמש:** נעשה שימוש במסך מגע (Touch) בגודל 2.8 אינץ' הכולל תפריט בעל שלושה לחצנים:
- 1. **ניטור תאורה:** שימוש בחיישן LDR לזיהוי רמת התאורה הסביבתית. בעת חשיכה, המערכת מפעילה באופן אוטומטי "פרוז'קטור" (שנבנה משדרוג פנס יד) המאיר את גזרת ה-180 מעלות. פעולה זו משפרת את שדה הראייה לכוחותינו ומסנוורת את האויב.
- 2. **מדידת טמפרטורה:** הצגת הטמפרטורה הסביבתית. נתון זה קריטי לדיוק המדידה, שכן מהירות הקול מושפעת ישירות מהטמפרטורה $V = 331.4 + 0.6 \times T$.
- 3. **סטטוס מכ"ם:** הצגת חיווי טקסטואלי המעדכן האם זוהה אובייקט בגזרה או לא.

לחצו כאן לחזרה לתוכן העניינים

סיכום

הפרויקט שלי ממחיש אינטגרציה בין חיישנים, מנועים, יחידות תצוגה ותקשורת בין בקרים. השילוב בין מערכת הגנה אקטיבית לבין ניטור סביבתי יוצר מוצר המדגיש עקרונות הגנה מודרניים, יעילות אנרגטית ויכולת תגובה טקטיקה בשטח.

תיאור הבעיה והצורך המבצעי

המצב הקיים והצורך המבצעי

בעולם המודרני, אבטחת מתחמים וזיהוי חדירות בזמן אמת מהווים אתגר משמעותי עבור כוחות הביטחון. מערכות הגנה סטטיות רבות סובלות ממגבלות של טווח ראייה מוגבל (במיוחד בתנאי חשיכה), צריכת אנרגיה גבוהה עקב הפעלה רציפה של כלל המערכות, וקושי בסנכרון מהיר בין שלב הזיהוי לשלב התגובה.

קיים צורך חיוני במערכת הגנה חכמה, המסוגלת לבצע ניטור רציף של השטח, לספק חיווי ויזואלי מדויק למפעיל, ולהגיב באופן אקטיבי וממוקד רק כאשר מזוהה איום ממשי, תוך אופטימיזציה של משאבי האנרגיה.

הפתרון המוצע: מערכת הגנה אקטיבית משולבת

הפרויקט מציע פתרון הוליסטי המשלב זיהוי, ניטור ותגובה בדגם אחד. המערכת נותנת מענה לבעיות המרכזיות שהוצגו באמצעות מספר רכיבים מסונכרנים:

- **ייעול צריכת האנרגיה (Energy Efficiency):** בניגוד למערכות הפועלות באופן מלא כל הזמן, המערכת שפיתחתי משתמשת ב"סנכרון מותנה". התותח והמערכות היקרות מבחינת אנרגיה נכנסים לפעולה ועוקבים אחר האיום רק כאשר המכ"ם מזהה אובייקט בטווח הקרוב (עד 100 ס"מ). בטווחים רחוקים יותר, המערכת נשארת במצב ניטור חסכוני.
- **התגברות על תנאי חשיכה:** המערכת כוללת יחידת ניטור סביבתי מבוססת LDR המזהה חשיכה ומפעילה פרוז'קטור עוצמתי. הפתרון משמש לצורך כפול: שיפור שדה הראייה של כוחותינו מחד, וטשטוש שדה הראייה של האויב (סנוור) מאידך.
- **דיוק בתנאי סביבה משתנים:** מאחר ומהירות הקול מושפעת מהטמפרטורה, שולב חיישן טמפרטורה המאפשר כיול בזמן אמת של המכ"ם האולטרסוני. פעולה זו מבטיחה שהמרחק המוצג למפעיל יהיה מדויק ככל הניתן, ללא תלות במזג האוויר.
- **ממשק שליטה ובקרה:** המערכת מנגישה את המידע המורכב למפעיל באמצעות מסכי מגע וחיווי צבעוני (צהוב/אדום), המאפשרים קבלת החלטות מהירה בשטח.

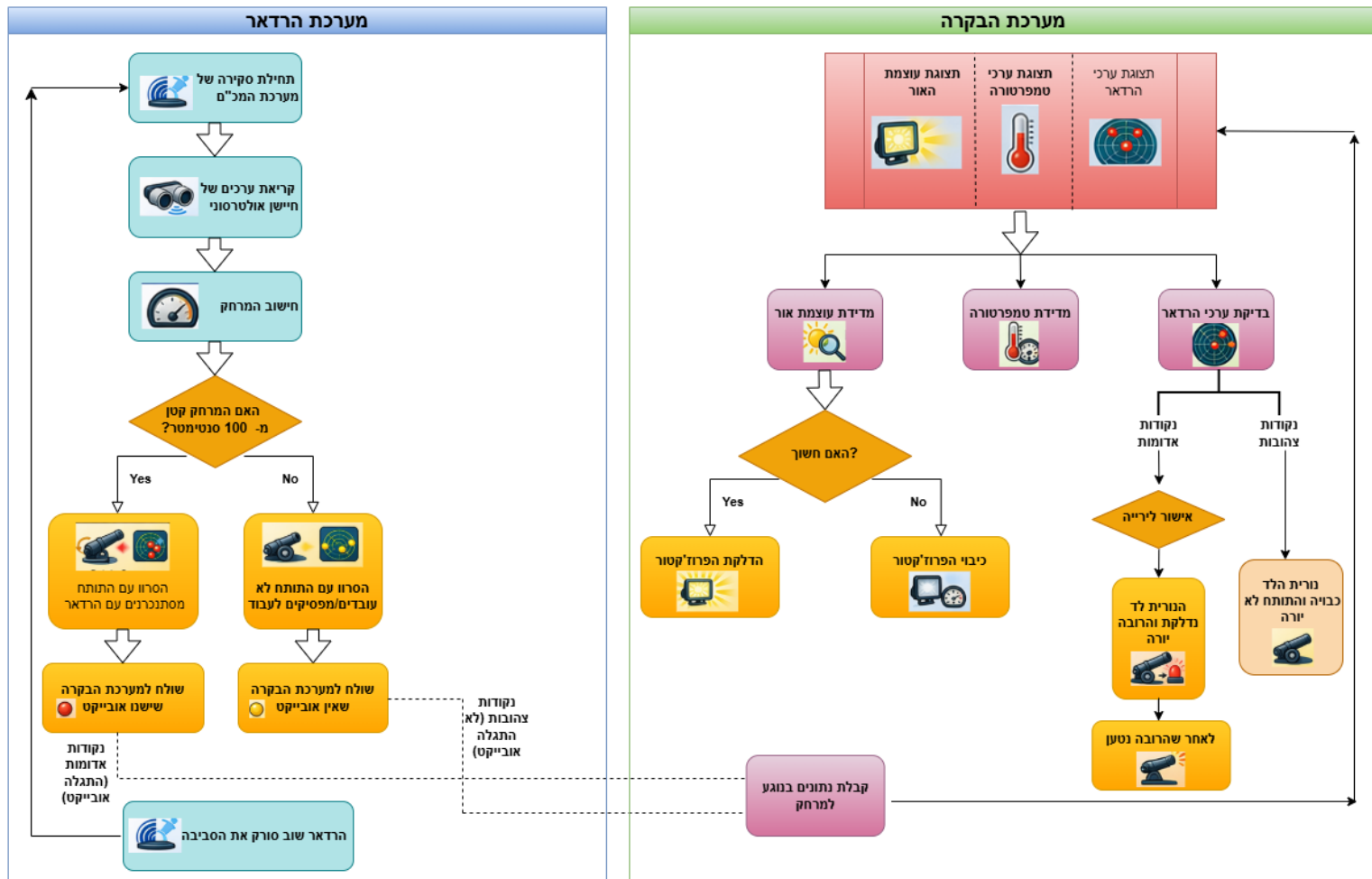
מטרות הפרויקט

1. פיתוח מערכת מכ"ם לסריקת שטח של 180 מעלות בזמן אמת.
2. יצירת סנכרון מכני ותוכנתי בין מערכת הזיהוי (אולטרסאונד) למערכת התגובה (תותח).
3. הטמעת אלגוריתם לחיסכון באנרגיה המבוסס על טווחי מרחק.
4. בניית מערכת ניטור סביבתית הכוללת פיצוי טמפרטורה ותאורת לילה אוטומטית.

הערך המוסף של הפרויקט: המערכת הופכת את האבטחה לפרודוקטיבית יותר: היא מצמצמת את הצורך בנוכחות פיזית של לוחם בעמדת התצפית, מונעת טעויות אנוש הנובעות מעייפות או תנאי ראות קשים, ומבטיחה תגובה מהירה ומדויקת לכל ניסיון חדירה לגזרה.

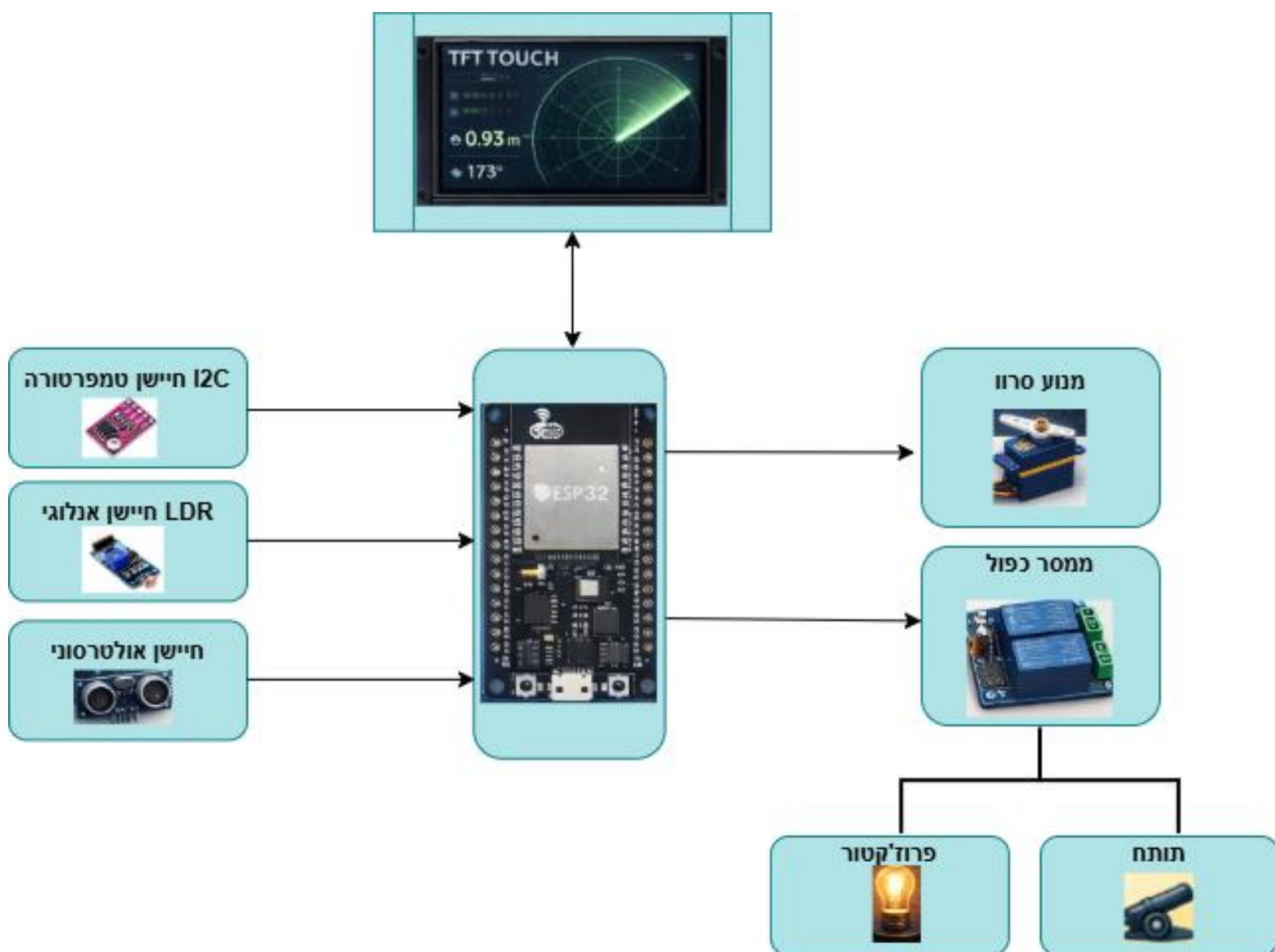
➤ לחצו כאן לחזרה לתוכן העניינים

תרשים זרימה של הפרוקיט מ- diagrams.net



לחצו כאן לחזרה לתוכן העניינים

תרשים מלבנים ראשוני diagrams.net



חיישן טמפרטורה

- **קלט:** נתוני חום מהסביבה.
- **פעולה:** מדידת הטמפרטורה בזמן אמת.
- **פלט:** נתון מספרי של הטמפרטורה → מועבר לבקר המרכזי.

חיישן אור

- **קלט:** עוצמת האור הקיימת בסביבה.
- **פעולה:** בדיקת האם יש תאורה מספקת או חושך מוחלט.
- **פלט:** ערך עוצמת אור → מועבר לבקר המרכזי לצורך החלטה (לדוגמה: הפעלת תאורה).

חיישן מרחק (Ultrasonic / Radar)

- **קלט:** אותות מהסביבה המשקפים מרחק ממכשולים או גופים זרים.
- **פעולה:** חישוב המרחק וזיהוי תנועה או חדירה לשטח.
- **פלט:** ערך מספרי של מרחק/איתור תנועה → מועבר לבקר המרכזי.

בקר מרכזי (Controller / MCU)

- **קלט:** נתונים משלושת החיישנים (טמפרטורה, אור, מרחק/ראדאר).
- **פעולה:**
 - עיבוד המידע המתקבל.
 - קבלת החלטות אוטונומיות בהתאם לתרחיש (איתור מטרה, חושך, תנאי סביבה).
 - הפעלת מנוע הסרוו לכיוון הרובה.
 - שליטה על ממסר כפול להפעלת רכיבים (מנורה/רובה).
 - שליחת נתונים למסך המגע.
- **פלט:** פקודות פעולה לרכיבים החיצוניים + הצגת נתונים במסך המגע.

מנוע סרוו (Servo Motor)

- **קלט:** פקודת מיקוד וכיוון מהבקר המרכזי.
- **פעולה:** מבצע תנועה מדויקת לכיוון המטרה שזוהתה.
- **פלט:** כיוון פיזי של הרובה למיקום המטרה.

ממסר כפול (Relay Module)

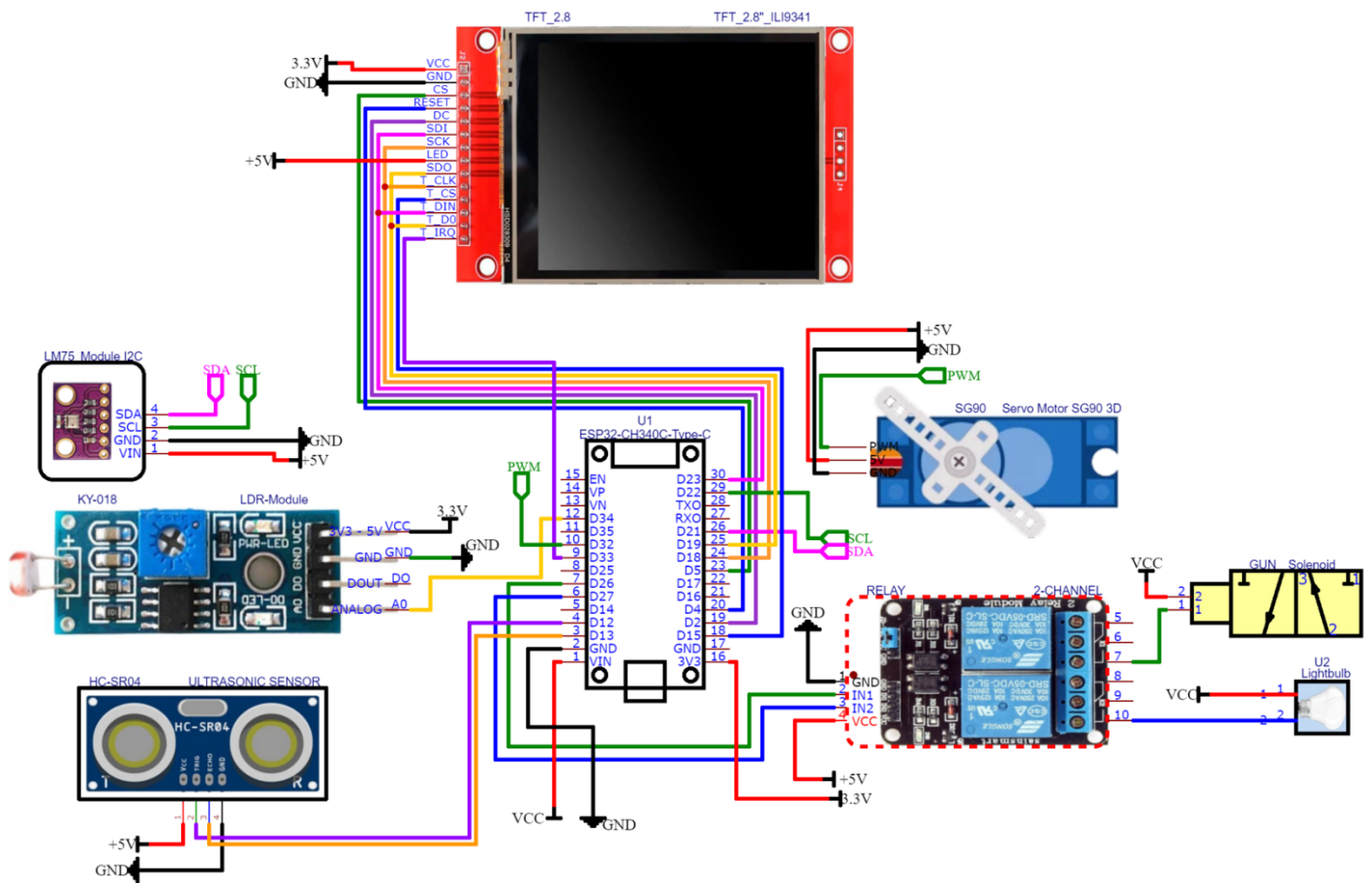
- **קלט:** פקודת הפעלה מהבקר המרכזי.
- **פעולה:** חיבור או ניתוק של רכיבים חשמליים חיצוניים.
- **פלט:** הפעלה של אמצעי תאורה או הפעלת הרובה.

מסך מגע (TFT Touch)

- **קלט:** נתוני מצב מהבקר המרכזי + קלט מהמשתמש (פקודות/מגע).
- **פעולה:**
 - הצגת נתוני החיישנים והמצב בזמן אמת.
 - קבלת פקודות ידניות מהמפעיל.
- **פלט:** החזרת פקודות לבקר המרכזי או חיווי למפעיל.

➤ לחצו כאן לחזרה לתוכן העניינים

שרטוט חשמלי ראשוני בתוכנת EASYEDA

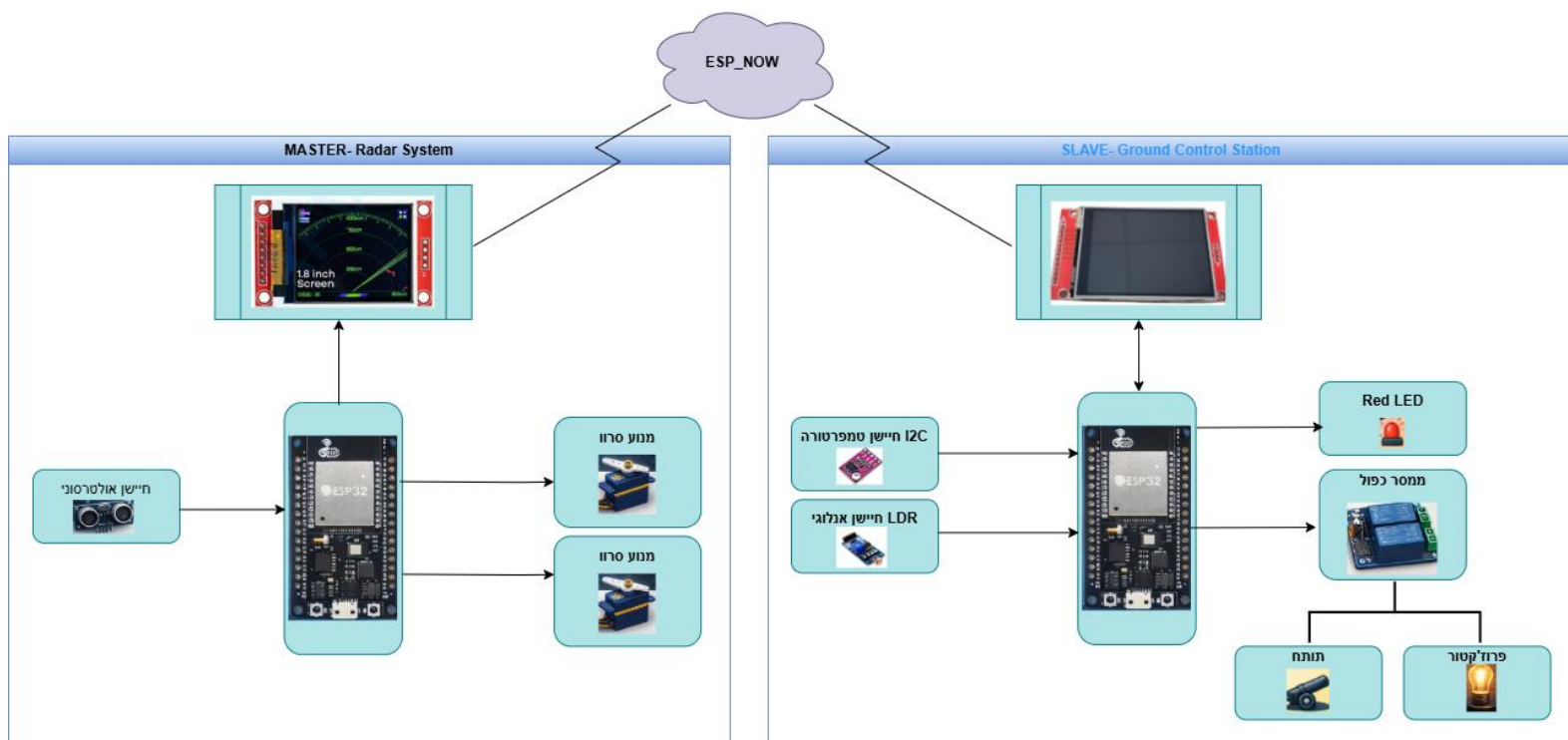


TITLE: Sheet_1		REV: 1.0
Company: Your Company		Sheet: 1/1
Date: 2023-10-19	Drawn By: ruybrook	

זהו למעשה השרטוט הראשוני עבור המערכת אליה התחייבתי בהצעת פרויקט. הפרויקט שעשיתי בפועל הוא יותר מורכב ומחולק לשתי מערכות שמתקשרות ביניהן בעזרת פרוטוקול תקשורת אלחוטי הנקרא ESP_NOW אני משתמש בשני מיקרו בקרים מסוג ESP32 ואתייחס לכך בשרטוט החשמלי הסופי שלי לפרויקט שם ניתן יהיה לראות בבירור את המערכות וכיצד הן מתחברות למערכת אחת גדולה.

➡ לחצו כאן לחזרה לתוכן העניינים

תרשים מלבני סופי לפרויקט diagrams.net



יחידה 1: מערכת המכ"ם (Master - Radar System)

חיישן מרחק אולטרסוני (HC-SR04)

- **קלט:** אותות מהסביבה המשקפים מרחק ממכשולים או גופים זרים.
- **פעולה:** חישוב המרחק וזיהוי תנועה או חדירה לשטח הסריקה.
- **פלט:** ערך מספרי של מרחק/איתור תנועה ← מועבר לבקר המרכזי.

מנוע סרוו (SG90 x2)

- **קלט:** פקודות מיקוד וכיוון מהבקר המרכזי.
- **פעולה:** מבצע תנועה מדויקת (צידוד והטיה) לכיוון המטרה שזוהתה.
- **פלט:** כיוון פיזי של המכ"ם למיקום המטרה.

מסך תצוגה (TFT "1.8")

- **קלט:** נתוני מצב מהבקר המרכזי.
- **פעולה:** הצגת נתוני החיישנים והסריקה בזמן אמת בצורה גרפית.
- **פלט:** חיווי ויזואלי למפעיל המקומי (מפת רדאר).

בקר מרכזי (Master - ESP32)

- **קלט:** נתונים מחיישן המרחק.
- **פעולה:** עיבוד המידע, ניהול תנועת המנועים ושליחת הנתונים באופן אלחוטי.
- **פלט:** פקודות למנועים ולמסך + שידור נתונים ליחידת המקלט.

יחידה 2: תחנת בקרת קרקע (Slave - Ground Control Station)

חיישן טמפרטורה (LM75)

- **קלט:** נתוני חום מהסביבה.
- **פעולה:** מדידת הטמפרטורה בזמן אמת.
- **פלט:** נתון מספרי של הטמפרטורה ← מועבר לבקר המרכזי.

חיישן אור (LDR)

- **קלט:** עוצמת האור הקיימת בסביבה.
- **פעולה:** בדיקת האם יש תאורה מספקת או חושך מוחלט.
- **פלט:** ערך עוצמת אור ← מועבר לבקר המרכזי לצורך החלטה (לדוגמה: הפעלת תאורה).

ממסר כפול (Relay Module)

- **קלט:** פקודת הפעלה מהבקר המרכזי.
- **פעולה:** חיבור או ניתוק של רכיבים חשמליים חיצוניים (נורה/סולנואיד).
- **פלט:** הפעלה פיזית של אמצעי תאורה או הפעלת הרובה.

מסך מגע (TFT Touch "2.8")

- **קלט:** נתוני מצב מהבקר המרכזי + קלט מהמשתמש (מגע).
- **פעולה:** הצגת נתוני החיישנים המרוחקים וקבלת פקודות ידניות מהמפעיל.
- **פלט:** חיווי למפעיל + החזרת פקודות לבקר המרכזי.

בקר מרכזי (Slave - ESP32)

- **קלט:** נתונים אלחוטיים מהמכ"ם ונתונים מחיישני הסביבה (אור וטמפרטורה).
- **פעולה:** עיבוד המידע שהתקבל, קבלת החלטות אוטונומיות (הפעלת התרעה) וניהול ממשק המשתמש.
- **פלט:** פקודות הפעלה לממסרים + הצגת מידע על מסך המגע.

מה מקשר בין המערכות?

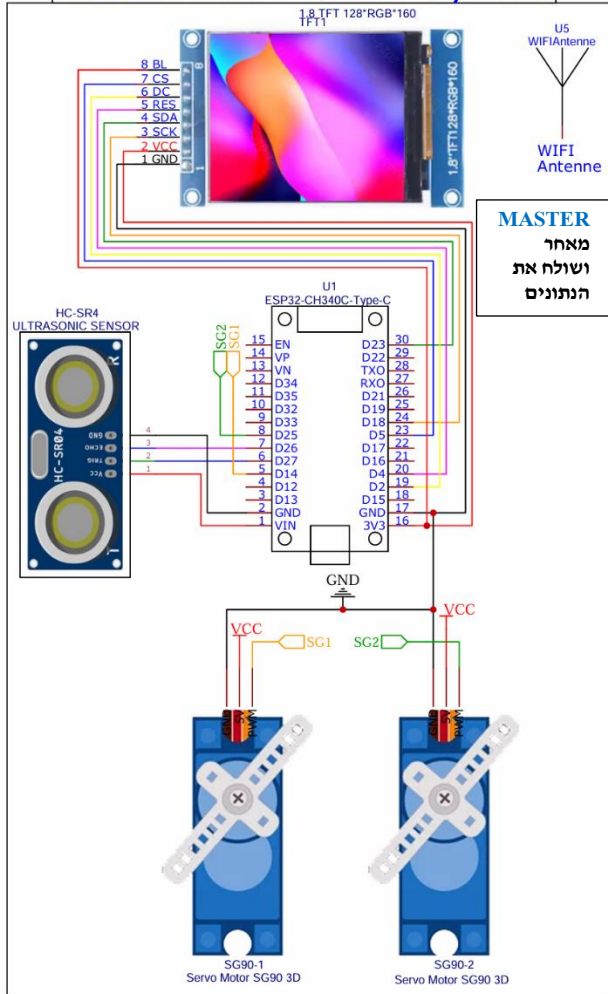
הגורם המקשר בין שתי היחידות הוא **ענן התקשורת האלחוטית (ESP_NOW)**.

- **מהות הקשר:** המערכות אינן מחוברות פיזית בחוטים. יחידת המכ"ם (Master) משדרת את נתוני המרחק והזווית באוויר.
- **זרימת המידע:** הנתונים נשלחים מיחידת ה Master אל יחידת ה Slave, מה שמאפשר למפעיל בתחנת הבקרה לראות את המתרחש בשטח ולהפעיל מרחוק את הממסרים במידת הצורך.

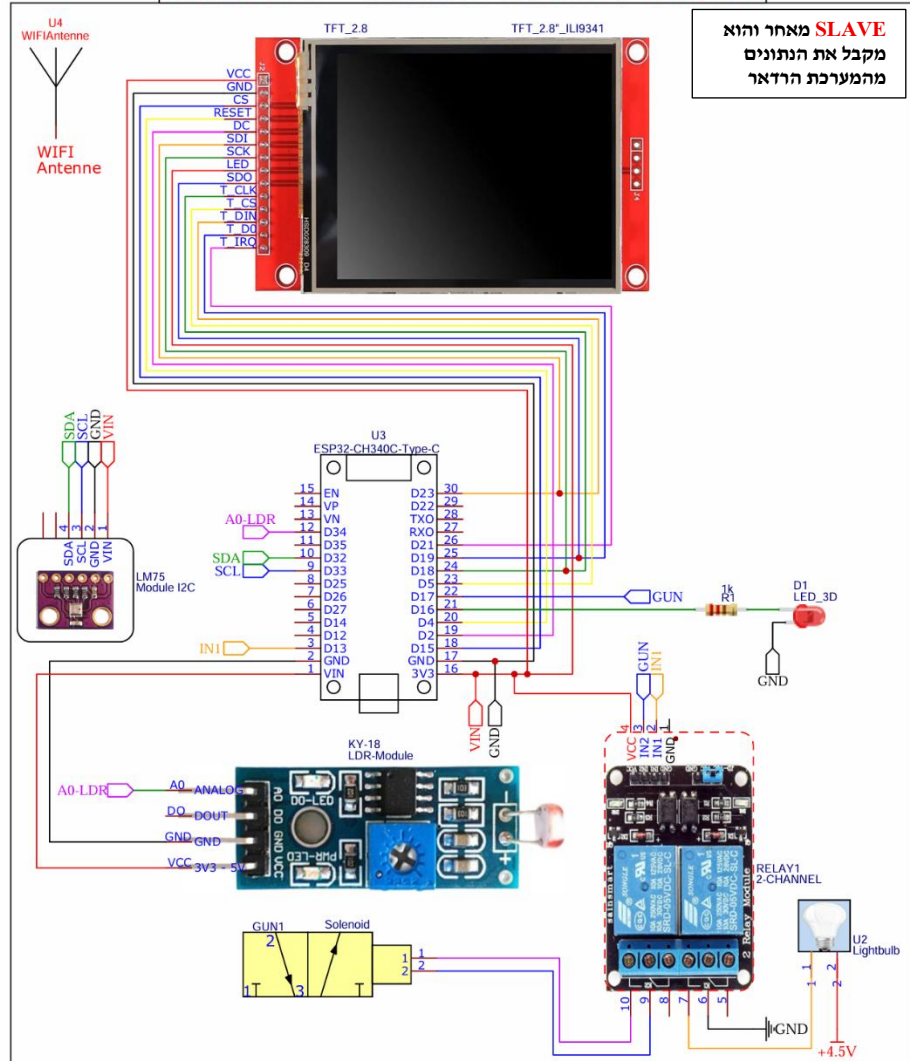
➤ לחצו כאן לחזרה לתוכן העניינים

שרטוט חשמלי סופי בתוכנת EASYEDA

Transmitter - Radar System



Reciever - Ground Control Station



ניתן לראות את השרטוט הסופי לפרויקט שעשיתי. המבלך הפרויקט למדתי המון דברים חדשים ונתקלתי במספר בעיות, אחת הבעיות הייתה עומס רב על מערכת אחת עם בקר אחד. מאחר וזה ESP32 הוא בקר המשמש לפרויקטים שיכולים לפעול עם עומס תמידי לא גדול הזדקקתי להפריד את הפרויקט לשני חלקים המתקשרים באמצעות פרוטוקול תקשורת ייחודי לבקר הנקרא NOW_ESP (עליו ארחיב המבלך הספר) למעשה המערכת הראשונה שכוללת את הרדאר ושני מנועי הסרוו כאשר על אחד מהם מסתובב החיישן אולטרסוניק ועל המנוע השני שמסנכרן יחד עם הראשון במידה והתגלו אובייקטים נמצא התותח האלקטרומגנטי שהלחמתי ידנית (במהלך הספר ניתן לראות את כל הדברים שהרכבתי וכמובן שיש סרטונים באופן מסודר באתר שבנתי באמצעות Visual Studio Code (Java Script+ CSS +HTML) תוך שימוש בתוכנת

במהלך עשיית הפרויקט למדתי המון על התחום האלקטרוניקה מעבר למה שידעתי לפניכן. לאט לאט התחלתי להתקל בקשיים ודברים שצרכו ממני מחשבה עמוקה ומקצועית יותר והגעתי לפתרונות ומסקנות שקידמו לי את רמת הפרויקט בכמעט כפליים מהפרויקט אליו התחייבתי. זה מה שקסום בתחום הזה הוא רחב ומאפשר ליצור כל מחשבה למציאות. מאחר ושימוש שלי הוא שיחידה אחת (מערכת הרדאר) מעבירה נתונים למערכת השנייה (הבקר) אז התקשורת היא תקשורת מסוג Simplex.

הסבר תרשים חשמלי מפורט – מערכת מכ"ם ובקרת קרקע

המערכת מורכבת משתי יחידות נפרדות המתקשרות ביניהן באופן אלחוטי בתצורת **Master** (משדר) ו-**Slave** (מקלט).

1. יחידת המשדר (Transmitter - Radar System)

יחידה זו מהווה את ה-**Master** במערכת היא אוספת את נתוני המרחק והזווית ושולחת אותם ליחידת המקלט.

- **בקר מרכזי : (ESP32-CH340C)** ה"מוח" של המערכת, האחראי על עיבוד האותות מהחיישנים ושליטת המנועים.
- **חיישן מרחק אולטראסוני : (HC-SR04)** מודד את המרחק לעצמים על ידי שליחת גלי קול. הוא מחובר לפינים הדיגיטליים של ה-ESP32.
- **מנועי סרוו : (SG90 x2)** אחראים על צידוד (סיבוב אופקי) והטיה (סיבוב אנכי) של חיישן המרחק כדי לסרוק את המרחב.
- **מסך תצוגה : (1.8" TFT)** מציג נתונים בזמן אמת ביחידת המכ"ם עצמה.
- **אנטנת : WiFi** מאפשרת את שידור הנתונים אלחוטית ליחידת המקלט.

2. יחידת המקלט (Receiver - Ground Control Station)

יחידה זו משמשת כ-**Slave** היא מקבלת את הנתונים מהמכ"ם ומבצעת פעולות בקרה והתראה.

- **בקר מרכזי : (ESP32-CH340C)** מקבל את המידע האלחוטי ומפעיל את רכיבי הקצה.
- **מסך תצוגה גדול : (2.8" ILI9341 TFT)** משמש כממשק המשתמש המרכזי (GCS) להצגת המפה והנתונים שהתקבלו.
- **חיישן טמפרטורה : (LM75 I2C)** מנטר את הטמפרטורה בסביבת תחנת הבקרה.
- **חיישן אור : (LDR Module)** מזהה את עוצמת התאורה בסביבה לצורך התאמת התצוגה או הפעלת תאורה.
- **מודול ממסר : (2-Channel Relay)** שולט על רכיבי מתח גבוה, כמו הנורה (U2) וסולנואיד (Solenoid).
- **אינדיקציות : (LED & Gun)** נורת LED אדומה ורכיב הדק (Gun) להפעלת תגובה פיזית במידה וזוהה עצם.

הסבר תהליכים עיקריים ושילוב חומרה

- 1. תהליך הסריקה :** ה-ESP32 ביחידת המשדר מניע את מנועי הסרוו בזוויות מוגדרות. בכל זווית, חיישן ה-HC-SR04 מבצע מדידה. המידע (זווית + מרחק) נשלח דרך פרוטוקול אלחוטי (WiFi/ESP-NOW) למקלט.
- 2. תקשורת ועיבוד :** יחידת המקלט קולטת את חבילת הנתונים. ה-ESP32 במקלט מעבד את המידע ומצייר "נקודה" על מסך ה-2.8 אינץ' בהתאם לזווית ולמרחק שדווחו.
- 3. מערכת תגובה : (Feedback Loop)** במידה והמרחק הנמדד קטן מסף מסוים (זיהוי מטר), המקלט מפעיל את הממסר שסוגר מעגל לנורה. בנוסף, חיישן ה-LDR מאפשר למערכת לדעת אם יש צורך להדליק תאורת עזר בחדר הבקרה.

➤ לחצו כאן לחזרה לתוכן העניינים

רשימת רכיבים – מפרט טכני של רכיבי המערכת

רשימת הרכיבים בהם נעשה שימוש במערכת כוללת את המפרט הטכני הבסיסי של כל רכיב כגון מתחי הפעלה, רזולוציה, סוגי תקשורת ותפקיד במערכת. המידע מבוסס על נתוני היצרן (Datasheet) של כל רכיב.

מס'	רכיב	מתח עבודה	מאפיינים טכניים	סוג תקשורת	תפקיד במערכת
1	ESP32	3.3V	תדר עד 240MHz, ADC 12bit, WiFi + Bluetooth	SPI, I2C, UART, WiFi	יחידת הבקרה המרכזית של המערכת
2	HC-SR04	5V	טווח מדידה 2-400cm רזולוציה 3mm, תדר 40kHz	Trig/Echo	מדידת מרחק וזיהוי אובייקטים
3	Servo Motor SG90	4.8-6V	זווית 0-180°, מהירות 0.12s/60°, 1.8kg/cm	PWM	הנעת חיישן הרדאר לצורך סריקה
4	TFT Display 1	3.3V	גודל "1.8", רזולוציה 160x128	SPI	הצגת נתוני הרדאר במערכת המשדר
5	TFT Display 2	3.3V	גודל "2.8", רזולוציה 320x240, מסך מגע	SPI	תצוגת נתונים וממשק משתמש
6	LM 75	2.8-5.5V	טווח -55°C עד 125°C, רזולוציה 0.125°C	I2C	מדידת טמפרטורת הסביבה
7	LDR Module	3.3-5V	יציאה אנלוגית, רגישות לאור	Analog	מדידת עוצמת תאורה
8	Relay Module	5V	זרם מיתוג עד 10A, מתח עד 250V AC	Digital	שליטה בעומסים חיצוניים
9	Solenoid	4.5-12V	פעולה אלקטרומגנטית לינארית	דרך ממסר	הפעלת מנגנון מכני
10	LED	2-3.2V	זרם עד 20mA	Digital	חיווי לזיהוי אובייקט
11	ESP-NOW	2.4 GHz	קצב עד 1Mbps, טווח עד 100m	אלחוטי	תקשורת בין היחידות

נתוני המפרט הטכני נלקחו מתוך דפי הנתונים (Datasheets) של היצרנים.

[לחצו כאן לחזרה לתוכן העניינים](#)

מיקרו בקר ESP32



ה- **ESP32** הוא הרבה יותר מסתם מיקרו-בקר פשוט ; הוא מערכת על שבב (SoC) חזקה ורב-תכליתית, המשלבת את כל מה שצריך כדי לבנות פרויקט חכם, מקושר וחסכוני באנרגיה. פרי פיתוחה של חברת **Espressif Systems**, ה- ESP32 בולט בזכות השילוב הייחודי של עוצמת עיבוד חזקה, קישוריות אלחוטית ויכולת צריכת חשמל נמוכה.

תכונות טכניות עיקריות:

1. **עיבוד בעל ביצועים גבוהים:** מצויד במעבד כפול-ליבה במהירות של עד 240 MHz, ה- ESP32 מסוגל לבצע משימות מורכבות ולנהל מספר פעולות במקביל במהירות גבוהה.
2. **קישוריות אלחוטית מובנית:** ה- ESP32 מגיע עם **Wi-Fi** ו- **Bluetooth** מובנים, מה שהופך אותו לבחירה אידיאלית לפיתוח פרויקטים מחוברים ללא צורך ברכיבים חיצוניים נוספים.
3. **יעילות אנרגטית:** עם מצבי שינה מרובים, ה- ESP32 מתוכנן לפעול לטווח ארוך על סוללה בודדת, מה שהופך אותו למושלם עבור יישומים המופעלים באמצעות סוללה.
4. **ממשקי קלט/פלט (I/O) גמישים:** ה- ESP32 מציע מגוון רחב של ממשקים היקפיים, כולל **ADC, DAC, I2C, SPI** ו- **GPIO**, ומספק כמעט אינסוף אפשרויות ליצירה.
5. **תמיכת קהילה נרחבת:** עם אלפי מפתחים ברחבי העולם המשתמשים ב- ESP32, קיים שפע של ספריות קוד, תיעוד ופרויקטים של קוד פתוח הזמינים לתמיכה בפיתוח שלך.

הסוד שמאחורי ה-ESP32: יחידת העיבוד המרכזית (CPU)

ה-ESP32 מצויד במעבד עוצמתי Xtensa LX6 dual-core, הפועל במהירות מרשימה של עד 240MHz. אבל מה זה בעצם אומר?

- דמיינו שיש לכם שני מוחים עובדים במקביל: בזמן שאחד מהם מטפל בקישוריות Wi-Fi, השני יכול לעבד נתונים מהחיישנים שלכם - הכול בבת אחת!
- **240MHz** זוהי מהירות השעון של המעבד. לשם השוואה, המעבד של ה-ESP32 מסוגל לבצע **240 מיליון פעולות בשנייה!**
- **ארכיטקטורת 32-ביט** זוהי אומרת שהמעבד יכול לעבד נתונים בכמויות גדולות יותר, מה שמאפשר חישובים מורכבים במהירות גבוהה.

הזיכרון: איפה כל המידע נשמר?

ה-ESP32 מגיע עם מערך מרשים של אפשרויות זיכרון:

- **SRAM (Static Random-Access Memory):** זיכרון מהיר לשימוש מידי של **520KB**. זהו המקום שבו המידע שהמעבד עובד עליו ברגע נתון נשמר.
 - **ROM (Read-Only Memory):** זיכרון קבוע של **448KB** המכיל את ההוראות הבסיסיות הנדרשות להפעלת המערכת.
 - **Flash (זיכרון חיצוני):** זיכרון של עד **16MB** שבו נשמרות תוכניות המשתמש שלכם והנתונים הקבועים.
- לשם השוואה, הזיכרון הכולל של ה-ESP32 הוא כל כך גדול, שתוכלו לאחסן בו ספר אודיו שלם!

קישוריות אלחוטית: החיבור לעולם:

ה-ESP32 הוא אלוף התקשורת האלחוטית:

- **Wi-Fi:** תומך בתקן **802.11 b/g/n**, מה שמאפשר חיבור כמעט לכל רשת אלחוטית מודרנית. הוא יכול לשמש גם כנקודת גישה ולספק חיבורים למכשירים אחרים.
- **Bluetooth:** עם תמיכה ב-**Bluetooth Classic** וגם ב-**Bluetooth Low Energy (BLE)**, ה-ESP32 יכול לתקשר עם מגוון רחב של מכשירים, מטלפונים חכמים ועד חיישנים זעירים.

ממשקים: חיבור לעולם הפיזי:

ה-ESP32 מציע מגוון רחב של אפשרויות חיבור כדי לתקשר עם העולם הפיזי:

- **GPIO (General Purpose Input/Output):** עד 36 פיינים, המשמשים כ"עצבים" של ה-ESP32, המאפשרים לו לחוש ולשלט בסביבה.
- **ממיר אנלוגי-דיגיטלי (ADC):** ישנם 18 ערוצים המאפשרים ל-ESP32 לקרוא ערכים אנלוגיים מחיישנים כמו טמפרטורה או עוצמת אור.
- **ממיר דיגיטלי-אנלוגי (DAC):** שני ערוצים המאפשרים לייצר אותות אנלוגיים כמו צלילים, או מתחים משתנים.
- **חיישני מגע (Touch Sensors):** 10 פייני מגע מובנים המאפשרים ל-ESP32 לחוש מגע ישיר.

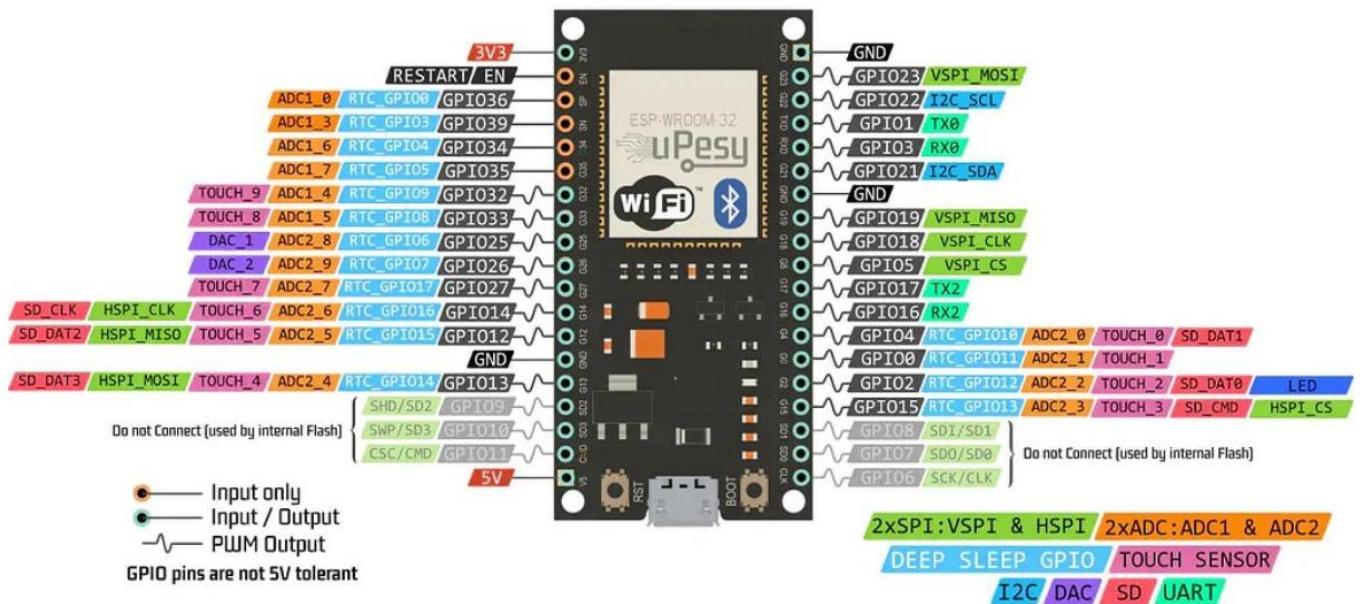
צריכת חשמל: יעילות אנרגטית מרשימה:

אחד היתרונות הגדולים ביותר של ה-ESP32 הוא היכולת שלו לחסוך באנרגיה:

- **מצב פעיל מלא:** במצב זה, צריכת הזרם היא עד 240mA.
 - **מצב שינה עמוקה:** במצב זה, הצריכה יורדת באופן דרמטי ל-10µA בלבד!
- זה אומר שגם עם סוללה קטנה, הפרויקט שלכם יכול לפעול במשך שבועות ואף חודשים ארוכים.

קעת נראה את צורת הפינים במיקרו בקר ESP32:

ESP32 Wroom DevKit Full Pinout



כפי שכבר ראינו, ה-ESP32 מוקף בפינים מרובים, שהם ה"שערים" דרכם הוא מתקשר עם העולם החיצוני. הבנה מעמיקה של פינים אלו היא המפתח ליצירת פרויקטים מדהימים. בואו נצלול לעולם המרתק של פיני ה-ESP32:

סוגי הפינים העיקריים:

פיני GPIO (General Purpose Input/Output)

- אלו הם הפינים הרב-תכליתיים של ה-ESP32.
- יכולים לשמש כקלט (לקריאת מצבים) או כפלט (להפעלת רכיבים).
- ה-ESP32 מציע עד 34 פיני GPIO, אך חלקם משמשים גם למטרות אחרות.
- **שימושים נפוצים:** חיבור נורות LED, לחצנים, מנועים ועוד.

פיני ADC (Analog to Digital Converter):

- מאפשרים קריאה של ערכים אנלוגיים והמרתם לערכים דיגיטליים.
- ה-ESP32 מציע שני מעגלי ADC עם סה"כ 18 ערוצים.
- **רזולוציה של 12 ביט** מאפשרת 4,096 רמות שונות של קריאה.
- **שימושים:** מדידת טמפרטורה, עוצמת אור, לחות קרקע ועוד.

פיני DAC (Digital to Analog Converter):

- ממירים ערכים דיגיטליים לאותות אנלוגיים.
- ה-ESP32 מציע שני ערוצי DAC ברזולוציה של 8 ביט.
- **שימושים נפוצים:** יצירת צלילים, בקרת מתח ועוד.

פיני Touch:

- ה-ESP32 מציע 10 חיישני מגע קפסיטיביים מובנים.
- מאפשרים זיהוי מגע אנושי ישיר, ללא צורך ברכיבים נוספים.
- **שימושים נפוצים:** יצירת ממשקי משתמש מבוססי מגע, לחצנים וירטואליים ועוד.

פיני תקשורת:

- **UART:** לתקשורת טורית. ה-ESP32 מציע 3 יחידות UART.
- **SPI:** לתקשורת מהירה. ה-ESP32 תומך במספר חיבורי SPI במקביל.
- **I2C:** לתקשורת עם מגוון רחב של חיישנים ורכיבים. ה-ESP32 מציע שתי יחידות I2C.

פינים מיוחדים:

- **פיין BOOT:** משמש לכניסה למצב **צריבת תוכנה (flashing mode)**.
- **פיין EN (Enable):** משמשים להפעלה וכיבוי של השבב.
- **פיני אספקת מתח:** פינים המספקים מתחים של 5V, 3.3V וכן GND.

נקודות חשובות לגבי השימוש בפינים:

1. **רמות מתח:** פיני ה-ESP32 עובדים ברמת מתח של 3.3V. חיבור למתח גבוה יותר עלול לגרום נזק.
2. **פינים מרובי תפקידים:** רבים מהפינים יכולים לשמש למספר מטרות. חשוב לתכנן את השימוש בהם בקפידה.
3. **הגנה על הפינים:** מומלץ להשתמש בנגדים מגבילי זרם בעת חיבור רכיבים חיצוניים, במיוחד עם נוריות LED.
4. **פינים מיוחדים:** חלק מפיני ה-GPIO (כמו 0, 2, 15) משמשים במהלך תהליך האתחול. שימוש לא נכון בהם עלול למנוע מה-ESP32 לפעול כראוי.
5. **פולאפ ופולאדאון (Pull-up and Pull-down):** חלק מהפינים כוללים נגדי Pull-Up או Pull-Down פנימיים. זה יכול להוות שימוש חשוב, אך חשוב להיות מודעים לכך בעת תכנון המעגל.

הבנה מעמיקה של מפת הפינים ותכונותיהם היא המפתח ליצירת פרויקטים מוצלחים ויעילים עם ESP32. זה מאפשר לנו לנצל את מלוא הפוטנציאל של השבב העוצמתי הזה.

ESP32: הכוח האלחוטי של Wi-Fi ו-Bluetooth:

אחד היתרונות הגדולים ביותר של ה-ESP32 הוא יכולתו המובנית לתקשר באופן אלחוטי.

Wi-Fi : החיבור לעולם הרחב

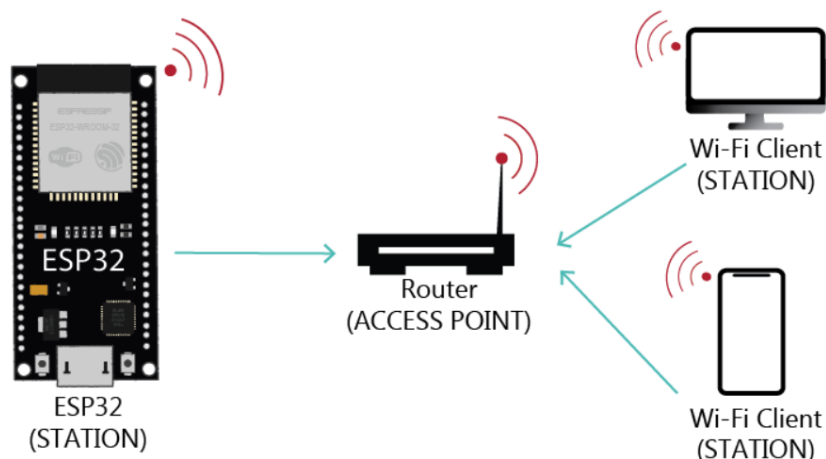
ה-ESP32 תומך בתקן Wi-Fi 802.11 b/g/n, מה שמאפשר לו להתחבר כמעט לכל רשת אלחוטית מודרנית.

מצבי פעולה עיקריים:

- **מצב תחנה (Station Mode - STA):**
 - ה-ESP32 מתחבר לרשת Wi-Fi קיימת, כמו כל מכשיר קצה רגיל.
 - שימושים נפוצים: התחברות לאינטרנט דרך הראוטר הביתי, שליחת נתונים למסד נתונים מרוחק ועוד.
- **מצב נקודת גישה (Access Point - AP):**
 - ה-ESP32 יוצר רשת Wi-Fi משלו, אליה יכולים להתחבר מכשירים אחרים.
 - מצוין ליצירת פרויקטים שמספקים ממשק אינטרנט או תקשורת ישירה עם מכשירים ניידים.
 - שימושים נפוצים במצבים בהם אין גישה לרשת Wi-Fi חיצונית.
- **מצב כפול (Dual Mode):**
 - ה-ESP32 יכול לפעול במקביל גם כתחנה וגם כנקודת גישה בו-זמנית!
 - מאפשר יצירת "גשר" בין רשתות או אספקת גישה לאינטרנט דרך ה-ESP32.

יכולות מתקדמות של Wi-Fi:

- **שרת אינטרנט (Web Server):** ה-ESP32 יכול לשמש כשרת אינטרנט, מה שמאפשר יצירת ממשקי משתמש מרחוק ודפי תצוגה נתונים.
- **MQTT:** תמיכה בפרוטוקול זה מאפשרת תקשורת יעילה בין מכשירי IoT ושרתים מרכזיים.
- **OTA (Over-The-Air) Updates:** עדכון תוכנה מרחוק, ללא צורך בחיבור פיזי למחשב.
- **ESP-NOW:** פרוטוקול תקשורת ייחודי של Espressif המאפשר תקשורת ישירה בין מכשירי ESP32 בצריכת אנרגיה נמוכה.



Bluetooth : תקשורת קצרת טווח עם יכולות ארוכות טווח

ה-ESP32 תומך הן ב-Bluetooth Classic והן ב-Bluetooth Low Energy (BLE), מה שפותח אפשרויות רבות :

Bluetooth Classic :

- מתאים לתקשורת עם מכשירים ישנים יותר או כשנדרשת העברת נתונים בנפח גדול.
- שימושים נפוצים : הזרמת אודיו, העברת קבצים, ותקשורת עם מכשירים שאינם תומכים ב-BLE.
- מאפשר יצירת פרופילים שונים כמו **Serial Port Profile (SPP)** לתקשורת טורית אלחוטית.

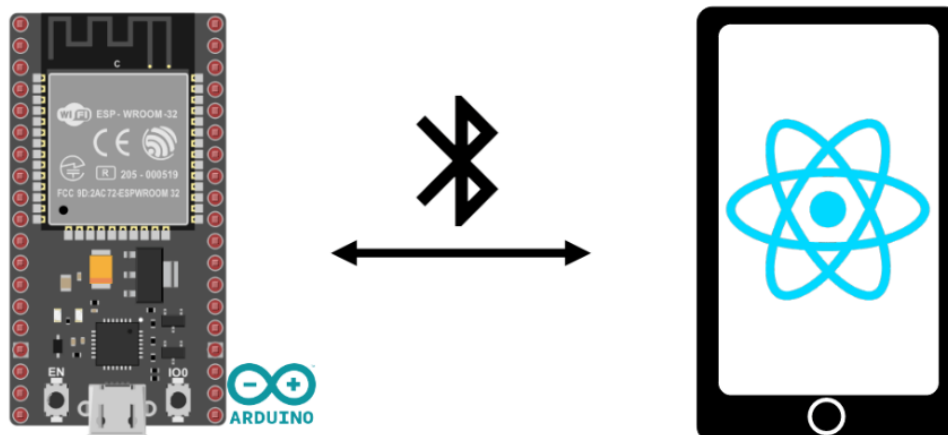
Bluetooth Low Energy (BLE) :

- צריכת אנרגיה נמוכה במיוחד, אידיאלי למכשירים המופעלים על סוללה.
- מתאים לשליחת נתונים קטנים באופן תדיר, כמו נתוני חיישנים.
- תומך במודל **GATT (Generic Attribute Profile)** המאפשר יצירת שירותים ומאפיינים מותאמים אישית.
- מאפשר יצירת "ביקונים" לשיווק מידע ללא צורך בחיבור מלא.

שילוב Wi-Fi ו-Bluetooth : יתרונות מרתקים :

אחד היתרונות הגדולים של ה-ESP32 הוא היכולת להשתמש גם ב-Bluetooth וגם ב-Wi-Fi במקביל. זה פותח אפשרויות מרתקות :

- **שער (Gateway) :** קבלת נתונים ממכשירי **Bluetooth** ושליחתם לענן דרך **Wi-Fi**.
- **שליטה מקומית ומרוחקת :** שליטה בפרויקט דרך **Bluetooth** כשקרובים, ודרך **Wi-Fi** כשמרוחקים.
- **רשתות היברידיות :** שימוש ב-BLE לתקשורת בין חיישנים קרובים, וב-Wi-Fi לשליחת נתונים מרובים לשרת מרכזי.



- 1. עוצמה חישובית:** עם מעבד כפול-ליבה במהירות של עד 240 MHz, ESP32 מאפשר ביצוע משימות מורכבות ועיבוד נתונים מהיר, מה שהופך אותו למתאים לפרויקטים IoT מתקדמים.
- 2. קישוריות אלחוטית מקיפה:** תמיכה מובנית ב-Wi-Fi ו-Bluetooth (כולל BLE) מאפשרת יצירת מגוון רחב של פרויקטים מחוברים, החל ממערכות בית חכם ועד ליישומים תעשייתיים.
- 3. גמישות בחיבורים:** מגוון רחב של ממשקים כמו GPIO, ADC, DAC ופרוטוקולי תקשורת כמו I2C, UART, SPI מאפשרים אינטגרציה קלה עם כמעט כל סוג של חיישן או מפעיל.
- 4. יעילות אנרגטית:** מצבי שינה מתקדמים וצריכת חשמל נמוכה מאפשרים יצירת מכשירים ניידים עם חיי סוללה ארוכים, מה שהופך אותו לאידיאלי למערכות ניטור מרוחקות.
- 5. פיתוח נגיש:** תמיכה במגוון סביבות פיתוח כמו ESP-IDF ו-Arduino IDE וקהילת מפתחים גדולה ופעילה מקלים על תהליך הפיתוח ומספקים שפע של ספריות תוכנה.
- 6. יכולות IoT מתקדמות:** תמיכה בפרוטוקולים כמו MQTT ויכולות עדכון תוכנה מרחוק (OTA) מאפשרות הקמת שרתי web מקומיים וניהול התקנים מכל מקום בעולם.
- 7. אבטחה משולבת:** תכונות אבטחה מובנות כמו הצפנת חומרה ו-secure boot חיוניות להגנה על מכשירי IoT ועל נתוני המשתמשים בעולם המחובר.
- 8. חיישנים מיוחדים מובנים:** ה-ESP32 כולל חיישני מגע קפסיטיביים, המאפשרים יצירת ממשקי משתמש חדשניים ללא צורך ברכיבים נוספים.
- 9. יכולות רשת מתקדמות:** תמיכה ביצירת רשתות Mesh ושימוש בפרוטוקול ESP-NOW מאפשרות יצירת רשתות תקשורת מרובות ורב-כיווניות.
- 10. התאמה למגוון יישומים:** מפרויקטים פשוטים ועד למערכות תעשייתיות מורכבות, ה-ESP32 מציע את הגמישות והעוצמה הדרושות למגוון רחב של יישומים.



[**➤ לחצו כאן לחזרה לתוכן העניינים**](#)

ESP_NOW

פרוטוקול ESP-NOW הוא טכנולוגיית תקשורת אלחוטית ללא חיבור (Connectionless) שפותחה על ידי חברת Espressif, המאפשרת למספר מכשירים לתקשר זה עם זה ללא צורך בתשתית Wi-Fi או בראוטר. הפרוטוקול פועל בתדר 2.4 GHz ומאופיין בצריכת הספק נמוכה, בדומה לקישוריות הקיימת בצידוד היקפי אלחוטי. על מנת ליצור תקשורת, נדרש ביצוע של תהליך צימוד (Pairing) בין המכשירים מראש. לאחר השלמת הצימוד, נוצר חיבור מאובטח בתצורת עמית-לעמית (Peer-to-Peer) אשר אינו מצריך תהליך "לחיצת יד" (Handshake) " בכל העברת נתונים. מאפיין זה מאפשר העברת חבילות מידע במהירות יוצאת דופן וביעילות אנרגטית גבוהה, מה שהופך את הפרוטוקול לפתרון אידיאלי עבור יישומי IoT וחיישנים המופעלים על סוללות.

מאפייני פרוטוקול ESP-NOW:

- **הגדרה:** על פי אתר האינטרנט של Espressif, זהו פרוטוקול המאפשר למספר מכשירים לתקשר זה עם זה מבלי להשתמש ב-Wi-Fi.
- **טכנולוגיה:** הפרוטוקול דומה לקישוריות אלחוטית בתדר { 2.4 GHz } בעלת הספק נמוך.
- **תהליך הצימוד:** נדרש ביצוע צימוד (Pairing) בין המכשירים לפני תחילת התקשורת ביניהם.
- **אבטחה וחיבור:** לאחר סיום הצימוד, החיבור הוא בטוח ובתצורת עמית-לעמית Peer to Peer, כאשר אין צורך בתהליך "לחיצת יד" (Handshake) ".
- **רציפות התקשורת:** החיבור בין המכשירים הוא קבוע (Persistent). אם אחד הלוחות מאבד מתח או מתאפס, הוא יתחבר מחדש באופן אוטומטי לעמית שלו עם הפעלתו מחדש כדי להמשיך בתקשורת.

יכולות הפרוטוקול:

- תמיכה בתקשורת Unicast מוצפנת ושאינה מוצפנת.
- תמיכה בשילוב של מכשירי עמית מוצפנים ושאינם מוצפנים.
- יכולת נשיאת מטען נתונים (Payload) של עד **250 בתים**.
- אפשרות להגדרת פונקציית Callback לשליחה, המיידעת את שכבת האפליקציה על הצלחה או כישלון של השידור.

מגבלות טכניות:

- **מגבלת עמיתים מוצפנים:** תמיכה ב-10 עמיתים מוצפנים לכל היותר במצב Station, ועד 6 עמיתים במצב SoftAP או מצב משולב (SoftAP + Station).
- **סך עמיתים כללי:** ניתן לצמד מספר עמיתים שאינם מוצפנים, אך המספר הכולל של המכשירים (כולל המוצפנים) חייב להיות נמוך מ-20.
- **נפח הודעה:** מטען הנתונים מוגבל ל- **250 בתים** בלבד.

במילים פשוטות **ESP-NOW**, הוא פרוטוקול תקשורת מהיר שניתן להשתמש בו כדי להחליף הודעות קטנות (עד 250 בתים) בין לוחות ESP32.

ESP-NOW הוא ורסטילי מאוד וניתן לקיים באמצעותו תקשורת חד-כיוונית או דו-כיוונית בתצורות שונות.

תקשורת חד-כיוונית ב-ESP-NOW

לדוגמה, בתקשורת חד-כיוונית, ניתן לקיים תרחישים כגון :

- לוח ESP32 אחד השולח נתונים ללוח ESP32 אחר.

תצורה זו קלה מאוד ליישום והיא מצוינת לשליחת נתונים מלוח אחד למשנהו, כמו קריאות חיישנים או פקודות הפעלה (ON) וכיבוי (OFF) לשליטה בפיני ה-GPIO.



שליחה מלוח אחד למספר לוחות (One-to-Many):

לוח ESP32 אחד השולח פקודות זהות או שונות ללוחות ESP32 שונים. תצורה זו אידיאלית לבניית מכשיר הדומה לשלט רחוק. ניתן להציב מספר לוחות ESP32 ברחבי הבית הנשלטים על ידי לוח ESP32 מרכזי אחד.



כמה נקודות לציון בשימוש בתצורה זו:

- **שליטה מרכזית:** לוח אחד משמש כ"מאסטר (Master)" והשאר כסלייבים (Slaves) ".
- **גמישות:** ניתן לשלוח פקודה אחת לכולם בו-זמנית או פקודות ייעודיות לכל לוח בנפרד.

איסוף נתונים ממספר לוחות ללוח אחד (Many-to-One):

תצורה זו אידיאלית אם ברצונך לאסוף נתונים ממספר צמתי חיישנים ללוח ESP32 אחד. ניתן להגדיר לוח זה כשרת אינטרנט (Web Server) כדי להציג נתונים מכל שאר הלוחות, למשל.



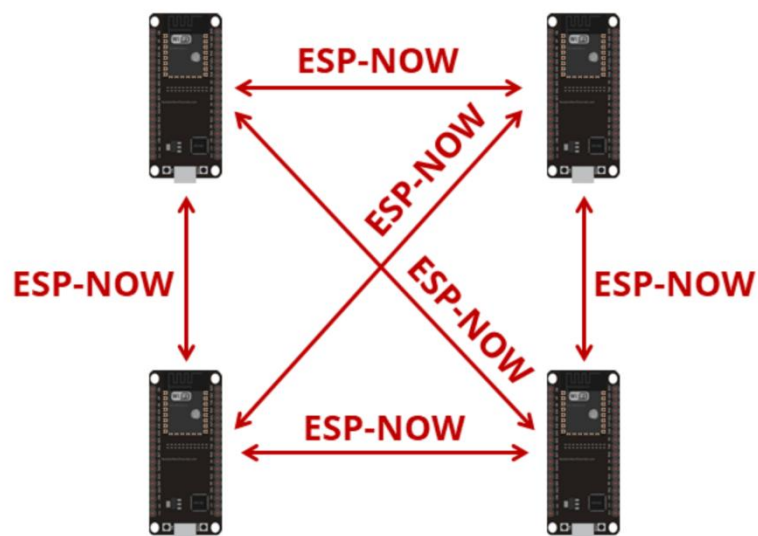
תקשורת דו-כיוונית ב-ESP-NOW (Two-Way Communication):

עם ESP-NOW, כל לוח יכול לתפקד כשולח וכמקבל בו-זמנית. כך שניתן:

- לקיים תקשורת דו-כיוונית בין שני לוחות: לדוגמה, שני לוחות ESP32 יכולים להחליף ביניהם הודעות, קריאות חיישנים או אישורי קבלה.
 - להגדיר רשת שבה כל לוח מתקשר עם האחרים: בתצורה זו, כל יחידה מסוגלת לשלוח נתונים וגם להאזין לנתונים המגיעים מלוחות אחרים ברשת.
- תצורה זו שימושית במיוחד כאשר יש צורך במשוב – (Feedback) למשל, לוח אחד שולח פקודה להדליק אור, והלוח השני מחזיר אישור שהפעולה בוצעה בהצלחה.



ניתן להוסיף לוחות נוספים לתצורה זו וליצור מבנה הדומה לרשת) כל לוחות ה- ESP32 מתקשרים זה עם זה בתצורה זו, כל לוח יכול לשלוח נתונים לכל לוח אחר בטווח הקליטה, מה שמאפשר גמישות מרבית במערכות מבוזרות או ביישומי בית חכם מורכבים.



השגת כתובת ה- MAC של הלוח ESP32:

כדי לתקשר באמצעות פרוטוקול ESP-NOW, עליך לדעת את כתובת ה- MAC של ה- ESP32 המקבל. כך תדע לאיזה מכשיר לשלוח את הנתונים.

לכל לוח ESP32 יש כתובת MAC ייחודית, וזו הדרך שבה אנו מזהים כל לוח כדי לשלוח אליו נתונים באמצעות ESP-NOW.

➤ לחצו כאן לחזרה לתוכן העניינים

תקשורת טורית UART

הקדמה:

תקשורת טורית (UART) היא שיטה נפוצה ופשוטה להעברת מידע בצורה טורית (סידורית) בין שני התקנים דיגיטליים, כגון בין מיקרו-בקר לבין מחשב או מודם. זהו אחד מפרוטוקולי התקשורת הבסיסיים ביותר הנמצאים בשימוש בתחום האלקטרוניקה הדיגיטלית.

בבסיסה, תקשורת UART מורכבת וכוללת פרוטוקולים המאפשרים שידור וקליטה של מידע בצורה אמינה ויעילה. היא משתמשת בשני חוטי תקשורת עיקריים: "קבל (RX)" ו-"שלח (TX)".

מהי תקשורת UART?

תקשורת טורית (UART) היא שיטה פשוטה ונוחה להעברת מידע בין שני התקנים דיגיטליים, למשל בין מיקרו-בקר כמו ארדואינו למחשב.

בתקשורת UART המידע מועבר בצורת **סיביות** (זרמים מתחלפים גבוהות ונמוכות - ראו תמונה בהמשך) ומועברות בחוטי תקשורת נפרדים בין שני המכשירים. כל מכשיר מחובר עם שני פינים לתקשורת - **קלט (RX)** ו**פלט (TX)**.

כדי לשדר מידע, המכשיר (השולח) שולח את הביטים על גבי קו ה-TX. בצד השני, המקבל מאזין לאותו קו ומזהה את המידע בכניסת ה-RX. שלו ומפענח אותו בחזרה למידע דיגיטלי.

עקרונות חשובים ב-UART הם **קצב התקשורת** - baud rate (כמה ביטים בשנייה), **אורך קבוע** - 8 (5 ביטים בשנייה), **ביט התחלה** (סינכרון לתחילת העברת מידע), **וביט סיום** (סימון סיום ההעברה).

היתרונות הגדולים של תקשורת טורית הם פשטות ומינימליות בחומרת המכשיר, מה שמאפשר להשתמש בה ליישומים רבים עם מגבלות תוכנה וחומרה.

לכן UART היא אחת השיטות הנפוצות להעברת נתונים בין מיקרו-בקרים לבין מחשבים במגוון יישומים כמו התקני אינטרנט של הדברים, מערכות בקרה תעשייתית, רובוטיקה ועוד.

לסיכום: תקשורת טורית מאפשרת לנו לשדר ולקלוט נתונים באופן אמין ופשוט יחסית בין כל סוגי ההתקנים הדיגיטליים.

מאפיינים תקשורת טורית:

- **תקשורת סינכרונית** (סריאלית) - המידע מועבר כביט, ביט אחרי ביט, על גבי קו תקשורת יחיד.
- **א-סינכרונית** - אין שעון משותף, כל התקן פועל לפי שעונו הפנימי.
- **תקשורת דו-כיוונית** - יכולה לשדר ולקלוט נתונים.
- **חד כיווני (Simplex)** - שידור רק בכיוון אחד.
- **חצי דו כיווני (Duplex-Half)** - בכל רגע נתון, רק צד אחד משדר והשני קולט ולהפך, זהו האופן הנפוץ ביותר.
- **שידור וקליטה בקצב זהה** (קבוע מראש לדוגמה 9,600 או 115200 ביט לשנייה).
- **מבנה מוגדר היטב** של מסגרת הנתונים (פריימים) - סיביות התחלה, נתונים, סיביות עצירה.
- **דרישות חומרה פשוטות** - בדרך כלל 2 קווים בלבד (TX ו-RX).

- **ממשק חומרה פשוט וסטנדרטי** (למשל UART 16550 במחשבים).
- **שימוש נרחב** במערכות מובנות (מחשב) ובתקשורת בין מכשירים.
- **פרוטוקולי חיבור** התקני קצה כמו מודמים, מדפסות, מודמים ועוד.
- **פרוטוקולים נפוצים** הבנויים מעל UART : RS-232, RS-422, RS-485.

מבנה חבילת נתונים בתקשורת טורית:

כאשר מתקן (למשל מיקרו-בקר) מעוניין לשדר נתונים, הוא בונה חבילת נתונים במבנה מוגדר הכולל מספר שדות:

- **ביט התחלה (Start Bit)** - ירידה מ-'1' ל-'0' לוגי, המסמן את תחילת השידור.
- **נתונים** - המידע עצמו במסגרת של ביטים (בדרך כלל 5-8 ביטים).
- **ביט זוגיות (Parity Bit)** - אופציונלי לבדיקת שגיאות.
- **ביט עצירה (Stop Bit)** 1 לוגי, המסמן את סוף השידור.

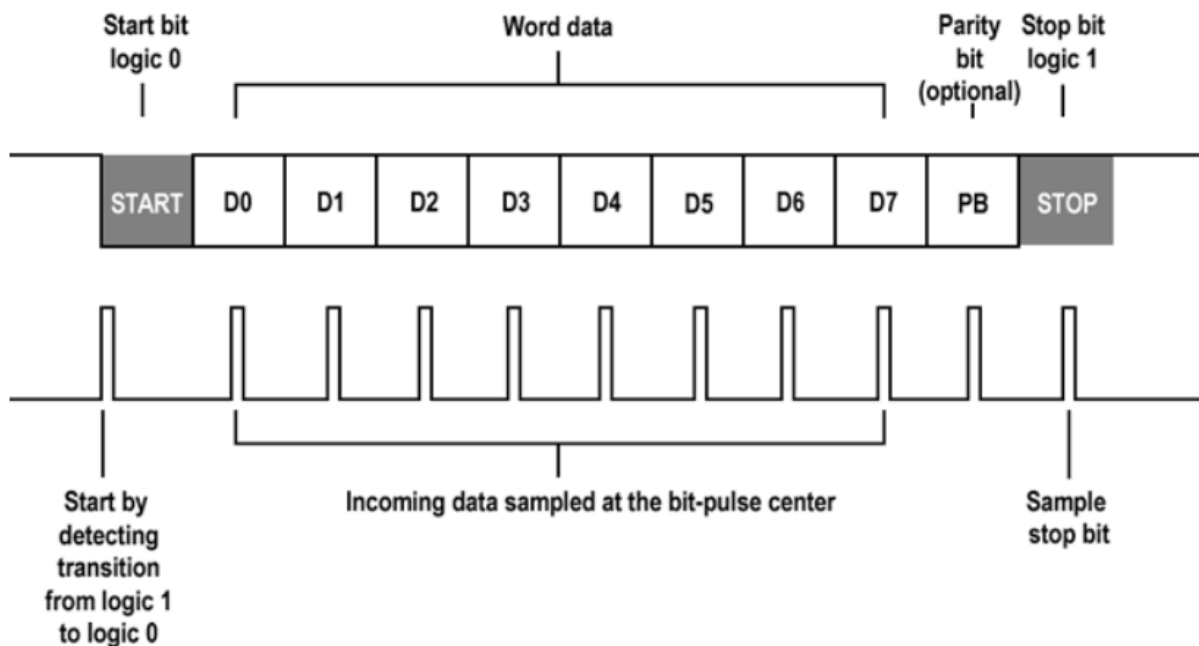
דוגמה:

במידה ונרצה לשלוח את התו 0x3F (המספר 63 בעשרוני) ייבנה מסר באורך 8 ביטים:

Start Bit (0) → 00111111 → Stop Bit (1)

כלומר, **ביט התחלה אחד (0)**, אחריו **(מימין לשמאל) הביטים של הנתון (00111111)**, ולבסוף **ביט עצירה אחד (1)**.

בין כל שידור של חבילה כזו יש מרווח זמן קצר ללא שידור, כך שהצד המקבל יוכל לזהות היטב את ביט ההתחלה הבא ובכך את תחילת הנתונים החדשים.



Parity bit (optional): ביט זוגיות (אופציונלי).

Start bit logic 0: ביט התחלה.

Stop bit logic 1: ביט סיום.

Word Data: המעיד שאנו שולחים.

Start by detecting transition from logic 1 to logic 0: תחילת שידור – מעבר מהמתח גבוה (1 לוגי) למתח הנמוך (0 לוגי) שמסמן את תחילת העברת המידע.

Incoming Data sampled at the bit – pulse center: נתונים מגיעים בפולסים של ביטים.

Sample stop bit: סיום שידור.

ביט זוגיות (Parity Bit):

ביט הזוגיות משמש לזיהוי שגיאות בסיסי בתקשורת טורית. הוא מוסיף ביט נוסף למסגרת הנתונים, כך שמספר הביטים עם ערך 1 יהיה תמיד זוגי (זוגיות זוגית) או אי-זוגי (זוגיות אי-זוגית). אם בקליטת המידע מספר הביטים עם ערך 1 לא מתאים לסוג הזוגיות שנקבע, המקלט יכול להניח שאירעה שגיאה בשידור. עם זאת, ביט זוגיות לא מאפשר לתקן שגיאות או לזהות שגיאות מרובות ביטים, ולכן הוא שיטה בסיסית בלבד לוודא שלמות הנתונים.

תקשורת טורית ב-ESP32:

ה-ESP32 – בניגוד לארדואינו אנו, מצויד במספר יחידות תקשורת טורית (UART). בדרך כלל, יש לו שלושה ממשקי UART, מה שהופך אותו למתאים במיוחד לפרויקטים הדורשים חיבור למספר התקנים חיצוניים כמו מודולים GPS, חיישני טמפרטורה, מסכי LCD ועוד.

הבדלים עיקריים בין תקשורת טורית בארדואינו ל-ESP32:

- **מספר יחידות UART:** בעוד שלארדואינו אנו יש רק יחידת UART אחת (בסיכות 0 ו-1), ל-ESP32 יש שלושה ממשקי UART: (UART0, UART1, UART2) זה מאפשר לפתח פרויקטים מורכבים יותר המשתמשים במספר רב של מכשירים הפועלים בשיטה טורית.
- **חופש בבחירת הפינים:** ה-ESP32 מאפשר למתכנת לבחור באופן חופשי את הפינים (GPIO) שישמשו לכל יחידת UART, בזכות יכולת מולטיפקסינג הפינים. המשמעות היא שניתן להגדיר את פיני ה-TX ו-RX של כל UART לכל פין GPIO פנוי בלוח, דבר המעניק גמישות רבה בתכנון החומרה.
- **פונקציונליות:** כל יחידת UART ב-ESP32 היא עצמאית לחלוטין וכוללת מאגרים (buffers) גדולים לקליטה ושידור. זה מבטיח שהמידע לא יאבד גם כאשר יש עומס גדול של נתונים, ומאפשר שימוש ב-UART0 (שמחובר גם ל-USB) לצורך ניפוי באגים (debug) בתוכנה, תוך כדי שימוש ב-UART1 ו-UART2 לצורך תקשורת עם רכיבים אחרים.

קצב שליחת נתונים:

אנו מגדירים את קצב שליחת הנתונים בתקשורת טורית בפקודה אחת בלבד בפונקציה void Setup באמצעות הפקודה:

Serial.begin(115200);

רשימת קצבי השידור הנפוצים בתקשורת טורית (UART):

- 300 ביט לשנייה
- 600 ביט לשנייה
- 1,200 ביט לשנייה
- 2,400 ביט לשנייה
- 4,800 ביט לשנייה
- 9,600 ביט לשנייה
- 19,200 ביט לשנייה
- 38,400 ביט לשנייה
- 57,600 ביט לשנייה
- 115,200 ביט לשנייה
- 230,400 ביט לשנייה
- 460,800 ביט לשנייה
- 921,600 ביט לשנייה

ניתן לראות שיש מגוון רחב של קצבי שידור, החל מ-300 ביט לשנייה במערכות ישנות יותר, ועד למיליוני ביטים לשנייה בממשקים טוריים מהירים יותר.

הקצבים הנפוצים ביותר הם: 9,600 ביט לשנייה ו-115,200 ביט לשנייה.

קצב השידור נקבע מראש ומחייב להיות זהה בין שני ההתקנים המתקשרים כדי לאפשר קליטה נכונה של הנתונים.

רכיבים שעובדים בתקשורת טורית:

- מודולי בלוטוס - HC-05, HC-06
- מודולי WiFi - ESP8266, ESP32
- מודולי LoRaWAN/LoRa - לתקשורת לטווח רחוק
- מודולי תקשורת סלולרית - SIM900, SIM800L
- מודולי GPS - NEO-6M, UBLOX
- צגי OLED ו-LCD טוריים (I2C)
- חיישני טמפרטורה, לחות, אור ותנועה
- מודולי קוראי RFID/NFC
- מודולי מצלמות טוריות
- כרטיסי SD וקוראי כרטיסי SD
- מדפסות ומפענחים טוריים

שליחת נתונים בתקשורת טורית:

ניתן לשלוח נתונים בתקשורת טורית באמצעות הפקודה:

Serial.print();

כל מה שנרשום בין הסוגריים () עם מרכאות ישלח בתור מחרוזת, ללא מרכאות ישלח בתור ערך של משתנה. בפקודה זו **אין ירידת שורה**. במידה ונרצה לרדת שורה אנו צריכים להשתמש בפקודה:

Serial.println();

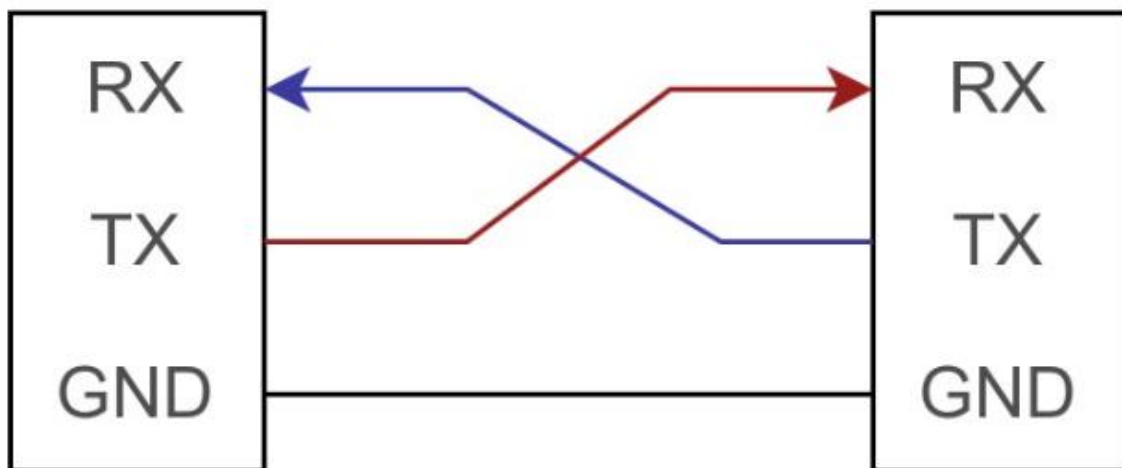
חיבור בין 2 רכיבים בתקשורת טורית:

בחיבור בין שני התקני UART נדרשים לפחות 3 חוטים - TX של צד אחד מתחבר ל-RX של הצד השני, וחוט הארקה משותף (GND).

לעתים נדרש גם חיבור של מתח ההזנה (V_{cc}) אם אחד ההתקנים מספק כוח לשני. יש לוודא התאמה של רמות המתח בין שני הצדדים (3.3V או 5V).

Device 1

Device 2



➤ לחצו כאן לחזרה לתוכן העניינים

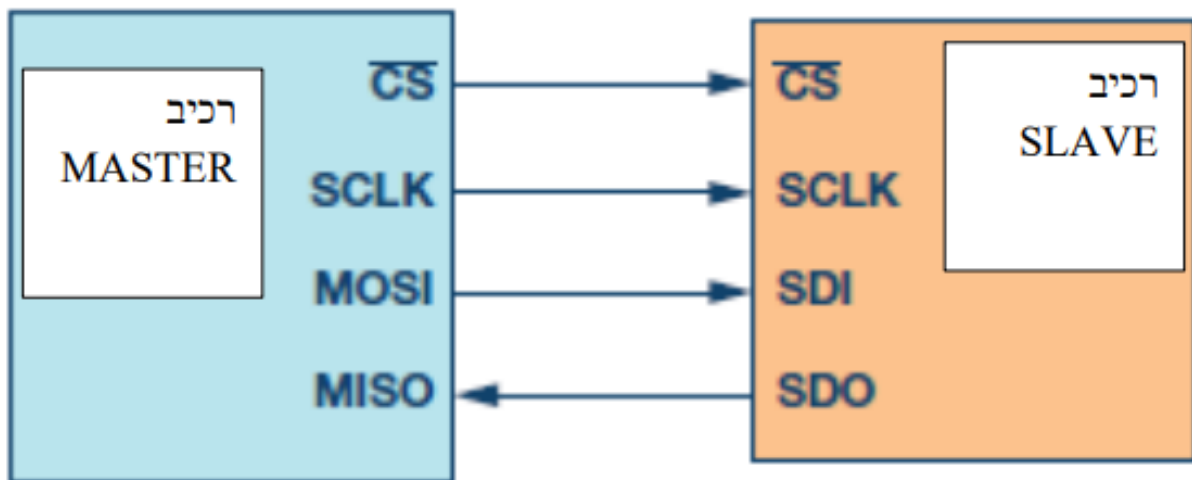
תקשורת SPI

התקשורת נוצרה על ידי חברת מוטורולה ב 1979 עם יציאת המיקרו פרוססור של חברת מוטורולה שנקרא 68000 .

השם SPI הוא - Interface Peripheral Serial – ממשק טורי היקפי . זוהי תקשורת טורית סינכרונית כי יש בה שעון שמסנכרן את הכנסת/ הוצאת הנתונים .

דוגמאות לרכיבי SPI הם מתגים, זיכרונות, רכיבי שמע להקלטה/השמעה , ממירים למיניהם (ADC), תצוגות לדס, TFT ועוד .

באיור הבא מתואר חיבור של תקשורת טורית SPI בין רכיב MASTER ורכיב SLAVE .



באיור רואים שבתקשורת SPI יש 4 קווים:

1. **Master out slave in – MOSI**: קו הנתון מהמסטר (המיקרו בקר) אל העבד (ברכיב העבד השם הוא SDI – Serial Data In).
2. **Master in slave out – MISO**: קו הנתון הטורי מהעבד אל המסטר. ברכיב העבד השם הוא SDO – Serial Data Out).
3. **Serial clock – SCLK**: שעון טורי. שני האותות MOSI ו-MISO מסונכרנים בעזרת קו השעון הטורי.
4. **Slave select**: זהו קו נוסף שדרכו אנו בוחרים את העבד ה-slave, בעזרת קו זה המיקרו בקר מודיע לאיזה עבד הוא פונה. הקו פעיל בנמוך – LOW ACTIVE. שמות נוספים לקו הם CS – בחירת רכיב – Enable, אפשר ועוד.

תהליך התקשורת מתבצעת בשלבים הבאים:

- כאשר ה-MASTER מתחיל תקשורת עם ה-SLAVE הוא מוריד את קו בחירת הרכיב עבד (Select slave) ל-0.
- ה-MASTER שולח בקו ה-MOSI ביט אחרי ביט, כאשר כל ביט מסונכרן בעזרת פולס שעון – SCLK – שמייצר ה-MASTER לתוך ה-SLAVE.
- הביטים הנשלחים נמצאים ברגיסטר הזזה הנמצא בתוך ה-SLAVE.
- הסנכרון לתוך ה-SLAVE יכול להיות בעליית פולס שעון או בירידה שלו.

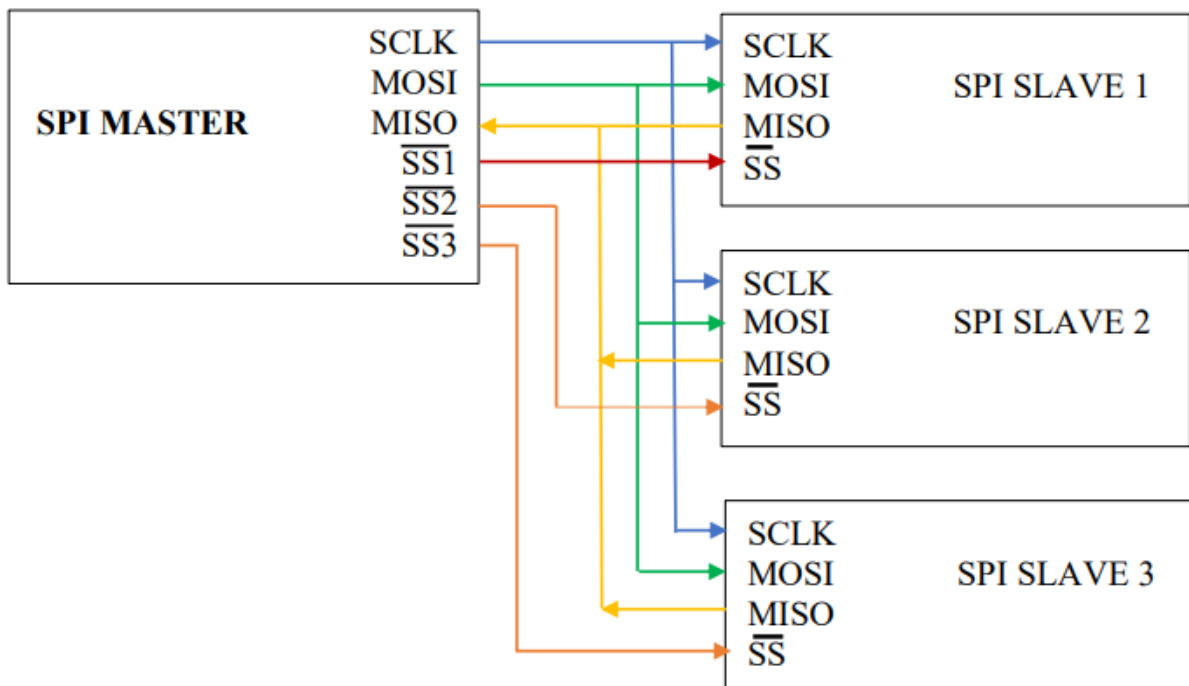
- תוך כדי שליחה של כל ביט מה-MASTER אל ה-SLAVE גם ה-SLAVE מוציא ביט אחרי ביט נתון אל קו ה-MISO.
- גם הביטים מה-SLAVE מסונכרנים בעזרת פולסי השעון ב-SCLK והם נכנסים לרגיסטר הזזה ב-MASTER.
- בסיום שליחת הביטים ה-MASTER מעלה את קו בחירת הרכיב עבד (Select slave) ל-1 ומסיים תקשורת.
- בסיום התקשורת ה-MASTER לא מייצר פולסי שעון, והקו של פולסי השעון SCLK נמצא במצב IDLE מצב סרק. הוא יכול להיות ב-0 או ב-1.

חיבור מספר SLAVES אל ה-MASTER:

ניתן לחבר ל-MASTER אחד מספר SLAVES, מה שיקרה למעשה שהקווים המשותפים לכל ה-SLAVES הם SCLK, MISO, MOSI. הקו שנבדל בין כולם הוא הקו לבחירת רכיב העבד (Slave select).

כל ה-SLAVES מקבלים במקביל את פולסי השעון ואת קווי ה-MOSI ו-MISO. קו ה-Slave select של כל רכיב יהיה ב-1 (כלומר הרכיב לא נבחר) וה-MASTER יוריד את קו בחירת רכיב העבד ל-0 רק עבור אותו ה-SLAVE שהוא רוצה לתקשר אתו.

נראה את האיור הבא שממחיש לנו חיבור של 3 SLAVES ל-MASTER:



חיבור של MASTER אחד לשלושה SLAVES שונים דרך תקשורת טורית SPI.

באיור זה נוכל לראות שלכל עבד יש קו (Slave select) משלו. ל-MASTER יש אותו מספר קווים בדומה למספר ה-SLAVES. לכל רכיב עבד יתחבר קו מסטר אחר, באיור הם נקראים:

(Slave select 1, Slave select 2, Slave select 3)

הם מסומנים עם קו מעליהם כדי להדגים שהם פעילים בנמוך (Active Low). לדוגמה אם ה-MASTER יחליט לתקשר רק עם רכיב SPI SLAVE 1 הוא יוריד את קו (Slave select 1) ל-0 ואז רק רכיב העבד הראשון ידע שה-MASTER מתקשר אתו. שאר העבדים יודעים שהמיקרו בקר אינו מתקשר איתם.

כמות ה-SLAVES שניתן לחבר אל ה-MASTER תלויים בכמות ההדקים הפנויים שיש לו במערכת בה הוא נמצא ובכיוון של דחיפת הזרם שלו. אפשרויות מתקדמות יותר על מנת לחסוך בהדקים של המיקרו בקר הן להתחבר אל הדקי Slave select של כל רכיב SPI בעזרת מפענח או מפלג DEMUX עם כניסה אחת, 3 הדקי בחירה ו-8 יציאות שונות.

אפשרויות העבודה עם תקשורת SPI:

במערכת ה-SPI (גם ה-MASTER וגם ה-SLAVE) יש 2 רגיסטרים, האחד הוא הרגיסטר הזזה המקבל את הדגימות ומזיז את הנתון, ועוד רגיסטר נתון שבסיום ההעברה הנתון שהתקבל נמצא בו.

בפולס שעון יש 2 מעברים (מגבוה לנמוך ולהפך) הכוללים גם עלייה וגם ירידה ויש לעשות 2 דברים:

- א. במעבר מ-0 ל-1 גם המסטר וגם העבד מוצאים את ביט הנתון לקו הנתון (MOSI במסטר MISO בעבד).
- ב. במעבר השני מתבצעת הזזה של הנתון ברגיסטר ההזזה שנמצא גם במסטר וגם בעבד.

שני פרמטרים/מאפיינים חשובים בתקשורת SPI:

- 1. CPOL (Clock POLarity – קוטביות השעון) הקובעת מה מצב הקו (נמוך או גבוה) במצב סרק – IDLE כאשר אין פולסי שעון (לפני התחלת תקשורת ובסיומה), כאשר $CPOL=0$ בקו יש 0 לוגי וכאשר $CPOL=1$ בקו יש 1 לוגי.
- 2. CPHA (Clock PHase – פאזה השעון) הקובעת באיזה מעבר (האם מ-0 ל-1 או מ-1 ל-0) יוצא ביט הנתון בקו ה-MOSI ובקו ה-MISO ובאיזה מעבר הוא מוזז ברגיסטר הזזה גם במסטר וגם בעבד. פרמטר נוסף חשוב הוא באיזה מצב נמצא קו השעון SCLK בסיום התקשורת (האם ב-0 או ב-1).

תבנית העברה של נתון עבור $CPHA=0$:

הקו Slave select פעיל בנמוך, ניתן לרשום אותו גם כך: NSS כדי שלא יהיה צורך לרשום גג SS. האיור הבא מתאר את תבנית העברה עם צורות הגל של תקשורת SPI כאשר $CPHA=0$ ועם פרמטרים של זמן אופייניים.

בחלק התחתון רואים את התחלת התקשורת כאשר קו NSS (Slave select) יורד ל-0 ואת סיומו כאשר קו זה עולה ל-1.

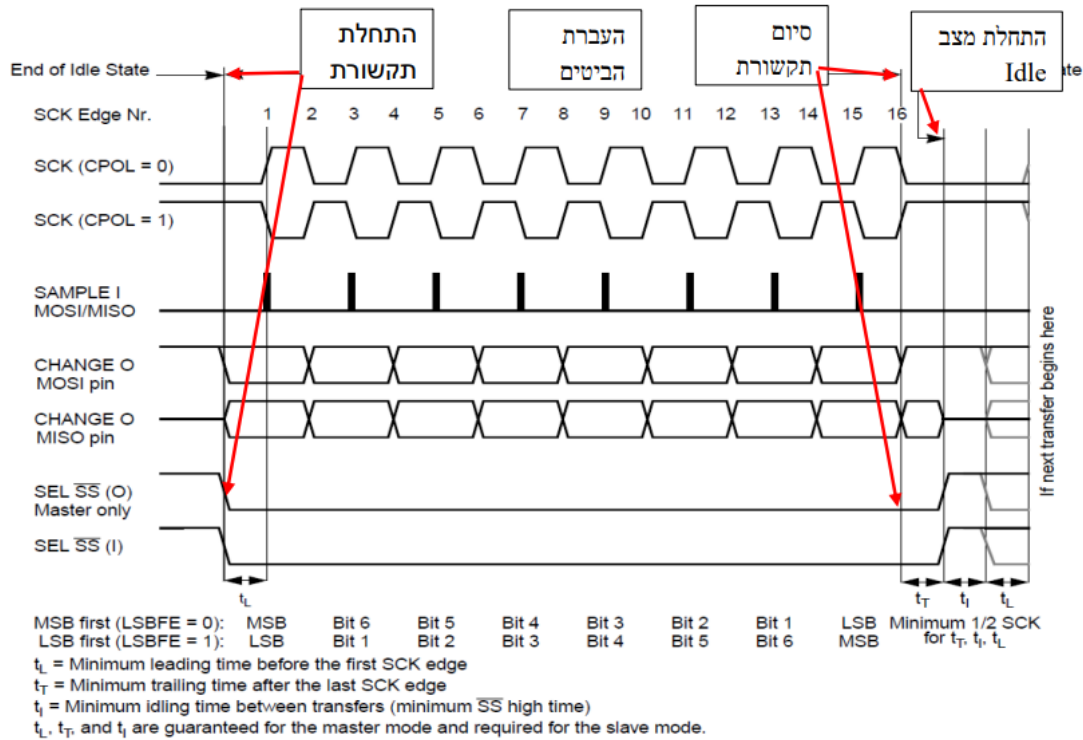
קו השעון SCK מתואר על ידי 2 צורות גלים. העליונה ביותר מתארת את פולסי השעון כאשר $CPOL=0$ (כאשר אין תקשורת יש 0 בקו פולסי השעון) והפולס הראשון הוא עלייה וצורת הגל שמתחתיה כאשר $CPOL=1$ ואז יש 1 לוגי בקו פולסי השעון והפולס הראשון הוא ירידה.

הראשון הוא ירידה.

במקום הרשום באיור O הכוונה ל-Output (יציאה) ובמקום שרשום I הכוונה ל-Input (כניסה).

הביטים של הנתון הטורי בהדקי MOSI ו-MISO נדגמים ומתבצעת הזזה ברגיסטרים של ההזזה באחת מ-2 אפשרויות. או בעליית השעון או בירידת פולסי השעון ואת זה נקבע לפי נתוני הרכיב שאליו מתחברים.

איור: צורת גל בתקשורת SPI כאשר $CPHA = 0$



המעבר הראשון של קו SCK (מ-0 ל-1 או מ-1 ל-0 תלוי ב-CPOL) משמש להכנסת ביט הנתון הראשון של העבד אל המסטר (בקו MISO) ואת ביט הנתון הראשון של המסטר אל העבד (בקו MOSI). באיור ניתן לראות במרכז את הקו שנקרא SAMPLE והוא משמש ככניסה גם למסטר בקו MISO וגם לעבד בקו MOSI.

במעבר השני, בחצי המחזור הבא, מופיע מעבר נוסף בקו SCK ואז הערך שנדגם מהעבד בקו MISO נכנס לתוך רגיסטר ההזזה שבמסטר. אחרי המעבר הזה משודר הביט הבא של המסטר על קו MOSI. התהליך נמשך עד 16 מעברים בקו SCK כאשר נתון נעל אל המסטר במעברים אי-זוגיים ומועבר אל העבד במעברים זוגיים.

אחרי מעבר מספר 16 הנתון שהיה ברגיסטר ה-SPI של המסטר צריך להיות ברגיסטר הנתון של העבד והנתון שהיה ברגיסטר הנתון של העבד צריך להיות במסטר.

באיור מופיעים זמנים כגון:

- t_L המראה מה הזמן המינימאלי מרגע הורדת קו NSS (Slave select) ל-0 ועד להתחלת פולסי השעון.
- t_T הוא הזמן המינימאלי בין פולס השעון האחרון והעלאת קו NSS (Slave select) ל-0.
- t_i הוא זמן הסרק - IDLE - בין העלאת קו NSS (Slave select) ל-1 ועד שידור נתון חדש על ידי הורדת NSS (Slave select) ל-0.

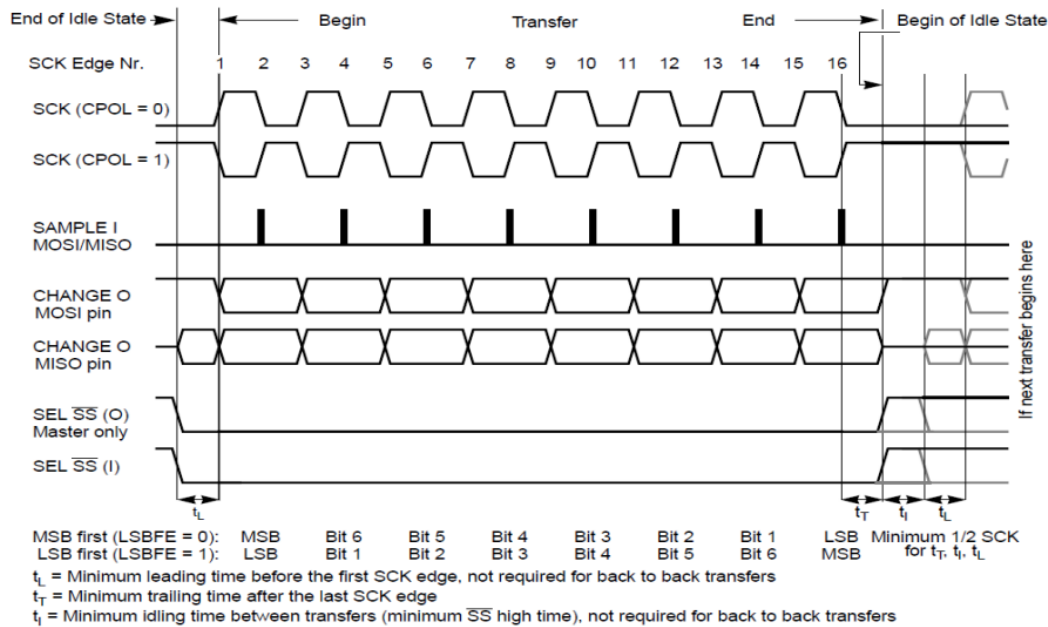
מתחת לצורות הגלים ניתן לראות שאפשר לשדר את הנתון מביט ה-LSB אל ה-MSB או להיפך. קיים ביט LSBFE (LSB First Enable) אפשוך LSB ראשון) שקובע מי נכנס ראשון. כאשר הוא 0 נכנס ביט ה-MSB ראשון וכאשר הוא 1 נכנס ה-LSB ראשון.

קבלת הנתון מתבצעת בשני שלבים. היא מוזזת לרגיסטר ההזזה של ה-SPI בזמן ההעברה ומועברת לרגיסטר הנתון של ה-SPI כאשר הביט האחרון הוזז פנימה.

תבנית העברה של נתון עבור $CPHA = 1$:

קיימים רכיבים שצריכים את המעבר של פולס השעון הראשון לפני שניתן יהיה לגשת לביט הנתון הראשון בקו היציאה של הנתון. המעבר השני מכניס את הנתון למערכת. פורמט זה מתקבל על ידי השמה של $CPHA = 1$.

האיור הבא מתאר את התקשורת עבור $CPHA = 1$.

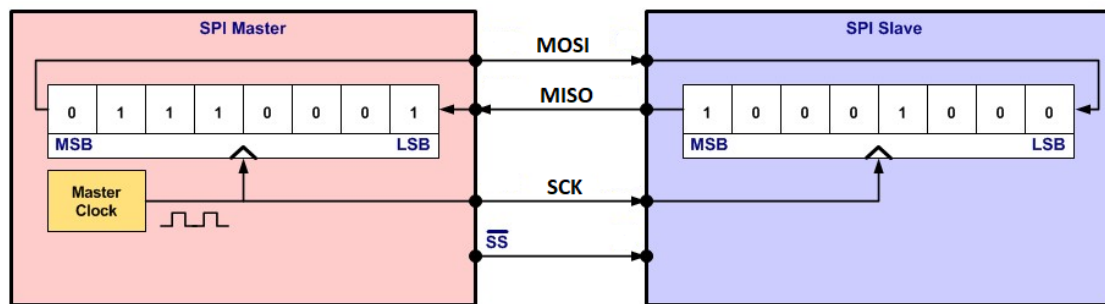


באיור רואים שהמעבר הראשון משמש כהשהיית סנכרון ואומר לעבד להוציא נתון בקו ה-MISO בחצי המחזור הבא, במעבר השני, יש נעילה של ביט הנתון גם במסטר וגם בעבד. כאשר מגיע המעבר השלישי מועבר הביט שננעל במעבר השני לתוך ה-LSB או ה-MSB של רגיסטר ההזזה (כתלות בביט LSBFE).

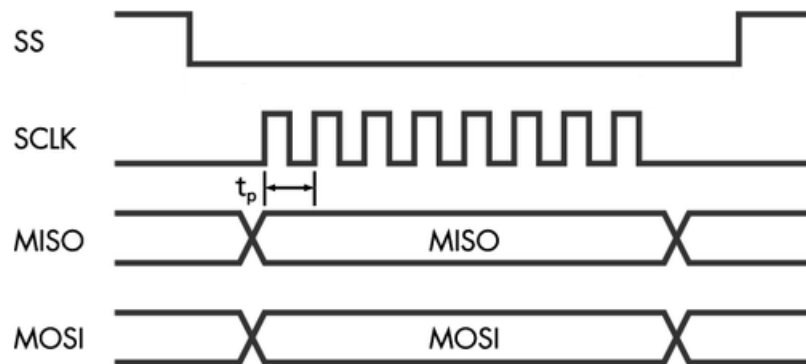
אחרי מעבר זה יוצא ביט הנתון הבא של המסטר בקו ה-MOSI. התהליך חוזר על עצמו עבור כל 16 המעברים, כאשר הנתון ננעל במעברים הזוגיים וההזזה קוראת במעברים האי זוגיים.

המידע מועבר בו זמנית בשני הכיוונים בכל עליית שעון באמצעות אוגרי הזזה.

בדוגמה הבאה: לאחר 8 מחזורי שעון מידע מוחלף בין ה-Master ל-Slave.



שליחת מילת בקרה של 8 סיביות.



בתחילת התקשורת, הדק SS יורדת ל-0. לאחר מכן מועבר המידע מה-Master דרך הדק MOSI במקביל. ה-Master יכול לקרוא את המידע מה-Slave דרך הדק MISO. בסיום התקשורת הדק SS חוזרת ל-1 לוגי.

t_p – זמן מחזור של השעון.

[לחצו כאן לחזרה לתוכן העניינים](#)

תקשורת I2C

הקדמה:

פרוטוקול I2C הוא שיטת תקשורת סריאלית נפוצה בין התקני אלקטרוניקה וחיישנים. הוא מאפשר העברת נתונים בין מספר התקנים, כאשר יתרונו הגדול הוא שהוא דורש רק שני חוטים לתקשורת – החוט לאותות השעון והחוט לאות הנתונים.

מהו הפרוטוקול I2C :

פרוטוקול I2C (אי-שתיים-סי) הוא שיטה סריאלית להעברת נתונים בין התקנים, שפותחה על ידי חברת פיליפס בשנות ה-80.

בניגוד לתקשורת UART שדורשת שני חוטי תקשורת לכל כיוון, ב-I2C יש רק שני חוטים משותפים לשני הכיוונים: SDA לנתונים ו-SCL לאות שעון. שני חוטים אלו מחוברים לכל ההתקנים בטופולוגיית BUS.

כל התקן ברשת I2C מזוהה בכתובת ייחודית. כאשר מתקן אחד רוצה לתקשר, הוא שולח הודעה עם הכתובת של המתקן היעד. רק המתקן עם אותה כתובת "יורס את השפופרת".

העברת הנתונים עצמה נעשית לפי עקרון מאסטר/עבד (לרוב מאסטר עובד), מפעיל את אות השעון ושולט מתי לשדר ומתי לקבל נתונים, ואילו המתקנים המשניים הם עובדים פסיביים.

ב-I2C כל ביט נשלח ברצף אחרי הביט הקודם, והוא תוקף כל עוד אות השעון SCL גבוה. ירדת SCL מהווה איתות לביט חדש.

יתרונותיו הגדולים של פרוטוקול I2C הן הפשטות והחיסכון בכמות החוטים לתקשורת, ניתן לחבר עשרות מתקנים קצה, וקל יחסית ליישם במיקרו-בקרים.

לכן זהו פרוטוקול נפוץ לתקשורת עם אלקטרוניקה כמו חיישנים, מסכי LCD, גרפים, זיכרונות וכד'.

מאפיינים פרוטוקול I2C:

- **תקשורת סדרתית (סריאלית)** - המידע מועבר בטור, ביט אחרי ביט, על גבי קו התקשורת SDA.
- **סינכרונית** - אות השעון SCL מסנכרן בין כל המכשירים.
- **תקשורת דו כיוונית** בין מאסטר למספר סלייבים.
- **שני קווי תקשורת** משותפים לכל המכשירים SDA ו-SCL.
- **קצב תקשורת** גבוה יחסית - עד 5 מגה ואף יותר.
- **כתובות מוגדרות** מראש לכל רכיב (7/10 ביט כתובת).
- **ממשק חומרה** פשוט וסטנדרטי.
- **מתאים במיוחד** לתקשורת בין מעבד להתקני קצה כמו חיישנים, מסכים וזיכרונות.
- **מצמצם מאוד** בכמות החיווט לעומת UART.

מבנה חבילת (מסגרת) נתונים בתקשורת טורית:

כאשר מתקן מאסטר (כמו מיקרו-בקר) מעוניין לשדר נתונים למתקן SLAVE, הוא בונה את החבילה במבנה מוגדר הכולל מספר שדות:

- **התחלת העברה (START)** - ירידה באות SDA מ-1 ל-0 בזמן שאות SCL נשאר גבוה, מה שמהווה "אות התחלה."
- **כתובת SLAVE** - 7 או 10 ביטים עם הכתובת הייחודית של המתקן SLAVE היעד. רק אותו מתקן "יריב את השפופרת."
- **נתונים** - המידע עצמו בסדרת ביטים הנשלחים על ידי המאסטר או שמתקבלים מהמתקן ה-SLAVE.
- **אישור 9 (ACK)** - פעולות שעון שבהן ה-SLAVE מאשר קבלת תקינה של העברת 8 ביטים בקו SDA.
- **סיום העברה (STOP)** - עלייה באות SDA מ-0 ל-1 בזמן שקו SCL נשאר גבוה, מה שמהווה "אות סיום."

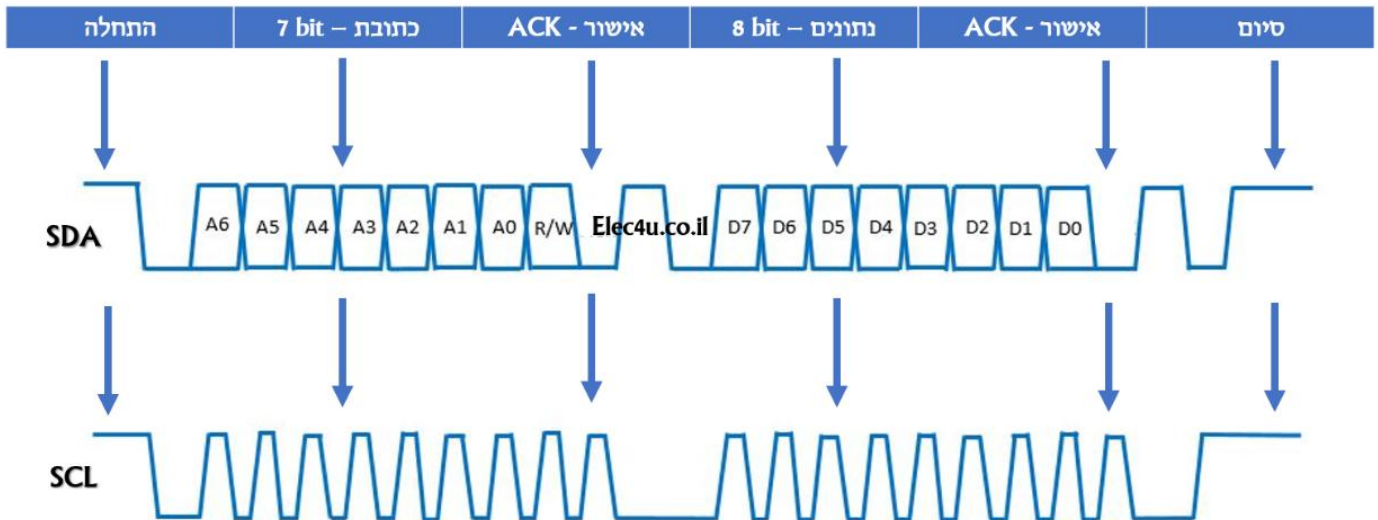
דוגמה: שליחת בייט בפרוטוקול I2C:

הנה פירוט דוגמה לשליחת הבייט 0x2F (מספר 47 עשרוני) למתקן בכתובת 0x38:

- 0111000 → START (0x38) כתובת → STOP → ACK → 00101111

פירוט שלבי התקשורת:

- **START**: מציין על התחלת העברה חדשה. נוצר על ידי ירידה של קו הנתונים SDA מ-1 ל-0, בעוד קו השעון SCL נשאר גבוה.
- 0111000 (כתובת 0x38): שבעת הביטים הבאים מציינים את כתובת ה-SLAVE (במקרה זה 0x38 או 56 עשרוני). הם נשלחים מהביט הנמוך (LSB) לביט הגבוה ביותר (MSB).
- 00101111: שמונה הביטים הבאים מציינים את הנתונים שאנחנו שולחים ל-SLAVE (במקרה זה 47 עשרוני או 0x2F). שוב, מ-LSB ל-MSB.
- **ACK**: לאחר הנתונים, ה-SLAVE חייב לשלוח ביט "אישור" אחד בחזרה למאסטר כדי לאשר שקלט את הנתונים כראוי.
- **STOP**: סיום תנועת ה-I2C. נעשה על ידי העלאת קו הנתונים SDA מ-0 ל-1, בעוד קו השעון SCL נשאר גבוה.



בחבילת נתונים של פרוטוקול I2C ישנה אפשרות לשלוח מסגרת אחת או מספר מסגרות ברצף באותה החבילה. מסגרת נתונים אחת מכילה את כמות המידע שאנו רוצים לשלוח באותו הרגע. הפרוטוקול מאפשר לנו לשלב מספר מסגרות נתונים ברצף בתוך אותה החבילה, ללא צורך לשלוח חבילה חדשה עבור כל מסגרת. זאת על ידי שליחת אישור (ACK) מהמתקן המקבל בין מסגרת למסגרת. כך ניתן לשלוח ביעילות נתונים ארוכים יותר באותה החבילה I2C.

יתרונות מרכזיים של פרוטוקול I2C :

- **חסכוני בכמות החיווט** - משתמשים רק בשני קווי תקשורת משותפים לכל ההתקנים - קו SDA לנתונים וקו SCL לאות השעון.
- **תקשורת דו-כיוונית** - מאפשר הן למאסטר לשלוח נתונים והן ל-SLAVE להחזיר נתונים.
- **פשוט לחיבור התקני קצה רבים** - ניתן לחבר עשרות התקנים שונים על אותה רשת I2C.
- **ממשק תקשורת פשוט וסטנדרטי** - כל רכיבי החומרה מתוכננים לתמוך בפרוטוקול.
- **מהירויות תקשורת גבוהות** - עד 5 מגה-הרץ, מתאים להעברת נתונים מחיישנים, שליטה ועוד.
- **אמינות גבוהה** - שימוש ב-ACK ומנגנוני זיהוי שגיאות.
- **כתובות מוגדרות מראש** - מאפשר תקשורת מכוונת בין התקני הרשת.

כמות רכיבים שניתן לחבר לפרוטוקול I2C:

בפרוטוקול I2C ניתן לחבר מספר רב של רכיבים, אך ישנה הגבלה על הכמות המקסימלית. בפועל, ניתן לחבר עד 128 רכיבי קצה (SLAVES) לכל רשת I2C. הסיבה להגבלה זו היא שימוש ב-7 ביטים לייצוג כתובת הרכיב בפרוטוקול, מה שמאפשר טווח כתובות של 0 עד 127 (כלומר 128 כתובות שונות). כמות זו של 128 רכיבים נחשבת גבוהה יחסית ומספיקה לרוב המערכות והיישומים הנוכחיים.

הרחבה ל-10 ביטים:

ישנה אפשרות להשתמש גם בכתובות עם 10 ביטים, מה שמרחיב את טווח הכתובות ומאפשר חיבור של מספר רב יותר של רכיבים. עם זאת, השימוש ב-10 ביטים מורכב יותר ופחות נפוץ.

הגבלות נוספות:

מלבד מגבלת הכתובות, קיימות גם מגבלות פיזיות שעלולות להשפיע על כמות הרכיבים המקסימלית שניתן לחבר בפועל. מגבלות אלו כוללות:

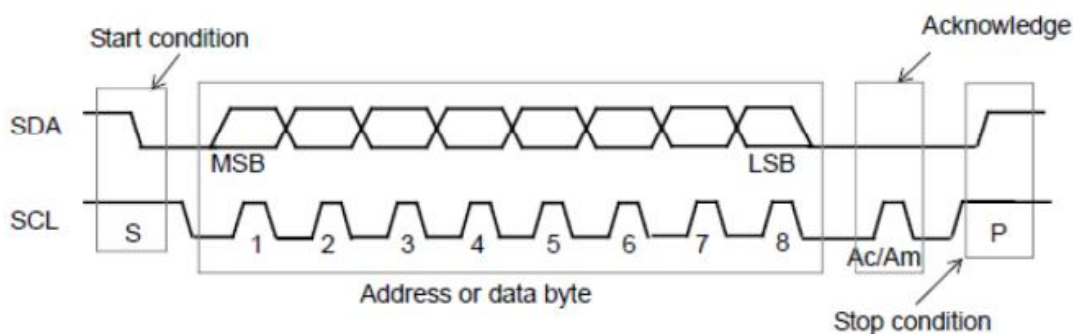
- **קיבוליות (Capacitance):** ככל שמוסיפים יותר רכיבים, כך גדלה הקיבוליות על קווי התקשורת SDA ו-SCL. קיבוליות גבוהה מדי יכולה לעוות את האותות החשמליים ולשבש את התקשורת.
 - **אורך החוטים:** ככל שהחוטים ארוכים יותר, כך גדלים הסיכון לרעש חשמלי ולירידת מתח.
 - **זיהום סביבתי:** סביבה רועשת מבחינה חשמלית עלולה להשפיע על האותות.
- בפועל, למרות התיאוריה של 128 כתובות, יש לקחת בחשבון את המגבלות הללו, ולרוב רשתות I2C פשוטות מוגבלות למספר קטן יותר של רכיבים.

פעולות בסיסיות בפרוטוקול התקשורת I2C:

כאשר שני האותות SCL ו-SDA, נמצאים במצב 1 לוגי ה-BUS במנוחה. השידור עצמו נעשה בשיטת MSB – First.

- האות SDA חייב להיות יציב כשהאות SCL במצב 1 לוגי.
- האות SDA יכול להשתנות רק כשהאות SCL במצב 0 לוגי.

נראה תמונה שמתארת העברת מידע בפרוטוקול בדומה לתמונה מהעמוד הקודם:



ניתן לראות שפעולת START מתבצעת כאשר SDA בירידה ו-SCL נמצא עדיין ב-1 לוגי. לעומת זאת פעולת STOP מתבצעת כאשר SDA בעלייה ו-SCL כבר ב-1 לוגי.

הרכיב שמקבל את המידע (ה-MASTER או ה-SLAVE) מחזירים Acknowledge באמצעות הורדת SDA ל-0 לוגי, רכיבי SLAVE חייבים תמיד להשיב באות Acknowledge לפנייה לכתובת שלהם.

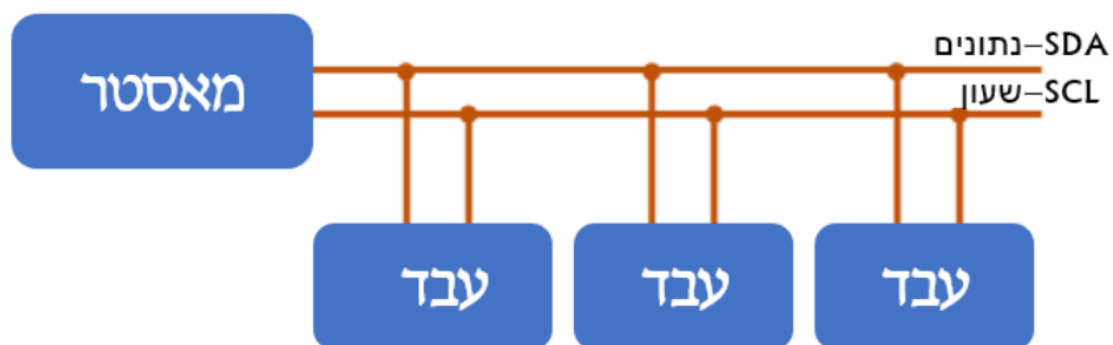
מצבים בהם רכיבי SLAVE לא מגיבים ב-Acknowledge :

רכיבי SLAVE יכולים שלא להגיב בתגובת Acknowledge (כלומר להגיב ב-NACK), במקרים הבאים :

- ה-SLAVE עסוק (במקרה כזה הוא יכול גם להאריך את ה-SCL).
- ה-SLAVE קיבל מידע לא חוקי.

לאחר קבלת NACK ה-MASTER חייב לבצע פעולת STOP. כאשר ה-MASTER הוא ה-Receiver הוא יכול שלא להגיב באות Acknowledge (כלומר להגיב ב-NACK) כדי לסמן שהגיע ל-Byte הסופי של המידע. מערכות שפועלות בפרוטוקול I2C יכולות לבצע פעולות Acknowledge גם ברמת ה-Byte.

הסבר כיצד הפרוטוקול I2C פועל:



פרוטוקול I2C הוא שיטת תקשורת טורית בין מיקרו-בקר לרכיבי ציוד היקפי, המשתמשת בשני קווים בלבד.

- **רגל - SDA (Serial Data)** אחראית על שליחת וקבלת המידע בין המכשירים.
- **רגל - SCL (Serial Clock)** אחראית על אות השעון, המסנכרן את התקשורת בין כל רכיב, כך שהם "מדברים" יחד ובזמן הנכון.

הארדואינו/ESP32, הם ה"מאסטרים" והוא שולט על הקווים, והשאר הם ה"עבדים" (SLAVES). כאשר הארדואינו/ESP32 רוצה לתקשר עם אחד החיישנים, הוא שולח אליו את הכתובת הייחודית שלו. לדוגמה :

- תצוגת LCD I2C 2X16 עובדת בכתובת 0x27.
- חיישן טמפרטורה LM75 עובד בכתובת 0x07 או 0x48.

UART (Universal Asynchronous Receiver/Transmitter)

פרוטוקול תקשורת טורית פשוט המשתמש בשני קווים - אחד לשידור ואחד לקליטת נתונים. משמש בדרך כלל לתקשורת בין מיקרו-בקרים וציוד היקפי כגון חיישנים או צגים. זהו פרוטוקול בסיסי מאוד שקל ליישום ותומך בתקשורת למרחקים ארוכים. עם זאת, הוא איטי יחסית, אין לו זיהוי או תיקון שגיאות ואינו מתאים להעברת נתונים במהירות גבוהה או בנפח גבוה.

- **יתרונות:** קל ליישום, תומך בתקשורת למרחקים ארוכים.
- **חסרונות:** העברת נתונים איטית, אין זיהוי או תיקון שגיאות, לא מתאים להעברת נתונים במהירות גבוהה או בנפח גבוה.

SPI (Serial Peripheral Interface)

פרוטוקול סינכרוני בדופלקס מלא, המאפשר תקשורת בין התקן מאסטר לרכיב עבד אחד או יותר. הוא משתמש בארבעה קווים - אחד להעברת נתונים, אחד לקליטת נתונים ושניים לשליטה בתקשורת. משמש בדרך כלל לתקשורת בין מיקרו-בקרים וציוד היקפי הדורשים העברת נתונים במהירות גבוהה, כגון זיכרון פלאש, צגי LCD ומודולי Wi-Fi. תומך בתקשורת במהירויות גבוהות מאוד, אבל מורכב יותר ליישום מאשר UART או I2C.

- **יתרונות:** העברת נתונים במהירות גבוהה, מתאים לתקשורת עם מספר רכיבים, תומך בתקשורת דופלקס מלא.
- **חסרונות:** מורכב יותר ליישום מאשר UART או I2C, דורש יותר קווים מאשר UART או I2C.

I2C (Inter-Integrated Circuit)

פרוטוקול סינכרוני, רב-מאסטר, מרובה עבדים המשתמש בשני קווים - אחד להעברת נתונים ואחד לסנכרון שעון. משמש בדרך כלל לתקשורת בין מיקרו-בקרים וציוד היקפי, כגון חיישנים, EEPROMs וצגי LCD. הוא איטי יותר מ-SPI. אך דורש פחות קווים וקל יותר ליישום. עם זאת, הוא אינו מתאים להעברת נתונים במהירות גבוהה או בנפח גבוה.

- **יתרונות:** דורש פחות קווים מ-SPI, קל יותר ליישום מאשר SPI, תומך בתקשורת עם מספר רכיבים.
- **חסרונות:** איטי יותר מ-SPI, לא מתאים להעברת נתונים במהירות גבוהה או בנפח גבוה.

מסך 1.8 – TFT (ST7735)



זהו המסך בו השתמשתי על מנת להציג את הרדאר בפרויקט שלי. מסך קטן וקומפקטי שיוצר תמונה ברורה וטובה עבור הרדאר. אשר מציג את המרחק החל מ 0 ועד 100 סנטימטרים. בנוסף כפי שניתן לראות את הקו שמסתובב 180 מעלות החל מצידו הימני/השמאלי של המסך ועד לצידו השני ומצייר נקודה בהתאם למרחק ככל שהאובייקט קרוב יותר נראה שצבעה של הנקודה הוא אדום, וככל שהאובייקט רחוק יותר הנקודה תהיה בצבע צהוב.

1 סוג המסך:

- דגם : ST7735.
- גודל : 1.8 אינץ'.
- רזולוציה : 128×160 פיקסלים.
- צבעים : צבעוני (RGB565).
- ממשק : SPI (Serial Peripheral Interface) מהיר ומדויק.

2 חיבורי פינים (SPI):

המסך מתקשר עם ה- ESP32 דרך SPI. בפועל בקוד שלי :

- GPIO 23 – MOSI (Master Out Slave In) (נתונים מה- ESP32 למסך).
- GPIO 18 – SCLK (Clock) (שעון SPI).
- GPIO 5 – CS (Chip Select) (בוחר איזה התקן SPI פעיל*).
- GPIO 2 – DC (Data/Command) (מספר למסר "פקודה או נתון").
- GPIO 4 – RESET (אתחול המסך).
- VCC / GND - מתח והארקה.

הערה: MOSI ו- SCLK משותפים אם יש יותר ממסך אחד CS ו- DC - חייבים להיות שונים לכל מסך.

3 Backlight (תאורת רקע):

- לרוב המסכים יש LED backlight פנימי.
- חיבור פיזי: BL / LED.
- אם מחברים ל- Backlight $5V \rightarrow$ או $3.3V$ דולק.
- ניתן לכבות או לשלוט בעוצמה דרך **טרנזיסטור PWM** כדי לחסוך צריכת חשמל.
- כיבוי Backlight חיסכון משמעותי בזרם (50–300 mA).

4 ספריות ותפקודים בקוד שלי:

- הספרייה Ucglib שולחת פקודות SPI למסך.
- פונקציות עיקריות בקוד:
 - `ucg.begin()` - אתחול המסך.
 - `setRotate90()` - סיבוב המסך לאופקי.
 - `setColor(...)` - הגדרת צבע לציור.
 - `drawLine, drawBox, drawDisc, drawCircle` - ציור צורות.
 - `setFont(...), setPrintPos(...), print(...)` - כתיבת טקסט.
 - `cls()` - ניקוי המסך באמצעות ריבועים שחורים.

5 זרם ומתח

- VCC: רוב המודולים מקבלים $5V$ חלק תומכים גם ב- $3.3V$.
- צריכה טיפוסית:
 - `backlight: 10–60 mA` ללא
 - `backlight: 50–300 mA` עם

מה המסך מציג בקוד שלי:

- **רקע גרפי**: עיגולים, קווים, רקע ירוק-שחור (גרדיאנט)
 - **טקסט**: שם הפרויקט, "Mini Radar", מרחקים בקנה מידה (25cm, 50cm...)
 - **נקודות**: מיקום אובייקטים לפי החיישן האולטראסוניק ($>100cm$ אדום, $<100cm$ צהוב)
 - **קווים**: קו סריקה שמסתובב עם הסרוו, כמו קרן רדאר.
- בקיצור, המסך הוא **הממשק הגרפי של הרדאר** – מציג מידע בזמן אמת בצורה צבעונית.

הסבר על תצוגת הרדאר:

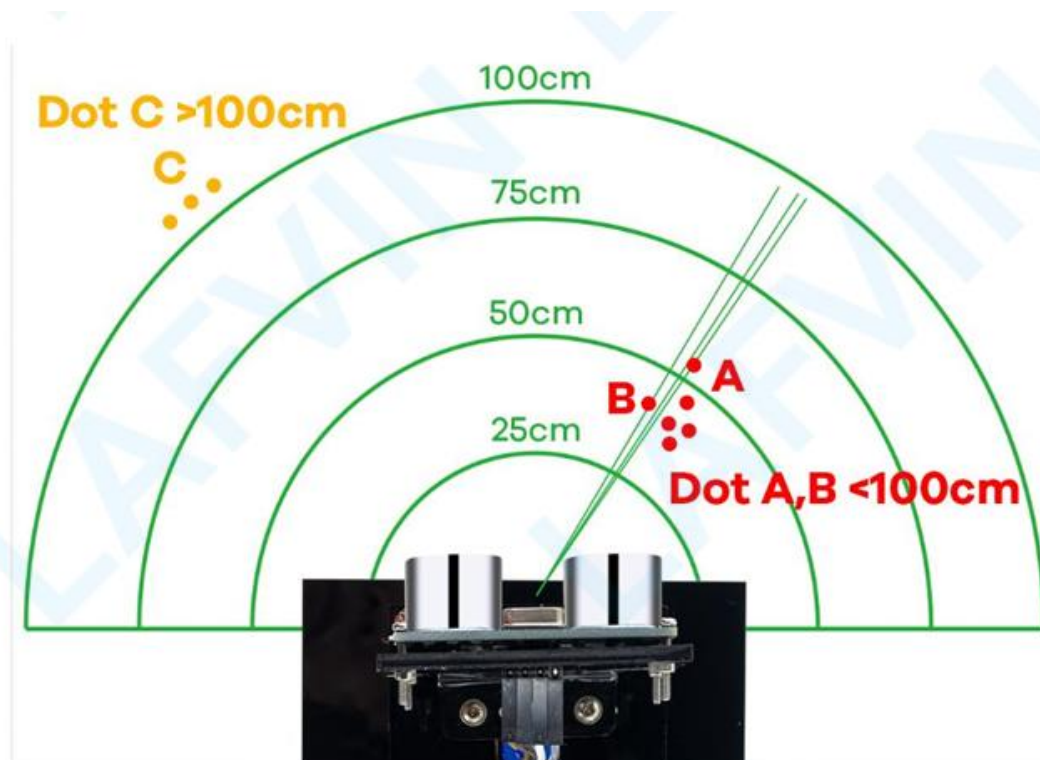
איך למעשה המערכת עובדת מה אנחנו רואים על המסך?

אז נתחיל בזה שהרובוט השומר שלנו צריך לשמור על סביבה שנסרקת 180 מעלות.

הפעולה של הסריקה עצמה מתבצעת באמצעות שני רכיבים חשובים מאוד במערכת הרדאר שלנו. הרכיב הראשון זהו המנוע סרוו שבכלל מאפשר לנו את כל העניין של התזוז של חצי הסיבוב החל מ0 מעלות ועד הגעתו ל180 מעלות. הרכיב השני שלנו שלמעשה מבצע את המדידה זהו החיישן האולטראסוניק שמבצע בדיקה של האובייקטים בסביבה בתווך של 100 פלוס סנטימטרים ומחזיר לנו "מידע" שלמעשה מתאפיין בנקודה על הרדאר.

כל המערכת מוצגת באופן חד וברור על המסך שלי ואנו יכולים לדעת האם האובייקט קרוב אלינו או רחוק מאיתנו, כאשר האובייקט נמצא בתווך בין 0 ל- 100 סנטימטרים נראה שהנקודה היא בצבע אדום. כאשר האובייקט נמצא בתווך שמעל ל- 100 סנטימטרים אז הנקודה תהיה בצבע צהוב.

נראה תמונה שממחישה לנו באופן ברור את מה שמתרחש במסך בזמן פעולת מערכת הרדאר:



כמו שאמרתי קודם לכן, ניתן לראות מהתמונה שכל עוד אנו נמצאים בתחום בין 0-100 סנטימטרים צבע הנקודות הוא אדום וברגע שאנו עוברים תחום זה צבע הנקודות הוא צהוב.

➤ לחצו כאן לחזרה לתוכן העניינים

מסך גרפי טאץ' TFT 9341

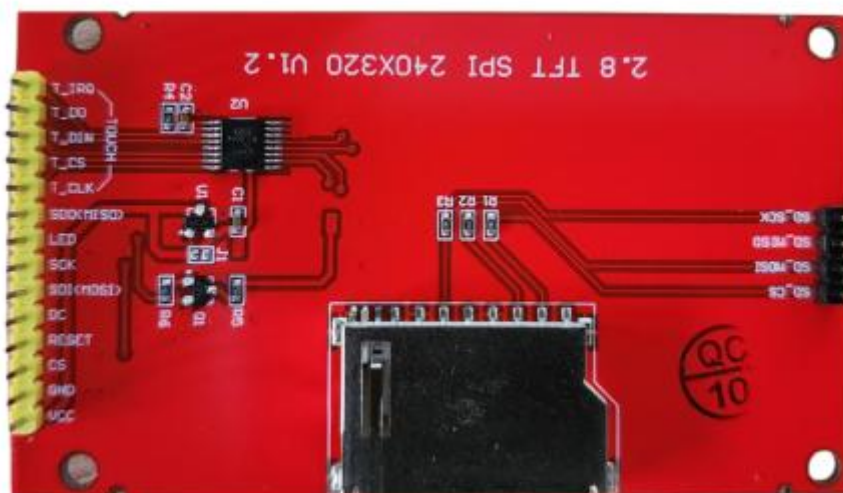


מסך גרפי TFT 9341 מאפשר הצגת תווים, מספרים, צורות גרפיות ותמונות על המסך.

מפרט טכני של מסך גרפי:

1. **סוג תצוגה:** מסך גרפי צבעוני מגע (Touch).
 2. **פרוטוקול עבודה:** SPI (Serial Peripheral Interface).
 3. **גודל פיזי 8:** אינץ'.
 4. **רזולוציה 240x320:** פיקסלים (נקודות).
 5. **ציר X רוחב:** (מכיל 320 פיקסלים).
 6. **ציר Y גובה:** (מכיל 240 פיקסלים).
 7. **מתח הפעלה:** 3V.
 8. **תכונת אחסון:** תצוגה כוללת קורא כרטיסי SD המאפשר שמירת קבצי תמונה בנפח זיכרון גדול.
- הערה:** אם נעדיף שהסדר יהיה רוחב ואז גובה (כפי שמקובל לעיתים ברזולוציה), ניתן לשנות את סעיפים 5 ו-6 בהתאם לצורה שבה אנו רוצים להציג את זה במסמך.

תיאור ותפקוד פניי חיבור – מסך TFT/Touch :



המסך הגרפי דורש מספר חיבורים מרכזיים כדי לאפשר תקשורת נתונים מהירה (SPI) ובקרת מגע. כל פין משרת מטרה ייחודית:

- 1. VCC מתח אספקה :** זהו פין אספקת המתח החיובי של התצוגה, הנדרש להפעלת הבקר והתאורה האחורית. עבור מסך זה, **המתח הנדרש הוא 3.3V**.
- 2. GND (הארקה):** חיבור ההארקה המשותף, סוגר את המעגל החשמלי.
- 3. CS בחירת רכיב - (Chip Select) :** משמשת לשליטה על שבב התצוגה (LCD). כאשר הפין הזה במצב נמוך (LOW), לוח הבקרה (כמו ESP32) מסמן לבקר התצוגה (ILI9341) שהוא הרכיב המיועד לקבל או לשלוח נתונים דרך קו ה-SPI המשותף.
- 4. DC נתונים/פקודה (Data/Command) :** – פין קריטי בבקרה. הלוח משתמש בו כדי להבדיל בין סוג המידע הנשלח: אם הפין במצב גבוה (HIGH) המידע הנשלח הוא **נתונים** (צבעי פיקסלים, תמונות); אם הפין במצב נמוך (LOW) המידע הוא **פקודה** (הוראות לבקר לשנות רזולוציה או אתחול).
- 5. SDI (MOSI) - Master Out Slave In :** קו **כניסת הנתונים הראשי** בפרוטוקול SPI. באמצעות פין זה, לוח הבקרה **שולח** את כל הנתונים הגרפיים והפקודות אל התצוגה.
- 6. שעון CLK - (Clock) :** רגל השעון. מספקת את **הקצב והתיזמון** הנדרשים לסינכרון תהליך העברת הנתונים ב-SPI. כל פיקסל או ביט נשלח בתיאום עם דופק השעון.
- 7. SDO (MISO) - Master In Slave Out :** קו **יציאת הנתונים** מפרוטוקול SPI. דרך פין זה, לוח הבקרה **מקבל נתונים** חוזרים מהתצוגה (למשל, קריאת סטטוס או זיהוי הבקר).
- 8. T_CS בחירת שבב מגע :** (זהו פין **Chip Select נפרד** המשמש **לשבב בקר המגע** לרוב (XPT2046). הוא מאפשר ללוח הבקרה לתקשר עם שבב המגע באמצעות אותה רשת SPI, בנפרד מבקר התצוגה הראשי.
- 9. T_IRQ - פסיקת מגע (Touch Interrupt) :** רגל זו נכנסת לפעולה כאשר **נגיעה מזוהה** על פני המסך. היא מאפשרת ללוח לעבד אירועי מגע באופן יעיל ומיידי (מבוסס פסיקה), מבלי צורך לבדוק באופן קבוע אם התרחשה נגיעה.

הסבר מקיף על פונקציות התצוגה (Adafruit_ILI9341): בה אני משתמש בפרויקט

1.הכללת הספריות (Headers):

בתחילת כל סקיצה (קוד) עלינו לכלול את הספריות הבאות, המגדירות את כל הפקודות הגרפיות והחומרתיות:

- `#include <Adafruit_GFX.h>` : זוהי ליבת הציור הגרפי (GFX Core). היא מספקת את כל הפקודות הגרפיות לציור צורות, קווים וטקסט, ואינה תלויה בבקר המסך הספציפי.
- `#include <Adafruit_ILI9341.h>` : זהו הדרייבר הספציפי התומך בבקר הפיזי ILI9341 של המסך שלך. הוא מתרגם את הפקודות הגרפיות של GFX לפקודות ברמה ממוכה (SPI) שהחומרה מבינה.

2.אתחול ותחילת עבודה:

פונקציות אלו נקראות לרוב רק פעם אחת בתוך פונקציית `setup()`:

- `tft.begin()` : זוהי הפקודה הראשונה והקריטית ביותר. היא שולחת את כל רצף ה**אתחול** הנדרש לבקר ה-ILI9341 כדי להביא אותו למצב עבודה וכיול ראשוני. (מקביל ל-`lcd.begin()` במערכות אחרות).
- `tft.setRotation()` : **כיוון** (פקודה זו קובעת את **כיוון התצוגה** (אוריינטציה). ישנם ארבעה מצבים אפשריים (0, 1, 2, או 3), כשאחד מהם יגדיר את המסך לרוחב (320x240) והאחר לאורך (240x320).
- `tft.fillScreen()` : **צבע** (זוהי הפקודה ל**ניקוי המסך**. היא ממלאת את כל שטח התצוגה בצבע אחיד, ומשמשת לרוב לקביעת צבע הרקע (למשל, `tft.fillScreen(ILI9341_BLACK)` מקביל ל-`lcd.clrscr()` במערכות אחרות).

3.עבודה עם טקסט ומיקום:

פונקציות אלו משמשות להצגת נתונים מספריים ומילוליים על המסך:

- `tft.setCursor(x, y)` : מגדירה את **מיקום הסמן** שבו הטקסט הבא יודפס. ה-X וה-Y מציינים את הפינה השמאלית העליונה של האות הראשונה, (מקביל ל-`lcd.goto(x, y)`).
- `tft.setTextSize()` : קובעת את **גודל הפונט**. גודל 1 הוא הפונט הבסיסי, וגדלים גדולים יותר מכפילים את הפיקסלים (לדוגמה, גודל 3 הוא פי 3 מהגודל הבסיסי).
- `tft.setTextColor()` : **צבע** (קובעת את **צבע הטקסט** (צבע החזית) שיוצג. ניתן להוסיף ארגומנט שני אופציונלי כדי להגדיר את צבע הרקע סביב הטקסט.
- `tft.print()/ tft.println()` : מבצעת את ה**דפסת הטקסט**, **המספרים** או **המשתנים** אל המסך במיקום הסמן הנוכחי (`println` מוסיפה ירידת שורה מקביל ל-`lcd.print()`).

4.פונקציות מגע (Touch):

הפונקציות הללו מטפלות בבקר המגע ככל הנראה (XPT2046) הנמצא במסך:

- `ts.begin()` : **אתחול שבב המגע** (כאשר ts הוא אובייקט של ספריית המגע הנפרדת, כמו `XPT2046_TouchScreen`). זהו המקביל לאתחול בקרת המגע.
- **קריאת מגע** : פונקציות ספציפיות בספריית המגע, (כגון: `ts.getTouch()`) מאפשרות **לזהות את הנגיעה** ולקבל את קואורדינטות ה-X וה-Y של נקודת הלחיצה על המסך.

סקירת הספרייה TFT_eSPI :

ספריית TFT_eSPI שפותחה על ידי Bodmer (נחשבת לסטנדרט בתעשייה עבור פרויקטים מבוססי ESP32 ו-ESP8266 בזכות הביצועים המהירים שלה והתמיכה במגוון רחב של בקרים ומסכים.

1. מאפיינים עיקריים:

- **מהירות יוצאת דופן:** הספרייה עושה שימוש ביכולות ה-DMA (Direct Memory Access) של בקר ה-ESP32, מה שמאפשר ריענון מסך מהיר מאוד ללא הכבדה על המעבד.
- **תמיכה רחבה בדרייברים:** הספרייה תומכת ברוב הדרייברים הנפוצים בשוק, כגון:
 - ILI9341, ST7735, ST7789, ST7796 ועוד.
- **ריבוי גופנים:** (Fonts) כוללת גופנים מובנים וגופני Anti-aliased (גופנים מוחלקים) שנראים מקצועיים בהרבה מהגופנים הסטנדרטיים של ארדואינו.
- **ניהול זיכרון:** (Sprites) תומכת ב- "Sprites" יצירת גרפיקה בזיכרון ה-RAM ושליחתה למסך בבת אחת, מה שמונע "הבהובים" (Flickering) "באנימציות".

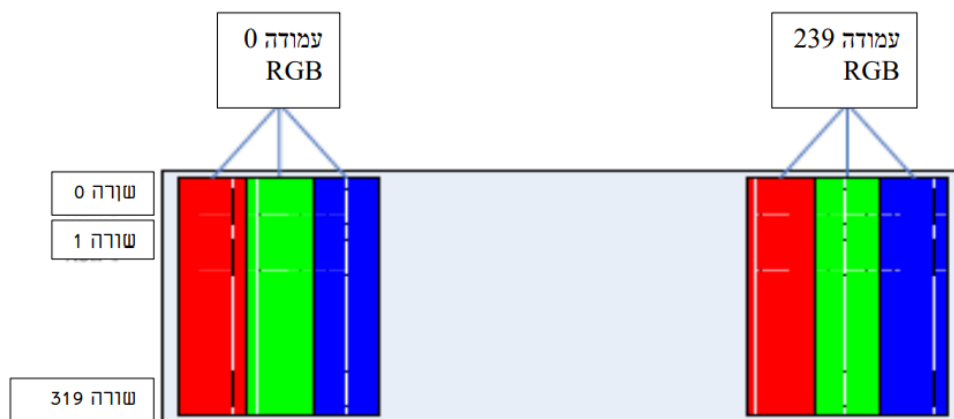
2. ארכיטקטורת ההגדרה (User Setup):

בשונה מספריות רגילות שבהן מגדירים את הפינים בתוך הקוד (Sketch), ב-TFT_eSPI ההגדרה מתבצעת בדרך כלל בקובץ הגדרות חיצוני הנמצא בתוך תיקיית הספרייה:

- **User_Setup.h:** כאן מגדירים את סוג הדרייבר של המסך, הרזולוציה ופיני ה-SPI כמו (MOSI, SCK, CS).
- שיטה זו מאפשרת לקוד המקור להישאר נקי וקבוע, בעוד ששינויי החומרה מתבצעים ברמת הספרייה.

תצוגות - TFT - Thin Film Transistor טרנזיסטור סרט דק – הן תצוגות אקטיביות:

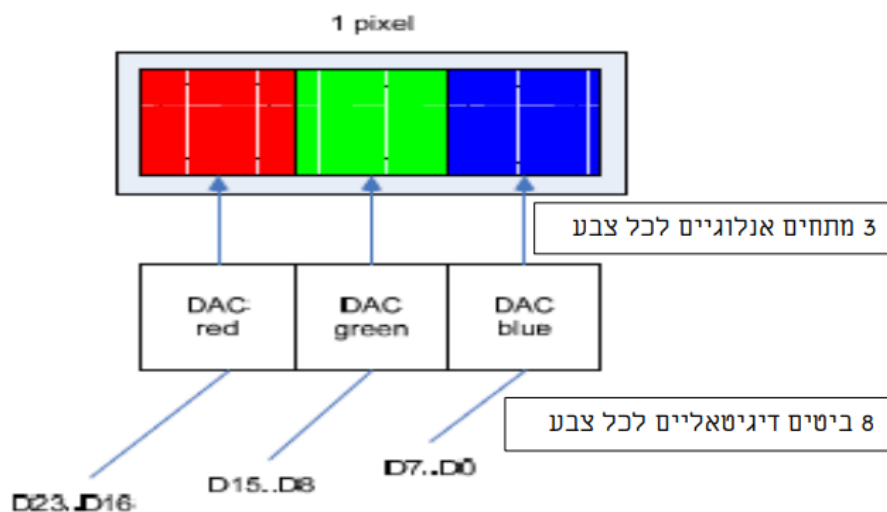
כל סגמנט בתצוגה מאפשר החזרה או העברה של אור. גם כאן יש מקור אור המקרין אור מאחורי התצוגה אל החזית. בעזרת הטרנזיסטור קובעים האם האור יוקרן החוצה או שהוא ייחסם ולא יועבר. כדי ליצור צבעים צריך 3 סגמנטים. 3 סגמנטים אלו מעבירים אור דרך מסננת אדומה, ירוקה או כחולה. כך נוצר פיקסל RGB. כלומר פיקסל מורכב מ- 3 סגמנטים. למעשה כל פיקסל מורכב מ- 2 מסננות עם קיטוב, 3 מסננות צבע ו 2 שכבות יישור וכך נקבע בדיוק כמה אור יעבור ובאיזה צבע. באיור רואים את סידור העמודות והשורות של תצוגת RGB של 320 שורות ו- 240 עמודות.



יצירת צבע בתצוגה TFT (Thin Film Transistor):

תצוגות TFT יכולות לדחוף 3 סגמנטים (פיקסל אחד) בפולס שעון אחד עם שדה חשמלי משתנה חזק. גודל המתח האנלוגי בכל צבע קובע את עצמת הצבע המסוים. תצוגה כזו תומכת בצבעים רבים. רמות הצבע נקבעות על ידי מספר קווי הנתון בפנל התצוגה ובמספר קווי יציאות הנתון של הבקר. האפשרויות הן: 24 ביט לפיקסל (bits per pixel), 18bpp, 16bpp, 15bpp, 8bpp. בממשק מקבילי נדרשים 320 פולסי שעון ל 320 פיקסלים.

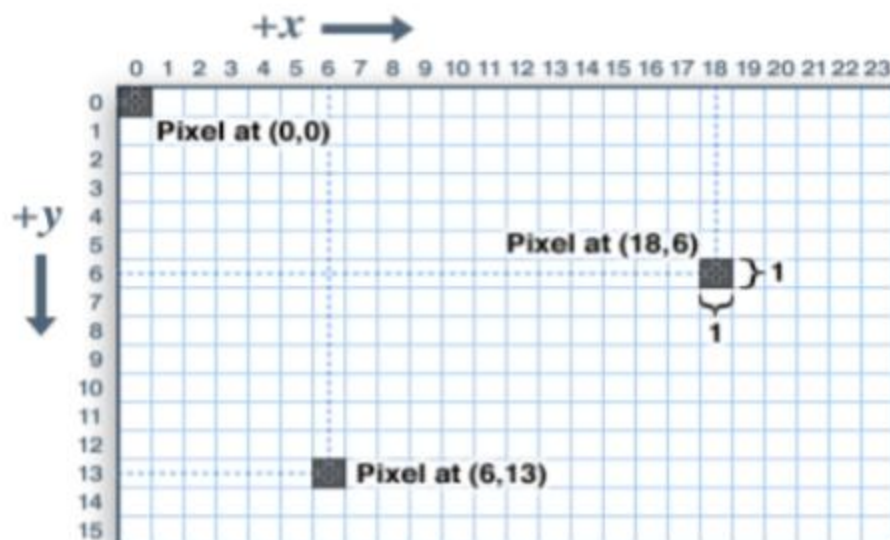
ניתן לראות בתמונה ממשק של 24 ביטים לפיקסל בשעון אחד:



תמונה זו מתארת תצוגה המתחברת בממשק של 24 ביט. כל 8 ביטים מתחברים ל DAC הקובע את גודל המתח שיגיע לכל צבע בפיקסל מסוים. הצבע הכחול נשלט על ידי 8 הביטים D0~D7, הצבע הירוק על ידי הביטים D8~D15 והצבע האדום על ידי D16~D23. 8 הביטים של המספר הדיגיטאלי קובעים את גודל המתח האנלוגי שיוצא מה DAC ואיתו את עצמת הצבע בכל אחד מ- 3 צבעי ה-RGB.

התמונה בתצוגת TFT מורכבת מפיקסלים. תצוגת TFT זו מורכבת ממכפלה של 2 המספרים 320 ו- 240 (הנותנת 153600 פיקסלים). ניתן לשלוט על הצבע של כל פיקסל ומכאן ניתן להבין כי ניתן להציג תצוגה מנקודה בודדת ועד תמונה מורכבת.

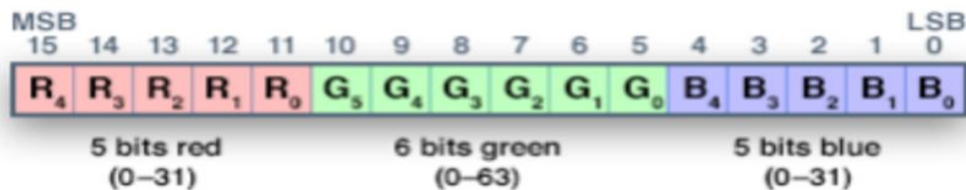
לכל פיקסל יש קואורדינטות (קואורדינטות הן קבוצת מספרים המציינת את מיקומו של גוף במרחב כלשהו) של x ושל y משלו. מערכת הקואורדינטות ממקמת את הראשית (0,0) בפינה השמאלית העליונה כאשר x גדל לכיוון ימין ו- y כלפי מטה, ניתן לראות באיור.



בתצוגות התומכות בצבע, הצבע נרשם כערך בן 16 סיביות לא מסומן – (unsigned כלומר רק חיובי). בחלק מהתצוגות ניתן לקבל יותר צבעים ובחלק פחות – יותר או פחות סיביות בערך. רוב הספריות שעובדות עם תצוגות כאלו עובדות עם 16 סיביות. גם לארדואינו קל לעבוד עם צבעים בני 16 ביטים. שלושת צבעי היסוד RGB – אדום (red), ירוק (green), כחול (blue) נכנסים בתוך ה-16 סיביות בצורה הבאה:

מהאיות רואים:

5 הביטים הגבוהים הם של הצבע האדום והם בני 5 ביטים (0-31).



6 הביטים הבאים הם של הצבע הירוק (0-63).

5 הביטים הנמוכים הם של הצבע הכחול (0-31).

לצבע הירוק יש 6 ביטים כי העין של האדם רגישה יותר לצבע הירוק,

לדוגמה, אם נרצה לצבוע פיקסל בצבע ירוק נשלח את הערך **07E0H = 00000 11111 00000**. הערך **F800H = 1111100000000000** יצבע את הפיקסל באדום והערך **FFFFH** יצבע את הפיקסל בלבן.

הצבעים הנפוצים הם:

- `#define BLACK 0x0000 // שחור`
- `#define BLUE 0x001F // כחול`
- `#define RED 0XF800 // אדום`
- `#define GREEN 0x07E0 // ירוק`
- `#define CYAN 0x07FF // טורקיז`
- `#define MAGNETA 0XF81F // גוון אדום`
- `#define YELLOW 0xFFE0 // צהוב`
- `#define WHITE 0xFF // לבן`

```

1. // Color definitions
2. #define BLACK    0x0000    // שחור
3. #define BLUE     0x001F    // כחול
4. #define RED      0xF800    // אדום
5. #define GREEN    0x07E0    // ירוק
6. #define CYAN     0x07FF    // כחול ירוק
7. #define MAGENTA  0xF81F    // גוון אדום
8. #define YELLOW   0xFFE0    // צהוב
9. #define WHITE    0xFF      // לבן

```

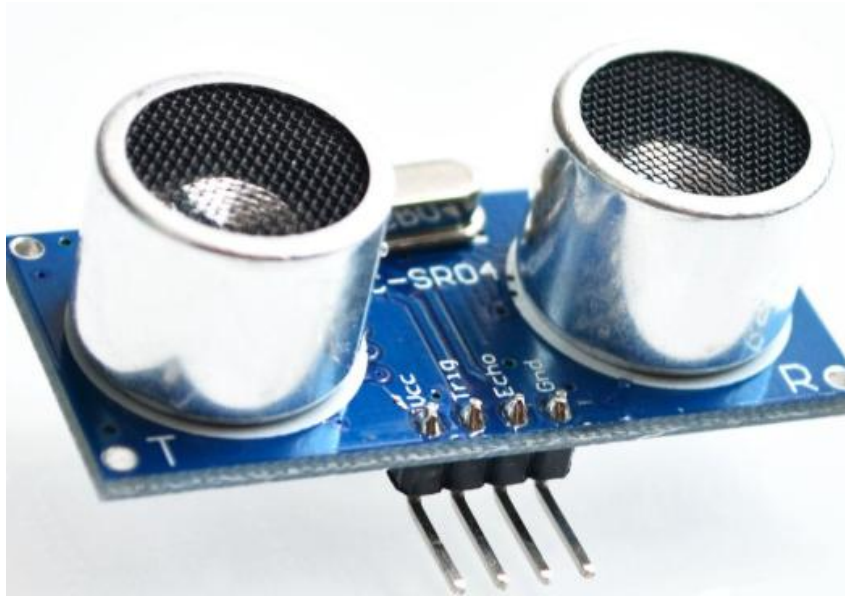
ברישום הקואורדינטות , קודם נרשמת הנקודה בציר ה X של הפיקסל ולאחריה בציר ה Y . הקואורדינטות מבוטאות ביחידות של פיקסל ולא בגדלים של אורך (מ"מ או ס"מ אינצ'ים וכו'). גודל מלבן שנצייר בהמשך יהיה במספר פיקסלים שלו בציר ה X ומספר הפיקסלים בציר ה Y ולא באורך של מ"מ או ס"מ. ניתן למדוד את אורך ורוחב המסך ואז לחלק בכמות הפיקסלים בכל ציר ולדעת את הגודל של כל פיקסל, אם כי הדבר פחות שימושי . ניתן לשנות את האוריינטציה של התמונה כך שהאורך יהיה רוחב והרוחב יהיה אורך.



התצוגה הימנית נקראת פורמט landscape (נוף) והשמאלית פורמט portrait (דיוקן). ראשית הצירים היא הנקודה השמאלית למעלה. ניתן לקבל 4 סוגי אוריינטציה, כלומר איזו אחת מ- 4 הפינות של המלבן תהיה ראשית הצירים (0,0) .

פונקציות הגרפיקה לכל תצוגת TFT יש בקר תצוגה פנימי המנהל את התצוגה עצמה ואת הקשר עם העולם שמחוץ לתצוגה שבו יש בדרך כלל מיקרו בקרים השולחים פקודות להפעלת התצוגה. לכל בקר יש פונקציית אתחול משלו. כדאי לראות בדפי הנתונים של התצוגה מי הבקר המפעיל אותה. במידה ולא יודעים אז בפונקציית.

חיישן המרחק – HC-SR04



ה-HC-SR04 זהו החיישן בו השתמשתי למדידת מרחק בפרויקט שלי. זהו חיישן מאוד נפוץ בפרויקטים של רובוטיקה, חיישני חנייה, ומכשירים חכמים. חיישן זה נפוץ בגלל מספר דברים, כמו למשל: הוא נחשב לזול, פשוט לשימוש ומדויק יחסית בטווחים קצרים ובינוניים.

איך החיישן עובד?

העיקרון מאחורי HC-SR04 מבוסס על גלי קול על-קוליים . Ultrasonic Waves - החיישן משדר גל קול בתדר גבוה, שאינו נשמע לאוזן האנושית (כ-40 קילו-הרץ). הגל הזה מתפשט בסביבה, וכאשר הוא פוגע במכשול או חפץ כלשהו, הוא חוזר אל החיישן. החיישן מודד את הזמן שלקח לגל להגיע חזרה. בעזרת מהירות הקול באוויר, ניתן לחשב את המרחק למכשול.

הנוסחה שמאפשרת לחשב את המרחק היא:

$$\text{מרחק} = \frac{\text{זמן החזרה} * \text{מהירות הקול}}{2}$$

החלק החשוב הוא חלוקה ב-2, שכן הזמן שנמדד הוא הזמן הכולל של ההלוך והחזור של הגל. **מהירות הקול באוויר אינה קבועה**, והיא תלויה בעיקר בטמפרטורה. ככל שהטמפרטורה גבוהה יותר, המולקולות באוויר נעות מהר יותר ומעבירות את גל הקול ביעילות רבה יותר. כתוצאה מכך, מהירות הקול עולה.

נוסחה לחישוב מהירות הקול

ניתן לחשב את מהירות הקול באוויר (במטרים לשנייה) בצורה מקורבת באמצעות הנוסחה הבאה :

$$\text{Speed} = 331.4 + 0.6 \times T$$

כאשר:

- T היא הטמפרטורה במעלות צלזיוס ($^{\circ}\text{C}$).
- 331.4 מ/ש היא מהירות הקול באוויר בטמפרטורה של 0°C .
- 0.6 מ/ש היא הקירוב לשינוי במהירות הקול עבור כל עלייה של מעלה אחת בטמפרטורה.

נוסחה לתיקון המרחק

נוסחה זו משמשת לתיקון "מתמטי" של המרחק שהחיישן כבר הציג (בהנחה שהחיישן כויל לטמפרטורה מסוימת, אך עובד בטמפרטורה אחרת):

$$\text{Corrected Distance} = \frac{\text{Measured Distance}}{1 + \alpha \times (T - T_0)}$$

- **שימוש:** תיקון שגיאות מדידה שנובעות מהפרשי טמפרטורה בין רגע הגיול לרגע המדידה בפועל.
- **α (אלפא):** מקדם התפשטות התרמית לאוויר (נע בין 0.0065 ל- 0.0075).
- **יתרון:** מאפשרת לתקן את התוצאה הסופית מבלי לחשב מחדש את מהירות הקול מאפס.

השפעה על הדיוק

אם לא מתחשבים בטמפרטורה, חישוב המרחק יכול להיות לא מדויק.

המערכות כה חשובות כגון כיפות ברזל או רובוט שומר בהתחשב בפרויקט שלי, יש צורך להתחשב בטמפרטורה הסביבתית על מנת להבין כיצד להגיע למרחק שמתקבל על פי החיישן האולטרסוני.

מהירות שינוי הקול: חיישנים אולטרסאונדים פועלים על ידי פליטת גלי קול ומדידת הזמן שלוקח לגלים לחזור אחורה לאחר פגיעה באובייקט. מהירות הקול באוויר משתנה בהתאם לטמפרטורה, ככל שהטמפרטורה משתנה כך גם מהירות הקול משתנה, מה שיכול להשפיע על הדיוק של מדידות המרחק. החישובים הפנימיים של החיישן מניחים מהירות צליל קבועה, ולכן שינויים משמעותיים בטמפרטורה עלולים להכניס שגיאות בקריאות המרחק.

כיוול חיישן: חיישנים אולטרסאונדים דורשים כיוול כדי לפצות על השפעות הטמפרטורה. שינויים בטמפרטורה יכולים להשפיע על ביצועי החיישן, כולל מעגל התזמון והדיוק של מדידות המרחק. כיוול חיוני כדי להבטיח קריאות מדויקות בטווחים טמפרטורות שונים. עם זאת, חשוב לציין שלא לכל החיישנים האולטרסאונדים יש יכולות פיצוי טמפרטורה או כיוול מובנים. כיוול הוא הפעולה שנועדה להבטיח שמכשיר המדידה (כמו חיישן המרחק שלך) מספק תוצאות התואמות את המציאות הפיזיקלית.

מבנה החיישן:



HC-SR04 מגיע עם ארבעה פינים עיקריים:

- VCC - מתח אספקה של 5 וולט.
- GND - חיבור לאדמה.
- Trig - פין טריגר, אליו שולחים פולס קצר (בדרך כלל 10 מיקרו-שניות) כדי להפעיל את שליחת הגל האולטרסוני.
- Echo - פין החזרה, שמפיק פולס שמתמשך לאורך זמן ההחזרה של הגל. אורך הפולס הזה מאפשר לחשב את המרחק.

מגבלות ויתרונות

מצוין למדידה בטווחים של 2 ס"מ עד 400 ס"מ. עם זאת, יש לו מגבלות: HC-SR 04 משטחים קטנים או סופגים עלולים להקשות על קריאת המרחק, כמו גם רעשים חזקים באוויר שמפריעים לגלים האולטרסוניים. בנוסף, הוא רגיש לזווית ההחזרה של הגל – החיישן עובד הכי טוב כאשר המשטח ניצב אליו.

1. דרישות פולס הטריגר והתזמון (Timing):

אף שצוין הצורך בפולס של $10\mu s$ בפין Trig, חשוב להדגיש את סדר הפעולות המדויק ואת התזמונים הקריטיים לתקשורת תקינה:

- **שליחת הפולס:** יש להקפיד שהפולס יהיה **$10\mu s$ בדיוק** (או ארוך יותר, אך $10\mu s$ זה המינימום הדרוש) כדי להפעיל את המשדר האולטרסוני.
- **זמן המתנה פנימי (Internal Delay):** לאחר קבלת פולס הטריגר, המודול ממתין כ- **$500\mu s$** לפני שהוא משדר בפועל את סדרת **שמונה פולסי $40kHz$** אלחוטיים.
- **פין ה-Echo:** פין זה משתנה למצב **HIGH** (גבוה) ברגע שהשידור מתחיל, ונשאר במצב זה עד לקליטת ההחזר או עד שזמן ה-"Time-Out" הפנימי (בסביבות 38ms חולף). המיקרו-בקר שלך (כמו ארדואינו) צריך למדוד את משך הזמן שבו פין ה-Echo נמצא במצב **HIGH**.

2. רזולוציה ודיוק מעשי (Resolution and Practical Accuracy):

מעבר לתלות בטמפרטורה, ישנן מגבלות נוספות המשפיעות על הדיוק:

- **הרזולוציה התאורטית:** ה-HC-SR04 משדר ומודד את הזמן. אם נניח מהירות קול של $340m/s$ ($0.034cm/\mu s$), פולס של $40kHz$ נמשך $25\mu s$. אורך גל בודד הוא כ- $8.5mm$. כדי לקבל רזולוציה של $3mm$ (0.3 ס"מ), נדרשת יכולת מדידת זמן מדויקת מאוד. **מעשית, הרזולוציה של המודול היא כ-3 מילימטר.**
- **היסט מינימלי (Blind Zone):** הטווח המינימלי הוא כ- $2cm$. הסיבה לכך היא שהחיישן זקוק לזמן מינימלי להשלמת הפעולה של שליחה, כיבוי והתחלת הקליטה, לפני שההד של גל הקול הראשון חוזר.
- **פיצול אלומת הקול (Beam Divergence):** החיישן אינו פועל כ"קרן לייזר" צרה. הוא משדר **בקונוס** (זווית פתיחה) של כ- 15° החיישן יזהה את **האובייקט הקרוב ביותר** שנמצא בתוך קונוס זה. משמעות הדבר היא שמדידת המרחק משקפת את הנקודה הקרובה ביותר על פני האובייקט ולא דווקא את המרכז. ככל שהמרחק גדל, שטח הכיסוי של הקונוס גדל, מה שמפחית את הדיוק המקומי (Local Accuracy).

3.מודל הנהיגה הפנימי ומתח הלוגי (Driving & Logic Level):

- **מתח פעולה ובקרה (VCC):** החיישן דורש 5V בפין ה-VCC והוא מתאים לעבודה עם מיקרו-בקרים של 5V כמו ארדואינו אונו (ישנם רגליים מתאימות עבור חיישנים העובדים ב-5 וולט גם במיקרו בקר ESP32).
- **תאימות לוגית עם 3.3V:** כאשר מחברים את החיישן למיקרו-בקרים של 3.3V כמו ESP32, (פין ה-Trig יכול לרוב להיות מופעל ישירות) מכיוון שה-3.3V נחשב למצב HIGH (מספיק), אך פין ה-Echo מוציא פולס של 5V. חיבור 5V ישירות לפין קלט של 3.3V עלול לגרום לנזק. לכן, נדרש שימוש במחלק מתח (Voltage Divider) או ממיר רמות לוגיות (Logic Level Shifter) בפין ה-Echo.
- **צריכת זרם:** החיישן צורך זרם נמוך במצב רגיל (פחות מ-2mA אך בזמן שידור הוא יכול לצרוך עד כ-15mA).

מגבלות החיישן הקולי HC-SR04

החיישן הקולי HC-SR04 נחשב לפתרון מצוין מבחינת דיוק ושימושיות כללית, במיוחד בהשוואה לחיישנים קוליים אחרים בעלות נמוכה. עם זאת, חשוב להכיר במגבלות הטכניות שלו כדי להבטיח עבודה תקינה:

- **טווח מקסימלי:** החיישן מוגבל למרחק מדידה מרבי. אם המרחק בין החיישן לחפץ או למכשול גדול מ-13 רגל (כ-4 מטרים), החיישן לא יצליח לקלוט את ההד החוזר והמדידה תיכשל.
- **זווית פגיעה:** החיישן פועל בצורה המיטבית כאשר גלי הקול פוגעים במשטח ישר בניצב לחיישן. אם המשטח נמצא בזווית חדה מדי, גלי הקול עלולים להשתקף לכיוון אחר ולא לחזור אל המקלט.
- **סוג המשטח:** חומרים רכים או סופגי קול (כמו בד, צמר סלעים או שטיחים) עלולים לבלוע את גלי הקול במקום להחזיר אותם, מה שיוביל למדידות שגויות או חוסר זיהוי של החפץ.
- **גודל האובייקט:** חפצים קטנים מדי עלולים שלא להחזיר מספיק אנרגיה קולית כדי שהחיישן יזהה אותם כעצם שנמצא בדרך.

➡ לחצו כאן לחזרה לתוכן העניינים

חיישן טמפרטורה LM75



חיישן הטמפרטורה LM75 אינו רק רכיב אלקטרוני פשוט אלא הוא מערכת-על-שבב (SoC) דיגיטלית קומפקטית שנועדה למדוד טמפרטורה ולהמיר אותה לאותות שמיקרו-בקרים כמו הארדואינו או ה-ESP32 יכולים להבין בקלות. בניגוד לחיישנים אנלוגיים (כמו ה-LM35) שמוציאים מתח משתנה, ה-LM75 מבצע את ההמרה לאות דיגיטלי בתוכו, מה שמפשט משמעותית את החיבור והתכנות.

מאפיינים ודיוק טכניים:

- **טווח מדידה :** החיישן מתוכנן למדוד טמפרטורות בטווח רחב של -55°C עד $+125^{\circ}\text{C}$. טווח זה הופך אותו למתאים ליישומים תעשייתיים, רפואיים וביתיים.
- **רזולוציה ודיוק :** החיישן מספק קריאה ברזולוציה של 9 ביט, המקבילה לדיוק 0.5°C . דגמים מתקדמים יותר כמו ה-LM75 מציעים רזולוציה של 11 ביט, מה שמספק דיוק גבוה יותר של 0.125°C (למרות שהדיוק המוחלט עדיין תלוי בטווח הטמפרטורה).
- **צריכת אנרגיה :** יתרון משמעותי של ה-LM75 הוא יעילותו האנרגטית. הוא צורך זרם נמוך מאוד במצב פעולה, וכולל מצב **כיבוי (Shutdown mode)** בו הוא צורך זרם של מיקרו-אמפרים בודדים בלבד (בסביבות $4\mu\text{A}$), מה שהופך אותו לאידיאלי ליישומים המופעלים על סוללות.
- **מתח עבודה :** החיישן יכול לפעול במתח שנע בין 2.7V ל- 5.5V , מה שהופך אותו לתואם למגוון רחב של מיקרו-בקרים, כולל ארדואינו (5V) ו-ESP32 (3.3V).

עקרון הפעולה והתקשורת הדיגיטלית (I2C): (כפי שהוסבר בתחילת הספר)

ה-LM75 מתקשר עם המיקרו-בקר באמצעות פרוטוקול, (Inter-Integrated Circuit) **I2C** הידוע גם בשם "TWI (Two-Wire Interface)". פרוטוקול זה משתמש בשני קווים בלבד לתקשורת:

- **SDA (Serial Data Line) :** קו דו-כיווני להעברת נתונים.
- **SCL (Serial Clock Line) :** קו שעון המסנכרן את התקשורת בין המיקרו-בקר לחיישן.

לכל רכיב I2C יש כתובת ייחודית של **7 ביט**. ה-LM75 מציע שלוש רגלי כתובת (A0, A1, A2) שניתן לחבר ל-VCC או ל-GND, מה שמאפשר להגדיר עד **שמונה חיישני LM75 שונים** על אותו אוטובוס I2C, ללא הפרעות.

כיצד זה עובד?

המיקרו-בקר (המאסטר) שולח בקשה לקריאת נתונים לכתובת הספציפית של ה-LM75 (העבד). החיישן מגיב בנתונים המאוחסנים בזיכרון הפנימי שלו, שם הטמפרטורה נשמרת באופן רציף לאחר המרה.

פונקציונליות "תרמוסטט" (Over-temperature):

אחת התכונות המיוחדות והשימושיות של ה-LM75 היא יכולתו לפעול כתרמוסטט דיגיטלי עצמאי. לחיישן יש יציאה ייעודית, OS (Over-temperature Shutdown), שיכולה לשמש בשני מצבים:

- **מצב השוואה (Comparator Mode):** המיקרו-בקר יכול להגדיר שני ספי טמפרטורה בחיישן עצמו:

- **TOS (Temperature Over-limit):** טמפרטורה מקסימלית. כאשר הטמפרטורה עולה מעל ערך זה, יציאת ה-OS מפעילה פלט דיגיטלי.
- **THYST (Hysteresis):** טמפרטורה שבה הפלט חוזר למצב רגיל. הדבר מונע הפעלה וכיבוי חוזרים ונשנים (נדנד) של היציאה כאשר הטמפרטורה נמצאת קרוב לסף.

- **מצב פסיקה (Interrupt Mode):** במצב זה, יציאת ה-OS יכולה לייצר פסיקה (interrupt) למיקרו-בקר, להתריע בפניו על חריגה מהטמפרטורה, ללא צורך בסקרים מתמידים של החיישן.

פונקציה זו שימושית במיוחד לבניית מערכות פשוטות של בקרת טמפרטורה, כמו מפוח שמופעל אוטומטית כאשר הטמפרטורה עולה על סף מסוים.

שלבי חיבור בסיסיים:

1. **VCC** (מתח) ו-**GND** (הארקה): חיבור למקור מתח של 5V או 3.3V.
2. **SDA** ו-**SCL**: חיבור לפיני ה-I2C המתאימים בארדואינו (ברוב הלוחות: A4 ו-A5 בהתאמה) או ל-ESP32.
3. **נגדי פול-אפ**: יש צורך בנגדי פול-אפ (בדרך כלל 4.7kΩ) על קווי ה-SDA ו-SCL כדי למשוך את הקווים למתח גבוה כאשר הם אינם פעילים. במודולים רבים, הנגדים הללו כבר מובנים על הלוח.

חיבור החומרה ב-ESP32:

החיבור הפיזי של חיישן LM75 ל-ESP32 הוא פשוט מאוד, כיוון ששניהם תואמים למתח עבודה של 3.3V.

1. **VCC** (מתח) של ה-LM75 מתחבר לפין **3.3V** ב-ESP32.
 2. **GND** (הארקה) של ה-LM75 מתחבר לפין **GND** ב-ESP32.
 3. **SDA** (קו נתונים) של ה-LM75 מתחבר לפין **GPIO21** ב-ESP32.
 4. **SCL** (קו שעון) של ה-LM75 מתחבר לפין **GPIO22** ב-ESP32.
- פיני **GPIO21** ו-**GPIO22** הם ברירת המחדל של I2C בלוחות ESP32 רבים, מה שמפשט את החיבור.

שלבי תכנות (פסאודו קוד):

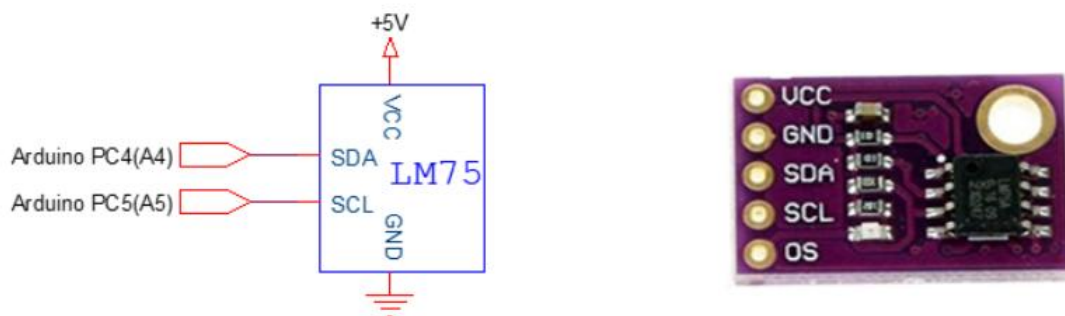
1. **הכללת ספרייה:** השתמש בספרייה ייעודית לחישה (למשל, Adafruit LM75) כדי לפשט את התקשורת.
2. **אתחול:** בפונקציית `setup()`, אתחל את החישה.
3. **קריאת נתונים:** בפונקציית `loop()`, קרא את ערך הטמפרטורה מהחישה. הספרייה כבר תמיר את הערך הדיגיטלי לטמפרטורה בצלזיוס.
4. **הצגת הנתונים:** הדפס את הנתונים לצג טורי (Serial Monitor) או לצג LCD.

יישומים נפוצים:

השילוב הייחודי של דיוק סביר, צריכה נמוכה וממשק דיגיטלי פשוט הפך את ה-LM75 לבחירה פופולרית במגוון רחב של תחומים:

- **מערכות קירור:** בקרת טמפרטורה בצידוד אלקטרוני, שרתים ומחשבים כדי למנוע התחממות יתר.
 - **טרמוסטטים:** בניית מערכות בקרת אקלים פשוטות לבית או למשרד.
 - **מכשירים ניידים:** שימוש במכשירים המופעלים על סוללות, כמו שעונים חכמים או מערכות ניטור אישיות.
 - **מדידת טמפרטורה כללית:** יישומים מדעיים, תעשייתיים וחינוכיים שבהם נדרשת מדידת טמפרטורה אמינה ולא יקרה.
- לסיכום, ה-LM75 מספק פתרון אלגנטי וחכם למדידת טמפרטורה דיגיטלית. הוא משחרר את המיקרו-בקר ממשמת המרת האות האנלוגי, מציע גמישות בשימוש בזכות ממשק ה-I2C ותכונות ה"תרמוסטט" המובנות, מה שהופך אותו לרכיב יסודי בפרויקטים רבים של IoT ואלקטרוניקה.

נראות ומאפיינים עיקריים של החישה:



SCL - קו זה מיועד לשאת אותות שעון וחייב להיות מחובר לנגד מושך, אך מאחר ולמודול זה יש נגדי משיכה מובנים הוא אינו חייב להיות מחובר לנגד חיצוני.

SDA - קו זה מיועד להעברת נתונים וחייב להיות מחובר לנגד משיכה גם כן, אך מאחר וכמו שאמרתי לפניכן יש נגדי משיכה מובנים במודול אז אין צורך לחבר לנגד חיצוני.

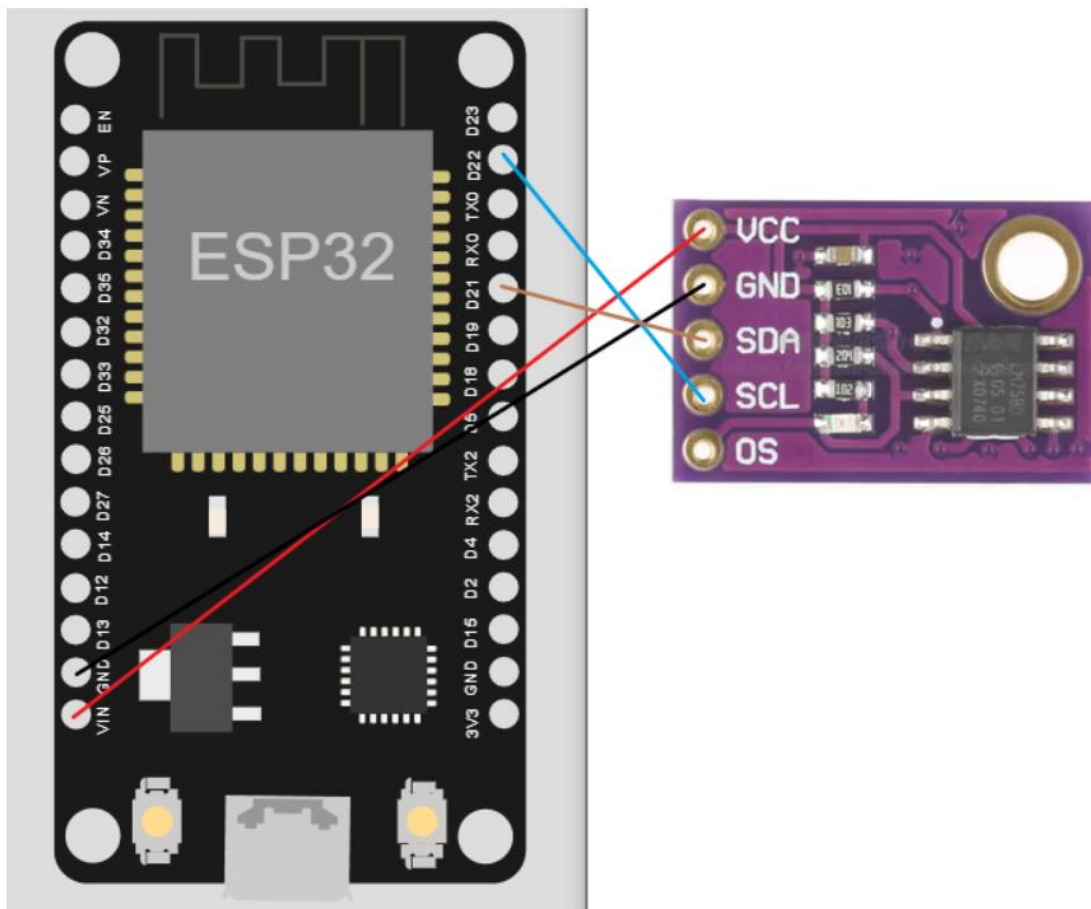
הסבר מדוע החיישן מחובר באמצעות הפרוטוקול I2C

I2C הוא פרוטוקול תקשורת שמאפשר לחבר התקנים שונים ללוח הארדואינו באופן פשוט יחסית. במקרה שלנו, החיישן LM75 מחובר דרך פני ה-SCL ו-SDA של הלוח. פנים אלה משמשים לתקשורת I2C דרך הפינים האלה, הארדואינו/ESP32 והחיישן "מדברים", באמצעות פרוטוקול I2C.

כשאנחנו רוצים לקרוא את הטמפרטורה, הארדואינו/ESP32 שולח בקשה לחיישן דרך פני ה-I2C. החיישן קורא את הטמפרטורה, ושולח חזרה את הערך דרך אותם פינים. כך אנחנו יכולים לקרוא את הטמפרטורה בקלות רבה, ללא צורך במעגל מורכב.

I2C מאפשר לנו לחבר מספר התקנים על אותם פינים SCL ו-SDA, ולתקשר איתם באופן עצמאי.

נראה חיבור של החיישן LM75 אל מיקרו בקר ESP32:

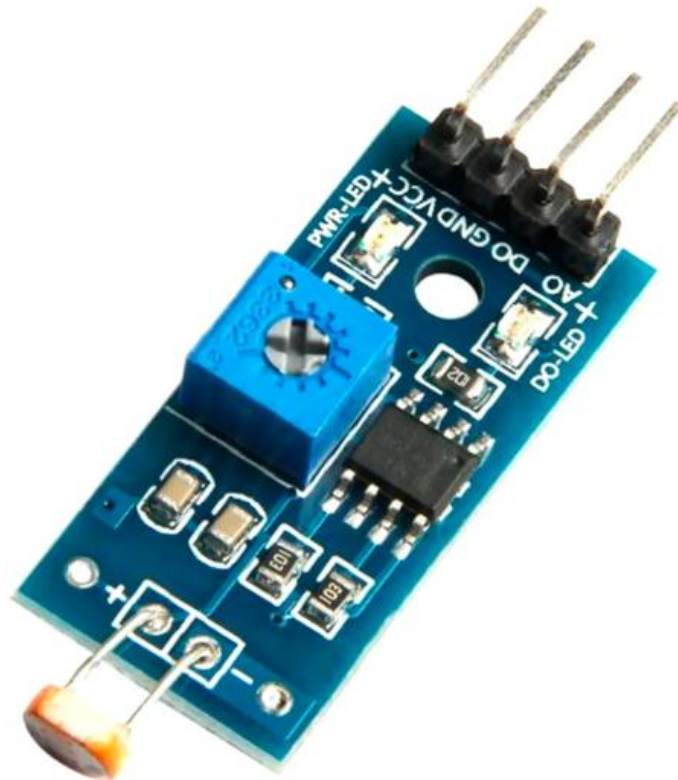


על מנת להגדיר כתובת של חיישן יש להגדיר רגליים A0,A1,A2:

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
1	0	0	1	A2	A1	A0	R/W

➡ לחצו כאן לחזרה לתוכן העניינים

מודול חיישן האור KY-018



מודול חיישן האור (KY-018 או "LDR Module") הוא רכיב אידיאלי למדידת עוצמת אור ובקרת תאורה אוטומטית באמצעות ה-ESP32. המודול משלב את החיישן הבסיסי עם רכיבי בקרה, מה שמאפשר שתי דרכי עבודה: קריאה מדויקת או קריאה בינארית פשוטה.

1.המבנה והרכיבים העיקריים:

המודול בנוי סביב שלושה רכיבים מרכזיים:

- נגד תלוי-אור: (LDR)** זהו החיישן הראשי שאחראי על המדידה. כשהאור הפוגע בו גדל, ההתנגדות שלו יורדת. המודול מכיל מעגל פנימי שממיר את שינוי ההתנגדות הזה לשינוי מתח.
- קומפרטור מתח: (LM393)** זהו הציפ האלקטרוני הראשי. תפקידו הוא להשוות את המתח המשתנה של ה-LDR מול **מתח ייחוס קבוע** (הסף). השוואה זו מניבה את הפלט הדיגיטלי (D0).
- פוטנציומטר כחול:** זהו נגד משתנה שמאפשר למשתמש **לכיל ידנית** את מתח הייחוס הקבוע של ה-LM393 ובכך לקבוע באופן חומרתי את סף ההדלקה/כיבוי של היציאה הדיגיטלית.

2. דרכי השימוש במודול (היציאות):

הגמישות של המודול נובעת משתי יציאות הפלט שלו :

א. יציאה אנלוגית - (A0) למדידה מדויקת

- **תפקיד :** פין זה מספק את המתח המשתנה המגיע ישירות ממחלק המתח הפנימי של ה-LDR.
- **היתרון :** הוא מעביר את כל טווח האור באופן רציף (ערכים בטווח של 0 עד 4095, ב-ESP32). שיטה זו מאפשרת לך לקבוע את סף ההדלקה **בתוך הקוד** שלך, מה שנותן שליטה ודיוק גבוהים יותר.
- **חיבור :** חבר את A0 לפין **ADC** של ה-ESP32 (למשל, GPIO34).

ב. יציאה דיגיטלית - (D0) להחלטה בינארית

- **תפקיד :** פין זה הוא הפלט של ה-LM393. הוא מספק רק שתי אפשרויות : HIGH או LOW.
- **היתרון :** השיטה היא "הכנס והפעל". ה-LM393 מבצע את ההחלטה האם חשך או מואר עבורך, בהתאם לכיול שקבעת באמצעות **הפוטנציומטר הכחול**.
- **חיבור :** חבר את D0 לפין **GPIO דיגיטלי** רגיל ב-ESP32.

חיישן אור - LDR - Light Dependant Resistor :

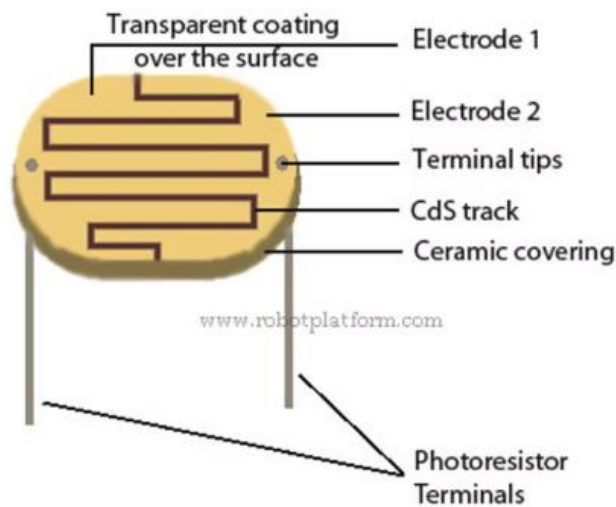


חיישן היכול למדוד את עוצמת האור הנמצאת בסביבתו. שמו באנגלית מסגיר את אופן פעולתו - נגד תלוי אור, כלומר נגד שמשנה את התנגדותו בתלות באור אליו הוא נחשף.

מבנה חיישן אור :

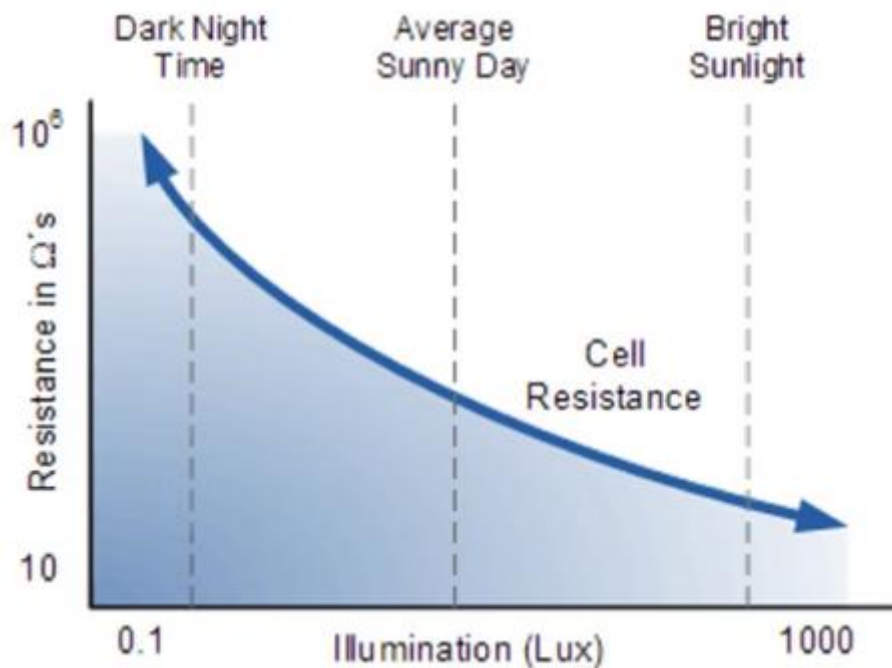
החיישן מורכב מדסקית עגולה העשויה חומר קרמי. הדסקית מחולקת לשני אזורים המופרדים ביניהם על ידי חומר הנקרא קדמיום סולפיד CdS. חומר זה אחראי להתנהגות החשמלית של החיישן. משני החצאי הקרמיים יוצאים גם שתי קטבים מוליכים המתחברים את המערכת.

- **בחושך (התנגדות גבוהה) :** במצב זה, האלקטרונים בחומר ה-CdS קשורים חזק לאטומים שלהם. כתוצאה מכך, ישנם מעט מאוד **נושאי מטען חופשיים** שיכולים לשאת זרם חשמלי, ולכן החיישן מציג **התנגדות גבוהה מאוד** (עשרות קילו-אוהם עד מגה-אוהם).
- **באור (התנגדות נמוכה) :** כאשר אור (המורכב מפוטונים) פוגע בשטח הפנים של ה-CdS, הפוטונים מעבירים אנרגיה לאלקטרונים ומשחררים אותם ממבנה האטום. שחרור זה יוצר מספר רב של **אלקטרונים חופשיים**.
- **תוצאה חשמלית :** ככל שנוצרים יותר אלקטרונים חופשיים, המוליכות של החומר גדלה, וההתנגדות החשמלית של החיישן **יורדת באופן דרסטי**.



אופייני שינוי התנגדות:

החיישן משנה את התנגדותו בהתאם לעוצמת האור אליה הוא נחשף. כאשר החיישן בחושך מוחלט אז התנגדותו מאוד גבוהה (ניתן לראות קטע מדף נתוני היצרן), ואילו כאשר הוא מואר התנגדותו יורדת באופן מהיר. עוצמת תאורה נמדדת ביחידות מידה הנקראות LUX.



אופן המדידה של מודול חיישן האור KY-018 מתבסס על עקרון **מחלק המתח**. הליבה של החיישן היא **נגד תלוי-אור (LDR)**, אשר התנגדותו משתנה באופן הפוך לעוצמת האור. המודול מחובר ל-ESP32 דרך אספקת מתח של 3.3V (VCC). המודול עצמו **כולל בתוכו נגד קבוע** בטור עם ה-LDR כדי ליצור מחלק מתח. כתוצאה משינוי ההתנגדות של ה-LDR, המתח בנקודה המשותפת (המסומנת A0) משתנה. בקר ה-ESP32 מודד את המתח המשתנה הזה באמצעות הפין האנלוגי (A0) ומתרגם אותו לערך מספרי בטווח של **0 עד 4095**, בהתאם לרזולוציית 12bit של הממיר האנלוגי-לדיגיטלי (ADC) שלו.

➤ לחצו כאן לחזרה לתוכן העניינים

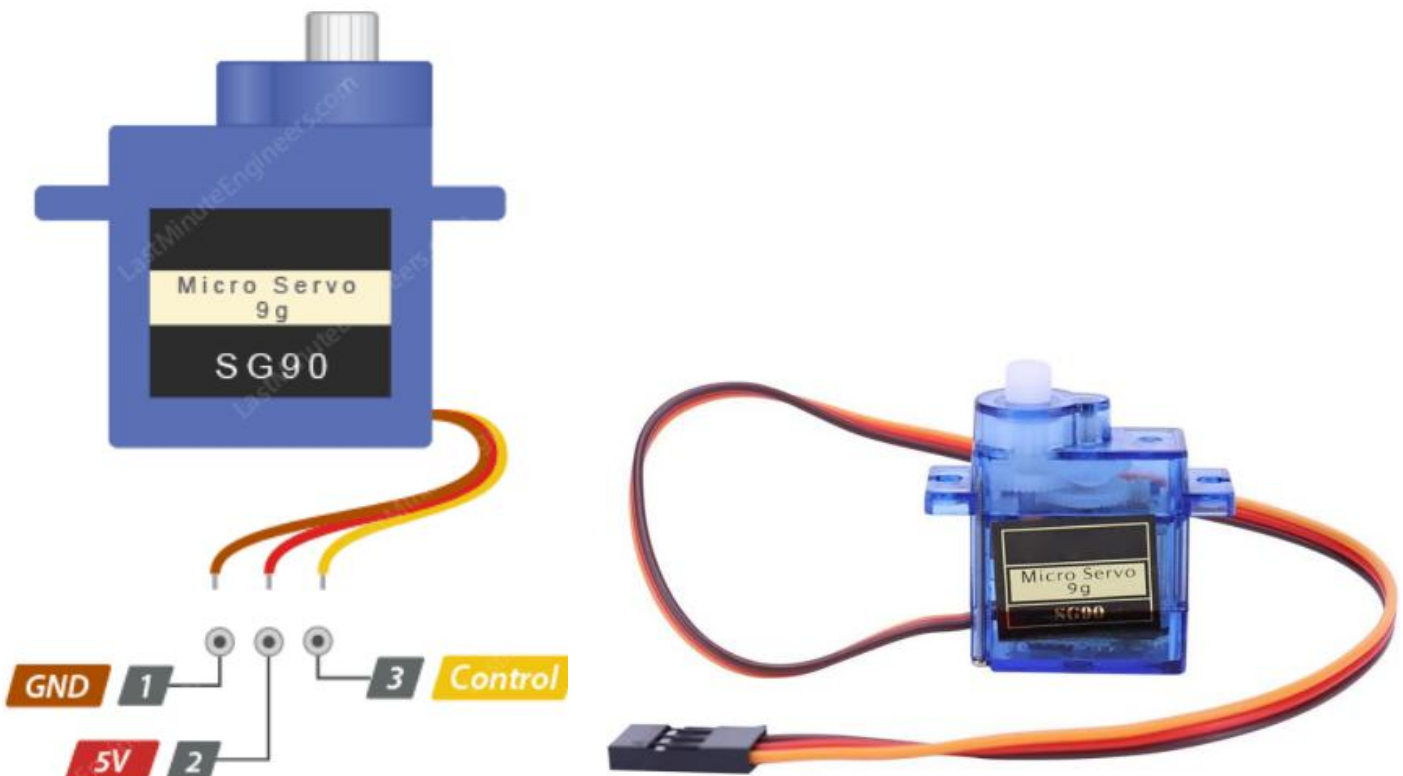
מנוע SERVO

הסבר כללי על המנוע סרוו:

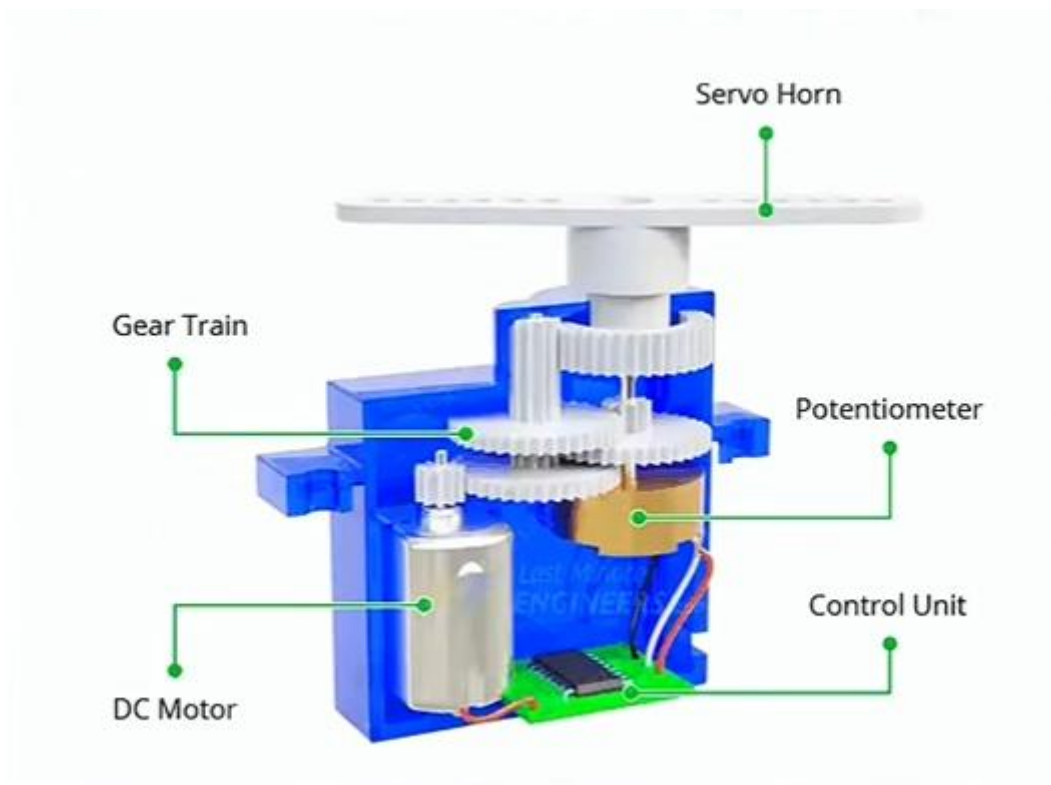
מנוע סרוו הוא מנוע שבו יש את המרכיבים הבאים:

- **מנוע זרם ישר (DC Motor)**
 - **מערכת תמסורת פנימית** של גלגלי שיניים
 - **בקרה אלקטרונית** על מיקום המנוע.
- מנוע SERVO לא מסתובב בצורה חופשית כמו מנועי DC, אלא נע עם פי זווית – לרוב בין 0 ל-180 מעלות (אם כי יש מנועי SERVO שמסתובבים גם עד 360 מעלות).
- מנועי SERVO נפוצים מאוד ומשמשים באפליקציות רבות. הסיבות לכך הן: קלות השליטה על **מנועי SERVO**, דרישות האנרגיה הנמוכות (**יעילות**), (**הכוח החזק יחסית** של המנוע, **רמת המתח** שמשתמשים להפעלתו, **הגודל**, **המשקל והמחיר** הנמוכים שלו).
- מנועי SERVO פועלים **בתוך מעגל סגור**, כלומר יש להם מערכת בקרה על מיקום המנוע והם יכולים לתקן הפרשים מהמיקום הרצוי.
- האיור הבא מתאר מנוע SERVO שניתן להשיג באינטרנט במחירים נמוכים יחסית ואת החיבורים שלו **המנוע נקרא: SG90**.

נראה כיצד נראה המנוע SERVO:



מנוע SERVO מכיל מנוע DC קטן המחובר לציר היציאה דרך גלגלי השיניים. על הציר מחובר פוטנציומטר המחזיר משוב למערכת הבקרה על המיקום הנוכחי בו הוא נמצא.



עקרון פעולה של מנוע SERVO:

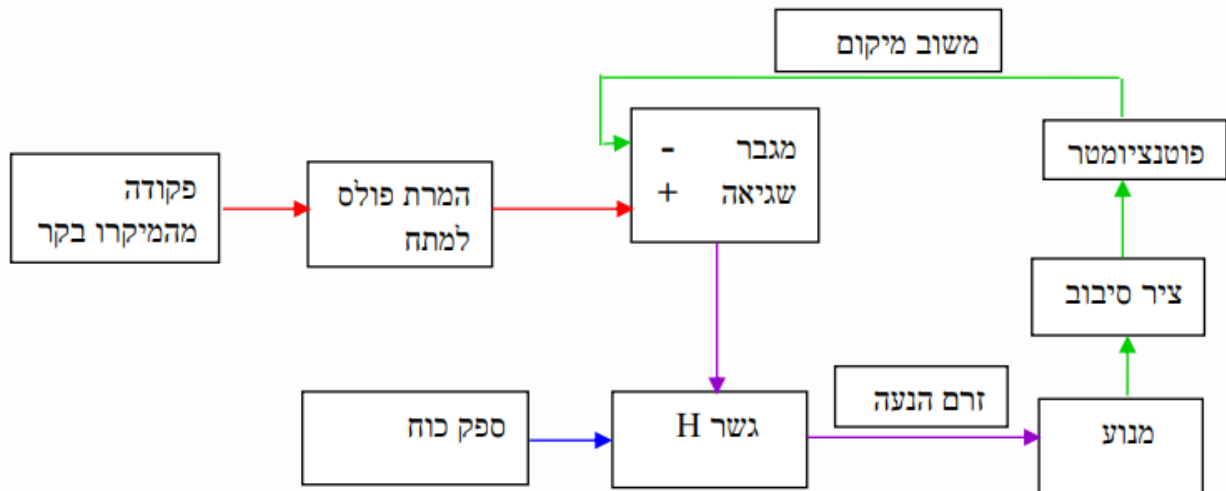
מנוע SERVO מופעל בדרך כלל על ידי **מיקום בקר** הגורם לו להגיע למיקום רצוי. פקודת התנועה מתקבלת בקר מתורגמת למתח **PWM** שמועבר ליחידת הבקרה הנמצאת בתוך המנוע. ערך זה הוא **הערך הרצוי** של המנוע.

הערך המצוי של המנוע, נמדד בעזרת **פוטנציומטר** המחובר לציר המניע של המנוע (הציר שיוצא מחלקו העליון של המנוע ואליו מחברים את המנגנון שיש להניע). סיבוב הפוטנציומטר גורם לשינוי ההתנגדות שלו ולשימושים של שינוי מתח המתח על רגלו של הפוטנציומטר. מתח זה הוא **הערך המצוי** שמשמש כמשוב ל-"בקר הסגור". כך **מתח שגיאה** בין המתח המצוי ולבין המתח הרצוי. מנוע ה-DC הנמצא בתוך ה-SERVO, ימשיך לנוע בכיוון המתאים.

סיבוב מנוע ה-DC מתבצע יחד עם **סיבוב גלגלי השיניים** -שמהווים תמסורת מפחיתה) **מאטה את מהירות הסיבוב ומגדילה את מומנט הכוח**. (גלגלי השיניים מניעים את ציר היציאה ואת הפוטנציומטר. ברגע שהפרש המתחים יהיה **קטן מערך סף (קרוב ל-0)**, יחידת הבקרה של המנוע **תנתק מתח** הנחוץ למנוע ה-DC והוא יחדל לנוע. עצירת המנוע תהיה **בדיוק** בזווית שנשלחה מהתוכנית ב"מיקרו בקר".

אם התוכנית שולחת למנוע פקודה לנוע לזווית מעבר ל-180 מעלות, המנוע יתחיל לרעוד מכיוון שהוא ינסה לסגור את השגיאה בין **הערך הרצוי למצוי** ולא יצליח. כמו כן יש ציר יציאה שמסומם מכני מהמנוע לעבור את זווית הסיבוב של 180 מעלות.

נראה כעת כיצד נראית מערכת הבקרה של מנוע SERVO:



מעולה, הנה גם הטקסט מהתמונה השלישית, מוכן להעתקה לוורד :

בצד שמאל מגיע אות הפקודה מהמיקרו בקר על הזווית הרצויה שנקרא **הערך הרצוי**. זהו פולס עם **Duty Cycle** כאשר הזמן בין ה-ON ל-OFF קובע את הזווית הרצויה. אות זה נקרא גם **מיקום מטרה**.

מעגל "מרת פולס למתח" מעביר למגבר השגיאה מתח שמוצא מה- **Duty Cycle** של אות הפקודה.

מגבר השגיאה מקבל מהפוטנציומטר מתח היחסי למיקום ציר המנוע, מתח זה הוא **המיקום הנוכחי של המנוע** ונקרא **ערך מצוי**.

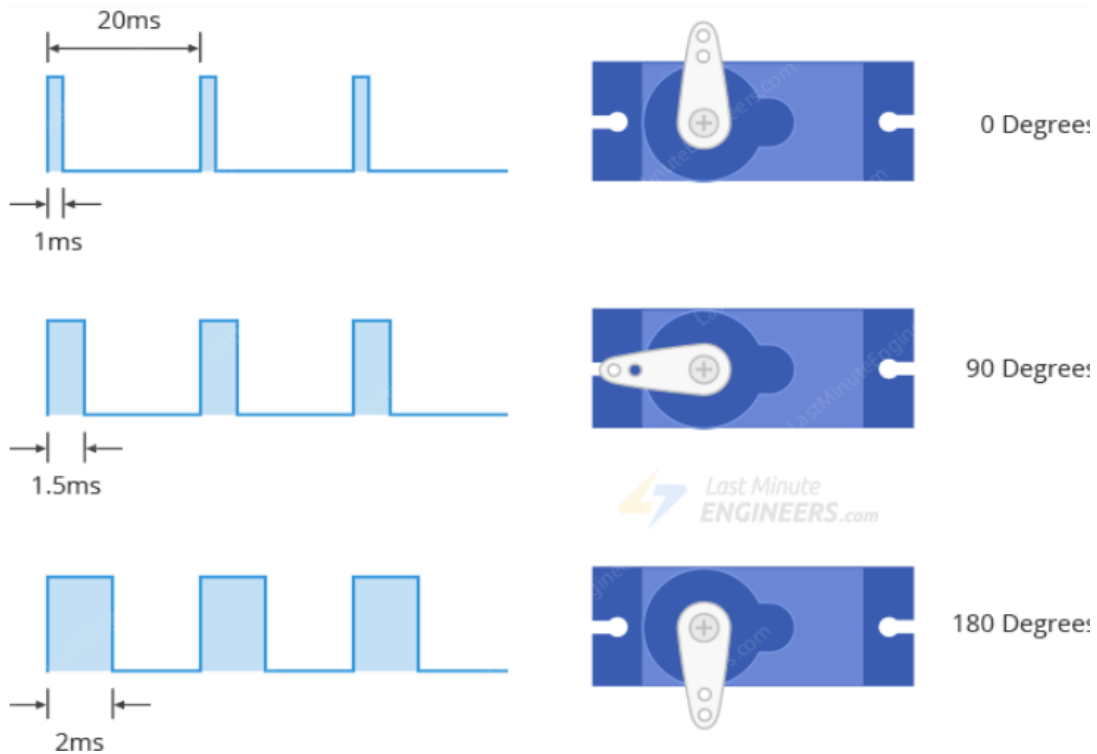
מגבר השגיאה מגביר את **הפרש המתח** בין הרצוי למתח המצוי. יציאת המגבר מחוברת לגשר **H-Bridge** שמעביר זרם המסובב את המנוע.

כאשר המנוע מסתובב הוא **מסובב את הפוטנציומטר** שמעביר שוב מתח שלילי אל **הכניסה המהפכת** (המינוס של המגבר) של המגבר. ככל שהמנוע מסתובב וציר המנוע מתקרב אל **הזווית הרצויה הפרש המתח הולך וקטן**. כאשר מגיעים לזווית הרצויה, **הפרש המתח למגבר הוא 0** ואין איננו מוציא פקודה לסיבוב המנוע והמנוע **עוצר**.

כיצד עובד מנוע סרוו?

עבודה עם פולסים:

אפשר לשלוט על מנוע SERVO על ידי שליחת סדרה של פולסים אליו. מנוע SERVO טיפוסי מחכה לפולס כל 20 אלפיות השנייה כלומר, תדר האות שנשלח למיקרו בקר צריך להיות 50HZ. רוחב הדופק קובע את מיקום מנוע ה-SERVO. האירור הבא מראה את הזוויות של המנוע עבור רוחבי Duty Cycles שונים.



מיקום המנוע כתלות ברוחב הדופק של ה-DUTY CYCLE.

מתיאור רוחב הפולס:

- פולס קצר ברוחב של 1מילישנייה או פחות מסובב את מנוע הסרוו ל- 0 מעלות.
 - פולס ברוחב של 1.5 אלפיות השנייה מסובב את מנוע הסרוו ל- 90 מעלות.
 - פולס ברוחב של 2 אלפיות השנייה בערך מסובב את מנוע הסרוו ל- 180 מעלות.
- פולסים בטווח של 1ms עד 2ms יסובבו את הסרוו למיקום פרופורציונלי לרוחב הפעימה. יש מנועי סרוו המסוגלים להסתובב 360 מעלות. רוחב הפולס במנועים אלו ייתן את הזוויות הבאות:

- 0.5 ms נותן זווית של 0 מעלות.
- 1 ms נותן זווית של 45 מעלות.
- 1.5 ms נותן זווית של 90 מעלות.
- 2 ms נותן זווית של 135 מעלות.
- 2.5 ms נותן זווית של 180 מעלות.

הערה: אין תאום מדויק בין רוחב הפולסים למיקום ולכן יהיה עלינו לשנות את התכנות שלנו כדי להתאים לטווח של מנוע ה-SERVO שלנו. כמו כן, **משך/רוחב הדופק** יכול להשתנות בין יצרני מנועי SERVO שונים; לדוגמה, זה יכול להיות **0.5ms עד 2.5 ms** עבור 180 מעלות ו- **1ms** עבור 0 מעלות.

הטבלה הבאה מאפיינת את המנוע ה-SERVO:

אנגלית	עברית	תיאור
Dimensions	מידות	23.2 x 12.5 x 22 mm
Weight	משקל	9 g
Operating Speed	פעולה מהירות	0.12sec/60 degrees (4.8V) 0.10sec/60 degrees (6V)
Stall Torque	מומנט מעכב	1.3kg.cm/18.09oz.in (4.8V) 1.5kg.cm/20.86oz.in(6V)
Operating Voltage	מתח הפעלה	4.8V~6V
Control System	מערכת בקרה	Analog
Direction	כיוון	CCW
Operating Angle	זווית פעולה	120degrees
Required Pulse	הפולס הנדרש	900us-2100us
Bearing Type	סוג מיסב	None
Gear Type	סוג גלגל השיניים	Metal
Motor Type	סוג המנוע	Plastic
Connector Wire Length	אורך חוטי החיבור	20 cm

עבודה עם מנוע סרוו ומיקרו בקר ESP32:

העבודה עם מיקרו בקר ESP32 בסביבת AEDUINO IDE דומה מאוד לעבודה עם כרטיס מיקרו בקר ארדואינו (אוונו, נאנו, מגה וכו').

התכנות כמעט זהה לתכנות עבור הארדואינו.

הערה חשובה: יש לשים לב כי המתח של מנוע ה-SERVO הוא בין 4.8V עד 6V ולא את מתח ה- 3.3V שברכיבים.

לחצו כאן לחזרה לתוכן העניינים

מודול ממסר כפול 5 וולט



מודול ממסר כפול 5 וולט (2 Channel 5V Relay Module)

מודול ממסר כפול 5 וולט הוא רכיב אלקטרוני חיוני המשמש כמתג אלקטרומגנטי. מטרתו העיקרית היא לאפשר לכם לשלוט במכשירים בעלי מתח זרם גבוהים) כמו נורות ביתיות ב-220V או מנועים (באמצעות מעגל בקרה בעל מתח נמוך ובטוח של **3.3 וולט כמו ה-ESP32**)

עקרון הפעולה של הממסר:

בלב כל ממסר נמצא **אלקטרומגנט** - סליל תיל. כשאתם מפעילים מתח של **5V** על הסליל, הוא הופך למגנט רב עוצמה. המגנט הזה מושך אליו **משך מתכתי** קטן (Armature). תנועת המשך גורמת למגעים **החשמליים** של המפסק להיסגר או להיפתח, ובכך היא סוגרת או מנתקת את המעגל החזק שבו אתם רוצים לשלוט. כלומר, זרם קטן בצד הבקרה משמש כמנוף להפעלת זרם גדול בצד העומס. המודול הזה הוא **כפול** מכיוון שהוא מכיל שני ממסרים כאלה, הפועלים **באופן עצמאי** אחד מהשני. זה מאפשר לשלוט בשני מכשירים שונים באמצעות מודול אחד.

התאמת ממסר 5V ל-ESP32 (3.3V):

השינוי העיקרי נובע מכך שה-ESP32 פועל ברמת מתח לוגי של **3.3V** בעוד סלילי הממסרים במודול דורשים **5V** להפעלה אמינה. הנה שני השיקולים המרכזיים וכיצד להתמודד איתם:

1. אספקת כוח מפוצלת הפרדת VCC ו-JD-VCC

הבעיה הגדולה ביותר היא שסלילי הממסר צורכים זרם גבוה (בדרך כלל 70-100 מיליאמפר לכל ממסר). ה-ESP32 לא יכול לספק את הזרם הזה מהציאות שלו, ואפילו לא מפין ה-5V שלו (אם הוא מופעל מ-USB) הפתרון הוא להפריד את אספקת הכוח.

- בדיקה קריטית: רוב מודולי הממסר מגיעים עם מגשר (Jumper Cap) קטן המחובר בין הפינים (VCC ל- JD-VCC או VCC ל- RY-VCC) הפעולה הנדרשת:

1. יש להסיר את המגשר הזה פעולה זו מפרידה בין מעגל השליטה (VCC) לבין מעגל הכוח של הסלילים (JD-VCC).
2. VCC צד הבקרה : (יש לחבר את VCC של המודול ל- 3.3V של ה-ESP32) זה נותן מתח לעיני הבקרה האופטיות (Optocouplers) שבמודול.
3. JD-VCC צד הכוח : (יש לחבר את JD-VCC למקור מתח חיצוני ויציב של 5V) ספק כוח נפרד או וסת 5V. זה מספק את הכוח הנדרש לסלילי הממסרים.
4. GND : יש לחבר את ההארקה (GND) של ה-ESP32 ואת ה-GND של ספק ה-5V החיצוני ביחד ל-GND של מודול הממסר.

2. רמת מתח השליטה 3.3V מול 5V

פיני הבקרה IN1 ו-IN2 מצפים בתיאוריה לאות 5V כדי להיות מופעלים/מכובים. למזלנו, ברוב המודולים החדשים יותר (שמכילים אופטוקופלרים) :

- אות (HIGH) כיבוי : (מתח של 3.3V היוצא מה-ESP32) בדרך כלל מספיק גבוה כדי להיחשב כ"אות HIGH ולכבות את הממסר (בהנחה שהמודול הוא "הדק רמה נמוכה").
- אות (LOW) הפעלה : (מתח של 0V מה-ESP32) תמיד ייחשב כ"אות LOW ויפעיל את הממסר.

ברוב המקרים, ה-ESP32 עובד ישירות עם מודול 5V ללא צורך בממיר רמות מתח נוסף, בתנאי שהפרדת הכוח (סעיף 1) בוצעה כהלכה.

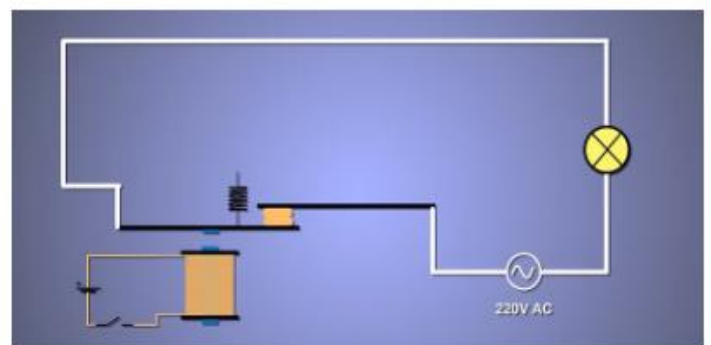
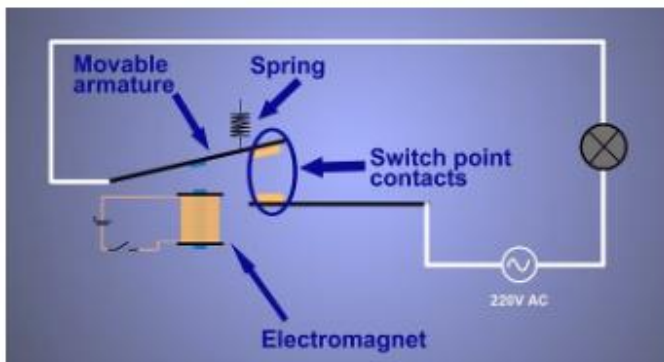
יתרונות מרכזיים

השימוש במודול זה מספק שני יתרונות קריטיים :

1. **בידוד ובטיחות**: הממסר יוצר **בידוד חשמלי מלא** בין מעגל הבקרה הרגיש והבטוח (5V) לבין מעגל הכוח המסוכן (כגון 220V). זה מגן על הבקר שלכם מפני קפיצות מתח ומבטיח שהשליטה תתבצע בבטחה.
2. **יכולת מיתוג**: הוא מאפשר למעגל אלקטרוני פשוט לשלוט על זרמים גבוהים שהבקר לבדו לא יכול היה לשאת, בדרך כלל עד **10 אמפר** (A) ב-250V- AC או 30V- DC (בהתאם למפרט הממסר הספציפי).

תיאור פשוט של אופן פעולת הרכיב

הממסר הוא רכיב חשמלי הבנוי משני חלקים : סליל אלקטרו-מגנטי ומפסק (מתג). כאשר מועבר זרם חשמלי בסליל, נוצר שדה מגנטי המושך את המגעים ופותח או סוגר את המעגל, דבר המוביל לניתוק או הפעלת המכונה/מערכת בהתאמה.



הסבר פנימי של אופן פעולת הרכיב:

מצבי המגעים בממסר

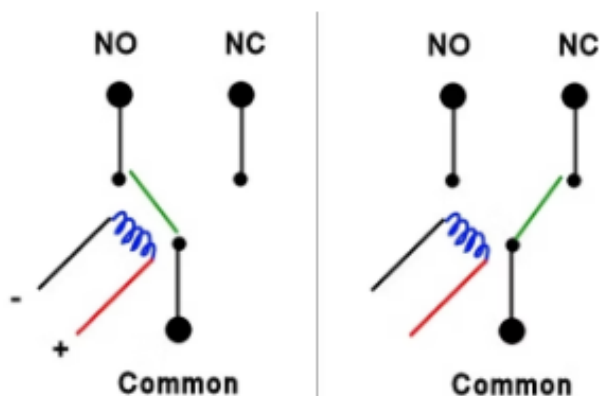
בממסר לרוב אפשר להשתמש בשני מצבים עיקריים :

- **מגע פתוח** : (NO - Normally Open) מצב שבו כשהממסר לא מופעל המגעים פתוחים ולכן לא עובר זרם במערכת (המעגל הנשלט פועל כשהממסר דולק).
- **מגע סגור** : (NC - Normally Closed) מצב שבו כשהממסר לא מופעל המגעים סגורים ועובר זרם במערכת (המעגל הנשלט פועל כשהממסר כבוי).

בממסרים רבים, כמו זה שאני עשיתי בו שימוש, למתכנן/ת המערכת יש אפשרות לבחור לחבר את הממסר בתצורת NO או NC בהתאם לצרכים שלו/ה.

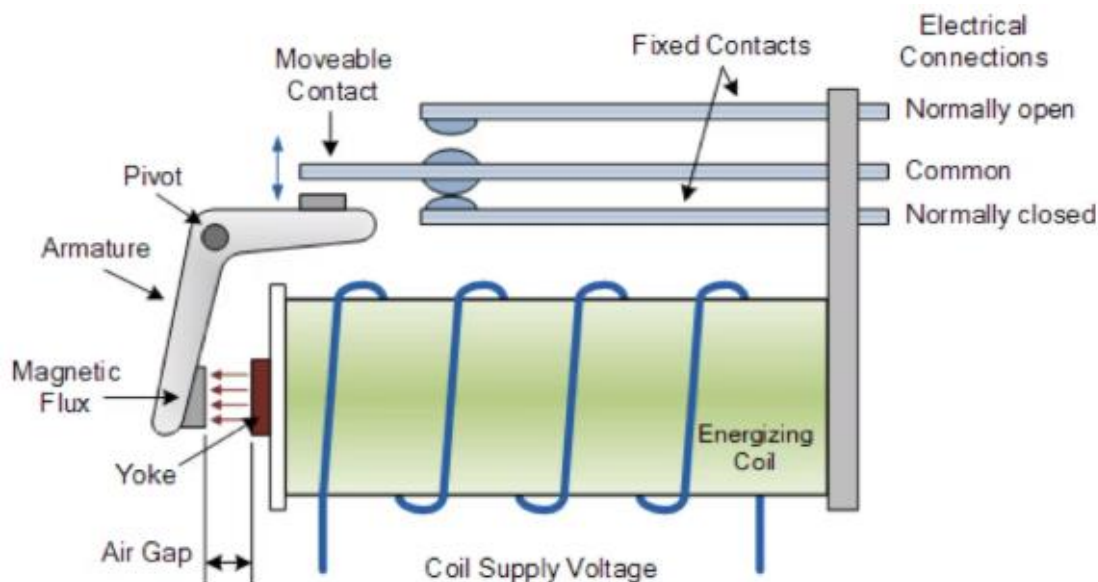
עקרון הפעולה האלקטרומגנטי:

מהסרטוט משמאל אפשר להבין פחות או יותר מה קורה בתוך הממסר עצמו ואיך הוא פועל.



- הסליל הכחול מייצג את מעגל השליטה הלוגי, כשהוא מופעל על ידי הבקר ESP32.
- כאשר מעבירים דרכו זרם נוצר שדה מגנטי שמושך את לשונית המתכת (בירוק).
- הלשונית סוגרת מעגל בין רגל הכניסה (COM - common) לרגל ה-NO.
- כשאין זרם, הלשונית (בעזרת קפיץ) חוזרת למצבה המקורי וסוגרת מעגל בין הכניסה לרגל ה-NC.

נבחין בשרטוט הבא:



בסרטוט מעל ניתן לראות כיצד הממסר האלקטרו-מכני הנפוץ נראה מבפנים ומאילו חלקים הוא בנוי. **סליל אלקטרו מגנטי** שכאשר עובר בו זרם מושך את המגנט ובכך מזיז את לשונית ה- **Common** לסגור מעגל עם לשונית ה- **NO** (Normally Open), וכשבסליל לא עובר זרם הלשונית סוגרת מעגל עם רגל ה- **NC** (Normally Closed).



← כדי להבין איך הממסר עובד מבפנים נחשוב על המפסק הכי פשוט שאפשר לדמיין.



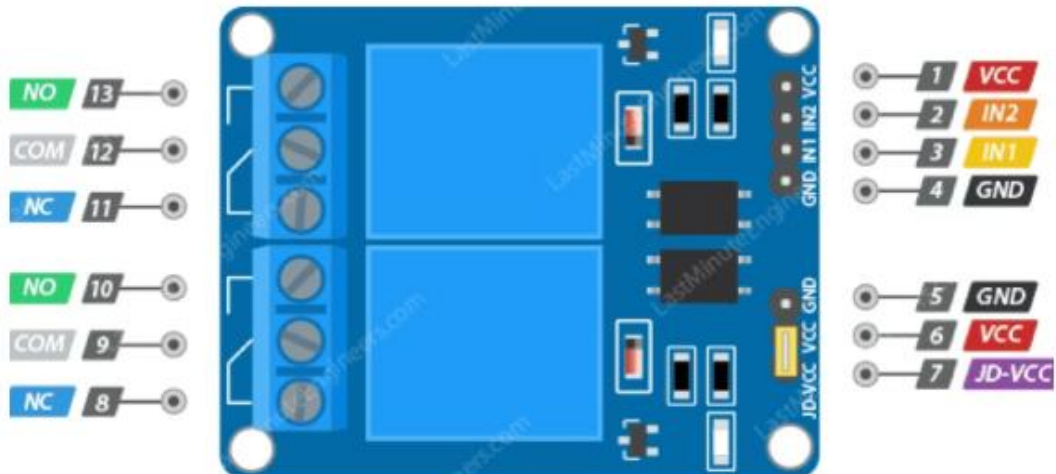
למפסק כזה יש רק שני מצבים, פתוח או סגור. לצורה זו קוראים SPST - single pole single throw והמשמעות היא שיש לו כניסה אחת ויציאה אחת.



לעומת זאת, בממסר שלנו יש כניסה אחת (**Common**) ושתי יציאות למעגל הנשלט, נקרא גם **SPDT (Single Pole Double Throw)**.

זאת אומרת שבכל זמן נתון, אחת היציאות סגורה והשנייה פתוחה. לכן אפשר להשתמש בממסר בשני מצבים נתונים: **NC (Normally Closed)** ו-**NO (Normally Open)**.

נתבונן באיור הבא ונסביר אותו:



מבנה וחיבורי מודול ממסר כפול 5V:

מודול הממסר הכפול מכיל שני ממסרים עצמאיים. החיבורים שלו מתחלקים לשני אזורים עיקריים: כניסות בקרה ויציאות כוח.

1. כניסות הבקרה (צד ה - ESP32 / ארדואינו)

אלו הפינים המקבלים מתח ושליטה מהבקר כגון ESP32 הפינים הנפוצים בצד זה הם:

- **VCC** חיבור למתח האספקה הלוגי של המודול 5V (במקור, או 3.3V כשמשמשים ב-ESP32).
- **GND**: חיבור להארקה המשותפת של המערכת.
- **IN1**: כניסה דיגיטלית לשליטה בממסר הראשון.
- **IN2**: כניסה דיגיטלית לשליטה בממסר השני.

2. יציאות הכוח (צד המעגל הנשלט)

יציאות אלו מחברות את הממסרים למכשיר החשמלי הנשלט. לכל אחד משני הממסרים יש סט של שלושה חיבורים (בורגים) זהים:

- **COM (Common) משותף**: (זוהי נקודת הכניסה המשותפת של הממסר, אליה מחברים קצה אחד של המעגל הנשלט).
- **NO (Normally Open) פתוח רגיל**: (זהו מגע שפתוח במצב מנוחה של הממסר. המעגל נסגר רק כשהממסר מופעל).
- **NC (Normally Closed) סגור רגיל**: (זהו מגע שסגור במצב מנוחה של הממסר. המעגל נפתח רק כשהממסר מופעל).

תיאור כללי של הרכיב:

הממסר הוא רכיב חשמלי המורכב משני חלקים: **סליל אלקטרומגנט ומפסק (מגען)**. מטרתו היא **להעביר או לנתק את הזרם** בין שתי נקודות במעגל חשמלי באופן נשלט. כאשר מועבר זרם חשמלי בסליל, נוצר שדה מגנטי המושך ופוחח/סוגר את המגעים, ובכך מפעיל או מנתק את המערכת בהתאמה.

סקירת סוגי ממסרים נוספים

ממסר מצב מוצק (SSR - Solid State Relay):

בממסר זה אין שימוש בחלקים מכניים, והוא עובד בצורה שונה מהממסר האלקטרו-מכני שאנו מכירים (EMR). הוא משתמש ברכיבים אלקטרוניים מוליכים למחצה, כמו **טרנזיסטורים ותיריסטורים**, כדי לחבר ולנתק את הזרם בין המעגלים (השולט והנשלט) בצורה מבודדת. הוא מחווט בצורה של (SPST כניסה אחת ויציאה אחת בלבד).

יתרונות:

- **עמידות גבוהה** : לעומת ממסר אלקטרו-מכני (EMR), חלקיו אינם נשחקים ומתבלים מהר, ולכן הוא עמיד ליותר זמן.
- **מהירות** : מתאפשרת החלפה מהירה יותר בין מצבי המפסק.
- **שקט** : אין רעש בעת קיום הפעולות.

ממסר עלה (Reed Relay):

ממסר זה, כמו ה-EMR, משתמש בסליל אלקטרומגנטי לפעולה, אך הפעם המוליך מושפע ישירות מהשדה המגנטי. המוליך הוא דק מאוד, ולכן הזרם הדרוש להפעלתו קטן יותר. בממסר זה שני חלקי המתג הם מגעים מוליכים דמויי "עלה". בזמן שהממסר מופעל, הסליל יוצר שדה מגנטי באזור המגעים. קצותיהם נהפכים לבעלי קטבים מגנטיים הפוכים כך שהם נמשכים זה לזה וסוגרים את המעגל. כשהממסר לא פועל, אין שדה מגנטי והמגעים חוזרים למצבם המקורי (מצב פתוח).

גם ממסר זה הוא מסוג SPST.

יתרונות:

- **חסכון בחשמל** : דורש מעט חשמל להפעלה.
- **מהירות** : כמות מופחתת של חלקים, גודלם הקטן והזרם המועט הדרוש מאפשרים לו להחליף מצבים בקצב מהיר יחסית.
- **שחיקה מופחתת** : ה"עלים" נמצאים בתוך שפופרת זכוכית אטומה שמלאה בגז אציל (או מרוקנת), מה שמקטין את הקורוזיה (חלודה) ומאריך את חיי הממסר.

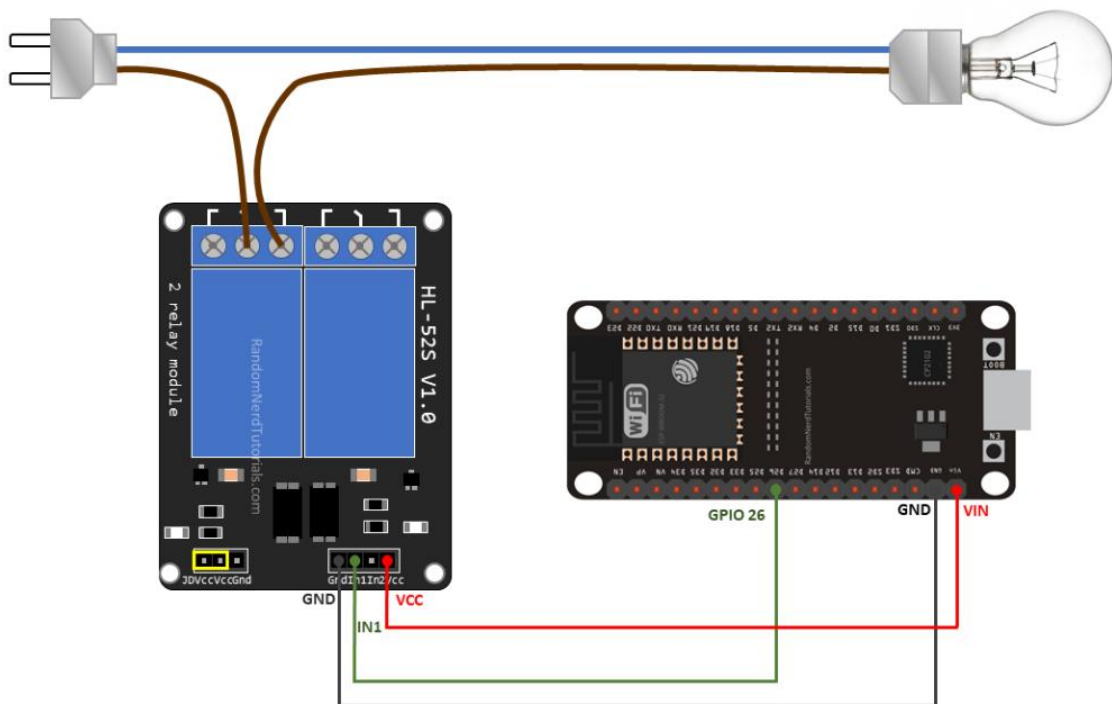
חסרונות:

- **רגישות להפרעות** : הפרעות חיצוניות עלולות להשפיע על תזוזת המגעים בממסר.
- **עדינות מכנית** : את המגעים מקיפה לרוב מעטפת זכוכית עדינה שעלולה להישבר מלחץ רב.

שגיאות שכדאי להימנע מהן וטיפים מועילים:

- כמובן שאלו רק שתי דוגמאות לממסרים נוספים שאפשר למצוא בנוסף לממסר אלקטרו-מכני. ישנם סוגים רבים נוספים של ממסרים הפועלים באופנים שונים שמבצעים פעולות דומות.
- ברירת מחדל:** חשבו היטב כיצד אתם רוצים שברירת המחדל של המעגל הנשלט שלכם תיראה, ככה תוכלו לדעת לאן לחבר אותו בממסר (ל- NO או ל- NC)
- קליק חזק:** כשהממסר מופעל שומעים קליק חזק. לא לפחד, זה סימן שהוא עובד.
- מקור מתח נפרד:** המעגל הנשלט לא מקבל מתח מהבקר, (ESP32) לכן צריך לדאוג שהוא יהיה מחובר למקור מתח נפרד! (אפשר לראות דוגמה בהסבר על החיווט).
- שימוש כפול:** אפשר להשתמש באותו ממסר לשני מעגלים שונים אם רוצים שהם אף פעם לא יופעלו במקביל ושיהיו תלויים באותם משתנים.
 - למשל: לחבר שני לד סטריפים בעלי מקור מתח משותף, אחד לרגל של ה-NC והשני לרגל של ה-NO, וכך כשהממסר יפעל – לד סטריפ אחד יאיר והשני יהיה כבוי, וכשהממסר יכבה – הראשון יכבה והשני יפעל.
- דוגמאות לשימוש:** מנורת לילה שנדלקת כשהחדר חשוך, מאורר שמופעל כשהטמפרטורה גבוהה מדי.
- מגבלות מתח:** בסדנה אנו משתמשים בדרך כלל בממסר לשליטה על מעגלים בעלי מתח של עד 24V, אבל בפועל הוא מסוגל להחזיק עד 10 אמפר במתח של 250V.
- התאמה לבקר:** הממסר דורש יחסית מעט מאוד מתח להפעלה, לכן הוא מצוין לשימוש בבקרים כמו ה-ESP32 שמסוגל להעביר לו את המתח הלוגי הדרוש (3.3V) לשליטה. מומלץ לשים לב עם אילו מתחים הממסר שלך פועל כדי לוודא שהוא יעבוד עם הבקר.

נראה חיבור של ממסר כפול המדליק מנורה דרך מיקרו בקר ESP32:



הסבר מפורט על החיבורים בסכמה בדף הקודם:

הסכימה מחולקת לשלושה חלקים עיקריים: בקר ה-ESP32 (המוח), מודול הממסר הכפול (המפסק) ומעגל העומס (הנורה).

1. חיבור צד הבקרה (ESP32 למודול הממסר)

חיבור זה משתמש במתח נמוך (V3.3) כדי לשלוט על הממסר.

- אספקת מתח לוגית (VCC): הפין VCC במודול הממסר מחובר לפין VIN (Voltage In) ב-ESP32. במקרה זה, ה-VIN בדרך כלל מספק 5V למודול (אם ה-ESP32 מופעל מ-USB), אך חשוב לציין שרמת המתח הלוגית של ה-ESP32 היא 3.3V.
- הארקה (GND): הפין GND במודול הממסר מחובר ל-GND ב-ESP32.
- כניסת שליטה (IN1): הפין IN1 במודול (השולט על הממסר הראשון) מחובר לפין GPIO 26 ב-ESP32. זהו האות הדיגיטלי (LOW/HIGH) שבו ה-ESP32 משתמש כדי להפעיל או לכבות את הממסר (בשיטת "הדק רמה נמוכה").
- חיבור JD-VCC: ניתן לראות שהחיבורים JDVCC ו-GND בצד המודול נשארים מנותקים (או שמחוברים לכוח V5 חיצוני באמצעות ספק כוח נפרד), וזהו החיבור הנכון והמומלץ עבור ESP32, שמבטיח שהסליל יקבל את הכוח הדרוש לו (V5) ולא יפגע בבקר.

2. חיבור צד העומס (הנורה למודול הממסר)

חיבור זה משתמש בממסר כדי לחבר או לנתק את זרם ה-AC מהנורה.

- מעגל ה-AC: כבל המתח (במקרה זה, כבל הפאזה, בדרך כלל חום/כחול) מהתקע נחתך.
- חיבור ל-COM: קצה אחד של הכבל החתוך (הבא מהתקע) מחובר לפין COM (Common) של הממסר הראשון.
- חיבור ל-NO: הקצה השני של הכבל החתוך (המוביל אל הנורה) מחובר לפין NO (Normally Open) של אותו ממסר.

3. משמעות התצורה

התצורה שנבחרה היא NO (Normally Open), שפירושה:

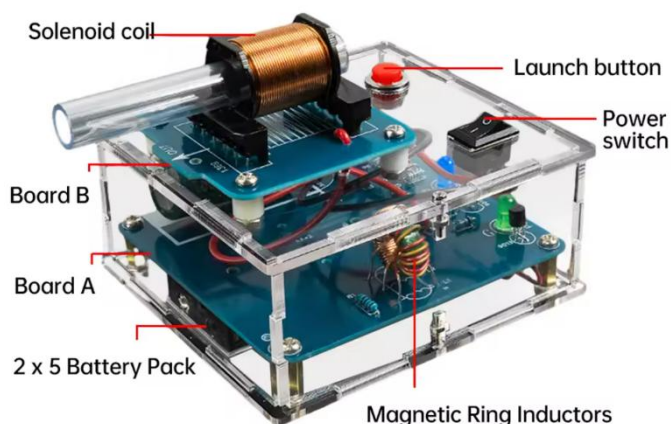
- ברירת מחדל: כאשר ה-ESP32 כבוי או שולח אות HIGH (3.3V), הממסר נמצא במצב מנוחה, המגעים פתוחים, והנורה כבוייה.
 - הפעלה: כאשר ה-ESP32 שולח אות LOW (V0) לפין GPIO 26, הממסר מופעל, סוגר את המעגל בין COM ל-NO, והנורה נדלקת.
- לסיכום, הסכימה מראה כיצד ה-ESP32 שולט בבטחה במכשיר חשמלי (נורה) באמצעות מודול ממסר כפול, תוך ניצול העיקרון האלקטרומגנטי לניתוק או חיבור של זרם ה-AC.

➤ לחצו כאן לחזרה לתוכן העניינים

תיעודים, בדיקות והרכבות

רובה אלקטרומגנטי (Electromagnetic gun) + הרכבה

תיאור כללי:



המערכת המתוארת בתמונה היא **רובה אלקטרומגנטי חד-שלבי** הפועל באמצעות שדה מגנטי שנוצר ע"י סליל (Solenoid Coil) המאיץ גוף מתכתי דרך קנה פלסטי. הרובה נשלט ידנית באמצעות כפתור שיגור, ומקבל את אספקת החשמל ממארז סוללות פנימי.

מטרת המערכת:

שיגור גופים מתכתיים גליליים קטנים (הנקראים כאן "Bombs") באמצעות כוח מגנטי – לצורכי הדגמה פיזיקלית, חינוכית או ניסיונית.

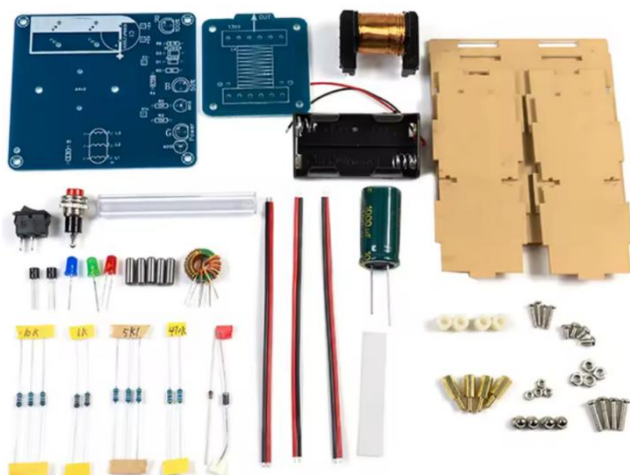
טבלת הסבר של התמונה:

תיאור

רכיב

Solenoid Coil סליל אלקטרומגנטי	סליל נחושת דרכו עובר זרם גבוה. יוצר שדה מגנטי חזק ברגע ההפעלה ומושך את הגוף המתכתי לתוכו במהירות גבוהה.
Launch Button כפתור שיגור	כפתור אדום המפעיל את המערכת ומזרים זרם לסליל. זהו רכיב השליטה הידני לשיגור.
Power Switch מתג הפעלה	מתג הפעלה/כיבוי ראשי של המערכת. מנתק ומחבר את אספקת המתח לכלל המעגלים.
Board B לוח עליון	מכיל את הסליל, קונקטורים, כפתור ההפעלה, נורות בקרה, והחיבורים הפיזיים.
Board A לוח תחתון	מכיל את הרכיבים החשמליים המרכזיים – קבלים, נגדים, מייצבים, לולאות השראה וכו'. אחראי על ויסות המתח וההגנה על המערכת.
2x5 Battery Pack מארז סוללות	שתי סדרות של 5 סוללות המספקות מתח כולל גבוה (לרוב 18-24 וולט) אחראי על אספקת האנרגיה לסליל.
Magnetic Ring Inductors חישוקי השראה מגנטיים	משמשים להחלקת הזרם, הפחתת רעשים אלקטרומגנטיים, ואחסון אנרגיה זמנית במעגל.
Bombs כדורים מתכתיים	גופים גליליים ממתכת אשר מוכנסים לקנה. בזמן שיגור הם נמשכים דרך הסליל ויוצאים במהירות מהקצה השני.

תמונה של הרכיבים ברובה האלקטרומגנטי:



עקרון פעולה:

1. המשתמש מפעיל את מתג ההפעלה.
2. בלחיצה על כפתור השיגור, הזרם מוזרם מהסוללות לסולנואיד.
3. נוצר שדה מגנטי חזק בתוך הסליל.
4. גוף המתכת נמשך פנימה ומואץ לאורך הקנה.
5. לאחר מכן, הגוף יוצא במהירות מהקצה – ללא חלקים נעים מכניים.

הטבלה הבאה מציגה את כל רכיבי הערכה לרובה האלקטרומגנטי כפי שמופיעים בתמונה למעלה, כולל תיאור תפקודי של כל רכיב במערכת:

תיאור תפקודי	רכיב
לוח המעגל המרכזי שעליו מולחמים כל רכיבי האלקטרוניקה, כולל נגדים, קבלים וסלילים.	לוח PCB ראשי
לוח עזר – לרוב משמש לקישוריות בין סליל הירי לבין הלוח הראשי או כקונקטור.	לוח PCB משני
סליל נחושת דרכו עובר זרם ליצירת שדה מגנטי חזק המאיץ את הקליע לאורך הקנה.	סולנואיד (Solenoid Coil)
מחזיק שני סטים של 5 סוללות AA – מספק מתח גבוה (לרוב 18V–24V) להפעלת הסולנואיד.	מחזיק סוללות (2x5)
חלקי גוף המערכת – חיתוך לייזר מחומר מוקשה / (MDF) אקריליק (להרכבת המארז).	פלטות מארז (Housing Panels)
כפתור אדום להפעלת השיגור – מזרים זרם מיידי לסליל לצורך ירי.	כפתור שיגור (Launch Button)
מפסק ראשי להפעלת או כיבוי המתח הכללי של המערכת.	מתג הפעלה (Power Switch)
נורת חיווי – מדליקה כשהמערכת פעילה או מוכנה לשיגור.	נורת לד
משמש כמסילה לירי – דרכו עובר הקליע במהלך השיגור.	צינור פלסטיק (קנה)
גופים גליליים ממתכת אשר נמשכים אל תוך הסליל ויוצאים במהירות – משמשים להדגמה.	קליעים מתכתיים (Bombs)

רכיבי השראה מגנטיים לאגירת אנרגיה ולשיפור יציבות הזרם.	טורואידים / סלילים טבעתיים
משמש לאגירת אנרגיה קצרה והעברת זרם חזק מידי בעת השיגור.	קבל אלקטרוליטי
רכיבים להגבלת זרם, קביעת מתח, או תיאום בין רכיבים שונים במעגל.	נגדים (Resistors)
חוטים אדומים/שחורים לקישוריות חשמלית בין הלוחות, הסוללות, הסולנואיד והכפתורים.	חוטים ומוליכים
חלקים מכניים להרכבת הלוחות, קיבוע הסוללה, בניית המסגרת וחיבור כללי של המערכת.	ברגים, אומים ומרווחים

יתרונות המערכת:

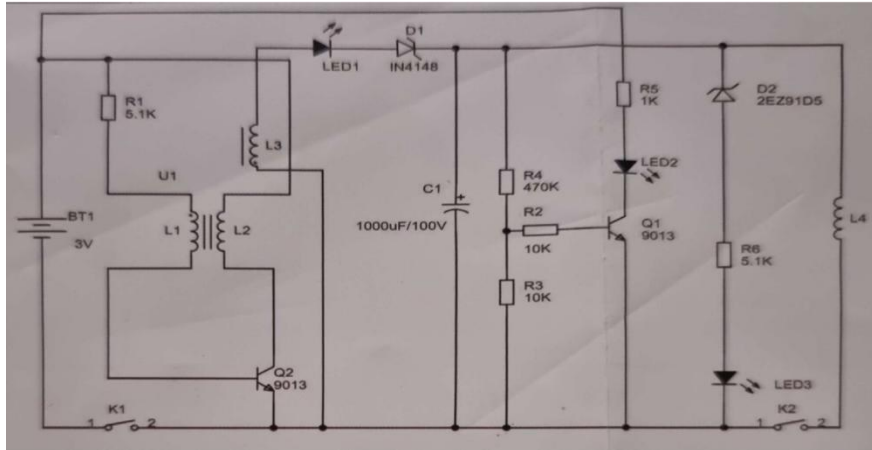
הסבר	יתרון
מדגים את עקרונות ההנעה המגנטית ללא צורך במכאניקה מסובכת.	שימוש בטכנולוגיה אלקטרומגנטית מתקדמת
מקטין בלאי, שקט יותר ובטוח יותר (יחסית).	אין חלקים נעים מכאניים
כפתור אחד לשיגור – מתאים ללמידה, הדגמה וניסויים.	תפעול פשוט
ניתן להפעיל את המערכת בכל מקום ולשאת אותה בקלות.	גודל קומפקטי וניידות
ניתן לשגר מספר פעמים עם אותן "פצצות".	רב פעמיות

חסרונות המערכת:

הסבר	חסרון
בשל מגבלות מתח, סליל יחיד וגוף קל – העוצמה והמהירות יחסית נמוכות.	עוצמת שיגור מוגבלת
חלק גדול מהאנרגיה מתבזבז כחום ולא תורם להאצת הגוף.	יעילות אנרגטית נמוכה
שימוש רצוף עלול לגרום להתחממות יתר ולפגיעה במעגלים.	חימום רכיבים
ללא חיישני מיקום – אין ניתוק מדויק של השדה ולכן תיתכן האטה בשלב מסוים.	אין בקרת תזמון מתקדמת
אם קיים שלב טעינת קבלים, יידרש זמן בין שיגור לשיגור.	טעינה איטית (אם משתמש בקבלים)

רובה אלקטרומגנטי הוא מערכת מרתקת ומשולבת – מדגים פיזיקה, חשמל, אלקטרוניקה ומכאניקה ברמה בסיסית. הוא מהווה פלטפורמה חינוכית מצוינת להבנת עקרונות השראה מגנטית, בקרה, ואנרגיה. מבנהו הקומפקטי והשימוש הנגיש הופכים אותו לכלי לימוד ולמעבדה ניידת לכל דבר.

הסבר על מעגל חשמלי - רובה אלקטרומגנטי



הסבר כללי:

המעגל משתמש ברכיבים כמו קבלים, סלילים (L), טרנזיסטורים, דיודות, ונורות LED, כדי לטעון קבל במתח גבוה ואז לפרוק אותו דרך סליל אלקטרומגנטי לצורך יצירת דחף (למשל, שיגור פרויקטיל מתכתי קטן).

הסבר לפי אזורים:

אזור הספק:

BT1 - סוללה 3(V) - מספקת את האנרגיה למעגל.
K1 - מתג הפעלה ראשי שמחבר את החשמל.

חלק טעינת הקבל:

C1 - קבל $1000\ \mu\text{F}$ – 100V נטען בהדרגה דרך המעגל, ואוגר אנרגיה שתשוחרר בבת אחת לשיגור.
D1 - דיודה 1 – (N4148) מגינה על המעגל מזרם חוזר.
LED1 - מציין שהקבל נטען.
L1, L2, L3 - סלילים כנראה כחלק ממעגל מתנד/רזוננס שמעביר אנרגיה לקבל.

חלק בקרה:

Q1, Q2 - טרנזיסטורים מסוג 9013 – מפעילים או חוסמים את מעבר הזרם.
R1-R6 - נגדים – קובעים את זרימת הזרם לבסיס הטרנזיסטורים, וכך שולטים עליהם.
LED2 - חיווי נוסף למצב ההפעלה של השלב הבא.

חלק הפעלת סליל השיגור:

D2 - דיודה Zener או SCR (2EZ91D5) מפעילה שחרור מהיר של האנרגיה מהקבל.
L4 - סליל השיגור – כאשר הקבל נפרק דרכו, נוצר שדה מגנטי חזק שיכול לדחוף אובייקט מתכתי קטן.
K2 - מתג הפעלה לשיגור.
LED3 - נדלק כאשר מתבצע השיגור.

טבלת רכיבים:

תפקיד	תיאור	רכיב
מקור מתח	סוללה V3	BT1
K1-הפעלה, K2-שיגור	מתגים	K1, K2
מגברים/מתגים לבקרה	טרנזיסטורים 9013	Q1, Q2
אגירת אנרגיה חשמלית	קבל $100\mu\text{F}$ - 100V	C1
הגנה מזרם חוזר	דיודה N41481	D1
פריקה פתאומית של הקבל	דיודה Zener / SCR	D2
קביעת זרמים לבקרה	נגדים	R1-R6
מראה מצבי פעולה שונים	נוריות חיווי	LED1-LED3
ייצוב, תדר, שיגור	סלילים	L1-L4

פירוט רכיבי המעגל ותפקודם האלקטרוני:

C1: קבל האגירה ($100\text{V}-1000\mu\text{F}$):

- **תפקיד ראשי:** המאגר הראשי של האנרגיה הפוטנציאלית החשמלית. הקבל נטען באופן יזום למתח גבוה (עד 100V) ומחזיק את האנרגיה הזו עד לשלב השיגור.
- **ניתוח מקצועי:**
 - **קיבול:** ($C=1000\mu\text{F}$) הקיבול נמדד בפאראדים (Farads) וקובע את כמות המטען (Q) הנדרשת כדי להגיע למתח נתון.
 - **מתח נקוב:** (100V) זהו המתח המקסימלי הבטיחותי שהקבל יכול לעמוד בו. במעגל דחף, חשוב שהמתח הנספג במהלך הטעינה לא יעלה על ערך זה.
- **אנרגיה אגורה:** האנרגיה הפוטנציאלית המקסימלית האגורה בקבל נתונה על ידי הנוסחה

$$E = \frac{1}{2} CV^2$$

עבור קבל זה בטעינה מלאה (100V):

$$E = \frac{1}{2} \cdot (1000 \cdot 10^{-6} \text{ F}) \cdot (100 \text{ V})^2 = \frac{1}{2} \cdot 0.001 \cdot 10,000 = 5 \text{ Joules}$$

זוהי אנרגיית השיגור המקסימלית העומדת לרשות המערכת.

D1: דיודת הגנה (1N4148)

- **תפקיד ראשי: חסימת זרם חוזר (Reverse Current Blocking).** הדיודה מוודאת שהזרם יזרום רק מהמעגל המגביר המעגל שמשתמש ב- (L1,L2,L3) אל הקבל (C1).
- **ניתוח מקצועי:**
 - **הגנה מפני פריקה:** בזמן השיגור (כאשר הקבל נפרק), המעגל חווה שינוי דרסטי במתח וזרם גדול. הדיודה מונעת מהזרם הפורק להזיק לרכיבי המגבר (כגון טרנזיסטורים) שעלולים להיות עדינים.
 - **דגם הדיודה: (1N4148) זוהי דיודת מיתוג מהירה וקטנה (Small Signal Switching Diode),** עם זרם קדימה מקסימלי נמוך יחסית (כ-150mA) לכן, יש להניח שהיא ממוקמת **במעגל הטעינה בלבד**, היכן שהזרם קטן יחסית (שכן הקבל נטען **בהדרגה**, ולא כחלק ממעגל הפריקה של ה-5J שם נדרשת דיודה חזקה בהרבה (כגון דיודת UF4007 או FR107).

LED1: נורית חיווי

- **תפקיד ראשי: אינדיקציה חזותית למצב הטעינה.**
- **ניתוח מקצועי:**
 - נורית זו מעידה כי המתח על הקבל (V) הגיע לרמה מספקת או מלאה. היא מחוברת בדרך כלל **במקביל לקבל** בטור עם נגד הגבלת זרם מתאים. כיוון שמתח הטעינה הוא 100V, יש צורך **בנגד הספק גבוה** או במנגנון מיתוג (כגון נורית ניאון קטנה או טרנזיסטור) כדי להבטיח שהיא לא תישרף בגלל המתח הגבוה.

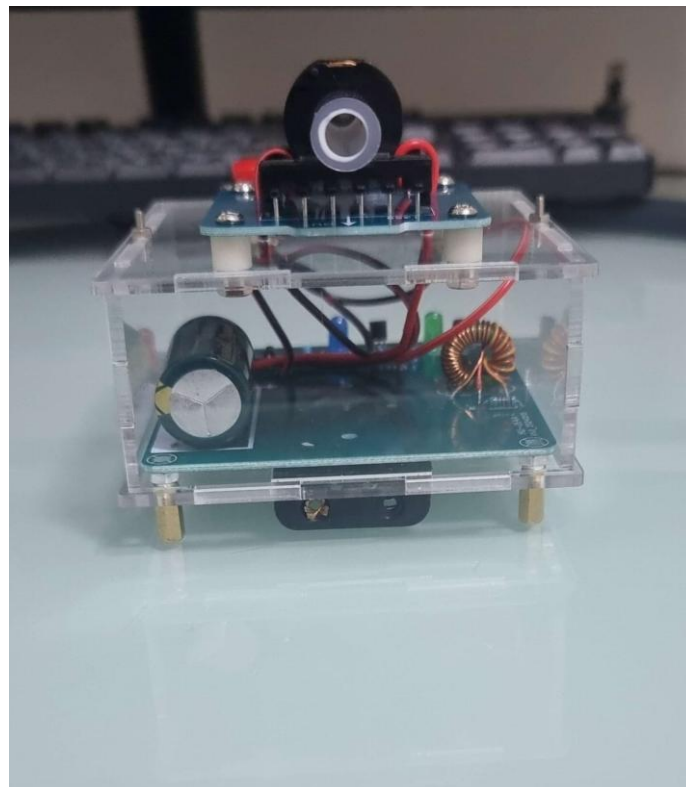
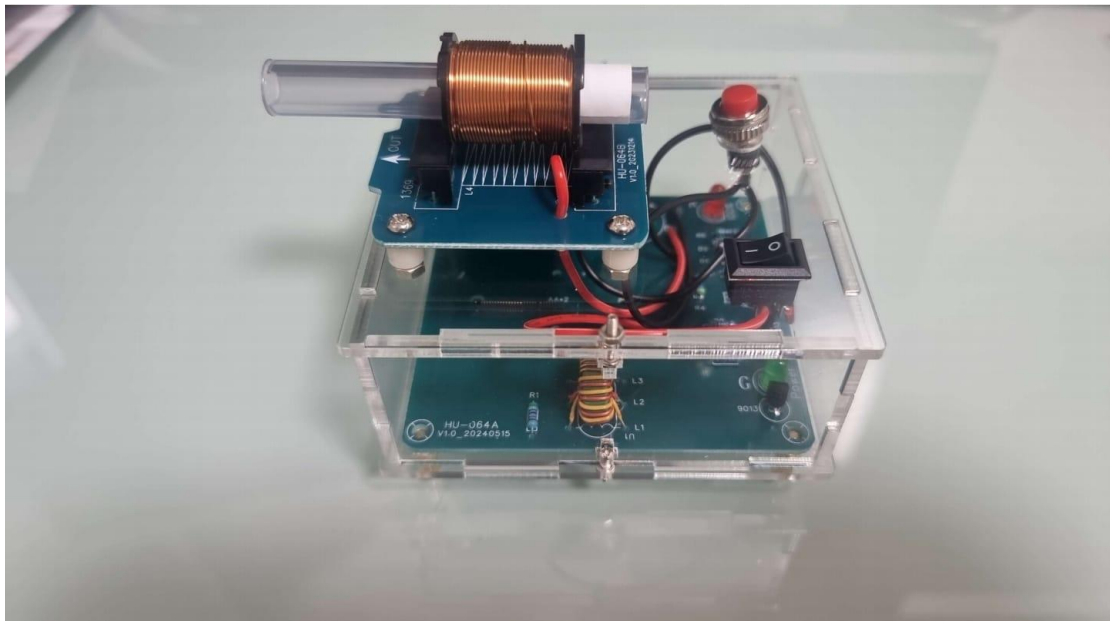
L1, L2, L3: סלילים (Inductors/Coils):

- **תפקיד ראשי:** רכיבים אלו הם ליבת מעגל המגביר (Boost Converter) או ממיר מתח מרום (Flyback Converter). תפקידם הוא להפוך מתח כניסה נמוך (כגון סוללה של 3V–12V למתח גבוה של 100V)
- **ניתוח מקצועי:**
 - **ממיר DC-DC:** סלילים אלה משמשים ליצירת שדה מגנטי. על ידי מיתוג מהיר של זרם דרך סליל ראשוני (L1) או (L2) באמצעות טרנזיסטור, נוצרת **השראה הדדית** שמייצרת **מתח גבוה** בסליל משני L2 או L3.
 - **דחיית זרם ישר:** המעגל המגביר פועל על העיקרון של **שינוי מהיר בשטף המגנטי**, שיוצר מתח גבוה על פי חוק פאראדיי. המתח הגבוה הזה מיושר באמצעות דיודה (אולי, D1 אם כי D1 היא חלשה מדי, או דיודה נוספת) ונשלח לטעינת C1.

סיכום ומסקנה (הקשר המערכת):

- המערכת כולה מתארת **מחולל דחף אלקטרומגנטי**, העובר את שלבי הפעולה הבאים:
1. **הגברה:** המתח הנמוך מוגבר ל-100V באמצעות מעגל המתנד/הממיר (L1,L2,L3).
 2. **טעינה:** המתח המוגבר עובר דרך דיודת הגנה (D1) ונטען לתוך **קבל האנרגיה** (C1).
 3. **מוכנות:** כאשר C1 מגיע ל-100V (אוגר 5J), נורית החיווי (LED1) נדלקת.
 4. **שיגור:** בשלב זה, באמצעות מתג מיוחד שאינו מצוין ברשימה, כגון טרנזיסטור MOSFET או SCR בעל זרם גבוה, משוחררת האנרגיה האגורה ב-C1 **בבת אחת** לתוך סליל שיגור (העומס), מה שמייצר את הדחף האלקטרומגנטי הדרוש לשיגור.

תמונות של התוצאה הסופית של הרובה:



קישור לסרטון תיעוד בו אני מרכיב את המערכת של הרובה האלקטרומגנטי ומדגים את אופן פעולתו וכיצד מתבצעת הירייה (יש לחץ CTRL + על קישור של הסרטון)

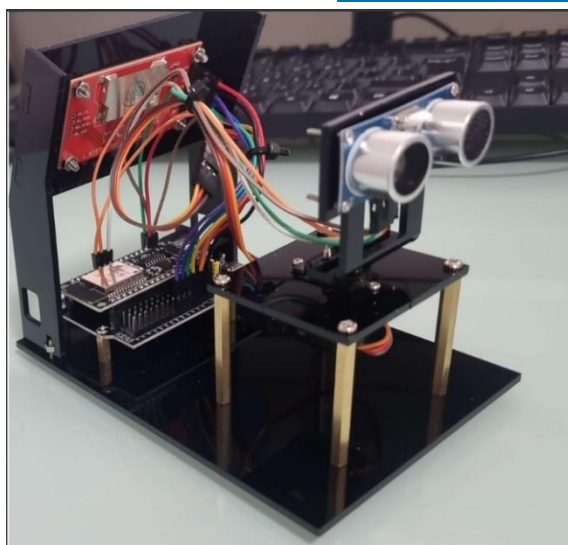
<https://youtu.be/5IMt-HXeCIk?feature=shared>

לחצו כאן לחזרה לתוכן העניינים

מיני רדאר (Mini Radar) + הרכבה

המטרה והעקרון:

המטרה של הפרויקט הייתה ליצור מערכת שתסרוק את הסביבה הקרובה שלה, תזהה מכשולים ותציג אותם על גבי מסך, בדיוק כמו מכ"ם אמיתי.



הרכיבים והתפקידים שלהם:

כמו בכל פרויקט מוצלח, גם המערכת הזו מורכבת מכמה חלקים שפועלים יחד:

- המוח: זהו המיקרו-בקר, הלב והמוח של המערכת. הוא שולט בכל הרכיבים ומבצע את החישובים המורכבים בזמן אמת.
- העיניים: החיישן האולטרה-סוני, שמזכיר קצת שתי עיניים קטנות. עין אחת משדרת פולס קולי שאנחנו לא יכולים לשמוע, והשנייה קולטת את ההד שחוזר. הנתון הקריטי שאנחנו מקבלים הוא הזמן שלקח לפולס הזה לצאת ולחזור.
- הזרוע: זהו מנוע הסרוו. הוא מאפשר לחיישן שלנו לנוע מצד לצד, מה שנותן למערכת את היכולת לסרוק שטח שלם ולא רק נקודה אחת.
- התוכנה: כל החלקים הפיזיים לא שווים כלום בלי הקוד שכתבתי. התוכנה היא זו שמנחה את מנוע הסרוו לנוע בזוויות שונות (למשל, 5 מעלות בכל פעם), ובכל עצירה היא מורה לחיישן למדוד את המרחק לעצם. לאחר מכן, הקוד שולח את נתוני המרחק והזווית למסך חיצוני או למחשב.



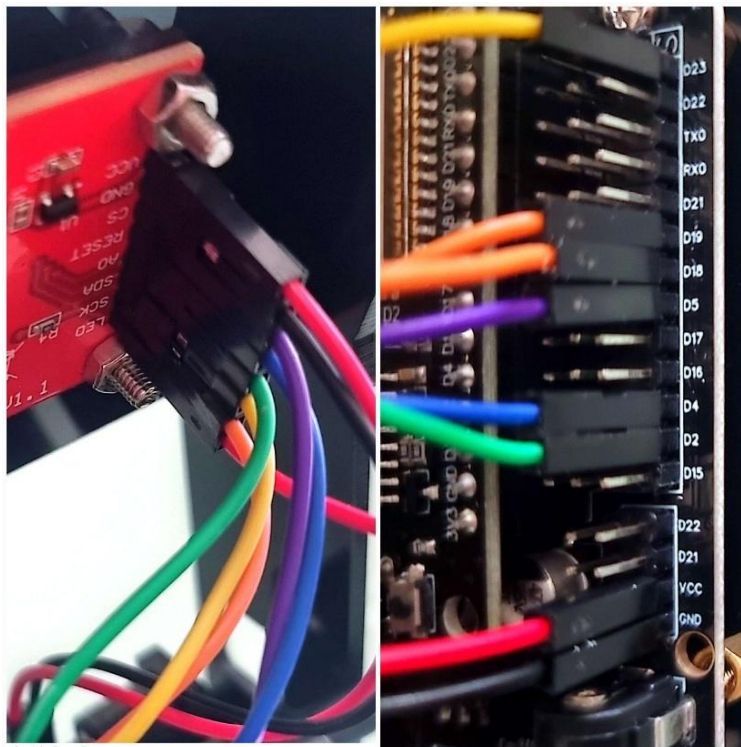
איך זה עובד בפועל?

התהליך פשוט ואלגנטי: המנוע מסתובב בזווית מסוימת, החיישן מודד את המרחק לעצם הקרוב מוזנים לקוד. התוכנה לוקחת את הנתונים – **הזווית והמרחק** – ביותר, ושני הנתונים האלו ומציגה אותם בצורה ויזואלית על מסך. ככל שהמכשול קרוב יותר, כך הנקודה שתופיע על ה"מכ"ם" תהיה קרובה יותר למרכז. על ידי חזרה על התהליך בכל זווית לאורך הקשת של 180 מעלות, אנחנו מקבלים תמונה שלמה של הסביבה שנמצאת מולנו.

הפרויקט הזה מדגים ששילוב פשוט של רכיבים יכול ליצור מערכת מורכבת ויעילה, והוא פותח את הדלת ליישומים מרתקים כמו רובוטים אוטונומיים, מערכות אבטחה ביתיות ועוד.

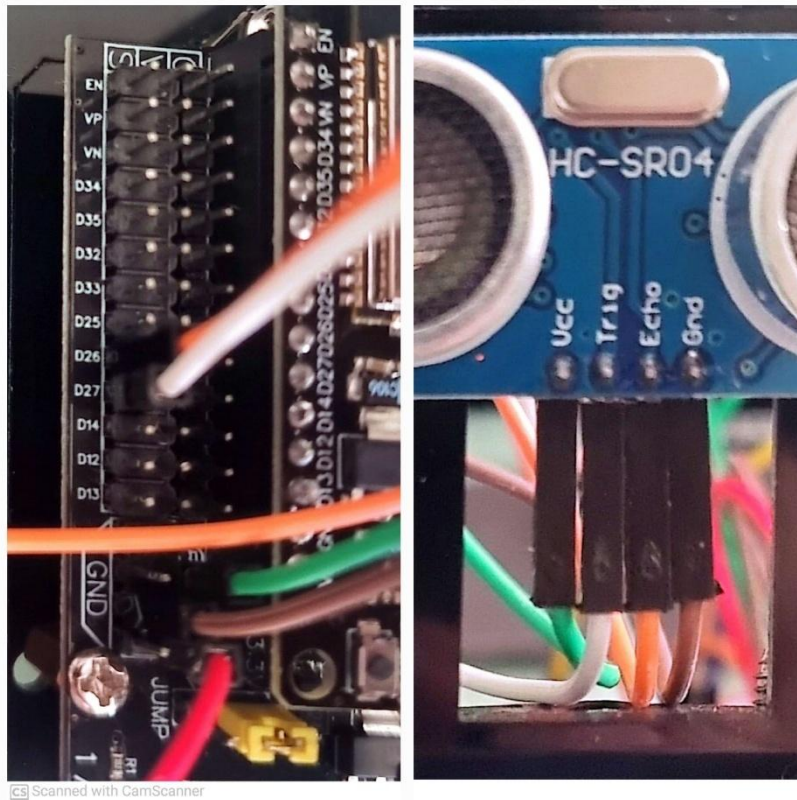
נראה את אופן צורת החיבורים של המסך והחיישן האולטראסוניק אל המיקרו בקר בצורה ברורה ומקורבת יותר:

המסך הקטן מחובר באופן הבא:



- ניתן לראות ש-VCC במסך מחובר ל-VCC שבמיקרו בקר.
- הפין GND שבמסך מחובר ל-GND במיקרו בקר.
- רגל CS שמוצגת במסך מחוברת לפין D5 שבמיקרו בקר.
- רגל ה-RESET שבמסך שלנו מחוברת לפין D4 שבמיקרו בקר.
- רגל A0 יכולה להיות מחוברת לרגל אנלוגית או דיגיטלית תלוי בפקודות שנרצה להשתמש, אני חיברתי רגל זאת לרגל דיגיטלית/סיפרתית D2 שבמיקרו בקר.
- רגל SDA שהיא למעשה הרגל SDI מאחר והמסך מתקשר עם המיקרו בקר דרך תקשורת טורית SPI, ישנו בלבול משום שאנו רגילים שרגל SDA שימושית בתקשורת I2C אך יבואנים סינייים החלו לייצר מסכים ולרשום גם במסכים המתקשרים דרך תקשורת SPI את השם של הרגל SDA והיא מאפיינת את התקשורת שבין המסך למיקרו בקר.
- רגל SCK מחוברת לפין D18 (החוט השני הוא של המנוע סרבו שמחובר לפין D19). למעשה רגל SCK זוהי הרגל שמעבירה את אות השעון.
- רגל ה-LED מחוברת ל-3.3V בצד השני של המיקרו בקר.

נראה את החיבור של החיישן האולטראסוניק:



- רגל ה-TRIG מחוברת לפין D27 שבמיקרו בקר.
- רגל ה-ECHO מחוברת לפין D26 שבמיקרו בקר.
- המתח VCC שבחיישן מתחבר לפין 5V שבמיקרו בקר משום שחיישן זה בדומה למנוע סרוו שלנו עובדים עם 5 וולט והמיקרו בקר ESP32 עובד על 3.3V.
- הארקה GND מחובר כמובן לאדמה GND שבמיקרו בקר שלנו.

קישור לסרטון בו אני מרכיב את המערכת של המיני ראדאר. (יש ללחוץ Ctrl ואז על הקישור):

<https://youtu.be/TOxmNhN4LiU>

➡ לחצו כאן לחזרה לתוכן העניינים

תיעודים-בדיקות רכיבי המערכת הראשונה

תיעוד של אופן פעולת החיישן האולטרסוניק ומדידת מתחים 5.9.25:

בתיעוד זה נוכל לראות את אופן פעולת החיישן האולטרסוניק, מדדתי את המתח שהספק מקבל מהמיקרו בקר ESP32 ובנוסף בדקתי כמה מתח המיקרו בקר מוציא מהפין של ה- 3.3V האם הוא באמת מוציא מתח זה או שלא, בבדיקות קיבלתי שהכל תקין והחיישן קיבל טיפה פחות מתח 5 וולט וזה עלול להיגרם ממספר סיבות הבאות:

מתח ה- USB הסטנדרטי מוגדר להיות 5.0V, אך יש לו **סבילות (Tolerance)**. קריאה של 4.5V נובעת ככל הנראה מהשילוב של הגורמים הבאים:

1. מפל מתח בכבל ה-USB (Voltage Drop):

זהו הגורם העיקרי:

- **התנגדות הכבל:** לכל כבל, גם הקצר והאיכותי ביותר, יש התנגדות חשמלית. כאשר ה- ESP32 והחיישנים מחוברים וצורכים זרם (הם **העומס**) נוצר **מפל מתח** לאורך הכבל, לפי חוק אוהם: $V_{drop} = I \times R$.
- **כבלי USB דקים וארוכים:** כבלים דקים יותר (בעלי מספר AWG גבוה יותר, כמו AWG 28 במקום AWG 20 עבה יותר) או ארוכים, הם בעלי התנגדות גבוהה יותר, מה שמגביר את מפל המתח.
- **הקריאה שלך נעשתה "תחת עומס":** אם מדדת 4.5V בזמן שה- ESP32, חיישן האולטרסוניק ומעגל ה- Wi-Fi שלו פעלו, המתח יורד עקב צריכת הזרם.

2. סבילות תקן ה-USB:

תקן USB 2.0 מאפשר למתח לרדת עד 4.4V בחיבור ל"Hub Port" (שקע פחות חזק) או לכל הפחות 4.75V בחיבור לשקע רגיל, עד לצריכה המקסימלית המותרת. ערך של 4.5V נופל בטווח הסביר, במיוחד אם אתה משתמש בכבל ארוך או ביציאת USB של מחשב נייד או רכזת (Hub).

3. מפל מתח ברכיבי לוח ה-ESP32:

המתח עובר דרך מספר רכיבים על לוח ה- ESP32 שלך:

- **הגנת זרם יתר (Over Current Protection):** לוחות רבים מכילים נתיך הניתן לאיפוס או רכיב הגנה אחר שמוסיף התנגדות קטנה וגורם למפל מתח קל.
- **דיודת הגנה (Protection Diode):** דיודת הגנה בכניסת ה-USB יכולה לגרום למפל מתח של כ- 0.2V עד 0.7V תלוי בסוג.

לסיכום, הקריאה של 4.5V היא תוצאה של הזרם שצורך ה- ESP32 (והחיישנים המחוברים אליו) בשילוב ההתנגדות של כבל ה-USB והרכיבים שעל הלוח. אם הלוח עובד יציב, המתח הזה **תקין לחלוטין** עבורו.

את המדידות שיצאו ניתן לראות בהסבר על החיישן הולטרסוניק בספר זה איפה שאני מסביר על החיישן ואופן פעולתו בסוף ההסברים הכנסתי תחת הכותרת "**מדידות מתח ואופן פעולה**" מצרף כאן סרטון מיוטיוב בוא תיעדתי את אופן הפעולה ואת המדידות וכמובן אכניס את הסרטון לאתר שלי תחת כותרת בדיקות.

[קישור לסרטון: https://youtu.be/qJxPFXFTNug](https://youtu.be/qJxPFXFTNug)

מידות מתח ואופן פעולה:

מידת מתחים של החיישן האולטרסוניק אל המיקרו בקר ESP32, אראה את המדידה אל המתח אליו אני מחבר את החיישן עצמו אל המיקרו בקר. בנוסף לכך אראה את המתח שמורה הפין של ה- 3.3V, האם הוא באמת מוציא מתח זה:

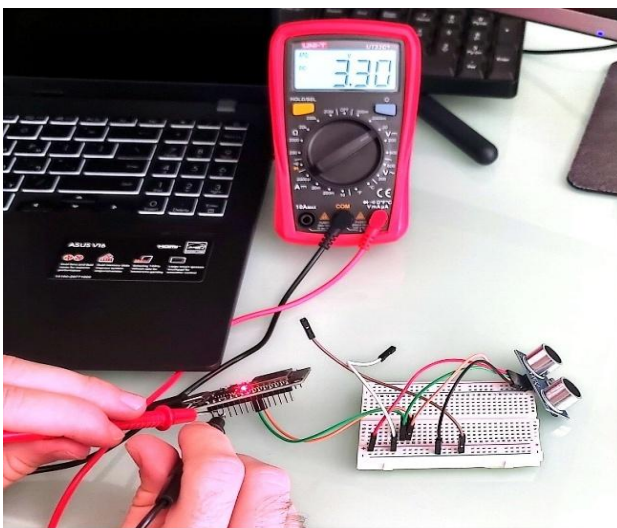
נראה תמונות:

```
1 // תוכנית תוצאות בסיריאל HC-SR04 ל-ESP32
2 // שולח פולס Trig-פין ה
3 #define TRIG_PIN 27
4 // מחזיר את הפולס Echo-פין ה
5 #define ECHO_PIN 26
6 // משתנה לשמירת הזמן שלוקח לפולס לחזור
7 long duration;
8 // מרחק בסנטימטרים
9 int distance;
10
11 void setup() {
12   // פתיחת תקשורת סיריאלית לצפייה בתוצאות
13   Serial.begin(115200);
14
15   // הגדרת פיינים
16   pinMode(TRIG_PIN, OUTPUT);
17   pinMode(ECHO_PIN, INPUT);
18
19   Serial.println("מידת מרחק עם חיישן אולטרסוניק");
20 }
21
22 void loop() {
23   // שליחת פולס קצר ל Trig
24   digitalWrite(TRIG_PIN, LOW);
25   delayMicroseconds(2);
26   digitalWrite(TRIG_PIN, HIGH);
27   delayMicroseconds(10);
28   digitalWrite(TRIG_PIN, LOW);
29
30   // HIGH היה Echo-קריאת משך הזמן שה
31   duration = pulseIn(ECHO_PIN, HIGH);
32
33   // חישוב מרחק ב"מ
34   distance = duration * 0.034 / 2;
35
36   // הדפסת המרחק למסך הסיריאלי
37   Serial.print("מרחק: ");
38   Serial.print(distance);
39   Serial.println(" מ"מ");
40
41   // המתנה קצרה בין מדידות
42   delay(500);
43 }
```



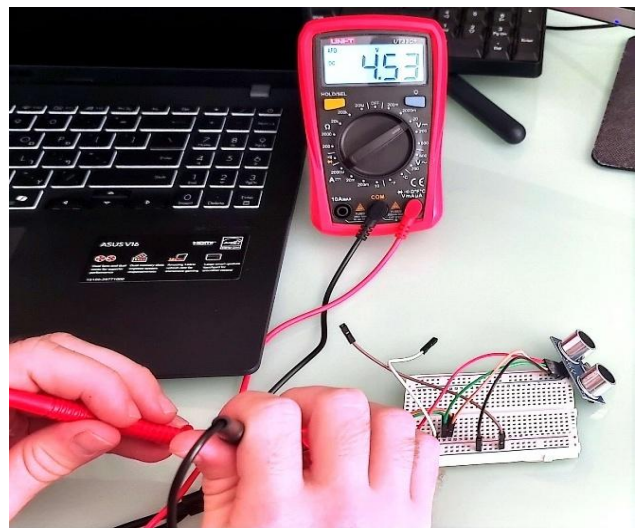
Scanned with CamScanner

המתח שמוציא המיקרו בקר מהפין 3.3V:



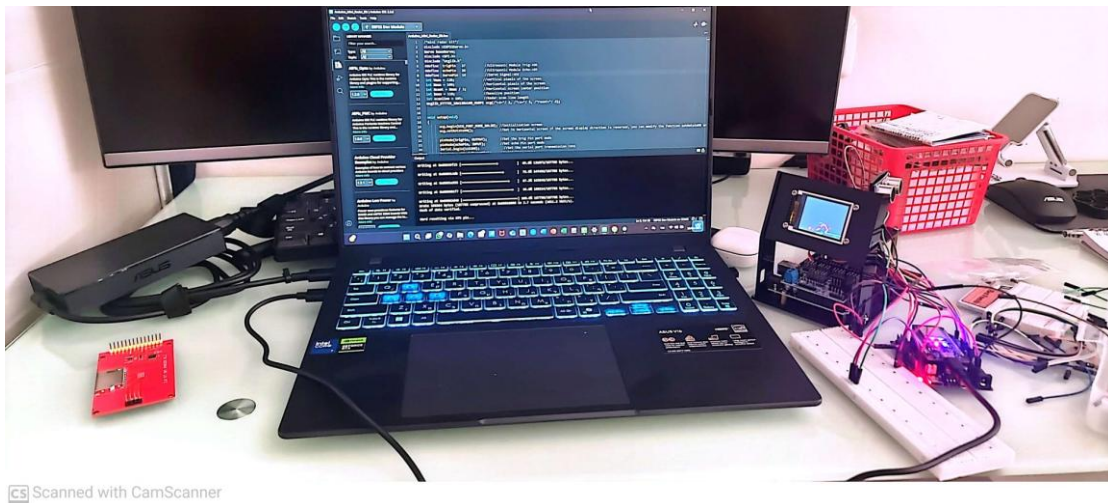
Scanned with CamScanner

המתח שאני מספק לחיישן:



Scanned with CamScanner

➡ לחצו כאן לחזרה לתוכן העניינים

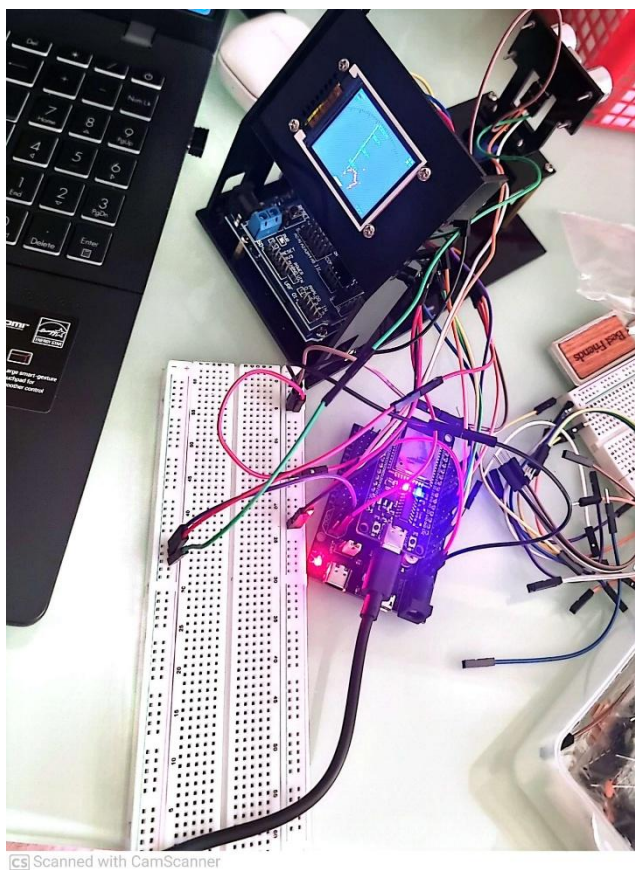


אחת הבעיות הגדולות שיצא לי להתמודד בפרויקט זה היה להפעיל את מערכת הרדאר.

נתקלתי בבעיות כמו למשל ספריות לא תואמות הגדרת פורטים לא נכונים. בעיות אלו פתרתי על ידי חקירה על הפונקציונליות של המיקרו בקר ובנוסף על ביסוס הפורט בעזרת הטרמינל במחשב, אלו היו הפרוצדורות המסובכות אך המועילות ביותר מאחר והן לימדו אותי להתמודד עם בעיות כאלו לפעמים הבאות במידה ואתקל בהן.

נוסף על כך אחת הבעיות שיצא לי להתמודד היה הפינים שהוגדרו להיות "בעייתיים" לשימוש, הכוונה שבמיקרו בקר אי אס פי ישנם פינים בהם כדאי להשתמש משום שנוחים יותר עבור פעולות מסוימות וישנם כאלו שמוגדרים לעבודה ספציפית יותר. לדוגמה יכול לומר שהמנוע שלי לא עבד כאשר חיברתי אותו לפין 15 ואז שיחקתי קצת עם הפינים ומתי שחיברתי SERVO לפין 19 הוא התחיל לעבוד והתסנכרן בעבודה יחד עם המסך והחיישן אולטרה סוני שלמעשה נמצא עליו. בנוסף אחת הבעיות הגדולות ביותר היה להפעיל את המסך. גרפיקה זה צעד גדול בפרויקט ויחסית מסובך, לא כל הפונקציות תואמות למיקרו בקרים ספציפיים וצריך לנסות כמה פונקציות על מנת למצוא את המתאימה ביותר עבור אופן הפעולה שלך.

מאחר ואני יוצר תמונת הרדאר שפועל בזמן אמת אז עלי לא להשתמש רק בתצוגה רגילה אלא ממש בתצוגת עיצוב גרפי שדורש ממני להשתמש בפונקציות ייחודיות עבור פעולות אלו. בסופו של דבר עם קצת כישלונות ולמידה הצלחתי להבין יכן טעיתי בכל הניסיונות הללו ולמדתי מזה המון.



גם כאן בתמונה ניתן לראות את הניסיון הפעלה על מיקרו בקר אי אס פי.

בהתחלה חיברתי הכול דרך מטריצה ולבסוף ארגנתי את זה על המערכת עצמה כדי לשפר את הנראות והנוחות במהלך השימוש בה.

בתמונה קצת קשה לראות את הרדאר שנוצר על המסך וכמובן שהתמונה לא משדרת את התזוזה של המנוע סרוו ואת הקליטה של החיישן אולטרסוני אך כדי להמחיש זאת אני צירפתי לכם סרטון שלי בודק את אופן הפעולה של המערכת.

בסרטון אציג את הבנייה כולה של המערכת ואת אופן פעולתה בסוף של הסרטון כדי שניתן יהיה לראות ולעקוב אחר השלבים צורת הבנייה של המערכת.

בנוסף הסרטון יעלה לאתר שניתן לגשת אליו דרך ספר זה בקישור שאצרף שיהיה תחת הכותרת "קישור לאתר"

תיעוד של אופן פעולת הרדאר 21.9.25:

בניסיון זה בדקתי את סימון הנקודות הצהובות שהן מופיעות עבור אובייקטים שרחוקים מ-100 סנטימטר. הנקודות האדומות שמופיעות על הרדאר אלו נקודות שמאפיינות אובייקטים הנמצאים בתווך של 0 עד 100 סנטימטרים מהחיישן אולטרסוניק.

תיעדתי ניסיון זה והוספתי סרטון שמראה כיצד בדקתי זאת הסרטון טיפה הפוך החלק של הבדיקה לקראת הסוף אבל זה לא לפרק זמן ארוך וניתן להבחין בכל זאת את הנקודות הצהובות. אני מצרף כאן קישור לסרטון שלי שנמצא ביוטיוב וגם הכנסתי את הסרטון תחת הקטגוריה "בדיקות" בתוך האתר שיצרתי שיופיע בעמודים האחרונים של הספר פרויקט שלי.

קישור לסרטון תיעוד של ניסיון יצירת ובדיקה של הנקודות הצהובות שמגדירות אובייקטים שנמצאים במרחק מעל 100 סנטימטרים כולל מהחיישן האולטרסוניק:

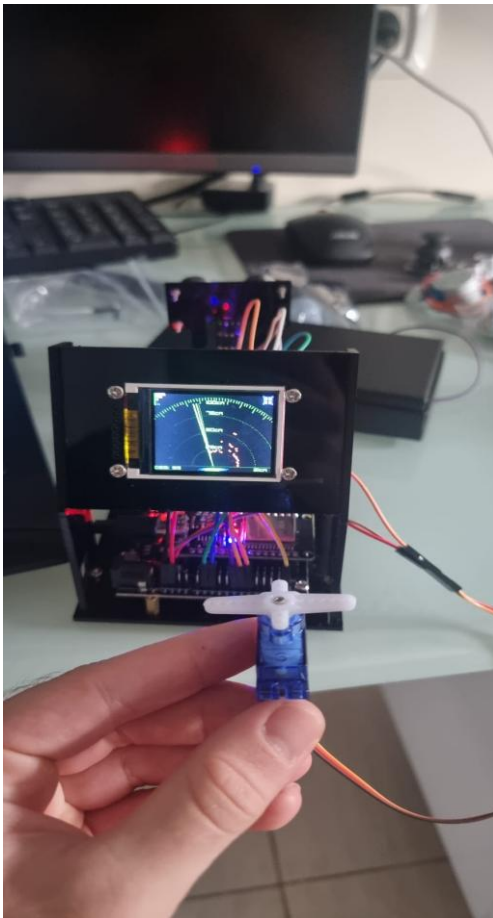
<https://youtu.be/8BiuVFxUo20> קישור לסרטון:

➡ לחצו כאן לחזרה לתוכן העניינים

תיעוד של אופן הפעולה כאשר חיברתי סרוו נוסף שיהיה אחראי על הסיבוב של התותח 23.9.25:

בניסוי זה קפצתי צעד יחסית משמעותי בפרויקט שלי משום שהצלחתי לבצע סנכרון בין שני המנועי SERVO שיש לי במערכת שלי. ה-SERVO הראשון אחראי על לסובב את החיישן האולטרסוני שסורק את הסביבה בטווח של 180 מעלות החל מ0 מעלות ועד ל180 מעלות. המנוע ה-SERVO השני שלי אחראי למעשה על הסיבוב של המשטח עליו אחבר את הרובה האלקטרומגנטי שלי. ברגע שמתגלות נקודות אדמות שבמשמעותן שהאובייקט נמצא בתווך של 100 סנטימטר ואף פחות אז המנוע ה-SERVO השני נכנס ישירות לפעולה מאותו המקום בוא המנוע SERVO הראשון שמסובב את החיישן האולטרסוני נמצא כעת וברגע שיתגלה שוב נקודה אדומה שמסמנת על אובייקט אז תתבצע ירייה מהרובה האלקטרומגנטי אך זה בשלבים מתקדמים יותר כרגע בצעתי את הסנכרון של המנוע ה-SERVO השני עם הראשון יחד.

נראה תמונה ואסביר את החיבורים הנוספים שבצעתי למיקרו בקר על מנת שהמנוע SERVO השני יתחיל להסתובב:



זאת למעשה התמונה בה אני מחזיק את המנוע SERVO השני, למעשה אי אפשר להבין שהמערכת אכן עובדת מתמונה פשוטה ולכן חשבתי אצרף סרטון ביוטיוב וקישור בתוך הספר וכמובן שאכניס את התיעוד לקטגוריית הבדיקות שבאתר שלי.

המנוע SERVO מחובר לרגל D25 שבמיקרו בקר ESP32 ואת שאר החוטים שבמנוע SERVO צירפתי את החוט האדום ל-VCC ואת החוט החום חיברתי לאדמה GND.

בקוד כל מה שעלי היה להוסיף זה את ההגדרה של ה-SERVO הנוסף ולאיוזו רגל הוא מחובר במיקרו בקר, ובנוסף לכך היה עלי להוסיף בתנועה הלוח והתנועה חזור בתוך התנאי איפה שרשום המרחק קטן מ-100 שזה למעשה הנקודות אדומות מה שאומר שהאובייקט נמצא בטווח של 100 סנטימטרים ומטה בשניהם היה עלי להוסיף פקודה שתפעיל את המנוע SERVO השני או במידה והמרחק של האובייקט גדול מ-100 אז התנועה של המנוע SERVO השני פוסקת והמנוע עוצר.

מצרף קישור לסרטון בערוץ היוטיוב שלי:

<https://youtu.be/eeDQquCGN5I>

➡ לחצו כאן לחזרה לתוכן העניינים

קוד סופי למערכת המכ"ם (MASTER):

```

1 // טפריית למערכת:
2 #include <ESP32Servo.h> // ESP32Servo - שליטה במנועי סרור
3 #include <Wire.h> // Wire - תקשורת I2C
4 #include <SPI.h> // SPI - תקשורת למסך
5 #include "Ucglib.h" // Ucglib - טפרייה לציור גרפיקה על מסך
6 #include <WiFi.h> // WiFi - הפעלת מודול WiFi של ESP32
7 #include <esp_now.h> // esp_now - תקשורת אלחוטית מהירה בין ESP32
8
9 // הגדרת פינים לחיבור הרכיבים:
10 #define trigPin 27 // trigPin - שליחת פולס לחיישן האולטרסוני
11 #define echoPin 26 // echoPin - קבלת ההחזר מהחיישן
12 #define ServoPin 14 // ServoPin - שמסובב את הרדאר
13 #define secondServoPin 25 // secondServoPin - (למשל לכיוון נשק / מצלמה)
14 #define RELAY_GUN_PIN 32 // RELAY_GUN_PIN - שליטה במסמר
15
16 // יצירת שני אובייקטים לשליטה במנועי הסרור
17 Servo baseServo; // baseServo - הסרור שמסובב את החיישן
18 Servo secondServo; // secondServo - סרור נוסף שמכוון לאובייקט
19
20 int targetAngle = 90; // targetAngle - הזווית שאליה הסרור צריך להגיע
21 int currentAngle = 90; // currentAngle - הזווית הנוכחית של הסרור
22 int lastDistance = 0; // lastDistance - המרחק האחרון שנמדד
23
24 // משמש כדי לשלוט מידע רק אם יש שינוי
25 bool lastState = false; // lastState - מצב קודם (אם היה אובייקט או לא)
26
27 // הגדרות גודל המסך
28 int Ymax = 128; // Ymax - גובה המסך
29 int Xmax = 160; // Xmax - רוחב המסך
30 int Xcent = Xmax / 2; // Xcent - מרכז המסך
31 int base = 118; // base - מיקום הבסיס של הרדאר
32 int scanline = 105; // scanline - אורך קו הסריקה של הרדאר
33
34 // משמש לשליטה על מהירות התנועה
35 unsigned long lastMoveTime = 0; // שומר את הזמן האחרון שהסרור זז
36
37 // ESP-NOW בעזרת ESP32 משמש לשליחת מידע בין שני
38 uint8_t receiverAddress[] = {0x6C, 0xC8, 0x40, 0x4E, 0x31, 0x78}; // המקבל ESP32 של ה-MAC כתובת
39
40 // מאפשר לציור גרפיקה של רדאר על המסך
41 Ucglib_ST7735_18x128x160_HWSPI ucg(/*cd=*/ 2, /*cs=*/ 5, /*reset=*/ 4); // ST7735 מסוג TFT יצירת אובייקט למסך
42
43 // ESP-NOW מבנה נתונים לשליחה ב
44 typedef struct {
45     uint8_t state;
46 } RadarMessage;
47 // מייצג אם זוהה אובייקט
48 // 0 = אין אובייקט
49 // 1 = יש אובייקט
50
51 void setup(void)
52 {
53     // התחלת תקשורת סריאלית עם המחשב לצורך בדיקות והדפסת מידע
54     Serial.begin(115200);
55     Serial.println("Setup started...");
56     // הגדרת פינים של חיישן המרחק האולטרסוני
57     pinMode(trigPin, OUTPUT); // trigPin מידה
58     pinMode(echoPin, INPUT); // echoPin הגל
59
60     // הגדרת מנוע הסרור הראשי שמסובב את הרדאר
61     // וחיבור לפין המתאים 50Hz תדר סטנדרטי לסרור 50
62     baseServo.setPeriodHertz(50);
63     baseServo.attach(ServoPin, 500, 2400);
64     // הגדרת הסרור השני שמכוון לכיוון האובייקט
65     // מתחיל בזווית 90 מעלות (אמצע)
66     secondServo.setPeriodHertz(50);
67     secondServo.attach(secondServoPin, 500, 2400);
68     secondServo.write(90);
69     // קביעת זווית התחלתית של הסרור
70     currentAngle = 90;
71     targetAngle = 90;
72
73     // והגדרת כיוון התצוגה TFT-הפעלת מסך ה
74     ucg.begin(UCG_FONT_MODE_SOLID);
75     ucg.setRotate(90);

```

שורות 1-7: כאן נטענות הספריית הדרושות להפעלת המערכת. הספרייה **ESP32 Servo** מאפשרת שליטה במנועי סרור, **Wire** משמשת לתקשורת I2C ו-**SPI** משמשת לתקשורת עם מסך ה-TFT. **Ucglib** מאפשרת ציור גרפיקה על המסך. בנוסף נטענות הספרייה **WiFi** ו-**esp_now** שמאפשרות תקשורת אלחוטית בין שני לוחות ESP32.

שורות 9-14: כאן מוגדרים הפינים בלוח ה-ESP32 שאליהם מחוברים הרכיבים. הפינים **trigPin** ו-**echoPin** משמשים לחיישן האולטרסוני למדידת מרחק. הפין **ServoPin** משמש לשליטה בסרור הראשי שמסובב את הרדאר, והפין **secondServoPin** לסרור נוסף. בנוסף מוגדר הפין **RELAY_GUN_PIN** לשליטה במסמר שמפעיל התקן חיצוני.

שורות 16-18: כאן נוצרים שני אובייקטים לשליטה במנועי הסרור. הסרור הראשון **baseServo** משמש לסיבוב החיישן האולטרסוני לצורך סריקת הרדאר. הסרור השני **secondServo** משמש לכיוון נוסף של המערכת לכיוון האובייקט שנמצא.

שורות 20-22: משתנים המשמשים לניהול תנועת הסרור. המשתנה **targetAngle** מייצג את הזווית שאליה הסרור צריך להגיע, בעוד **currentAngle** מייצג את הזווית הנוכחית. המשתנה **lastDistance** שומר את המרחק האחרון כדי לזהות שינוי במדידה.

שורות 24-25: משתנה זה שומר את מצב הזיהוי הקודם של האובייקט. משווה עם המצב הנוכחי כדי לשלוט נתונים רק כאשר יש שינוי בזיהוי האובייקט או במרחק.

שורות 27-32: משתנים אלו מגדירים את ממדי המסך ומיקום האלמנטים הגרפיים של הרדאר **Xmax** ו-**Ymax** הם רוחב וגובה המסך **Xcent**. מייצג את מרכז המסך **base**, הוא מיקום בסיס הרדאר, ו-**scanline** הוא אורך קו הסריקה של הרדאר.

שורות 34-35: משתנה זה שומר את הזמן שבו הסרור זז בפעם האחרונה. באמצעותו ניתן להגביל את קצב תנועת הסרור כדי למנוע תנועה מהירה ולשמור על פעולה יציבה למערכת.

שורות 37-38: משתנה זה מכיל את כתובת ה-MAC של הבקר המקבל. הכתובת משמשת לשליחת נתונים אל הלוח השני באמצעות פרוטוקול **ESP-NOW** שמאפשר תקשורת אלחוטית מהירה

שורות 40-41: בשורה זו נוצר אובייקט שמאפשר שליטה במסך ה-TFT מסוג **ST7735**. האובייקט משמש לציור גרפיקה של מערכת הרדאר על המסך באמצעות תקשורת **SPI**.

שורות 43-49: בחלק זה מוגדר מבנה נתונים בשם **RadarMessage**. המבנה מכיל משתנה אחד בשם **state** מסוג **uint8_t**. משתנה זה מייצג את מצב הזיהוי של האובייקט ברדאר, כאשר הערך 0 מציינ שאין אובייקט והערך 1 מציינ שאובייקט זוהה. מבנה זה משמש לשליחת נתונים בין לוחות ESP32 באמצעות תקשורת **ESP-NOW**.

שורה 51: פונקציית **setup()** היא פונקציה שרצה פעם אחת בלבד כאשר המערכת מופעלת. בתוכה מתבצעות כל פעולות התחול של המערכת כגון הגדרת הפינים, הפעלת הסרורים, אתחול המסך והתחלת תקשורת בין הרכיבים.

שורות 53-58: מפעילות את התקשורת הסריאלית בין לוח ה-ESP32 למחשב. המהירות מוגדרת ל-115200 ביט לשנייה. בנוסף מודפסת הודעה למסך הסריאלי כדי לציין שתהליך האתחול של המערכת התחיל. כאן מוגדרים הפינים של החיישן האולטרסוני. הפין **trigPin** מוגדר כפלט ומשמש לשליחת פולס לחיישן לצורך מדידת מרחק. הפין **echoPin** מוגדר כקלט והוא מקבל את זמן החזרת הגל הקולי מהחיישן.

שורות 60-63: קטע זה מגדיר את מנוע הסרור הראשי שמסובב את החיישן האולטרסוני לצורך סריקת הרדאר. התדר מוגדר ל-50 Hz שהוא התדר הסטנדרטי להפעלת סרור. לאחר מכן הסרור מחובר לפין שהוגדר במערכת ומוגדר טווח אות ה-PWM שלו.

שורות 64-68: בחלק זה מוגדר מנוע הסרור השני במערכת. הסרור פועל גם הוא בתדר של 50 Hz ומחובר לפין שהוגדר קודם. לאחר החיבור הוא מוזז לזווית של 90 מעלות, שהיא בדרך כלל נקודת האמצע של הסרור.

שורות 69-71: שני משתנים אלו מגדירים את הזווית ההתחלתית של הסרור **currentAngle**. מייצג את הזווית הנוכחית של הסרור, ו-**targetAngle** מייצג את הזווית שאליה הסרור צריך להגיע. שניהם מוגדרים ל-90 מעלות כדי להתחיל את הפעולה ממצב מרכזי.

שורות 73-75: מפעילות את מסך ה-TFT ומגדירות את מצב הצגת הפונטים. הפונקציה **ucg.begin()** מאתחלת את המסך ומאפשרת להתחיל לצייר עליו גרפיקה וטקסט. הפקודה **setRotate(90)** מסובבת את התצוגה ב-90 מעלות כדי להתאים את כיוון המסך למיקום הפיזי של המסך במערכת.


```

149
150 // פונקציה לניקוי המסך
151 void cls()
152 {
153     ucg.setColor(0, 0, 0); // הגדרת צבע שחור לצורך מחיקת התצוגה
154
155     // מעבר על כל שטח המסך באמצעות לולאות
156     for(int s=0;s<128;s+=8) // מעבר על הציר האנכי של המסך
157     for(int t=0;t<160;t+=16) // מעבר על הציר האופקי של המסך
158     {
159         ucg.drawBox(t,s,16,8); // ציור מלבן שחור קטן שמוחק את האזור במסך
160     }
161 }
162
163 // פונקציה למדידת מרחק בעזרת החיישן האולטרסוני
164 int calculateDistance()
165 {
166     long duration; // משתנה לשמירת זמן החזרת הגל מהחיישן
167     digitalWrite(trigPin, LOW); // Trigg איפוס פין ה
168     delayMicroseconds(2); // השהייה קצרה
169
170     digitalWrite(trigPin, HIGH); // שליחת פולס לחיישן האולטרסוני
171     delayMicroseconds(10); // פולס של 10 מיקרו-שניות
172     digitalWrite(trigPin, LOW); // סיום הפולס
173
174     duration = pulseIn(echoPin, HIGH, 25000); // מדידת זמן החזרת הגל מהחיישן (Timeout 25 usms)
175     if (duration == 0) return 200; // אם אין החזר מהחיישן מניחים שהאובייקט רחוק
176     return duration * 0.034 / 2; // חישוב המרחק לפי מהירות הקול והחזרת המרחק בסימ
177 }
178
179 void fix_font()
180 {
181     ucg.setColor(0, 180, 0); // הגדרת צבע ירוק לטקסט
182     ucg.setPrintPos(70,128-120+7); // מיקום הטקסט על המסך
183     ucg.print("100cm"); // הדפסת סימון מרחק 100 ס"מ
184
185     ucg.setPrintPos(70,128-85-11); // מיקום סימון נוסף
186     ucg.print("75cm"); // הדפסת סימון מרחק 75 ס"מ
187
188     ucg.setPrintPos(70,128-60-8); // מיקום סימון נוסף
189     ucg.print("50cm"); // הדפסת סימון מרחק 50 ס"מ
190
191     ucg.setPrintPos(70,128-35-4); // מיקום סימון נוסף
192     ucg.print("25cm"); // הדפסת סימון מרחק 25 ס"מ
193 }
194
195 void fix()
196 {
197     ucg.setColor(0, 40, 0); // הגדרת צבע ירוק כהה לרקע הרדאר
198     ucg.drawDisc(Xcent, base+1, 3, UCG_DRAW_ALL); // ציור מרכז הרדאר
199     // ציור מעגלי הרדאר (טווחי מרחק שונים)
200     ucg.drawCircle(Xcent, base+1, 115, UCG_DRAW_UPPER_LEFT); // ציור חצי מעגל שמאלי גדול - מייצג את הטווח הרחוק ביותר של הרדאר
201     ucg.drawCircle(Xcent, base+1, 115, UCG_DRAW_UPPER_RIGHT); // ציור חצי מעגל ימני גדול - משלים את המעגל העליון
202     ucg.drawCircle(Xcent, base+1, 86, UCG_DRAW_UPPER_LEFT); // ציור חצי מעגל שמאלי לטווח בינוני
203     ucg.drawCircle(Xcent, base+1, 86, UCG_DRAW_UPPER_RIGHT); // ציור חצי מעגל ימני לטווח בינוני
204     ucg.drawCircle(Xcent, base+1, 58, UCG_DRAW_UPPER_LEFT); // ציור חצי מעגל שמאלי לטווח קרוב יותר
205     ucg.drawCircle(Xcent, base+1, 58, UCG_DRAW_UPPER_RIGHT); // ציור חצי מעגל ימני לטווח קרוב יותר
206     ucg.drawCircle(Xcent, base+1, 29, UCG_DRAW_UPPER_LEFT); // ציור חצי מעגל שמאלי קטן - טווח קרוב מאוד
207     ucg.drawCircle(Xcent, base+1, 29, UCG_DRAW_UPPER_RIGHT); // ציור חצי מעגל ימני קטן - משלים את המעגל
208     ucg.drawLine(0, base+1, Xmax,base+1); // ציור קו בסיס של הרדאר
209     ucg.setColor(0, 120, 0); // שינוי צבע לקווי הסקאלה
210
211     // ציור קווי סקאלה של זוויות הרדאר
212     for(int i= 40;i < 140; i+=2)
213     {
214         if (i % 10 == 0)
215             ucg.drawLine(105*cos(radians(i))+Xcent,base - 105*sin(radians(i)) , 113*cos(radians(i))+Xcent,base - 113*sin(radians(i))); // קו גדול כל 10 מעלות
216         else
217             ucg.drawLine(110*cos(radians(i))+Xcent,base - 110*sin(radians(i)) , 113*cos(radians(i))+Xcent,base - 113*sin(radians(i))); // קו קטן בין הסימונים
218     }
219
220     // ציור אלמנטים גרפיים לקישוט המסך
221     ucg.setColor(0,200,0);
222     ucg.drawLine(0,0,0,18);
223
224     for(int i= 0; i < 5; i++)
225     {
226         ucg.setColor(random(255),random(255),random(255)); // צבע אקראי
227         ucg.drawBox(2,i*4,random(14)+2,3); // ציור מלבנים קטנים
228     }
229
230     // ציור מסגרת קטנה בפינה של המסך
231     ucg.setColor(0,0,180); // הגדרת צבע כחול למסגרת
232     ucg.drawFrame(146,0,14,14); // ציור מסגרת ריבועית קטנה בפינה הימנית העליונה של המסך

```

שורות 150-161: פונקציה זו משמשת לניקוי המסך לפני ציור חדש של גרפיקת הרדאר. הקוד עובר על שטח המסך באמצעות שתי לולאות ומצייר מלבנים קטנים בצבע שחור שמכסים את כל המסך. כך נמחקת הגרפיקה הקודמת ומוכן שטח נקי לציור הסריקה הבאה.

שורות 163-177: פונקציה זו מודדת את המרחק באמצעות החיישן האולטרסוני. תחילה נשלח פולס קצר דרך פין ה Trigg-שמופעיל את החיישן. לאחר מכן נמדד הזמן שבו האות חוזר דרך פין Echo. הזמן שנמדד מוכפל במהירות הקול כדי לחשב את המרחק מהאובייקט. אם לא מתקבל אות חזרה, הפונקציה מחזירה ערך גדול שמציין שאין אובייקט בטווח המדידה.

שורות 179-193: פונקציה זו מציגה על המסך סימונים של מרחקים בממשק הרדאר. הטקסטים 25cm, 50cm, 75cm, 100cm מוצגים במקומות שונים על המסך כדי לציין את הטווחים של מעגלי הרדאר. סימונים אלו עוזרים למשתמש להבין מה המרחק של האובייקט שמופיע בסריקה.

שורה 195: פונקציה זו מציירת את הרקע הגרפי של מערכת הרדאר על המסך. היא אחראית לציור מרכז הרדאר, מעגלי המרחק, קווי הסקאלה ואלמנטים גרפיים נוספים שמרכיבים את ממשק התצוגה.

שורות 197-198: כאן מוגדר צבע ירוק כהה ולאחר מכן מצויר עיגול קטן במרכז הרדאר. עיגול זה מסמן את נקודת המוצא של הסריקה, כלומר המקום שבו נמצא החיישן האולטרסוני.

שורות 200-207: בחלק זה מצוירים חצאי מעגלים שמייצגים את טווחי המרחק של הרדאר. כל מעגל מסמן מרחק שונה מהחיישן, וכך ניתן להבין על המסך כמה רחוק נמצא האובייקט שנמדד.

שורות 208-209: מציירים קו אופקי בתחתית הרדאר. הקו משמש כקו בסיס שממנו מתחילה הסריקה של הרדאר. בנוסף מתבצע שינוי צבע לקווי הסקאלה

שורות 211-218: לולאה זו מציירת קווי סימון של הזוויות של הממשק הרדאר. כל כמה מעלות מצויר קו קטן, ובכל 10 מעלות מצויר קו ארוך יותר. כך נוצרת סקאלה שמראה את זווית הסריקה של הרדאר.

שורות 220-228: בחלק זה מצוירים אלמנטים גרפיים קטנים בצד המסך שמוסיפים עיצוב לממשק הרדאר. הקוד מצייר מספר מלבנים בצבעים אקראיים כדי ליצור אפקט גרפי דינמי.

שורות 229-231: כאן מצוירת מסגרת קטנה בפינה הימנית העליונה של המסך. מסגרת זו היא אלמנט גרפי שמוסיף עיצוב לממשק הרדאר ויכול לשמש גם כאזור אינדיקציה.

```

232
233   ucg.setColor(0,0,60);           // צבע כחול כהה לקווי המסגרת הפנימיים
234   ucg.drawLine(148,0,10);         // ציור קו אופקי עליון
235   ucg.drawLine(146,2,10);         // ציור קו אנכי שמאלי
236   ucg.drawLine(148,13,10);        // ציור קו אופקי תחתון
237   ucg.drawLine(159,2,10);         // ציור קו אנכי ימני
238
239   // ציור אינדיקטורים צבעוניים
240   ucg.setColor(random(255),random(255),random(255)); // צבע אקראי
241   ucg.drawRect(148,2,4,4);        // ציור ריבוע קטן (אינדיקטור)
242   ucg.setColor(0,220,0);          // צבע ירוק
243   ucg.drawRect(148,8,4,4);        // ציור ריבוע נוסף
244   ucg.setColor(random(255),random(255),random(255)); // צבע אקראי
245   ucg.drawRect(154,8,4,4);        // ציור ריבוע נוסף
246   ucg.setColor(random(255),random(255),random(255)); // צבע אקראי
247   ucg.drawRect(154,2,4,4);        // ציור ריבוע נוסף
248
249   // ציור אלמנטים גרפיים בתחתית המסך
250   ucg.setColor(0,0,90);           // צבע כחול כהה
251   ucg.drawTetragon(62,123,58,127,98,127,102,123); // ציור צורה גיאומטרית בתחתית המסך
252   ucg.setColor(0,0,160);          // צבע כחול בהיר יותר
253   ucg.drawTetragon(67,123,63,127,93,127,97,123); // ציור שכבה נוספת של הצורה
254   ucg.setColor(0,255,0);          // צבע ירוק
255   ucg.drawTetragon(72,123,68,127,88,127,92,123); // ציור צורה נוספת להשלמת העיצוב
256 }
257
258 // == כאן ניצור את הפונקציה החדשה ==
259 bool redPointDetected(int distance) {
260     return distance < 100; // כולר true בודק אם האובייקט קרוב מ-100 ס"מ, אם כן מחזיר
261 }
262 void sendRadarStatus(int distance, bool detected) {
263     // רק אם יש שינוי במצב או במרחק ESP-NOW שולח נתונים ב
264     if (detected != lastState || distance != lastDistance) {
265         RadarMessage msg; // יצירת מבנה הודעה לשליחה
266         msg.state = distance; // הכנסת המרחק להודעה (ניתן גם לשלוח רק מצב 0 או 1)
267         esp_now_send(receiverAddress, (uint8_t*)&msg, sizeof(msg)); // שליחת הנתונים ל-ESP32 באמצעות
268
269         // הדפסת מידע למסך הסריאלי לצורך בדיקה וניטור
270         Serial.print(detected ? "Object detected: " : "No object: ");
271         Serial.println(distance);
272
273         // עדכון המצב האחרון והמרחק האחרון שנשלחו
274         lastState = detected;
275         lastDistance = distance;
276     }
277 }
278
279 פונקציית loop() לולאה הראשית של התוכנית. היא רצה שוב ושוב כל עוד ה-ESP32 פועל,
280 ומבצעת את פעולות הסריקה של הרדאר. מדידת המרחק מהחיישן והצגת הנתונים על המסך.
281 void loop(void) {
282     // משתנה לשמירת המרחק שנמדד מהחיישן
283     int distance;
284     // ציור הרקע של הרדאר (מעגלים, קווים וכו')
285     fix();
286     // ציור סימוני המרחק על המסך
287     fix_font();
288
289     // משתנה שמציין אם זוהה אובייקט
290     bool objectDetected = false;
291
292     // הסרוו מסתובב מ-180 מעלות לכיוון 0 (סריקה ימינה לשמאל)
293     for (int x=180; x > 4; x-=2)
294     {
295         // הזזת הסרוו לזווית הנוכחית
296         baseServo.write(x);
297         // השהייה קצרה כדי לתת לסרוו זמן לזוז
298         delay(5);
299
300         // ציור קווי הסריקה של הרדאר
301         // זווית מעט לפני הקו הנוכחי ליצירת אפקט תנועה
302         int f = x - 4;
303         // קו סריקה ראשי בצבע ירוק
304         ucg.setColor(0, 255, 0);
305         // ומצויר בזווית (Xcent, base) ציור קו הסריקה של הרדאר: הקו מתחיל במרכז הרדאר
306         // כדי לחשב את המיקום לפי הזווית והאורך של קו הסריקה sin ו-cos בעזרת פונקציות
307         ucg.drawLine(Xcent, base, scanline*cos(radians(f))+Xcent,base - scanline*sin(radians(f)));
308
309         // שינוי הזווית מעט קדימה
310         f+=2;
311         // קו ירוק כהה מאחוריו (אפקט גרפי)
312         ucg.setColor(0, 128, 0);
313         // ומצויר בזווית (Xcent, base) ציור קו הסריקה של הרדאר: הקו מתחיל במרכז הרדאר
314         // כדי לחשב את המיקום לפי הזווית והאורך של קו הסריקה sin ו-cos בעזרת פונקציות
315         ucg.drawLine(Xcent, base, scanline*cos(radians(f))+Xcent,base - scanline*sin(radians(f)));
316
317         // התקדמות נוספת בזווית
318         f+=2;
319         // מחיקת הקו הישן כדי ליצור אנימציה
320         ucg.setColor(0, 0, 0);
321         // ומצויר בזווית (Xcent, base) ציור קו הסריקה של הרדאר: הקו מתחיל במרכז הרדאר
322         // כדי לחשב את המיקום לפי הזווית והאורך של קו הסריקה sin ו-cos בעזרת פונקציות
323         ucg.drawLine(Xcent, base, scanline*cos(radians(f))+Xcent,base - scanline*sin(radians(f)));
324
325         // שינוי צבע לפני ציור הנתונים
326         ucg.setColor(0,200, 0);

```

שורות 237-233: קטע קוד זה מצייר קווי מסגרת קטנים בתוך הריבוע שבפינת המסך. הקווים האופקיים והאנכיים יוצרים מסגרת פנימית שמוסיפה עיצוב לממשק הרדאר.

שורות 247-239: בחלק זה מצוירים ארבעה ריבועים קטנים בתוך המסגרת. הריבועים מוצגים בצבעים שונים (חלקם אקראיים) ומשמשים כאינדיקטורים גרפיים שמוסיפים אפקט עיצובי לממשק הרדאר.

שורות 255-249: קטע זה מצייר צורות גיאומטריות בתחתית המסך. הצורות מצוירות בצבעים שונים ויוצרות אפקט גרפי שמדמה את לוח הבקרה של הרדאר.

שורות 261-258: פונקציה זו בודקת אם אובייקט נמצא בטווח של פחות מ-100 ס"מ מהחיישן. אם המרחק קטן מ-100 הפונקציה מחזירה true, כלומר זוהה אובייקט. אם המרחק גדול יותר הפונקציה מחזירה false.

שורה 262: פונקציה זו אחראית לשליחת נתוני הרדאר ללוח ESP32 אחר באמצעות תקשורת ESP-NOW. הפונקציה שולחת את המרחק שנמדד ואת מצב הזיהוי של האובייקט.

שורה 262: הקוד בודק האם מצב הזיהוי או המרחק השתנו מאז המדידה הקודמת. רק אם יש שינוי, המערכת תשלח נתונים חדשים, וכך נמנעת שליחה מיותרת של מידע.

שורות 275-265: כאן נוצרת הודעה חדשה מסוג RadarMessage שמכילה את המרחק שנמדד. הנתונים נשלחים ללוח ESP32 אחר באמצעות פרוטוקול ESP-NOW. בנוסף מודפס המידע למסך הסריאלי לצורך בדיקה וניטור. אם זוהה אובייקט תודפס ההודעה "Object detected", ואם לא – תודפס ההודעה "No object", יחד עם המרחק שנמדד. בסוף הפונקציה נשמרים הנתונים האחרונים שנמדדו. כך ניתן להשוות אותם למדידות הבאות ולשלוח נתונים רק כאשר מתרחש שינוי.

שורות 282-280: distance משתנה ששומר את המרחק שנמדד מהחיישן ומשמש להצגת מיקום האובייקט על מסך הרדאר. fix(), fix_font() הפונקציות מציירות את הרדאר על המסך, כולל המעגלים וסימוני המרחקים.

שורה 284: משתנה זה משמש כדי לציין האם זוהה אובייקט במהלך הסריקה. בתחילת כל מחזור סריקה הוא מוגדר ל-true, ומתעדכן ל-false אם החיישן מזהה אובייקט בטווח שנקבע.

שורות 311-286: קטע קוד זה אחראי על סריקת הרדאר והצגת קו הסריקה על המסך. תחילה מתבצעת לולאה שמסובבת את מנוע הסרוו מזווית 180 מעלות לכיוון 0 מעלות בקפיצות של 2 מעלות, וכך נוצרת סריקה הדרגתית של האזור. בכל שלב הסרוו מופנה לזווית החדשה, ולאחר מכן מתבצעת השהייה קצרה כדי לאפשר לו להגיע למיקום הרצוי. לאחר קביעת הזווית מחושב ערך עזר המשמש לציור קו הסריקה על המסך. בעזרת פונקציות טריגונומטריות (sin ו-cos) מצויר קו הסריקה שמתחיל במרכז הרדאר ומצביע לכיוון הזווית הנוכחית של הסריקה. כדי ליצור אפקט גרפי של תנועה, מצוירים מספר קווים בגווי ירוק שונים שמדמים את תנועת קו הרדאר, ולאחר מכן קו שחור שמוחק את הקודם וכך נוצרת אנימציה של סריקה רציפה. לבסוף מוגדר שוב צבע ירוק כדי להמשיך בציור הנתונים הבאים על המסך.

```

312
313 distance = calculateDistance(); // מדידת המרחק מהחיישן האולטרסוני
314 objectDetected = distance < 100; // אם יש אובייקט בטווח של מ-100 ס"מ נחשב שיש אובייקט
315 // עדכון והדפסה רק אם יש שינוי
316 if (objectDetected != lastState || distance != lastDistance) {
317     sendRadarStatus(distance, objectDetected); // השני ESP32-שליות המידע ב
318     lastState = objectDetected; // שמירת המצב האחרון של זיהוי האובייקט
319     lastDistance = distance; // עדכון מרחק אחרון
320 }
321
322 // ציור נקודה על המסך לפי המרחק שנמדד
323 if (distance < 100)
324 {
325     ucg.setColor(255,0,0); // נקודה אדומה כאשר האובייקט קרוב
326     ucg.drawDisc(1.15*distance*cos(radians(x))+Xcent,-(1.15*distance*sin(radians(x)))+base, 1, UCG_DRAW_ALL);
327
328     // כאן מוסיפים את הסרוו השני
329     targetAngle = x; // כיוון הסרוו השני לאותו כיוון של האובייקט
330     objectDetected = true; // חשוב! מעדכן שהתגלה אובייקט
331 }
332
333 else
334 { // אם המרחק גדול מ-1 מטר מציירים נקודה צהובה בקצה הרדאר
335     ucg.setColor(255,255,0); // צבע צהוב מסמן שאין אובייקט קרוב
336     ucg.drawDisc(116*cos(radians(x))+Xcent,-116*sin(radians(x))+base, 1, UCG_DRAW_ALL);
337 }
338
339 if (x > 70 and x < 110) fix_font(); // לא יימחקו
340 ucg.setColor(0,0,155, 0); // הגדרת צבע הטקסט שמוצג בתחתית המסך
341
342 ucg.setPrintPos(0,126); // מיקום הטקסט בצד שמאל בתחתית המסך
343 ucg.print("DEG: "); // הדפסת הכיתוב שמציג את הזווית של הסרוו
344
345 ucg.setPrintPos(24,126); // מיקום הזווית
346 ucg.print(x); // הדפסת הזווית הנוכחית של הסרוו
347 ucg.print(" "); // רווחים כדי למחוק תווים ישנים
348
349 ucg.setPrintPos(125,126); // מיקום המרחק בצד ימין
350 ucg.print(" "); // ניקוי תווים ישנים
351 ucg.print(distance); // הדפסת המרחק שנמדד
352 ucg.print("cm "); // הוספת יחידת מידה סנטימטר
353
354 // == שלב 5: הזזת הסרוו השני במקום אחד בלבד ==
355 if (millis() - lastMoveTime > 20) { // בדקה שעברו לפחות 20 מילישניות מאז התנועה האחרונה
356     אם יש הבדל בין הזווית הרצויה לנוכחית { //
357         currentAngle = targetAngle; // עדכון הזווית הנוכחית
358         secondServo.write(currentAngle); // הזזת הסרוו השני לכיוון האובייקט
359     }
360     lastMoveTime = millis(); // שמירת זמן התנועה האחרונה
361 }
362 }
363 delay(50); // השהייה קצרה בין סריקות
364
365 cls(); // ניקוי המסך
366 fix(); // ציור מחדש של רקע הרדאר
367 fix_font(); // ציור מחדש של סימוני המרחק
368
369 for (int x=1; x < 176; x+=2) { // הסרוו מסתובב חזרה מ-0 מעלות לכיוון 180 (סריקה משמאל לימין)
370     הזזת הסרוו לזווית הנוכחית //
371     delay(5); // נותן לסרוו זמן פיזי לזוז
372
373     // ציור קווי הסריקה של הרדאר
374     int f = x + 4; // זווית מעט קדימה ליצירת אפקט תנועה של הסריקה
375     ucg.setColor(0, 255, 0); // קו סריקה ראשי בצבע ירוק
376     ucg.drawLine(Xcent, base, scanline*cos(radians(f))+Xcent,base - scanline*sin(radians(f)));
377
378     f-=2; // הזזה קטנה של הזווית
379     ucg.setColor(0, 128, 0); // קו ירוק כהה מאחור (זנב של קו הסריקה)
380     ucg.drawLine(Xcent, base, scanline*cos(radians(f))+Xcent,base - scanline*sin(radians(f)));
381
382     f-=2; // הזזה נוספת
383     ucg.setColor(0, 0, 0); // מחיקת הקו הישן כדי ליצור אנימציה
384     ucg.drawLine(Xcent, base, scanline*cos(radians(f))+Xcent,base - scanline*sin(radians(f)));
385     ucg.setColor(0, 200, 0); // שינוי צבע לפני ציור הנתונים
386
387     distance = calculateDistance(); // מדידת המרחק מהחיישן האולטרסוני
388     objectDetected = distance < 100; // בדיקה אם יש אובייקט בטווח של פחות מ-100 ס"מ
389
390     // עדכון והדפסה רק אם יש שינוי
391     if (objectDetected != lastState || distance != lastDistance) {
392         sendRadarStatus(distance, objectDetected); // השני וגם הדפסה לסריאל ESP32-ל ESP-NOW-שליות המידע ב
393         lastState = objectDetected; // שמירת המצב האחרון של זיהוי האובייקט
394         lastDistance = distance; // שמירת המרחק האחרון שנמדד
395     }
396 }

```

שורות 313-314: הפונקציה calculateDistance() מודדת את המרחק מהחיישן האולטרסוני ושומרת אותו במשתנה distance. לאחר מכן מתבצעת בדיקה האם המרחק קטן מ-100 ס"מ. אם כן, המשתנה objectDetected מקבל ערך true ומציין שזוהה אובייקט.

שורות 316-320: קטע זה בודק האם מצב הזיהוי או המרחק השתנו מהמידה הקודמת. אם יש שינוי, הפונקציה sendRadarStatus() שולחת את הנתונים ללוח ESP32 שני באמצעות תקשורת ESP-NOW לאחר מכן נשמרים הערכים החדשים להשוואה בסריקה הבאה.

שורות 322-326: כאשר המרחק קטן מ-100 ס"מ, המערכת מציירת נקודה אדומה על מסך הרדאר. מיקום מחושב לפי המרחק שנמדד והזווית של הסרוו וכך מתקבל מיקום האובייקט על המסך.

שורות 328-330: כאן מתעדכנת הזווית שאלה צריך להסתובב הסרוו השני. הזווית נקבעת לפי כיוון הסריקה שבו נמצא האובייקט, כך שהסרוו יכול להצביע לכיוון המטרה.

שורות 333-337: אם המרחק גדול מ-100 ס"מ, מצוירת נקודה צהובה בקצה אזור הסריקה. נקודה זו מציינת שאין אובייקט קרוב בטווח המדידה.

שורה 339: כאשר קו הסריקה עובר באזור שבו מופיעים מספרי המרחק על המסך, הפונקציה fix_font() מציירת אותם מחדש כדי שלא יימחקו במהלך הסריקה.

שורות 342-343: הקוד קובע את מיקום הטקסט בצד השמאלי התחתון של המסך ולאחר מכן מציג את הכיתוב DEG שמציין שהנתון שיוצג אחריו הוא זווית הסרוו.

שורות 345-347: כאן מוצגת הזווית הנוכחית של מנוע הסרוו (x). הזווית משתנה במהלך הסריקה וכך המשתמש יכול לראות לאיזה כיוון הרדאר סורק בכל רגע.

שורות 349-352: אלו מציגות בצד הימני התחתון של המסך את המרחק שנמדד מהאובייקט. הערך מוצג בסנטימטרים ומעודכן בזמן אמת במהלך הסריקה.

שורות 354-362: קטע קוד זה אחראי על הזזת הסרוו השני לכיוון האובייקט. הבדיקה עם millis() מגדירה לך שהסרוו יזוז רק פעם ב-20 מילישניות כדי למנוע תנועה מהירה מדי. אם יש הבדל בין הזווית הנוכחית לזווית היעד, הסרוו מסתובב לזווית החדשה.

שורות 363-367: הקוד יוצר השהייה קצרה בין סריקות כדי לייצב את האנימציה. לאחר מכן מתבצע ניקוי של המסך (cls) וציור מחדש של הרקע והסקאלה של הרדאר באמצעות הפונקציות fix() וfix_font().

שורות 369-371: לולאה זו מסובבת את הסרוו מהכיוון ההפוך (מיס לכיוון 180 מעלות). כך הרדאר מבצע סריקה הלוד וחוזר של כל האזור.

שורות 373-385: קטע קוד זה אחראי על ציור קו הסריקה של הרדאר על המסך. תחילה מחושב משתנה עזר המשמש לקביעת זווית קו הסריקה. לאחר מכן מצוירים מספר קווים באמצעות פונקציות טריגונומטריות (sin ו cos) כדי לקבוע את מיקום הקו לפי הזווית. הקווים מצוירים בגווני ירוק שונים כדי ליצור אפקט של תנועת רדאר, ולאחר מכן קו שחור מוחק את הקו הקודם וכך נוצרת אנימציה של סריקה רציפה. בסיום מוגדר שוב צבע ירוק להמשך ציור הנתונים על המסך.

שורות 387-388: נמדד המרחק מהחיישן האולטרסוני באמצעות הפונקציה calculateDistance(). הערך נשמר במשתנה distance. לאחר מכן מתבצעת בדיקה האם המרחק קטן מ-100 ס"מ. אם כן, המשתנה objectDetected מקבל ערך אמת והמערכת מזהה שיש אובייקט בטווח הסריקה.

שורות 390-394: קטע קוד זה בודק האם מצב זיהוי האובייקט או המרחק שנמדד השתנו מהמידה הקודמת. אם יש שינוי, הפונקציה sendRadarStatus() שולחת את הנתונים ללוח ESP32 נוסף באמצעות תקשורת ESP-NOW, ולאחר מכן נשמרים הערכים החדשים לצורך השוואה בסריקה הבאה.

שורות 398-

402: קטע

קוד זה בודק
האם המרחק
שנמדד קטן
מ-100 ס"מ.

אם כן, המערכת מזהה אובייקט קרוב ומציירת נקודה אדומה על מסך הרדאר במיקום המתאים לפי המרחק והזווית.

שורות 403-404: קביעת הזווית שאליה הסרוו השני מסתובב כדי להצביע לכיוון האובייקט שנמצא.

שורות 407-411: אם המרחק גדול מ-100 ס"מ, מצוירת נקודה צהובה בקצה הרדאר כדי לציין שאין אובייקט בטווח הקרוב.

שורה 412: כאשר קו הסריקה עובר באזור שבו נמצאים המספרים על המסך, הפונקציה fix_font() מציירת אותם מחדש כדי שלא יימחקו בזמן הסריקה.

שורות 413-420: הצגת הזווית הנוכחית של מנוע הסרוו בתחתית המסך. תחילה נקבע מיקום ההדפסה בצד השמאלי של המסך באמצעות הפונקציה setPrintPos, ולאחר מכן מודפס הכיתוב "DEG" שמציין שהערך שיופיע הוא זווית. לאחר מכן נקבע מיקום נוסף על המסך ומודפסת הזווית הנוכחית של הסרוו באמצעות המשתנה x, אשר משתנה במהלך סריקת הרדאר. לבסוף מודפס רווח קטן כדי לנקות תווים ישנים מהתצוגה כאשר הערך משתנה.

שורות 422-425: הצגת המרחק שנמדד מהאובייקט בתחתית המסך. תחילה נקבע מיקום ההדפסה בצד הימני של המסך באמצעות הפונקציה setPrintPos. לאחר מכן מודפס המרחק שנמדד באמצעות המשתנה distance, ולבסוף מודפסת יחידת המידה cm כדי לציין שהמרחק מוצג בסנטימטרים. הנתון מתעדכן בזמן אמת במהלך סריקת הרדאר.

שורות 428-435: קטע קוד זה אחראי על הזזת הסרוו השני לכיוון האובייקט שנמצא. תחילה מתבצעת בדיקה שעברו לפחות 20 מילישניות מאז התנועה האחרונה של הסרוו, כדי למנוע תנועה מהירה מדי.

לאחר מכן נבדק האם קיים הבדל בין הזווית הנוכחית (currentAngle) לבין הזווית הרצויה (targetAngle). אם כן, הסרוו מסתובב לזווית החדשה וכך מכוון לכיוון האובייקט שנמצא במהלך הסריקה. בסיום מתעדכן זמן התנועה האחרון של הסרוו.

```
397 // ציור נקודה על המסך לפי המרחק שנמדד
398 if (distance < 100)
399 {
400   ucg.setColor(255,0,0); // צבע אדום כאשר זוהה אובייקט בטווח
401   ucg.drawDisc(1.15*distance*cos(radians(x))+Xcent,-(1.15*distance*sin(radians(x))+base, 1, UCG_DRAW_ALL); // ציור נקודה אדומה במיקום האובייקט על הרדאר
402   // כאן מוסיפים את הסרוו השני
403   targetAngle = x; // קובע את הזווית שאליה הסרוו השני צריך להסתובב
404   objectDetected = true; // מעדכן שהתגלה אובייקט
405 }
406
407 else
408 { // אם המרחק גדול מ-100 ס"מ מציירים נקודה צהובה בקצה הרדאר
409   ucg.setColor(255,255,0); // צבע צהוב מצייין שאין אובייקט קרוב
410   ucg.drawDisc(116*cos(radians(x))+Xcent,-116*sin(radians(x))+base, 1, UCG_DRAW_ALL); // ציור נקודה בקצה תחום הרדאר
411 }
412 if (x > 70 and x < 110) fix_font(); // אם קו הסריקה עובר באזור המספרים מציירים אותם מחדש כדי שלא יימחקו
413 ucg.setColor(0,0,155, 0); // צבע הטקסט בתחתית המסך
414
415 ucg.setPrintPos(0,126); // מיקום הטקסט בצד שמאל למטה
416 ucg.print("DEG: "); // הדפסת הכיתוב שמציג את הזווית
417
418 ucg.setPrintPos(24,126); // מיקום הזווית
419 ucg.print(x); // הדפסת הזווית הנוכחית של הסרוו
420 ucg.print(" "); // רווחים לניקוי תווים קודמים
421
422 ucg.setPrintPos(125,126); // מיקום המרחק בצד ימין
423 ucg.print(" "); // ניקוי תווים ישנים
424 ucg.print(distance); // הדפסת המרחק שנמדד מהתיישן
425 ucg.print("cm "); // הצגת יחידת המידה בסנטימטרים
426
427 // שלב 5: הזזת הסרוו השני במיקום אחד בלבד
428 if (millis() - lastMoveTime > 20) { // בדיקה שעברו לפחות 20 מילישניות מאז התנועה האחרונה של הסרוו
429   // בודק אם יש הבדל בין הזווית הנוכחית לזווית היעד
430   if (abs(targetAngle - currentAngle) > 1) {
431     currentAngle = targetAngle; // עדכון הזווית הנוכחית לזווית שאליה רוצים להגיע
432     secondServo.write(currentAngle); // הזזת הסרוו השני לכיוון האובייקט שזוהה
433   }
434   lastMoveTime = millis(); // שמירת הזמן שבו הסרוו זז לאחרונה
435 }
436
437 delay(50); // השהייה קצרה בין סריקות הרדאר
438 cls(); // ניקוי המסך לפני ציור הסריקה הבאה
439 }
```

דברים שהוספתי על מנת לתקשר בין המכש מהמערכת רדאר הראשונה לבין מערכת המכש שנוצרת בעזרת תוכנת PROCESSING שיצרתי שמסונכרן עם הרדאר המובנה:

```
316 Serial.print(x);
317 Serial.print(",");
318 Serial.print(distance);
319 Serial.println(".");

394 // Processing-שליחת נתונים ל
395 Serial.print(x);
396 Serial.print(",");
397 Serial.print(distance);
398 Serial.println(".");
```

למעשה על מנת שהרדאר יתקשר עם המכש הראשון עלי להריץ את הקוד למערכת הרדאר דרך פורט מסוים במחשב ואז בתוכנת PROCESSING לציין את הפורט שחיברתי וכך בעצם המערכות נמצעות על ספורט משותף ומתקשרות בתקשורת טורית אסור להריץ את הקוד של הרדאר בזמן שה-PROCESSING מחובר משום שהפורט משמש להרצה של מערכת אחת בלבד לכן קוד אריץ את הקוד של הרדאר ואז אריץ את הקוד עבור הרדאר במחשב.

מה שבעצם עובר זה המרחק שהחיישן אולטרסוני קלט וגם הזווית שמאפיינת את הזווית שהסרוו נמצא בה.

השורה שמאפשר תקשורת בין מערכת הרדאר לרדאר שנוצר במחשב שיהיו מסונכרנים כמערכת אחת משולבת:

```
port = new Serial(this,"COM9",115200); // Open serial communication with device
```

ייתכן כי יבוצעו שינויים קטנים בקוד לקראת מועד הבחינה, כגון התאמות בערכי החיישנים או בטווחי הסיבוב של המנועים.

➡ לחצו כאן לחזרה לתוכן העניינים

תיעודים עבור רכיבים למערכת השנייה

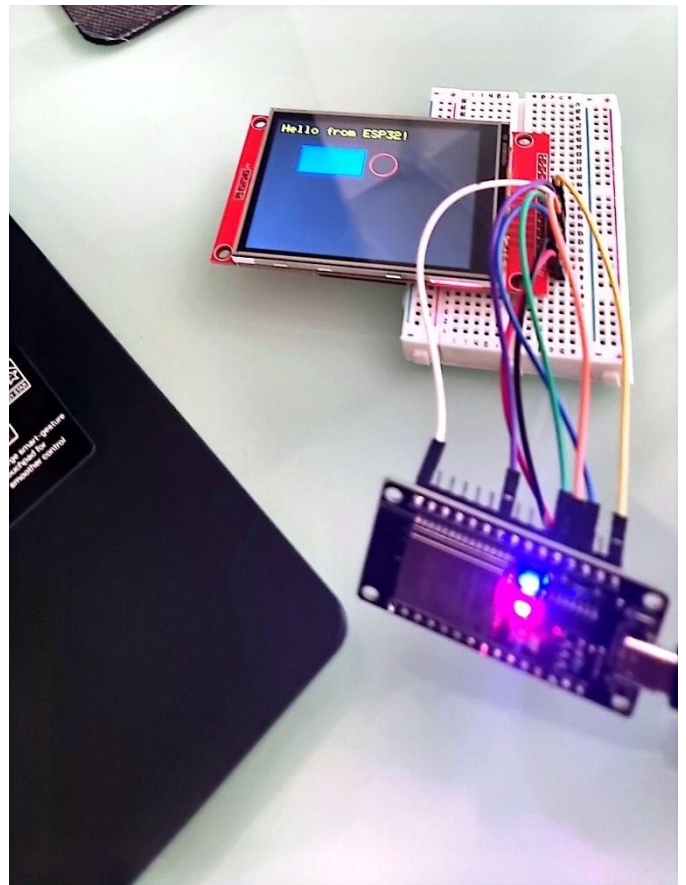
תיעוד של אופן פעולת המסך טאץ' המסך השני בו אני משתמש בפרויקט 27.9.25

בתיעוד זה אציג את אופן פעולת המסך השני זהו מסך טאץ' בו אשתמש כדי להראות את מצב התאורה והטמפרטורה ואוכל לעבור בין מצבים שונים שאחליט לצרף על המסך. הקשיים איתם התמודדתי על מנת לצרף את המסך השני בעיקר יחד עם פעולת המסך הראשון היו בעיקר מאופיינים בהגדרת ספריות אחרות בהן המסך שלי לא תמך, לאחר זמן ממושך וחקירה לגבי הנושא מצאתי את הספריות המתאימות שעליהן הרחבתי והסברתי בספר זה כאשר הסברתי על המסך טאץ'.

מצרף קישור לסרטון בו אני מבצע מדידת מתחים יחד עם אופן פעולת המסך השני. הקוד שצירפתי זהו קוד בסיסי שמדפיס את המשפט Hello from ESP32 ומתחתיו יש מלבן שכולו כחול ועיגול שהיקפו אדום והפנים הוא בצבע של המסך (בצבע שחור). זהו אותו הקוד שהצגתי בהסבר על המסך טאץ' (המסך השני בפרויקט שלי), לכן ניתן לראות אותו בצורה מלאה וברורה יותר בספר זה תחת הכותרת וההסברים של "מסך טאץ' TFT 9341".

נראה כעת את אופן פעולת המסך ומדידת המתחים בפינים אליהם אני מחבר את המסך:

```
1 #define TFT_CS 15 // Chip Select (CS)
2 #define TFT_DC 2 // Data/Command (DC)
3 #define TFT_RST 4 // Reset (RST)
4 // הכללת הספריות
5 #include <Adafruit_GFX.h>
6 #include <Adafruit_ILI9341.h>
7 // יצירת אובייקט המסך
8 // CS, DC, RST) אנו מעבירים לו את הגדרות הפינים הנוספים
9 Adafruit_ILI9341 tft = Adafruit_ILI9341(TFT_CS, TFT_DC, TFT_RST);
10
11 void setup() {
12   Serial.begin(115200);
13   // אתחול המסך
14   tft.begin();
15
16   // הגדרת כיוון המסך (0, 1, 2 או 3)
17   tft.setRotation(1);
18
19   // ניקוי המסך ומילוי בצבע שחור
20   tft.fillRect(ILI9341_BLACK);
21
22   // הגדרת גודל וצבע הטקסט
23   tft.setTextSize(2); // גודל טקסט: 2
24   tft.setTextColor(ILI9341_YELLOW); // צבע טקסט: צהוב
25
26   // הדפסת טקסט במיקום (x, y) = (10, 10)
27   tft.setCursor(10, 10);
28   tft.print("Hello from ESP32!");
29
30   // רוחב, גובה, צבע, ציור מלבן מלא
31   tft.fillRect(50, 50, 100, 50, ILI9341_BLUE);
32
33   // רדיוס, צבע, ציור מעגל
34   tft.drawCircle(180, 75, 20, ILI9341_RED);
35 }
36 void loop() {
37 }
```



CS Scanned with CamScanner

מתמונה זו ניתן לראות שהדפסתי על המסך דברים בסיסים על מנת לראות שהוא פועל יחד עם המיקרו בקר ESP32 מה שניתן לראות למעשה על המסך מה שמודפס עליו זה Hello from ESP32 בנוסף מלבן ממולא בצבע כחול ומעגל שרק ההיקף שלו צבוע בצבע אדום ואינו צבוע מבפנים. הקוד הוא פשוט ובסיסי רק כדי להמחיש את אופן הפעולה של המסך יחד עם המיקרו בקר ולראות שהוא מדפיס ואכן מתבצעת תקשורת בין המסך למיקרו בקר.

כעת נראה את מדידת המתחים:

מדידת מתח הנכנס לפין VCC:

מדידת מתח הנכנס לפין LED:



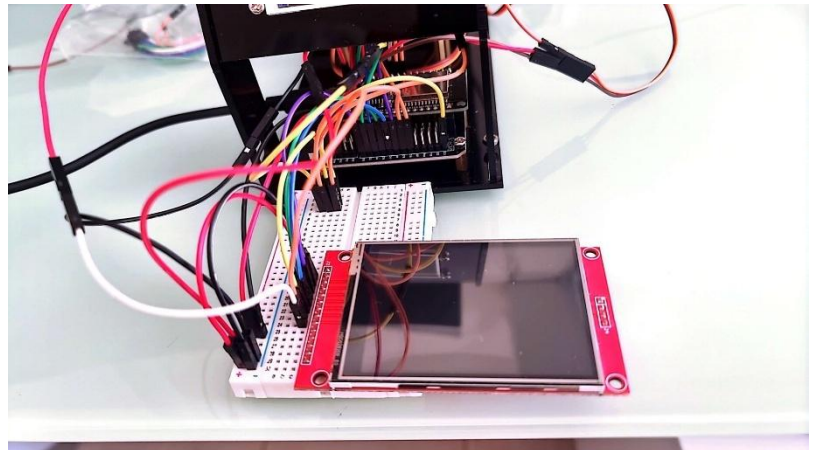
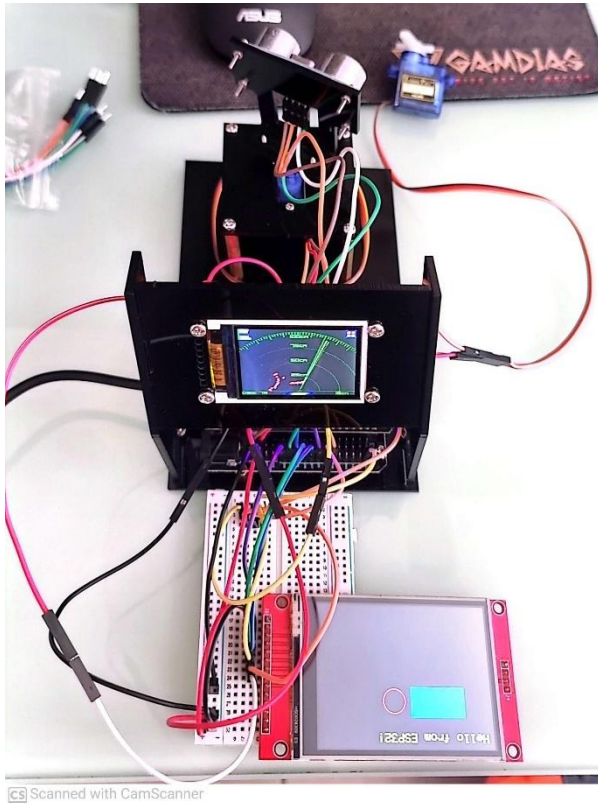
ניתן לראות הסבר מדוע המתח אינו 5 וולט אלא בקירוב 4.5 וולט. בחישן אולטרסוניק כאשר ביצעתי מדידה של המתחים מהפינים האלו במיקרו בקר אליו נכנסים הפינים VCC ו-GND ברגע שמדדתי בפין ה-VN קיבלתי גם מתח דומה למתח זה ובפין ה-3.3V קיבלתי גם בקירוב למתח 3.3V חקרתי מדוע תופעה זאת קורת והסברתי בתיעוד הראשון תחת הכותרת "תיעודים- בדיקות" בתיעוד בתאריך 5.9.25, הסברתי מדוע תופעה זאת עלולה לקרות ומה הסיבות שגורמות ל"איבוד" המתח בכניסה VN.

<https://youtube.com/shorts/a2xaMos41Qk?feature=share> : קישור לסרטון

➡ לחצו כאן לחזרה לתוכן העניינים

תיעוד של חיבור המסך השני למערכת כולה ואופן פעולת המערכת 28.9.25:

כאן תיעדתי את אופן החיבור של המסך השני למערכת כולה, המסך מציג בניסיון זה את אותו הדבר כמו שהראה בתיעוד הקודם אך ניסוי זה היה מדרגה משמעותית עבודי בהתקדמות הפרויקט אציג את אופן פעולת המערכת בסרטון נוסף וכמובן אציג מספר תמונות להמחשה כדי שניתן יהיה לראות זאת. הסרטון מראה בצורה ברורה ומקצועית יותר את אופן הפעולה, ניתן לראות את התזוזה של המנועי SERVO ואת ריצת הרדאר במסך השני.



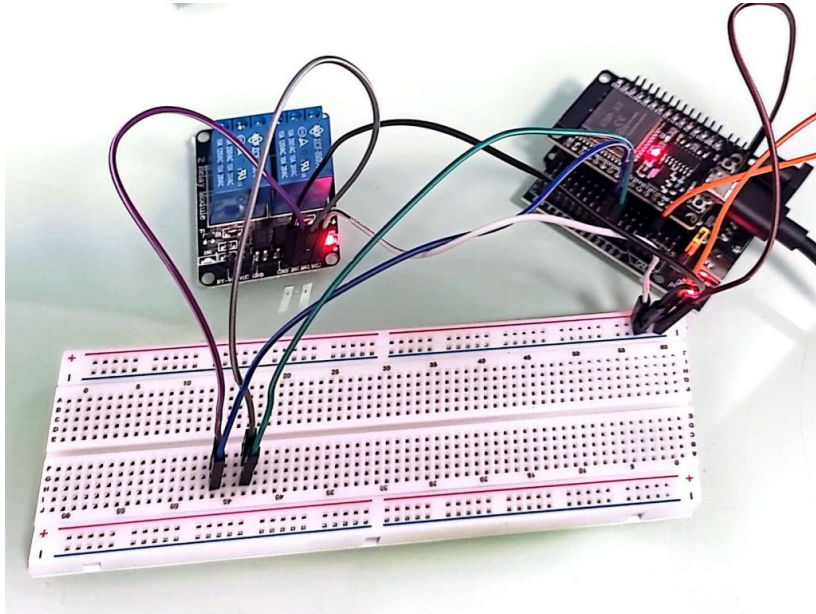
שני המסכים עובדים בפרוטוקול SPI לכן היו לי שני רגליים משותפות בשני המסכים. בעזרת שני הרגליים האלו המסכים מתקשרים בפרוטוקול SPI עם המיקרו בקר. ניתן לראות הסבר מורחב בנוגע לפרוטוקול זה בספר שלי תחת הכותרת "פרוטוקול SPI"

<https://youtube.com/shorts/rToKhwszY90?feature=share>: קישור לסרטון

לחצו כאן לחזרה לתוכן העניינים

תיעוד של אופן פעולה של הממסר הכפול 3.10.25:

בניסוי זה חיברתי ממסר 5 וולט כפול בעזרת מטריצה אל המיקרו בקר ESP32. נתתי קוד בסיסי שמדפיס לי במוניטור שהממסר הראשון דולק ואז לאחר זמן קצר נכבה, ובנוסף הממסר השני גם כן פועל לזמן מסוים ולאחר מכן נכבה. ההודעות מופיעות ומוצגות במוניטור. כמו כן, ניתן לראות שהלדים פועלים על הממסר ואתיחס לכך גם במהלך הניסויים הבאים.



כאן ניתן לראות את החיבור שביצעתי יחד עם המטריצה אל המיקרו בקר.

המטריצה שימשה ככלי עזר על מנת לסדר את חיבור החוטים בין הממסר הכפול אל המיקרו בקר ESP32.

הקוד והפלט:

```
Turning RELAY 1 OFF
Turning RELAY 2 ON
Turning RELAY 2 OFF
Turning RELAY 1 ON
Turning RELAY 1 OFF
Turning RELAY 2 ON
Turning RELAY 2 OFF
Turning RELAY 1 ON
Turning RELAY 1 OFF
Turning RELAY 2 ON
Turning RELAY 2 OFF
Turning RELAY 1 ON
Turning RELAY 1 OFF
Turning RELAY 2 ON
Turning RELAY 2 OFF
```

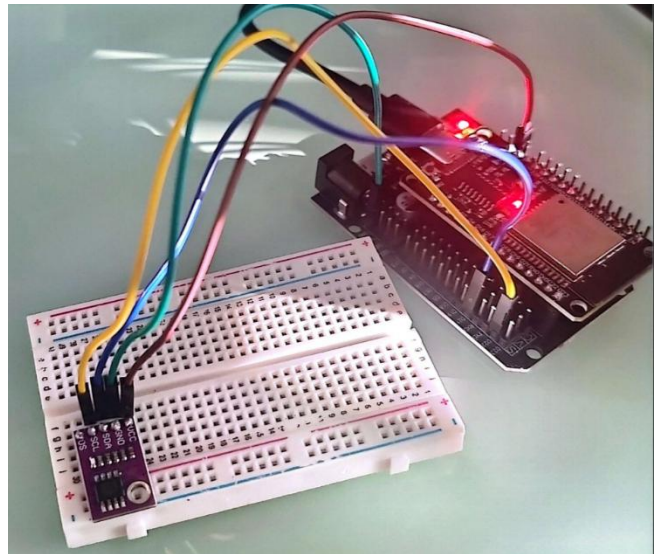
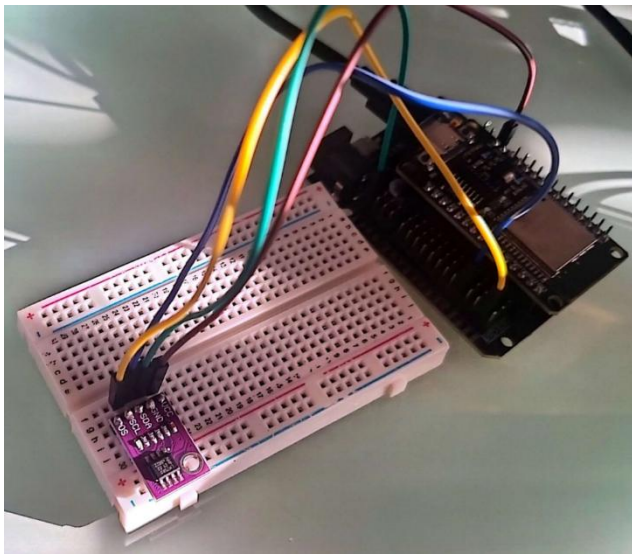
```
1  const int RELAY_PIN_1 = 26;
2  const int RELAY_PIN_2 = 27;
3
4  const int DELAY_TIME = 2000;
5
6  void setup() {
7      pinMode(RELAY_PIN_1, OUTPUT);
8      pinMode(RELAY_PIN_2, OUTPUT);
9
10     digitalWrite(RELAY_PIN_1, HIGH);
11     digitalWrite(RELAY_PIN_2, HIGH);
12
13     Serial.begin(115200);
14     Serial.println("Dual Relay Test Started...");
15 }
16
17 void loop() {
18     Serial.println("Turning RELAY 1 ON");
19     digitalWrite(RELAY_PIN_1, LOW);
20     delay(DELAY_TIME);
21
22     Serial.println("Turning RELAY 1 OFF");
23     digitalWrite(RELAY_PIN_1, HIGH);
24     delay(DELAY_TIME);
25
26     Serial.println("Turning RELAY 2 ON");
27     digitalWrite(RELAY_PIN_2, LOW);
28     delay(DELAY_TIME);
29
30     Serial.println("Turning RELAY 2 OFF");
31     digitalWrite(RELAY_PIN_2, HIGH);
32     delay(DELAY_TIME);
33 }
```

קישור לסרטון:

<https://youtu.be/T7QlbvXVhSo>

לחצו כאן לחזרה לתוכן העניינים

תיעוד של אופן פעולת החיישן LM75 תאריך 10.10.25:



ניתן לראות את אופן החיבור של החיישן טמפרטורה LM75 הפועל בפרוטוקול תקשורת I2C. מה שלמעשה שונה בחיישן זה האופן חיבור המיוחד שלו למיקרו בקר ESP32, יש שתי רגליים אליהן חיברתי את הפינים שאחרים על התקשורת של החיישן עם המיקרו בקר. הרגליים על המיקרו בקר הם 22,21 כאשר פין מספר 21 מתחבר לפין SCL בחיישן הטמפרטורה והפין 22 מתחבר ישירות לפין SDA שבחיישן. על מנת שאני אוכל לתקשר בפרוטוקול תקשורת I2C אני רושם Wire.begin לאחר מכן אני רשאי למעשה לתקשר בפרוטוקול תקשורת זה ולהגיר אפילו פינים אחרים ולאו דווקא את הפינים 22 ו 21. אם ארצה לשנות רגליים נראה זאת בעזרת הפקודה הזאת: Wire.begin(32, 33); // SDA=32, SCL=33.

```
1  #include <Wire.h>
2
3  #define LM75_ADDR 0x48 // של החיישן I2C כתובת
4
5  void setup() {
6      Wire.begin();
7      Serial.begin(9600);
8      Serial.println("LM75 אמת טמפרטורה");
9  }
10
11 void loop() {
12     Wire.beginTransmission(LM75_ADDR);
13     if (Wire.endTransmission() == 0) { // אם החיישן מגיב
14         Wire.requestFrom(LM75_ADDR, 2); // מבקשים 2 בתים
15         if (Wire.available() == 2) {
16             byte msb = Wire.read();
17             byte lsb = Wire.read();
18
19             int temp = ((msb << 8) | lsb) >> 5; // המרת בתים
20             float celsius = temp * 0.125;
21
22             Serial.print("טמפרטורה: ");
23             Serial.print(celsius);
24             Serial.println(" °C");
25         }
26     } else {
27         Serial.println("LM75 נמצא");
28     }
29
30     delay(200); // מספיק לראות שינוי מהיר במגע
31 }
```

ניראה את התוצאות המודפסות ב serial monitor :

```
15:05:01.370 -> LM75 נמצא .
15:05:03.381 -> LM75 נמצא .
15:05:05.370 -> LM75 נמצא !
15:05:05.370 -> 29.75 °C טמפרטורה:
15:05:07.364 -> LM75 נמצא !
15:05:07.364 -> 29.00 °C טמפרטורה:
15:05:09.363 -> LM75 נמצא !
15:05:09.363 -> 28.50 °C טמפרטורה:
15:05:11.358 -> LM75 נמצא !
15:05:11.358 -> 28.25 °C טמפרטורה:
15:05:13.364 -> LM75 נמצא !
15:05:13.364 -> 28.00 °C טמפרטורה:
15:05:15.346 -> LM75 נמצא !
15:05:15.346 -> 27.75 °C טמפרטורה:
15:05:17.368 -> LM75 נמצא !
15:05:17.368 -> 27.75 °C טמפרטורה:
15:05:19.346 -> LM75 נמצא !
15:05:19.346 -> 27.50 °C טמפרטורה:
15:05:21.346 -> LM75 נמצא !
15:05:21.391 -> 27.75 °C טמפרטורה:
15:05:23.368 -> LM75 נמצא !
15:05:23.368 -> 28.00 °C טמפרטורה:
15:05:25.376 -> LM75 נמצא !
15:05:25.376 -> 28.12 °C טמפרטורה:
15:05:27.346 -> LM75 נמצא !
15:05:27.346 -> 28.25 °C טמפרטורה:
```

```
14:53:59.841 -> LM75 נמצא !
14:53:59.841 -> 23.00 °C טמפרטורה:
14:54:01.858 -> LM75 נמצא .
14:54:03.839 -> LM75 נמצא .
14:54:05.851 -> LM75 נמצא .
14:54:07.855 -> LM75 נמצא !
14:54:07.855 -> 23.25 °C טמפרטורה:
14:54:09.861 -> LM75 נמצא !
14:54:09.861 -> 23.12 °C טמפרטורה:
14:54:11.837 -> LM75 נמצא !
14:54:11.837 -> 23.00 °C טמפרטורה:
14:54:13.838 -> LM75 נמצא !
14:54:13.838 -> 23.12 °C טמפרטורה:
14:54:15.818 -> LM75 נמצא !
14:54:15.818 -> 23.00 °C טמפרטורה:
14:54:17.845 -> LM75 נמצא !
14:54:17.845 -> 23.00 °C טמפרטורה:
14:54:19.861 -> LM75 נמצא !
14:54:19.861 -> 22.87 °C טמפרטורה:
```

ניתן לראות את ההדפסות המתקבלות ב- serial monitor. "הנמצא" או "לא נמצא" זה כאשר אני מכסה את החיישן וברגע שמכסים אז ניתן לראות שזה לא מוצא את החיישן וברגע שלא מסתירים אז החיישן נמצא. בנוסף ברגע שנגעתי בחיישן אז הטמפרטורה שהוא בדק עלתה בעקבות החום גוף שלי וניתן לראות את זה ממש בהדפסות למעילה.

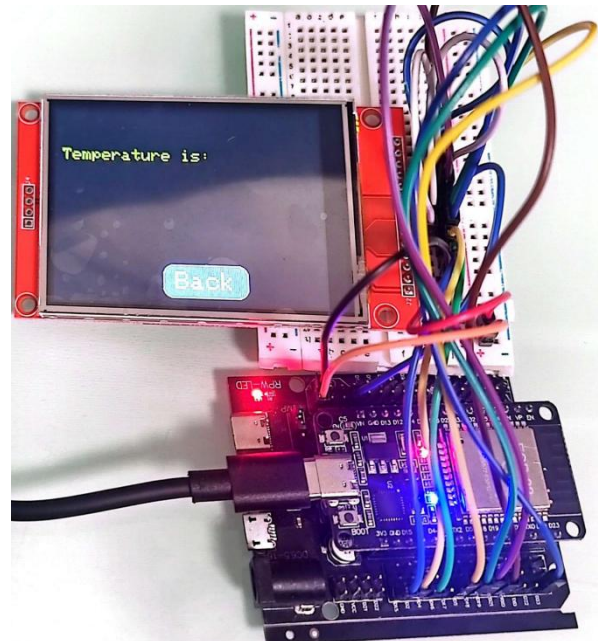
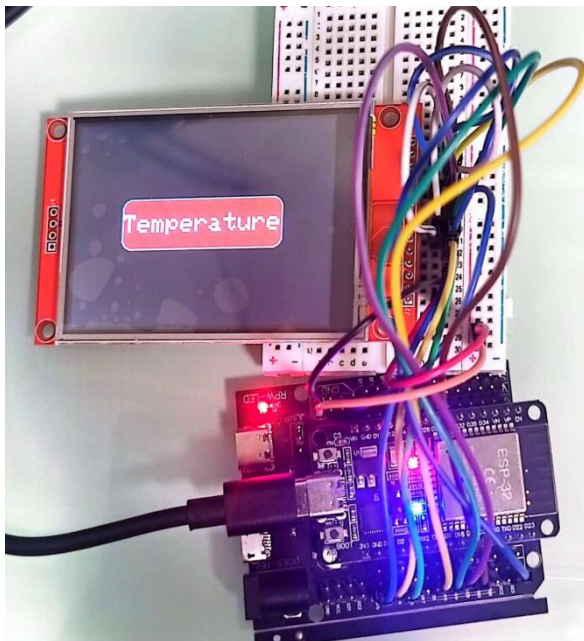
<https://youtube.com/shorts/kOMxanI2hQE> קישור לסרטון אופן פעולה של הרכיב:

[▶ לחצו כאן לחזרה לתוכן העניינים](#)

תיעוד עבור טאץ' במסך 9431 TFT 17.10.25:

כעת אבנה את המערכת השנייה בפרויקט שלי. הפרויקט הוא פרויקט גדול וכמות העומס שיש על כל רכיב/ מסך בקוד עלול לגרום להפרשי זמנים עצומים בין פעולתם. לכן החלטתי להפריד את הפרויקט לשתי מערכות נפרדות, כאשר לכל מערכת יש מיקרו בקר משלה והמיקרו בקרים יתקשרו ביניהם באמצעות הפרוטוקול ESP_NOW.

כעת אראה את אופן הפעולה של המסך טאץ' כיצד יצרתי את הטאץ' ומצאתי למעשה את הקואורדינטות עבור כל כפתור בצורה ידנית. לחצתי בעזרת קוד פשוט לחישוב קואורדינטות ובדיקת טאץ' באופן כללי, רשמתי את הקואורדינטות בקוד חדש שמשלב יצירת תצוגה לכפתורים ואדגים בעזרת תמונות וסרטון בעמוד יוטיוב שלי שיכנס גם לאתר שלי שבניתי עבור הפרויקט וניתן יהיה ממש לראות את האופן פעולה של המסך טאץ' בצורה היקפית וברורה יותר מאשר בתמונות.



בתמונות אלו ניתן לראות את האופן בוא חיברתי את המסך. מאחר והמסך מוגדר כברירת מחדל להדפסה בצורה אנכית אז על מנת שאראה הכל בצורה אופקית נדרשתי לשוב את התצוגה בעזרת rotation 90 מעלות שזה למעשה סיבוב פעם 1, כל 90 מעלות זה סיבוב 1 נוסף.

ESP32	מסך ILI9341
3.3V	VCC
GND	GND
GPIO 15	CS
GPIO 2	DC
GPIO 4	RST
GPIO 23	MOSI (DIN / SDA)
GPIO 19	MISO
GPIO 18	SCK / CLK
3.3V	LED / BL
ESP32	Touch
GPIO 5	T_CS
GPIO 21	T_IRQ
GPIO 23	D_IN
GPIO 19	D_OUT
GPIO 18	SCK

נראה את הקוד:

```

1 #include <TFT_eSPI.h>
2 #include <SPI.h>
3 #define ILI9341_DRIVER
4 #define TFT_MOSI 23
5 #define TFT_MISO 19
6 #define TFT_SCLK 18
7 #define TFT_CS 15
8 #define TFT_DC 2
9 #define TFT_RST 4
10 #define TOUCH_CS 5
11 #define TOUCH_IRQ 21
12 TFT_eSPI tft = TFT_eSPI();
13 struct Button {
14     int x, y, w, h;
15     String label;
16     uint16_t fillColor;
17     uint16_t textColor;
18 };
19 // במרכז Temperature כפתור
20 Button btnTemp = {0,0,200,60,"Temperature",TFT_RED,TFT_WHITE};
21 // בצד ימין למטה, גדול יותר Back כפתור
22 Button btnBack = {tft.width()-110, 193, 100, 40, "Back", TFT_DARKGREY, TFT_WHITE};
23 enum Screen { MAIN, TEMP };
24 Screen currentScreen = MAIN;
25 // ציור כפתור עם אפקט לחיצה
26 void drawButton(const Button &btn, bool pressed=false){
27     uint16_t color = pressed ? TFT_ORANGE : btn.fillColor;
28     tft.fillRoundRect(btn.x, btn.y, btn.w, btn.h, 12, color);
29     tft.drawRoundRect(btn.x, btn.y, btn.w, btn.h, 12, TFT_WHITE);
30     tft.setTextColor(btn.textColor);
31     tft.setTextSize(3);
32     int txtW = btn.label.length() * 6 * 3; // רוחב הטקסט
33     int txtH = 8 * 3; // גובה הטקסט
34     int cx = btn.x + (btn.w - txtW)/2;
35     int cy = btn.y + (btn.h - txtH)/2;
36     tft.setCursor(cx, cy);
37     tft.print(btn.label);
38 }
39 // ציור טקסט חופשי
40 void drawText(String text, int x, int y, uint16_t color, int size){
41     tft.setTextColor(color);
42     tft.setTextSize(size);
43     tft.setCursor(x, y);
44     tft.print(text);
45 }
46 // מסך ראשי
47 void drawMainScreen(){
48     tft.fillScreen(TFT_BLACK);
49     // במרכז מיקום Temperature כפתור
50     btnTemp.x = (tft.width() - btnTemp.w)/2;
51     btnTemp.y = (tft.height() - btnTemp.h)/2;
52     drawButton(btnTemp);
53 }

```

```

54 // מסך בדיקת טמפרטורה
55 void drawTempScreen(){
56     tft.fillScreen(TFT_BLACK);
57     drawText("Temperature is:", 10, 50, TFT_YELLOW, 2);
58     drawButton(btnBack); // Back עם הגדלה למטה, עם הגדלה למטה
59 }
60 void setup(){
61     Serial.begin(115200);
62     tft.init();
63     tft.setRotation(1); // סיבוב 90° ימין
64     pinMode(TOUCH_IRQ, INPUT);
65     drawMainScreen();
66 }
67 void loop(){
68     uint16_t tx=0, ty=0;
69
70     if(digitalRead(TOUCH_IRQ) == LOW){
71         if(tft.getTouch(&tx,&ty,600)){
72             Serial.print("Touch X: "); Serial.print(tx);
73             Serial.print(" | Y: "); Serial.println(ty);
74             // MAIN screen - לחיצה על כפתור Temperature
75             if(currentScreen == MAIN){
76                 if(tx >= btnTemp.x && tx <= btnTemp.x + btnTemp.w &&
77                     ty >= btnTemp.y && ty <= btnTemp.y + btnTemp.h){
78                     drawButton(btnTemp, true); // אפקט לחיצה
79                     delay(100);
80                     currentScreen = TEMP;
81                     drawTempScreen();
82                     while(tft.getTouch(&tx,&ty,600)); // המתנה לשחרור אצבע
83                 }
84             }
85             // TEMP screen - לחיצה על כפתור Back
86             else if(currentScreen == TEMP){
87                 if(tx >= btnBack.x && tx <= btnBack.x + btnBack.w &&
88                     ty >= btnBack.y && ty <= btnBack.y + btnBack.h){
89                     drawButton(btnBack, true); // אפקט לחיצה
90                     delay(100);
91                     currentScreen = MAIN;
92                     drawMainScreen();
93                     while(tft.getTouch(&tx,&ty,600)); // המתנה לשחרור אצבע
94                 }
95             }
96             delay(50); // האטה קצרה למניעת חזרות מיותרות
97         }
98     }
99 }
100

```

הקוד למעשה יוצר לנו שני כפתורים כאשר הכפתור הרשון Temperature הוא כפתור אדום וברגע שאלחץ עליו אעבור לחלונית חדשה במסך שתכיל את הטקסט Temperature is וגם כפתור Back שיחזיר אותי ישירות לחלונית עם הכפתור Temperature. הלוח שאני בוחר בהרצת התוכנית זה **ESP32 DEV Module**.

ניתן לראות סרטון בוא אני מציג את זה באופן ברור ולהבחין באופן הפעולה של המסך יחד עם הטאץ. את הסרטון אוסיף גם ישירות לאתר שלי שעוזר לעקוב אחר הפרויקט.

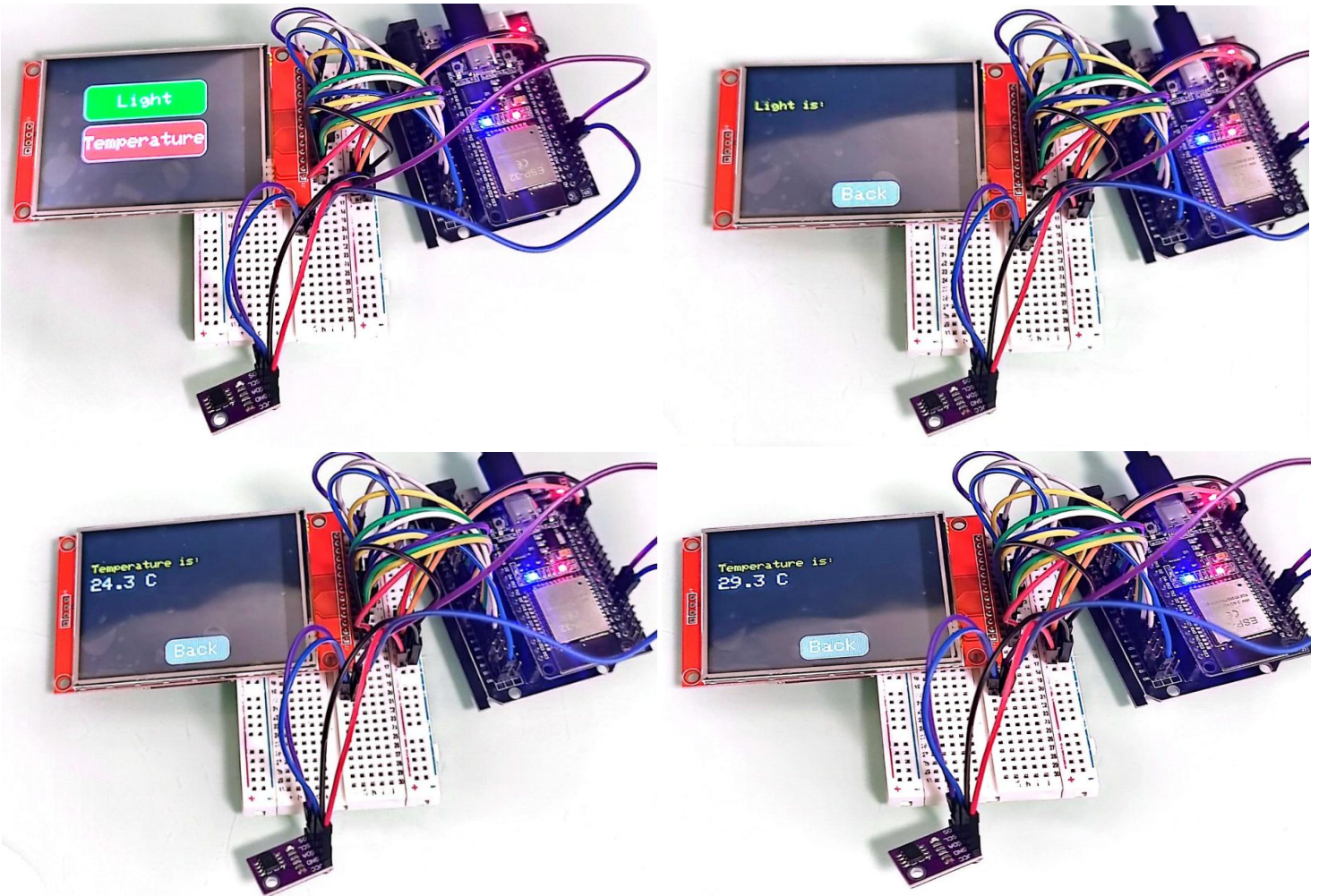
<https://youtube.com/shorts/wPKWHskcr4U> **קישור לסרטון:**

➡ **לחצו כאן לחזרה לתוכן העניינים**

חיבור חיישן טמפרטורה למערכת השנייה והוספת כפתור עבור החיישן תאורה, 25.10.25:

כאן אציג את ההתקדמות שעשיתי, הוספתי כפתור נוסף שיציג את מה שהחיישן תאורה יציג את ערכיו, כעת לא חיברתי אותו עדיין לכן בניסוי זה רק נראה את הכפתור ומה שקורה לאחר שלוחצים עליו. הבדיקה העיקרית בניסוי זה החיבור של החיישן טמפרטורה LM75 אל המערכת כולה. אצטרף למעשה תמונות המשלבות את החיבור של החיישן טמפרטורה ואציג שני סרטונים בנפרד אחד עבור עיצוב המסך יחד עם הכפתור השני, וסרטון נוסף עבור חיבור של החיישן עם המערכת השנייה.

נראה את החיבורים:



חיברתי את החיישן טמפרטורה את המתח ל-3.3 וולט את האדמה לאדמה ואת SDA לפין 32 ואת SCL לפין 33. ניתן לראות את התצוגה החדשה על המסך שלי יחד עם הכפתור הירוק החדש שרשום עליו תאורה באנגלית וכאשר לוחצים עליו אז ניתן לראות בתמונה למעלה מצד ימין שזה מעביר אותי לחלונית כזאת ומשם ניתן לחזור ולראות את הערכים רק שמכיוון שלא חיברתי את החיישן עדיין אז לא קורא שום ערכים וכרגע אי אפשר לראות.

סרטון ראשון של הוספת כפתור נוסף:

<https://youtube.com/shorts/AhsRNy-44W4?feature=share>

סרטון שני של הוספת חיישן טמפרטורה LM75:

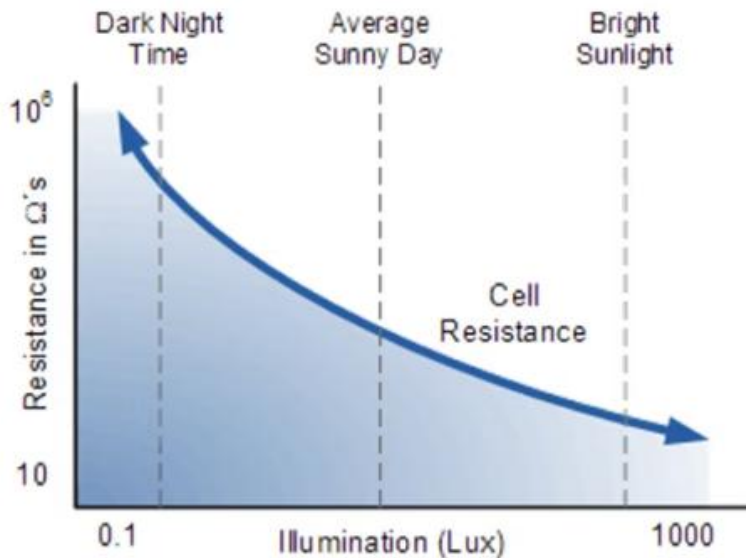
<https://youtube.com/shorts/1TjnN75Hobc?feature=share>

➡ לחצו כאן לחזרה לתוכן העניינים

תיעוד של אופן פעולת חיישן LDR עם המודול מודול חיישן האור KY-018:

בניסוי זה בדקתי למעשה את אופן פעולת החיישן LDR בתוך המודול שאני משתמש בו עבור פרויקט זה. למעשה אני זקוק לבדיקת תאורה משום שאם מדובר בתנאי חושך עלי לפעיל משהון שידמה כמקור אור על מנת להגדיל את שדה הראיה ואת השטח מסביב למערכת.

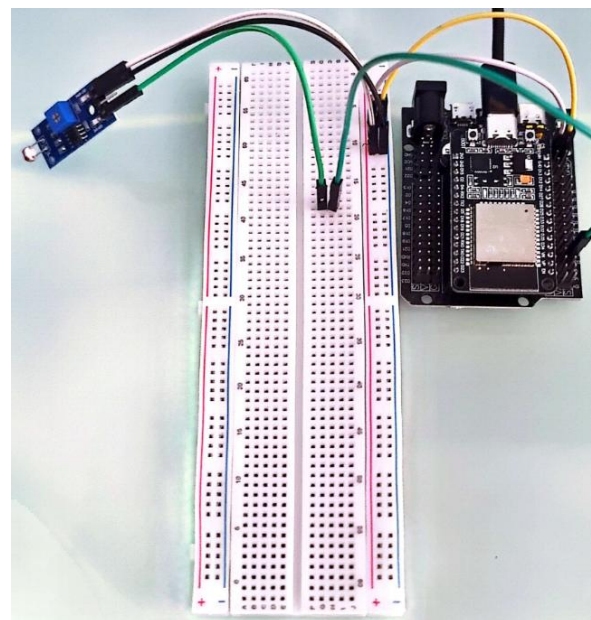
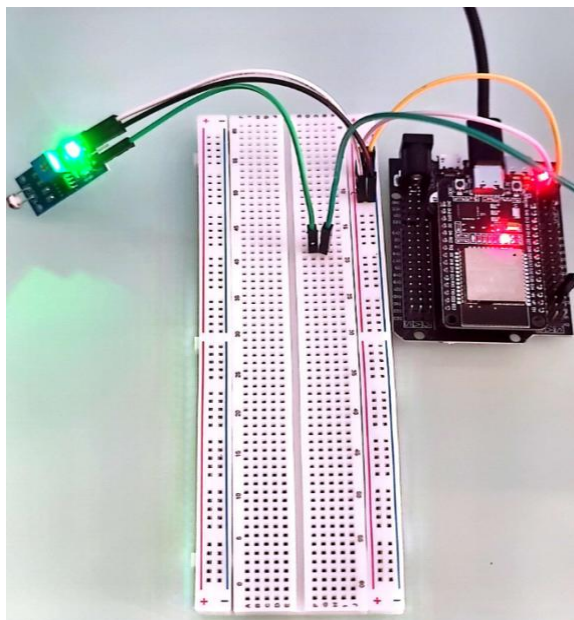
בניסוי זה ראיתי שככל שיש יותר אור בסביבת החיישן LDR הערך האנלוגי צונח ומנגד כאשר אני מכסה את החיישן עם האצבעות אז הערך האנלוגי גדל. אוכל להשתמש בבדיקת ערכים אלו ולהציב ערך סף שממנו אדליק או אכבה את ה"מקור אור" בפרויקט שלי.



כאן למעשה ניתן לראות את הגרף שהוא בצורה הקספוננציאלית. ככל שיותר חושך בסביבה נראה שהערכים גדלים משום שההתנגדות גדלה ככל שיותר חושך בסביבה.

לעומת זאת כאשר בהיר בסביבה של החיישן LDR נראה שההתנגדות קטנה וכך גם הערכים שמציג החיישן קטנים.

נראה את אופן החיבור :



החיבורים שעשיתי: חיברתי את המתח ל-3.3 וולט ואת האדמה לאדמה את הפין A0 חיברתי לפין 34 במיקרו בקר.

סרטון בוא אני מציג את אופן הפעולה:

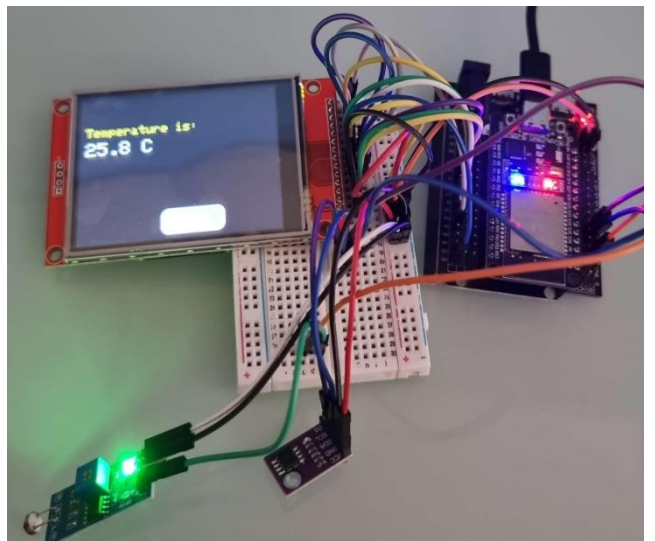
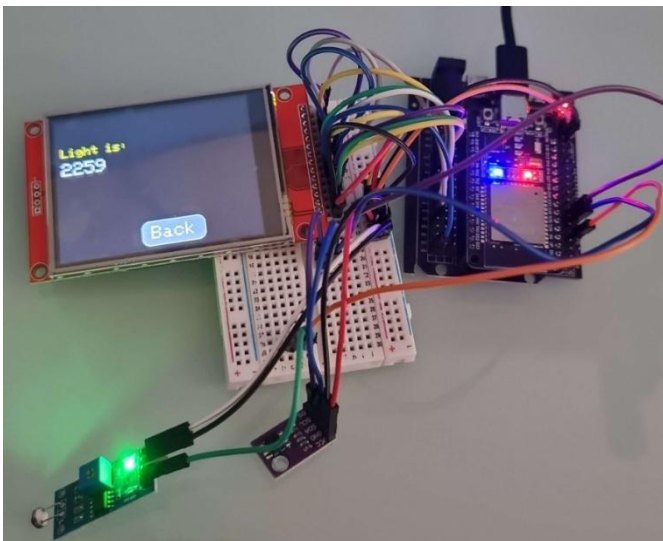
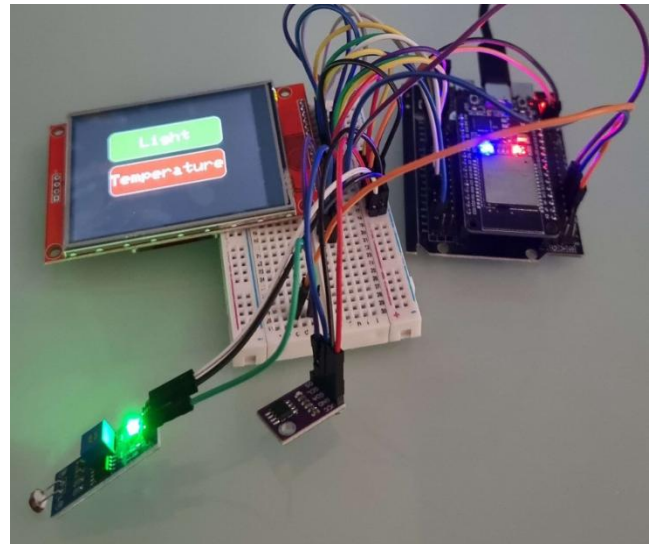
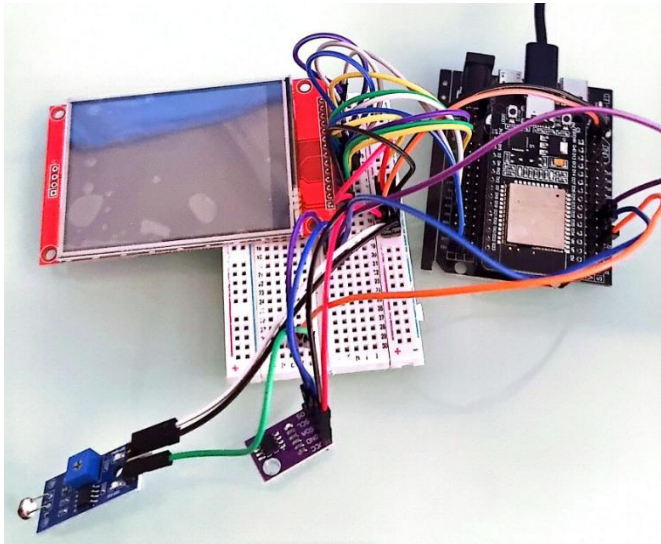
<https://youtube.com/shorts/BH0AUaHqhAI?feature=share>

לחצו כאן לחזרה לתוכן העניינים

תיעוד של חיבור החיישן למערכת השנייה עם המסך והחיישן טמפרטורה, 8.11.25:

בניסוי זה למעשה בדקתי האם המערכת עם הבקר השני תעבוד כולה המסך יחד עם כל החיישנים. לאחר שחיברתי את החיישן LDR עם המודול אל המערכת כולה שיניתי את הקוד כך שיציג לי את הערכים האנלוגיים שמגלה החיישן LDR הערכים מוצגים בחלונית ישר לאחר שלוחצים על הכפתור LIGHT וזה מעביר לחלונית נוספת ששם יוצא הערך והכפתור חזור לתפריט הראשי.

נראה את החיבור של המערכת ומצבים שונים:



ניתן לראות את המצבים השונים המסך פתיחה ואת החלוניות הנפתחות בהתאם לכל כפתור שנלחץ. את אופן הפעולה קל יותר לראות בתורה ברורה בסרטון שאצרף ממש כאן עבור בדיקה זו וכמובן שאצרף גם לאתר שלי עם כותרת מתאימה שיהיה קל לעקוב אחרי התקדמות הפרויקט.

[קישור לסרטון בערוץ יוטיוב שלי:](https://youtube.com/shorts/ooBv1BanwWM?feature=share)

<https://youtube.com/shorts/ooBv1BanwWM?feature=share>

[לחצו כאן לחזרה לתוכן העניינים](#)

תיעוד של תקשורת בין שני המיקרו בקרים ומציאת כתובת MAC של המקבל, 15.11.25

בניסוי זה בדקתי את הדבר החשוב ביותר התקשורת בין שני המיקרו בקרים. למעשה הפרויקט שלי הוא פרויקט גדול והחלטתי לחלק אותו לשתי מערכות נפרדות המתקשרות זו עם זו בעזרת תקשורת אל-חוטית הנקראת ESP_NOW. תקשורת זו עדיפה על פני תקשורת טורית רגילה UART משום שמאפשרת לנו מרחק רחב יותר של שימוש ואף יותר מיקרו בקרים לתקשורת.

נראה את הקודים והפלטים בסריאל מוניטור שיצאו לי:

בצד השולח:

```
1  #include <WiFi.h>
2  #include <esp_now.h>
3  uint8_t receiverAddress[] = {0x6C, 0xC8, 0x40, 0x4E, 0x31, 0x78}; // של המקבל MAC כתובת
4  typedef struct {
5      uint8_t value;
6  } Message;
7  void setup() {
8      Serial.begin(115200);
9      delay(1000);
10     Serial.println("Sender setup started...");
11     WiFi.mode(WIFI_STA);
12     if (esp_now_init() != ESP_OK) {
13         Serial.println("Error initializing ESP-NOW");
14         while(true);
15     } else {
16         Serial.println("ESP-NOW initialized");
17     }
18     esp_now_peer_info_t peerInfo = {};
19     memcpy(peerInfo.peer_addr, receiverAddress, 6);
20     peerInfo.channel = 0;
21     peerInfo.encrypt = false;
22     if (esp_now_add_peer(&peerInfo) != ESP_OK) {
23         Serial.println("Failed to add peer");
24     } else {
25         Serial.println("Peer added successfully");
26     }
27 }
28 void loop() {
29     Message msg;
30     msg.value = random(0, 256);
31     esp_err_t result = esp_now_send(receiverAddress, (uint8_t *)&msg, sizeof(msg));
32     if (result == ESP_OK) {
33         Serial.print("Sent value: ");
34         Serial.println(msg.value);
35     } else {
36         Serial.println("Send failed");
37     }
38     delay(1000); // שולח פעם בשנייה
39 }
40
```

פלט בסריאלי מוניטור:

```
19:33:05.014 -> Sent value: 140
19:33:06.004 -> Sent value: 39
19:33:07.014 -> Sent value: 208
19:33:08.031 -> Sent value: 201
19:33:09.007 -> Sent value: 244
19:33:10.015 -> Sent value: 225
19:33:11.011 -> Sent value: 88
19:33:12.020 -> Sent value: 177
```


בצד המקבל:

```
1 #include <WiFi.h>
2 #include <esp_now.h>
3 typedef struct {
4     uint8_t value;
5 } Message;
6 // לקבלת הודעה Callback
7 void onDataRecv(const esp_now_recv_info_t *info, const uint8_t *incomingData, int len) {
8     Message msg;
9     memcpy(&msg, incomingData, sizeof(msg));
10    Serial.print("Received value: ");
11    Serial.println(msg.value);
12 }
13 void setup() {
14     Serial.begin(115200);
15     delay(1000);
16     Serial.println("Receiver setup started...");
17     WiFi.mode(WIFI_STA);
18     if (esp_now_init() != ESP_OK) {
19         Serial.println("Error initializing ESP-NOW");
20         while(true);
21     } else {
22         Serial.println("ESP-NOW initialized");
23     }
24     esp_now_register_recv_cb(onDataRecv);
25 }
26 void loop() {
27     // כלום, ההודעות מתקבלות ב callback
28 }
29
```

```
19:33:05.012 -> Received value: 140
19:33:06.002 -> Received value: 39
19:33:07.012 -> Received value: 208
19:33:08.029 -> Received value: 201
19:33:09.005 -> Received value: 244
19:33:10.013 -> Received value: 225
19:33:11.010 -> Received value: 88
19:33:12.018 -> Received value: 177
```

פלט בסריאל מוניטור:

ניתן לראות שהערכים הנשלחים מהצד של הבקר השולח אלו בדיוק הערכים שאנו מקבלים בצידו של הבקר המקבל.

ESP-NOW הוא פרוטוקול תקשורת אלחוטי שמובנה בתוך שבבי ESP32 ו-ESP8266, ומאפשר למיקרו-בקרים לתקשר זה עם זה ישירות באמצעות Wi-Fi, ללא צורך בנתב, רשת אלחוטית או חיבור לאינטרנט. בניגוד ל-Wi-Fi רגיל שבו יוצרים רשת, מתחברים ל-SSID ומעבירים נתונים בפרוטוקולים כמו TCP/IP, ב-ESP-NOW המידע נשלח בצורה של חבילות קצרות (packets) ישירות בין מיקרו-בקרים על בסיס כתובת החומרה שלהם. היתרון הגדול של ESP-NOW הוא מהירות גבוהה, השהיה נמוכה מאוד, צריכת חשמל נמוכה, ופשטות – מה שהופך אותו לאידיאלי לפרויקטים של חיישנים, רובוטיקה, רדארים ומערכות בזמן אמת כמו בפרויקט שלך.

במערכת שלי קיימים שני תפקידים ברורים: משדר (Sender) ומקבל (Receiver). המשדר מודד ערך כלשהו (ערך שמייצג מצב או מרחק), אורז אותו בתוך מבנה נתונים (struct), ושולח אותו למקבל באמצעות ESP-NOW. המקבל אינו "מבקש" את המידע אלא מקבל אותו אוטומטית

ברגע שהוא נשלח, באמצעות מנגנון שנקרא callback. זהו רעיון חשוב: הפונקציה onDataRecv אינה רצה בלולאה (loop), אלא מופעלת אוטומטית על ידי המערכת בכל פעם שחבילת נתונים מגיעה מהאוויר.

בקוד של המקבל רואים שהוגדר מבנה בשם Message, שבתוכו שדה אחד מסוג uint8_t. זהו מבנה הנתונים שמייצג את ההודעה שנשלחת. כאשר מתקבלת הודעה, המידע הגולמי (bytes) מועתק לתוך המבנה הזה באמצעות memcpy וכך אפשר לעבוד עם הנתונים בצורה מסודרת וברורה. לאחר מכן הקוד מדפיס לסריאלי את הערך שהתקבל, דבר שמאפשר בדיקה וניטור של התקשורת בזמן אמת. חשוב להבין שב-ESP-NOW אין "חיבור מתמשך" כמו ב-Bluetooth, אלא כל הודעה היא חבילה עצמאית שמגיעה, מעובדת, ונגמרת.

בצד המשדר מתבצע תהליך הפוך. גם כאן מוגדר אותו מבנה Message, כדי ששני הצדדים "ידברו באותה שפה". המשדר מאתחל את ESP-NOW, ואז מוסיף מקבל (peer) לרשימת היעדים שאליהם מותר לשלוח נתונים. ההוספה הזו מתבצעת באמצעות כתובת ה-MAC של המקבל. לאחר שה-peer נוסף בהצלחה, המשדר יכול לשלוח הודעות באמצעות הפונקציה esp_now_send. בדוגמה הזאת, בכל שנייה נשלח ערך אקראי, והמשדר מדפיס לסריאלי האם השליחה הצליחה. זה שלב חשוב מאוד בפיתוח, כי כך ניתן לדעת שהבעיה אינה בתקשורת אלא בלוגיקה של התוכנית.

כתובת ה-MAC היא מזהה חומרה ייחודי של כל רכיב Wi-Fi, בדומה למספר שלדה של רכב. כל ESP32 מגיע מהמפעל עם כתובת MAC ייחודית, והיא משמשת את ESP-NOW כדי לדעת למי לשלוח את הנתונים. בפרויקט שלי מצאנו את כתובת ה-MAC של המקבל על ידי הדפסתה בסריאלי מוניטור באמצעות פונקציה כמו WiFi.macAddress() או על ידי קריאת הפלט שמופיע בעת צריבת הקוד (כפי שניתן לראות בלוג של esptool). לאחר שמצאנו את הכתובת, העתקנו אותה בצורה של מערך בתים (hexadecimal) אל קוד המשדר, למשל

{0x6C, 0xC8, 0x40, 0x4E, 0x31, 0x78}. זהו שלב קריטי: אם כתובת ה-MAC אינה נכונה, הנתונים פשוט לא יגיעו, גם אם הקוד תקין לחלוטין.

לסיכום ESP-NOW, מאפשר לנו לבנות מערכת מבוזרת שבה מיקרו-בקר אחד מודד ומעבד נתונים, ומיקרו-בקר אחר מציג או מגיב אליהם, בצורה מהירה ואמינה וללא תלות בתשתית חיצונית. השימוש ב-struct משותף callback, לקבלת נתונים, וכתובת ה-MAC לזיהוי היעד, יוצר מערכת תקשורת נקייה, מודולרית ומקצועית.

קישור לסרטון של אופן תקשורת בין שני הבקרים:

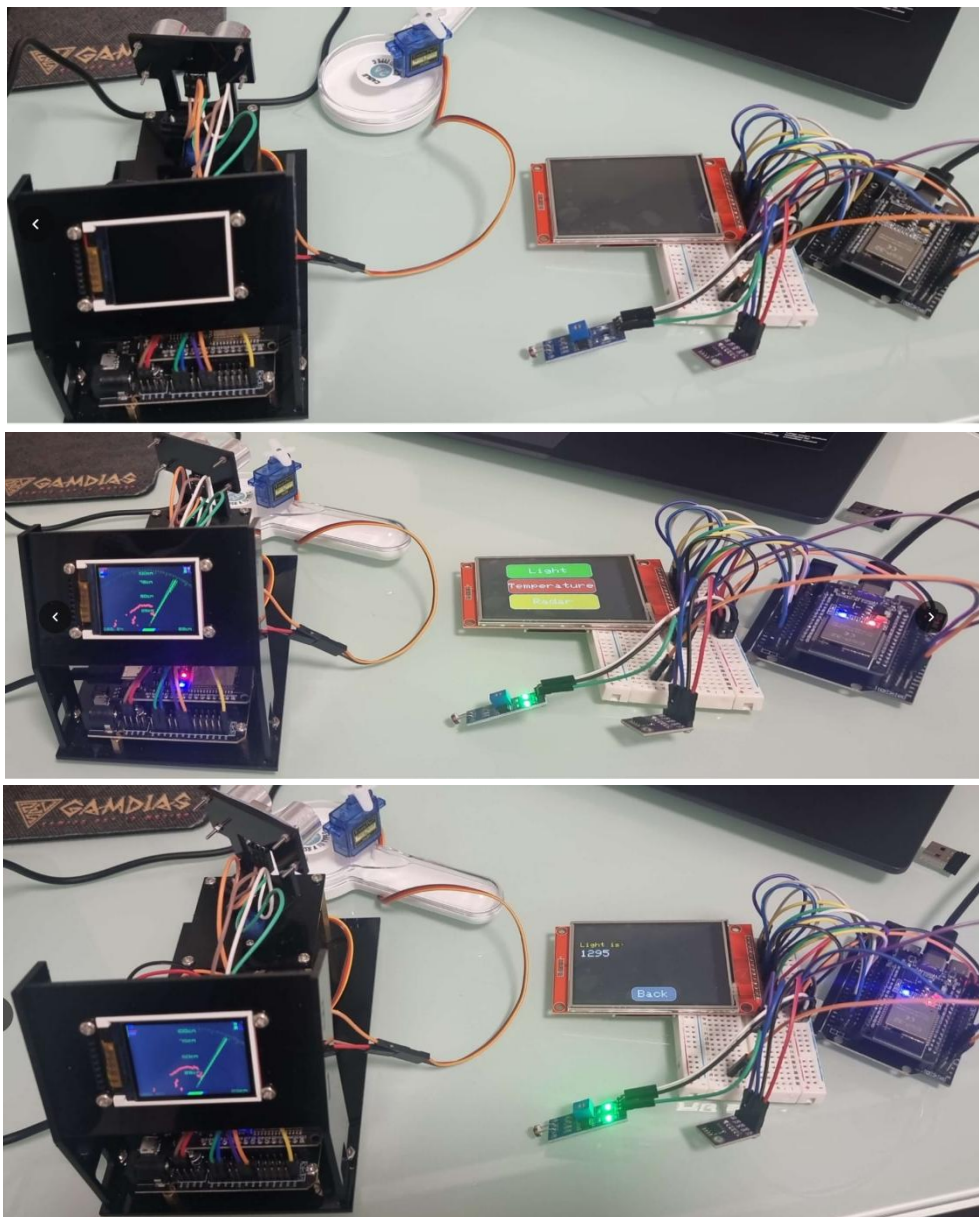
https://youtu.be/oa2_R1PLNHI

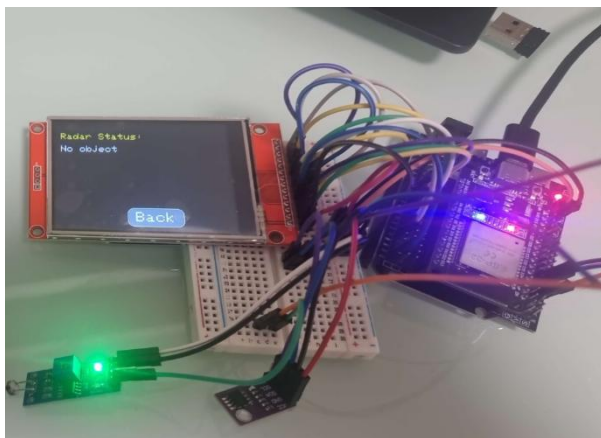
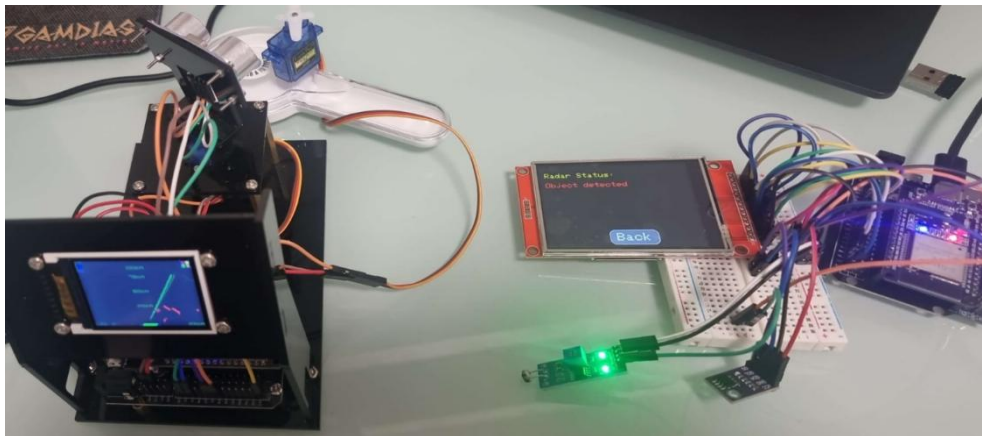
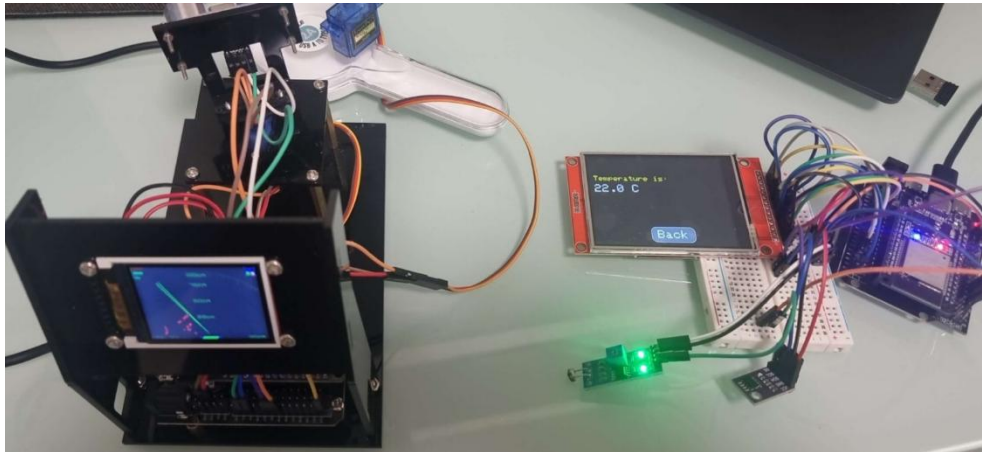
➤ לחצו כאן לחזרה לתוכן העניינים

תיעוד של אופן התקשורת בין שתי המערכות, 22.11.25:

בניסוי זה החלטתי לבדוק את האופן בו המערכות פועלות כל אחת בפני עצמה ובנוסף לכך את התקשורת בין שני המיקרו בקרים. כיצד עשיתי זאת? למעשה חיברתי כל מערכת לפורט אחר במחשב דרך חיבור USB לאחר מכן בחרתי את ה-COM המתאים עבור כל מערכת והרצתי את התוכניות. מה שלמעשה עשיתי זה הוספתי במערכת של המערכת עם המסך תצוגה הגדול והחיישנים, הוספתי כפתור נוסף בצבע צהוב ובו רשום RADAR לאחר מכן כאשר לוחצים עליו זה מעביר אותנו לחלונית נפרדת שבה רואים האם התגלה אובייקט במידה והמערכת הראשונה עם הראדאר גילה אובייקט והנקודה היא אדומה אז במערכת השנייה רשום לנו שהתגלה אובייקט בצבע אדום. במידה ולא התגלה אובייקט שזה 100 סנטימטר ומעלה ויש נקודה צהובה במערכת הראשונה של הראדאר אז המערכת השנייה מראה שאין אובייקט. אראה איך הכול עובד ואצרף סרטון שמראה הכול בצורה ברורה ומקצועית יותר. כמובן שכל הסרטונים עולים לאתר שלי שמהווה חלק בלתי נפרד לתצפית על התקדמות הפרויקט שלי.

נראה חיבורים:





ניתן לראות את אופן הפעולה של התצוגה ואת המצבים השונים לאחר שלוחצים על כל כפתור. מאחר ואלו תמונות קצת קשה להבין כיצד הדברים פועלים בפועל לכן אצרף סרטון להמחשה ברורה שיציג את אופן הפעולה של המערכות והפרויקט. בנוסף ניתן לראות את המצבים השונים של הראדאר שכאשר יש נקודות אדומות אז שום שהתגלה אובייקט וזה רשום בצבע אדום וכאשר יש נקודות צהובות רשום שאין אובייקט בטווח של 100 סנטימטר.

נבחין גם בסריאל מוניטור של כל אחד מהמיקרו בקרים. צירפתי גם הדפסות בסריאל מוניטור על מנת להבטיח תקשורת בטוחה ולראות שאכן מתקיימת תקשורת. למעשה השולח שזאת המערכת של הרדאר מעבירה את המרחק שהרדאר מדפיס על המסך והחיישן אולטרסוניק קולט. לאחר מכן אנו נראה בהדפסה הסריאלית של המיקרו בקר המקבל את אותם הערכים שנשלחו אליו.

המקבל:

```
22:09:34.887 -> Object detected: 35
22:09:34.922 -> Object detected: 36
22:09:34.988 -> Object detected: 34
22:09:35.116 -> Object detected: 16
22:09:35.162 -> Object detected: 25
22:09:35.196 -> Object detected: 30
22:09:35.278 -> Object detected: 41
22:09:35.359 -> Object detected: 39
22:09:35.445 -> Object detected: 38
22:09:35.492 -> Object detected: 36
22:09:35.654 -> Object detected: 37
22:09:35.740 -> Object detected: 36
22:09:35.858 -> Object detected: 37
22:09:35.903 -> Object detected: 36
```

השולח:

```
22:09:34.887 -> Object detected: 35
22:09:34.922 -> Object detected: 36
22:09:34.988 -> Object detected: 34
22:09:35.115 -> Object detected: 16
22:09:35.161 -> Object detected: 25
22:09:35.196 -> Object detected: 30
22:09:35.278 -> Object detected: 41
22:09:35.358 -> Object detected: 39
22:09:35.445 -> Object detected: 38
22:09:35.491 -> Object detected: 36
22:09:35.653 -> Object detected: 37
22:09:35.740 -> Object detected: 36
22:09:35.857 -> Object detected: 37
22:09:35.903 -> Object detected: 36
```

ניתן להבחין שישנם הפרשים קטנים בזמני שליחה וקבלה וכך אנו למעשה מבחינים מי מהם מי. השולח שולח את ההודעה והמקבל יכול לקבל את ההודעה בעיקוב ממש קטן ניתן להבחין בזאת בכמה מקומות כמו למשל 22:09:35.857 בצידו של השולח, והמקבל מקבל את זה ב 22:09:35.858.

קישור לסרטון:

<https://youtube.com/shorts/cFMXjWujSgI?feature=share>

▶ לחצו כאן לחזרה לתוכן העניינים

בניית הנורה/מיני פרוז'קטור לפרויקט, 29.11.25:

למעשה הרעיון מאחורי בניית המיני פרוז'קטור הוא שאם חשוך בסביבה אנחנו זקוקים לתאורה לשטח אליה מכוון הרדאר והתותח. האור החזק שיכוון אל פני השטח אותו אנו סורקים יטשטש את שדא הראייה של האובייקטים שמנסים להתקרב אל יחידת ההגנה וגם מגדיל את שדא הראייה של האנשים הנמצאים באזור שמאחורי התותח למשל חיילים או אנשים בכללי מכוחותינו.

על מנת לבנות פרוז'קטור שיתאים לרמת מתח אותו עובדת המערכת ואינו זקוק לכוח רב כמו נורות למיניהם הצורכות מתח וזרם נורא גבוהים השתמשתי בפנס שקניתי ואז פירקתי אותו, לאחר שפירקתי השתמשתי בבית הסוללות ובחלק של הלדים והלחמתי לכל מקום שהייתי צריך חוטים שאותם חיברתי או ישירות לבקר או דרך מטריצה. לאחר שפירקתי את הפנס וחיברתי הכול כפי שהייתי צריך דרך הממסר זה יפשר לי לשלוט מתי להדליק או לח=כבות את הפנס למשל כאשר החיישן תאורה מצא ערכים גבוהים מדי אז זה מסמל לנו שיש חושך בסביבה ולהפך כאשר יש ערכים נמוכים אז זה אומר שיש אור בסביבה וידעתי לפי זה להדליק או לכבות את החלק של הלדים מהפנס.

נראה תחילה את הפנס וכיצד פירקתי והחלקים שהשתמשתי:



ניתן לראות את המבנה של הפנס פנס רגיל שקניתי ולאחר מכן פירקתי והוצאתי ממנו את החלקים שאני רוצה להשתמש בהם. השתמשתי בבית הסוללות ובחלק של הלדים.

לאחר שפירקתי ולקחתי את החלקים שאני רוצה להשתמש הייתי חייב לבדוק את הקוטביות של בית הסוללות לכן בדקתי בעזרת המולטי מטר את המתח שבית הסוללות מוציאות והיכן נמצא הפלוס והמינוס של בית הסוללות (ניתן יהיה לראות זאת בצורה הטובה ביותר בסרטון שאצרף לאחר ההסברים).

בדיקת המתח של בית הסוללות והקוטביות שלו:



ניתן לראות שבית הסוללות מוציא בערך 4.5 וולט כך גם היה רשום שכל סוללה עם מתח של 1.5 וולט. הייתי חייב לוודא זאת על מנת לא לגרום נזק למערכת שלי וגם על מנת למצוא קוטביות של בית הסוללות.

השאלה הגדולה מדוע שאשתמש בכלל בבית סוללות נפרד אם ה-esp32 מסוגל גם ככה להוציא מתח 4.5-5 וולט? אז התשובה לזה שאני לא רציתי להכביד על המערכת יותר על המידה וצריכת הזרם הזקוקה להדלקת המערכת גדולה לכן לא רציתי להכביד על המערכת כך שאם הייתי מחבר ישירות זה עלול היה להכביד על הבקר כולו ועל אופן פעולת המערכת בכללי.

נראה את החיבורים שביצעתי לפנס:



לאחר מכן הוספתי חוט נוסף לשדה המתכתי של הפנס שנמצא ממש במקום איפה שהחוט החום שמאפיינים את הפלוס של הפנס, עשיתי זאת על מנת לסגור את המעגל החשמלי ורק כך המיני הפרויקטור יוכל לפעול.

מצרף את הקוד שהרצית על מנת לבדוק שהכול עובד כמו שצריך:

```
1  const int relayPin = 13;
2  void setup() {
3      pinMode(relayPin, OUTPUT);
4  }
5  void loop() {
6      // הדלקת הפנס
7      digitalWrite(relayPin, LOW);
8      delay(5000); // נשאר דולק ל-5 שניות
9      // כיבוי הפנס
10     digitalWrite(relayPin, HIGH);
11     delay(2000); // נשאר כבוי ל-2 שניות
12 }
```

הקוד למעשה מדליק את הפנס למשך 5 שניות ולאחר מכן מכבה את הפנס למשך 2 שניות נוספות.

מצרף סרטון שבו אני מראה את כל התהליך יחד עם אופן הפעולה:

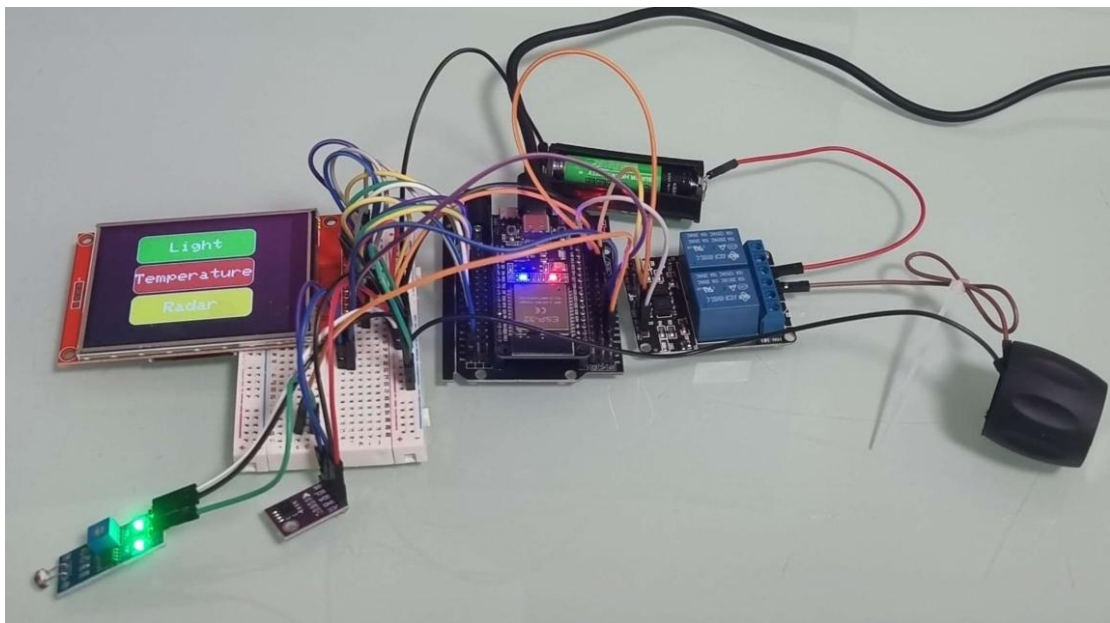
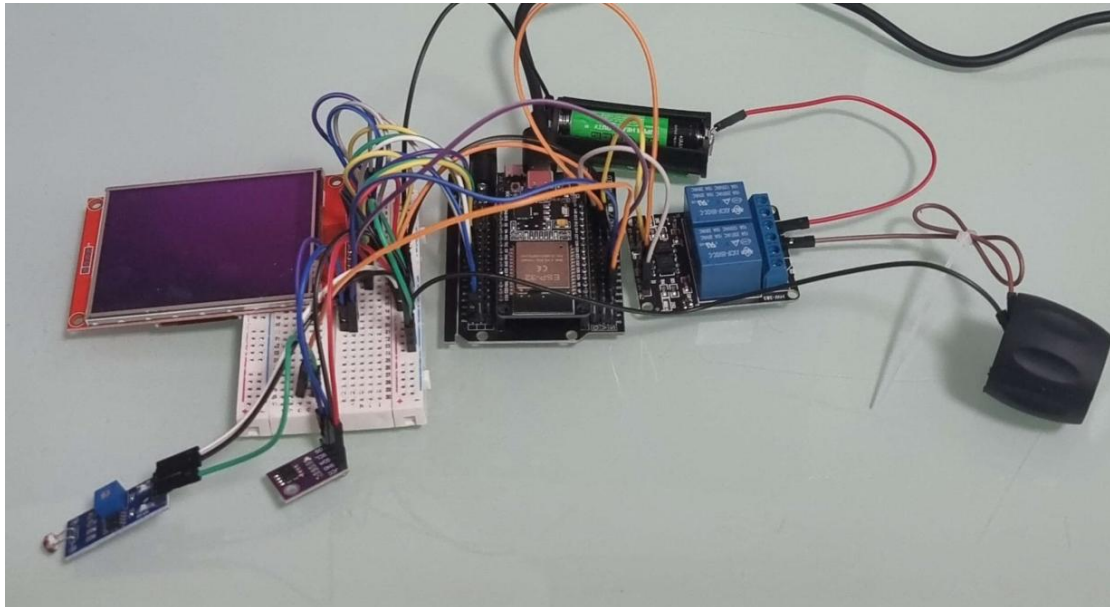
<https://youtu.be/JK8zP62lnlc>

➤ לחצו כאן לחזרה לתוכן העניינים

תיעוד של אופן פעולת המערכת יחד עם חיבור המיני פרוז'קטור, 5.12.25:

לאחר שפירקתי את הפנס יד ובניתי את המיני פרוז'קטור מהחלקים שלקחתי ממנו ובדקתי שהכול אכן עובד חיברתי את המיני פרוז'קטור למערכת השנייה בפרויקט שלי ששם מוצגים כל הערכים של החיישנים ושל הגילוי אובייקט מהראדאר .

נראה את החיבור המלא של המערכת השנייה :



חיבור המיני פרוז'קטור אל המערכת כולה. ניתן יהיה לראות בסרטון שאצרף ישירות לאחר ההסברים וכמובן כל הסרטונים גם עולים לקטגוריה "בדיקות" באתר שלי. החיבור נעשה על ידי ממסר כפול שאליו אחבר גם את הרובה

[קישור לסרטון בו אני מציג את אופן הפעולה של המערכת:](#)

<https://youtube.com/shorts/0NTX0XyeGn0>

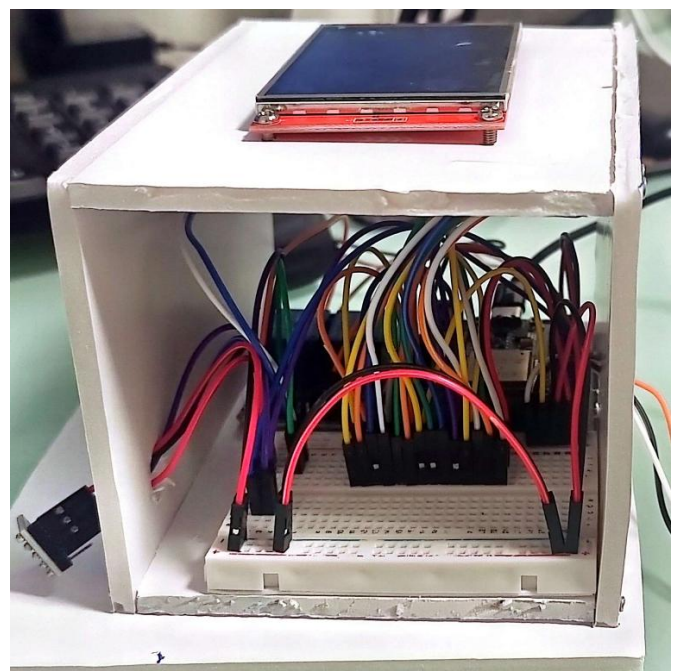
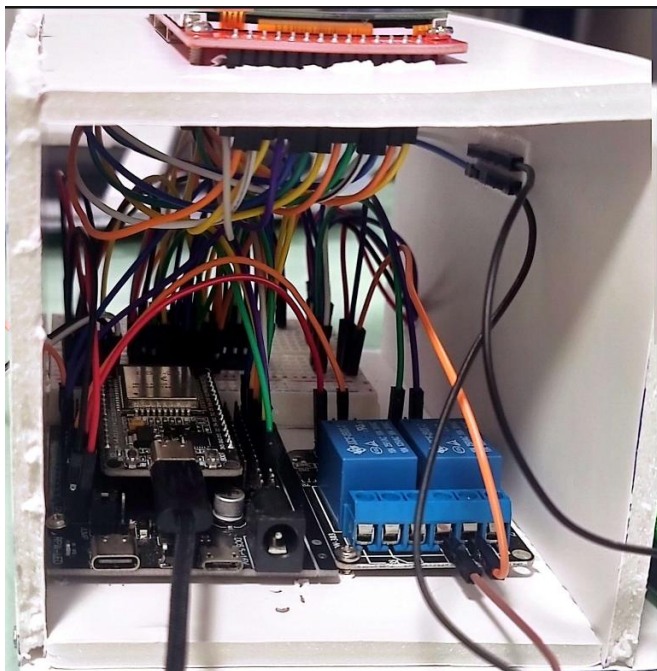
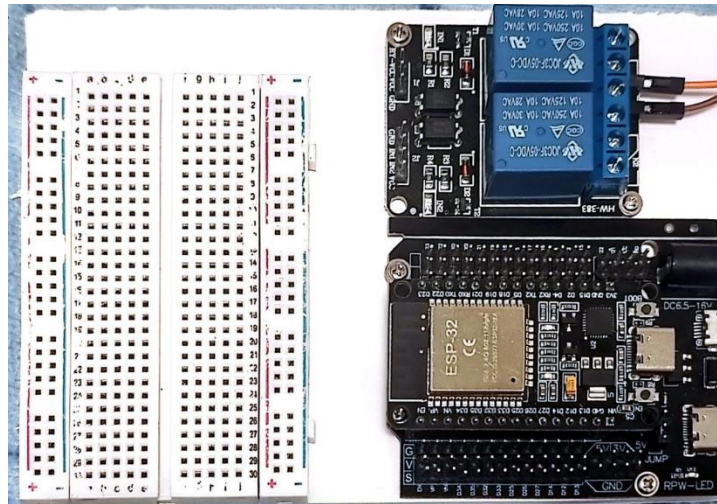
[לחצו כאן לחזרה לתוכן העניינים](#)

תיעוד ראשוני של בניית הדגם, 13.12.25:

הדבר הראשון שהרכבתי היה האזור שבוא נמצאת המערכת השנייה "מערכת הבקרה" יחד עם המסך, חיישנים והממסר שאחראי לתפעל את הפנס ובהמשך את הרובה.

כרגע זהו שלב ראשוני לכן המערכת עדיין לא הכי מעוצבת אך בהמשך אני אשפר את המשנה ואת הנראות שתראה ממש כמו תא בקרה עצמאי שמתקשר עם המערכת השנייה.

כעת נעבור לתמונות ולאידך שהכל נראה בנוסף אשתף את החיווט החדש שעשיתי בממסר וכיצד הוא מועיל יותר עבור הפרויקט שלי מאשר החיווט הקודם לו.



ניתן לראות את המערכת השנייה שלי של המערכת בקרה עם החיישן טמפרטורה והחיישן אור יחד עם המודול. בנוסף ניתן לראות את הממסר שמחובר במערכת שלי ודרכו אני אפעיל את הפנס והרובה שיירט את המטרה.

https://youtu.be/TfWN_WLRvNg סרטון בו אני מציג את המערכת לאחר הבנייה:

נראה את החיבור החדש שביצעתי עבור המערכת של הממסר למעשה בידדתי את הספקת המתח שתהיה אך ורק מהבית סוללות. ביטלתי את האדמה המשותפת שהייתה בין הבית סוללות לאדמה של המיקרו בקר ובכך זה עוזר למנוע קפיצות מתח ורעשים חשמליים.

פין בממסר	לאן מתחבר	הערה חשובה
VCC של הממסר	5V של המיקרו בקר	מספק כוח לצד האופטי בלבד
IN1	פין 13 בבקר	אות הפעלה
GND של הממסר	נשאר ריק	כדי לשמור על בידוד מהבקר
JD-VCC	פלוס של בית הסוללות	מספק כוח לסליל הממסר
GND ליד ה-JD-VCC	מינוס של בית הסוללות	סגירת מעגל הסליל
COM	פלוס של בית הסוללות	המתח שיעבור לפנס כשהממסר ייסגר
NO	לפלוס של הפנס	היציאה לפנס

1. הגנה מפני "קפיצות מתח (Back-EMF)"

בתוך הממסר יש סליל מגנטי. ברגע שהממסר מתנתק, השדה המגנטי קורס ויוצר "מכת" מתח גבוהה מאוד (Spike).

- **ללא בידוד:** הקפיצה הזו עלולה לעבור דרך קווי המתח או האדמה ישירות לתוך המיקרו-בקר ולשרוף את היציאה או את הבקר כולו.
- **עם בידוד:** הקפיצה נשארת בתוך המעגל של הסוללות ואינה יכולה "לקפוץ" לצד של הבקר.

2. מניעת רעשים חשמליים ושיבושי תוכנה (EMI)

פנסים) במיוחד אם הם חזקים או מבוססי LED עם דרייבר (וממסרים יוצרים רעש אלקטרומגנטי.

- רעש זה עלול לגרום למיקרו-בקר "להשתגע": לבצע Reset פתאומי, להיתקע (Freeze), או לקרוא נתונים שגויים מחיישנים אחרים שמחוברים אליו.
- ההפרדה מבטיחה שהבקר יעבוד בסביבה "סטריילית" ושקטה.

3. מניעת עומס יתר על מייצב המתח של הבקר

המיקרו-בקר (כמו ארדואינו) לא נועד לספק זרם גבוה.

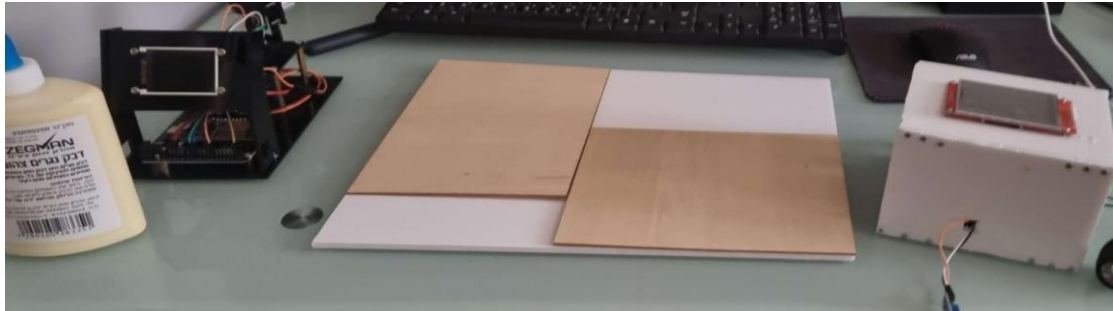
- סליל של ממסר צורך זרם משמעותי בזמן ההפעלה. אם תמשוך את הזרם הזה מה-5V של הבקר, המתח עלול לרדת רגעית (Brownout), מה שיגרום לבקר לאתחל את עצמו בכל פעם שאתה מדליק את האור.
- על ידי חיבור לבית סוללות חיצוני, הבקר מספק רק כמה מילי-אמפר בודדים להדלקת נורת ה-LED-הקטנה שבתוך האופטו-קופלר (הרכיב המבודד).

[▶ לחצו כאן לחזרה לתוכן העניינים](#)

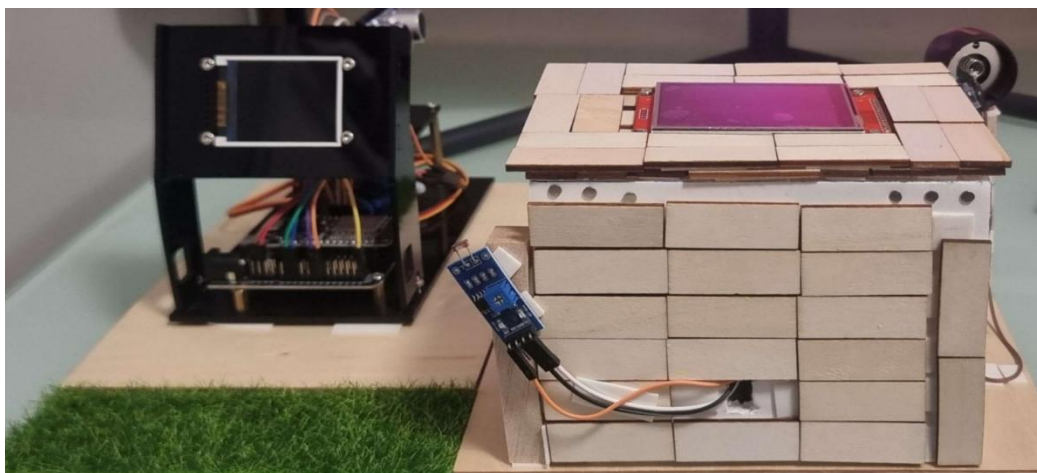
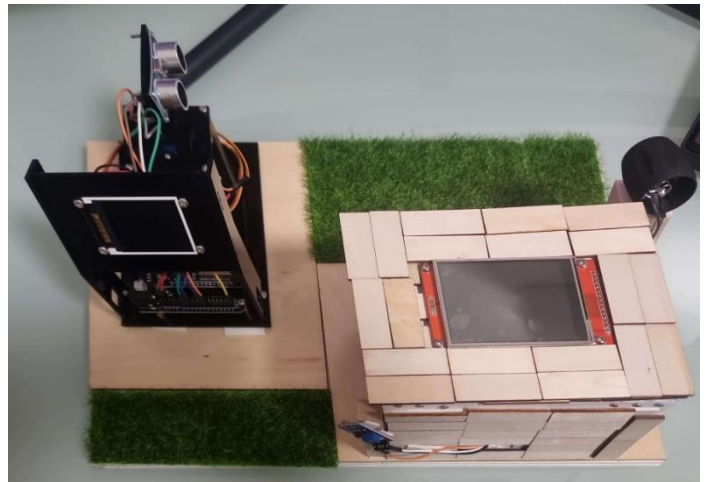
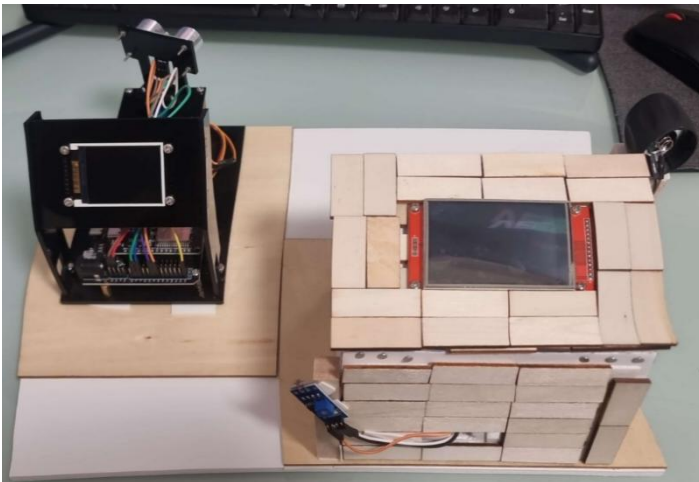
תיעוד בהתקדמות הפרויקט 21.12.25:

בשלב זה התחלתי לבנות את המערכת ולחבר בין המערכות על דגם מלא. עשיתי זאת שלב אחר שלב תוך הקדשה רבה של מחשבה עבור הדגם ואיך כדאי לבצע את הבנייה. אראה כעת את השלבים של הבנייה וכמובן שצריך גם את הסרטון של הבנייה המלא ואעדכן את הסרטון אל האתר שבניתי עבור הפרויקט שלי.

נראה כעת תמונות של הדגם מההתחלה ועד איך שהוא נראה נכון לעכשיו:



שלב הבא שעשיתי הוא הרכבה של חלק הבקרה והראדאר במיקום שתכננתי ובנוסף בניית מסגרת מעץ עבור חלק הבקרה של הפרויקט פלוס הוספת דשא סינטי:



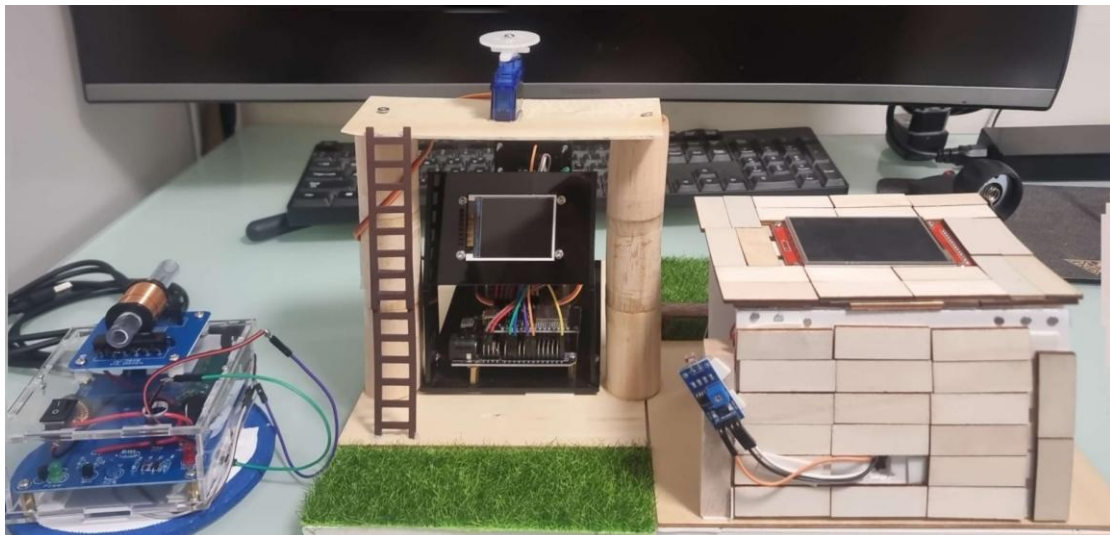
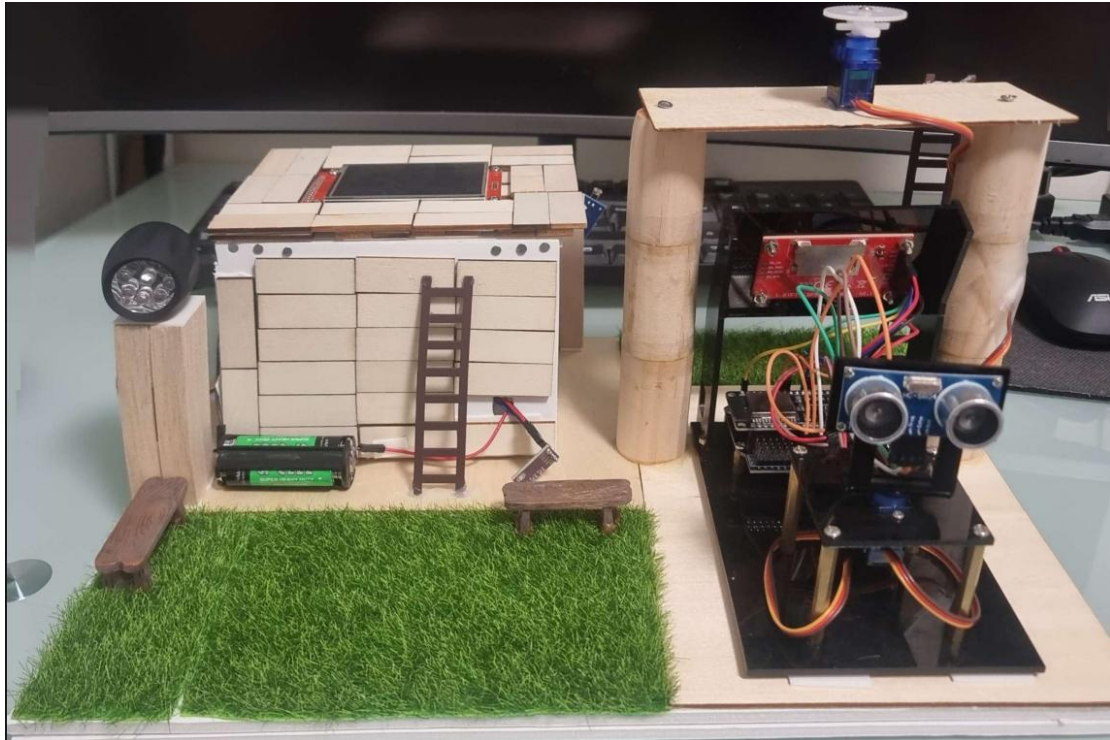
מצרף קישור לסרטון בערוץ היוטיוב שלי ששם אני מסביר ומראה את אופן הפעולה של שתי המערכות : <https://youtu.be/xRKDXMoh2Vc>

➤ לחצו כאן לחזרה לתוכן העניינים

תיעוד ההתקדמות 22.12.25:

המשכתי בבניה של הפרויקט והוספתי קומה נוספת שעלייה העמדתי את המנוע סרבו השני שעליו אני אשים את הרובה האלקטרומגנטי. בנוסף קישטתי מעט את הפרויקט הוספתי ספסלים קטנים מעץ וסולמות במקומות הנחוצים כמו למשל סולם לתותח במידה ויש צורך לתקן אותו (כמו בחיים אמיתיים). הוספתי בסיס סיבובי למנוע סרבו שתהיה יציבות גדולה יותר במהלך הסיבוב.

כך נראית מערכת נכון לעכשיו:



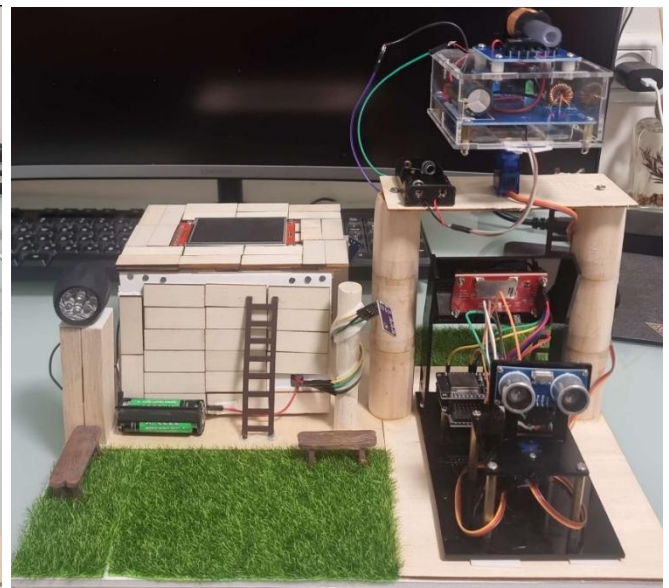
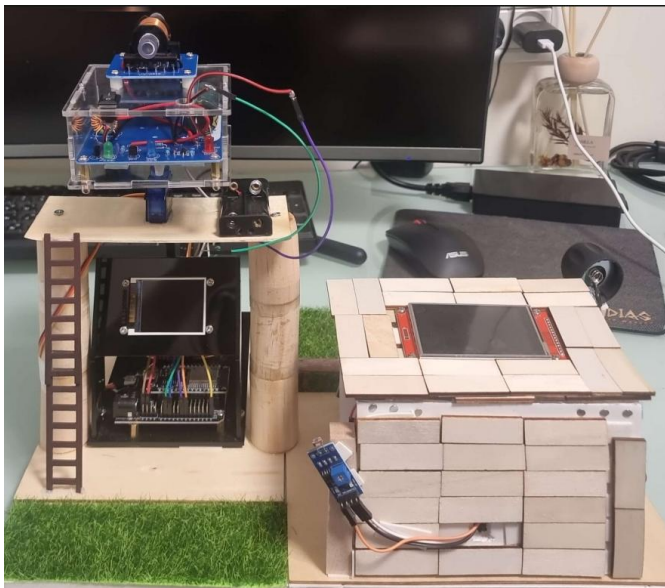
השלב הבא להוסיף את הרובה האלקטרומגנטי ולחבר אותו לממסר ושירה ישירות ברגע שמתגלה אובייקט והמנוע סרבו השני שעליו הרובה יהיה יסנכרן יחד עם הרדאר. אני חושב להוסיף לד או לייזר קטן שיהיה קל לראות את הירייה אך העיקרון הוא להוסיף ממש ירייה באמצעות היריות המתכתיות המגיעות יחד עם הרובה.

➡ לחצו כאן לחזרה לתוכן העניינים

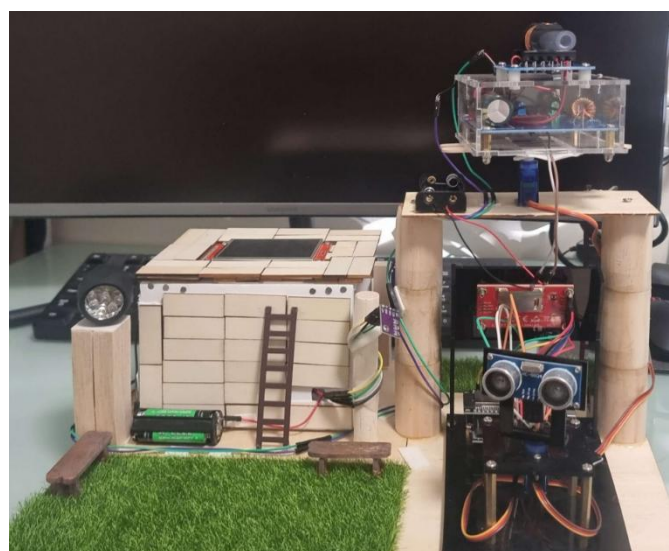
תיעוד מתאריך 3.1.26:

כעת אציג בפניכם את המערכת יחד עם הרובה מחובר על המנוע סרוו שנמצא למעלה. חשוב לי לציין שזה לא הסופי משום שאחרי זה ניתקתי בין המנוע לרובה וחילפתי מחדש את המרכז כובד של בתותח האלקטרומגנטי בנוסף עשיתי בסיס יציב יותר מעל הפלסטיק של המנוע סרוו כך שהתזוזות לא יחרסו את תזוזת הרובה והמנוע לא יושפע מעומס לא יציב במרכזו.

את הרובה חיברתי למערכת השנייה שמקושרת יחד עם המערכת לרדאר. ברגע שאני מקבל שישנו אובייקט בסביבה אז הרובה יורה. דבר חשוב נוסף שאני שמתי לב עוד לפני הקוד ששולט לי עכשיו על הירייה במקום הכפתור של הרובה מכיוון שהקבל נטען זמן רב משום שהוא גדול מאוד בערכו אז לוקח לו להיטען סופית 20-40 שניות אז אני שוקל להוסיף חיישן לייזר או לד שידלקו/יתחילו לפעול כל רגע שיש נקודות אדומות והרובה ירה בכל פעם שהקבל טעון עד מלואו. כמובן שאני אעלה תיעודים נוספים לאחר ההוספה אך כרגע המערכת נראת כך :



לאחר חיבור התותח למערכת הבקרה וגם ייצוב חדש של התותח על הסרוו על מנת לקבל יציבות גבוהה יותר :



אני מצרף סרטון לערוץ היוטיוב ובנוסף כמובן לאתר שלי שם הגישה הרבה יותר נוחה ונגישה.

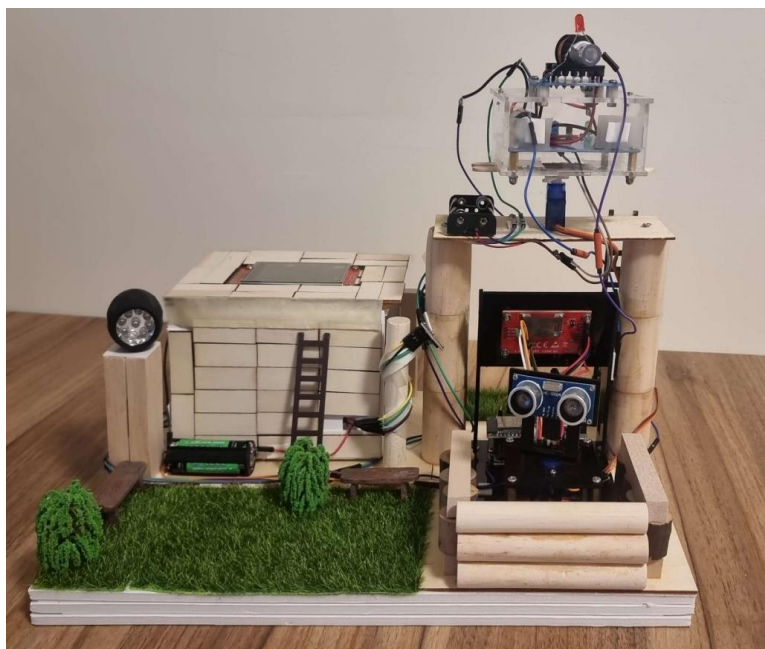
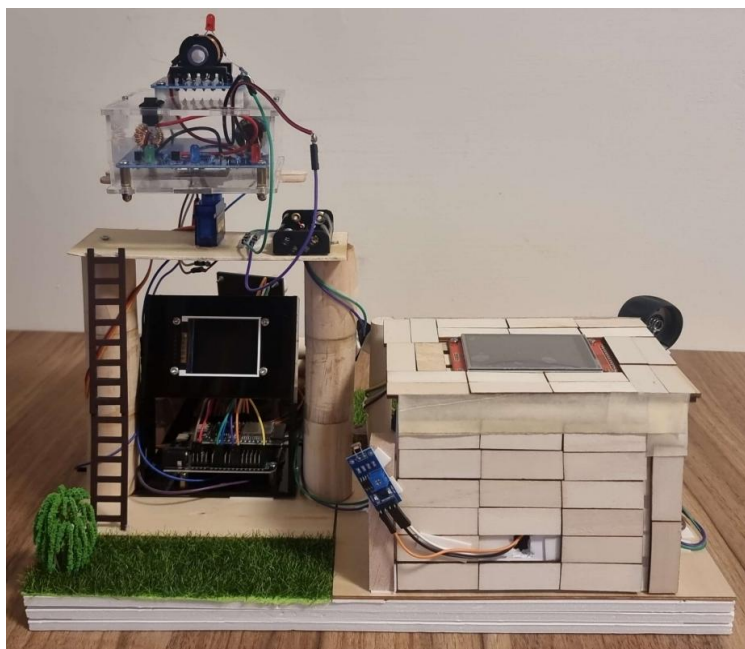
קישור לסרטון: https://youtu.be/iM_ebDkwi1s

לחצו כאן לחזרה לתוכן העניינים

תיעוד 10.1.26:

בחלק זה חשבתי יותר כיצד לקשט את הדגם מה להוסיף כדי שידמה באמת כמו תחנת בסיס קטנה משום שהמערכת רדאר והרובה שמעליה והמערכת בקרה בצד מדמים בסיס שמירה ל-180 מעלות לכיוון שטח אויב משמע חשבתי להוסיף כמו שלפני כן הוספתי ספסלים אז כרגע הוספתי גם כמה עצים וגם סגרתי פינות במערכת בקרה היכן שראו את הברגים והחלק הלבן של הכפה לכן כדי לשפר את הנראות כיסיתי את זה עם שכבה של דבק בצבע דומה לצבע של העץ שכיסיתי את כל המערכת בקרה עצמה.

נראה את התמונות עם רקע ברור יותר ולא עם הרקע של המסך מחשב וכך זה יראה ייצוגי ויפה יותר לעין.

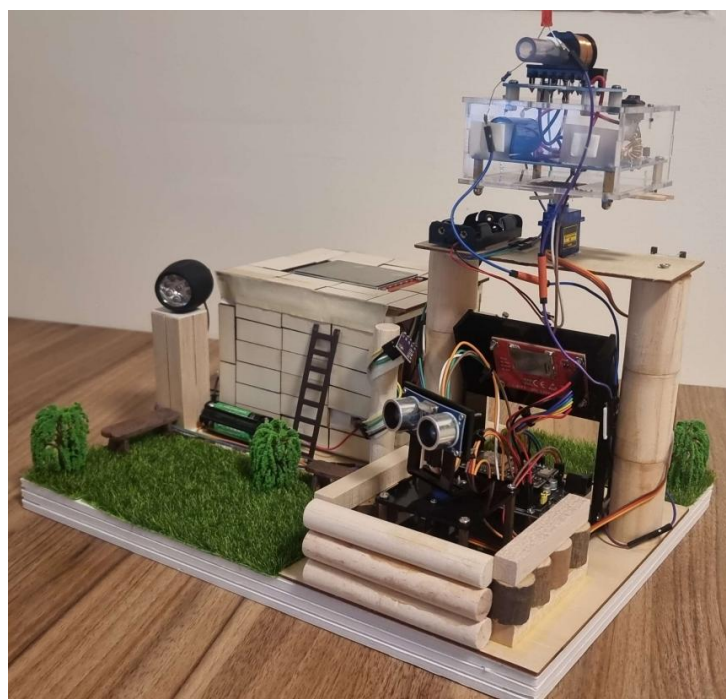
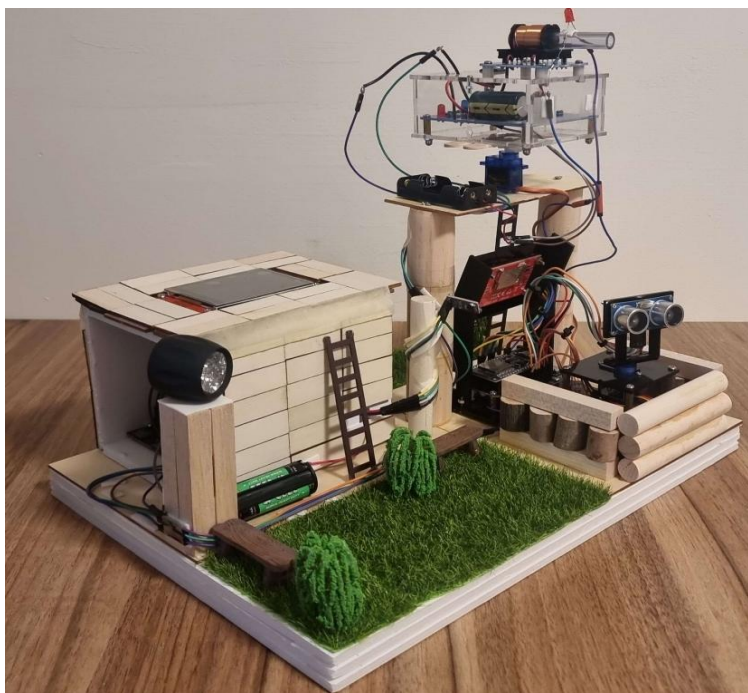


➡ לחצו כאן לחזרה לתוכן העניינים

תיעוד 17.1.26:

בשלב זה חשבתי כיצד אוכל לשפר את הפרויקט כך שיהיה אפקטיבי וברור יותר ובנוסף יראה הרבה יותר טוב ויפה. אחד הדברים שחשבתי עליהם היה להוסיף לד אדום שיידלק במידה והרדאר שולח אות על כך שהתגלה אובייקט ונכבה לאחר שבוודאות אין אובייקט משמע לאחר כמה שניות בודדות שאפילו לא שמים לב ברוב המקרים לכך במהלך הפעולה. לד זה מדמה יריות רצופות משמע מאחר והתותח לוקח לו זמן להטען ונראה שמתבצעת ירייה אך לאחר 20-40 שניות (בקוד שלי הגדרתי ל-40 שניות משום שרציתי שיגיע לטעינה מלאה). כאן אציג את המבנה הסופי נכון לעכשיו וגם את אופן הפעולה, אצרף קישור לסרטון בערוץ היוטיוב וכמובן שאצרף ואעדכן את כל האתר בדברים החדשים והשרטוט החשמלי הסופי לפרויקט.

נראה את התמונות:



מצרף קישור לסרטון היוטיוב וכמובן שמעלה את השרטון בצורה נגישה לאתר שניתן יהיה לראות זאת תחת הכותרת תיעוד בתאריך 17.1.26 :

<https://youtu.be/bjLNvIrKfxU> קישור לסרטון:

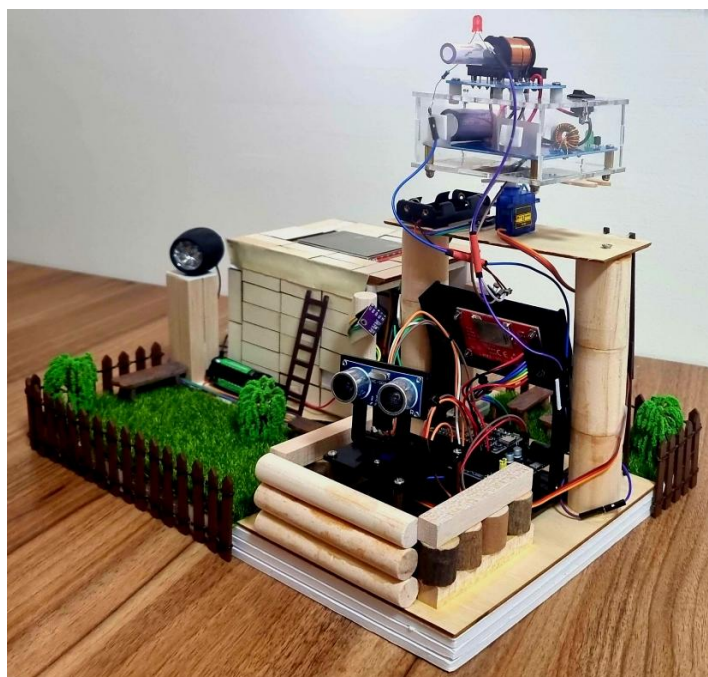
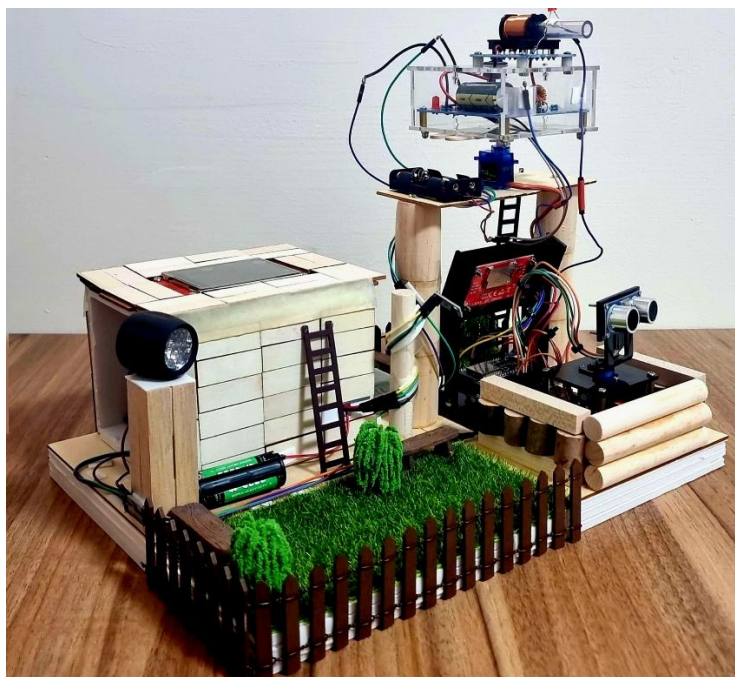
לחצו כאן לחזרה לתוכן העניינים

תיעוד 30.1.26

בתיעוד זה הוספתי עיצוב יפה למערכת כמו גדר וספסלים נוספים כך שיראה יפה וייצוגי יותר בנוסף עשיתי סרטון מסכם של כל המערכת, התהליך הארוך של בניית הפרויקט ואיך שהכול התחיל רק מחשיבה וחיישנים בנפרד למערכת מורכבת ועובדת, ממש הצלחתי להפוך מחשיבה למציאות גם בנראות הדגם וגם באופן הפעולה של המערכת ההגנה.

מתרגש בתהליך שעשיתי ובדרך שעברתי ומודה לכל מי שהיה לצידי ועקב אחר הפרויקט, תודה לכולם

אראה את הדגם עצמו הסופי נכון לעכשיו (אולי יהיו תוספות קטנות לשיפור הנראות בלבד).



מצרף סרטון בו אני מביג את אופן הפעולה של הפרויקט יחד עם התצוגה של הרדאר במחשב. אסביר ואראה איך שהכל עובד ותקשורות נוספות שהתווספו כתוצאה מיצירת המכס במחשב. אצרף את הסרטון ל"בדיקות" באתר שלי ובנוסף לעמוד הראשי של האתר שניתן יהיה לראות ישירות את הסרטון עם הטמונות של הפרויקט

<https://youtu.be/3BDUI5t1Y2Q> קישור לסרטון ביוטיוב:

[לחצו כאן לחזרה לתוכן העניינים](#)

תיעוד 7.2.26:

ניתוח בעיה ותצפיות במהלך הבדיקות

במהלך שלבי הבדיקה של מערכת הרדאר (MASTER), זוהתה תופעה קריטית המשפיעה על יציבות המערכת: מאחר ושני המנועים היו מחוברים ישירות אל הבקר, במהלך שיא העבודה של המערכת כולה, ניתן היה להבחין במקרים מסוימים שבהם מסך הרדאר הופך לשחור לכמה רגעים.

אבחנה הנדסית: תופעה זו נגרמה כתוצאה מנפילות מתח וצריכת זרם גבוהה של המנועים ברגעי השיא, שגרמו ל"גירעון" חשמלי בקו ההזנה של רכיבי הלוגיקה והתצוגה.

הפתרון המיושם: הפרדת נתיבי כוח (Dual Bus)

כדי לפתור את בעיית הניתוקים במסך ולייצב את המערכת, בוצעה הפרדה מלאה:

- **הזנת הבקר והמסך:** מתבצעת דרך חיבור ה-USB המבטיח מתח נקי ויציב לרכיבים העדינים.
- **הזנת מנועי הסרוו:** מתבצעת דרך מערך חיצוני של 3 סוללות AAA 4.5V. הזנה ממקור מתח חיצוני לצורך יעול המערכת.
- **ייחוס משותף (Common\ Ground):** חיבור כלל קווי המינוס לנקודה אחת במטריצה לשם סגירת מעגל אותות הבקרה.

מערך ייצוב זרם - בנק קבלים (Capacitor\ Bank)

כדי לתת מענה לדרישות הזרם הרגעיות של המנועים ולמנוע מהם "למשוך" מתח משאר חלקי המערכת, הוקם בנק קבלים מקבילי על גבי מטריצת המיני.

חישוב הקיבול המצטבר

נעשה שימוש ב-4 קבלי אלקטרוליט בעלי ערך אישי של 100uF ודירוג מתח של 50V החיבור בוצע בתצורה מקבילית על מנת לסכום את ערכי הקיבול:

$$C_{total} = C_1 + C_2 + C_3 + C_4$$

$$C_{total} = 100\mu F \times 4 = 400\mu F$$

יתרונות המערך:

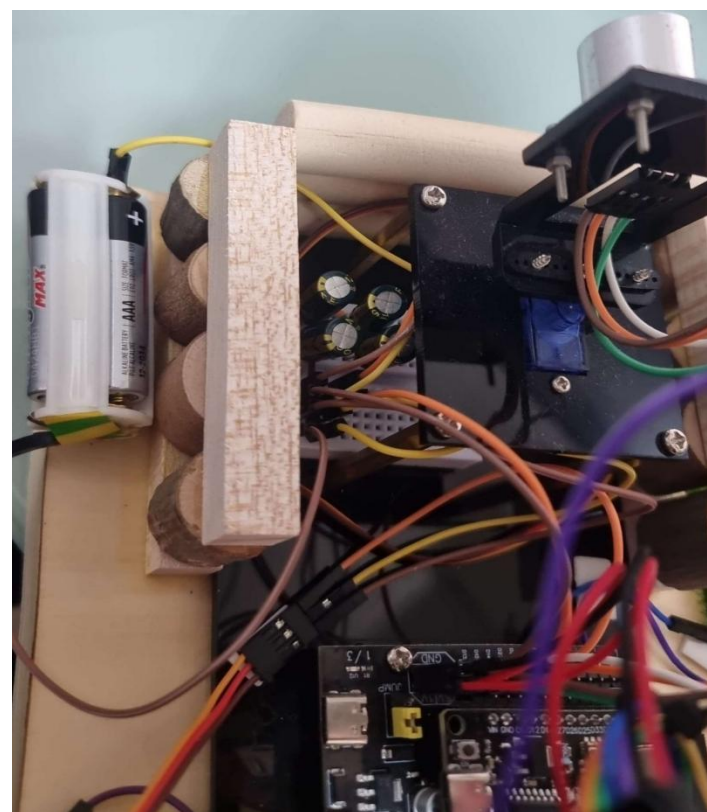
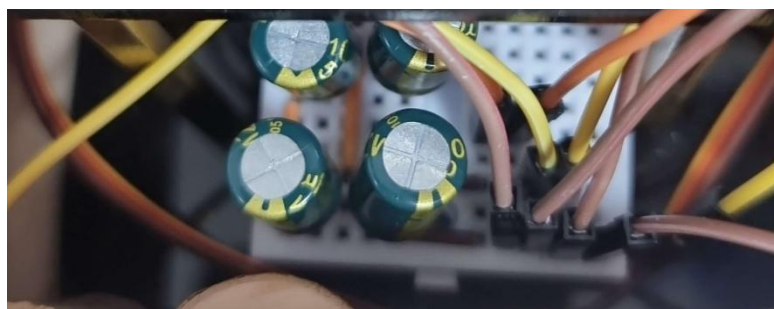
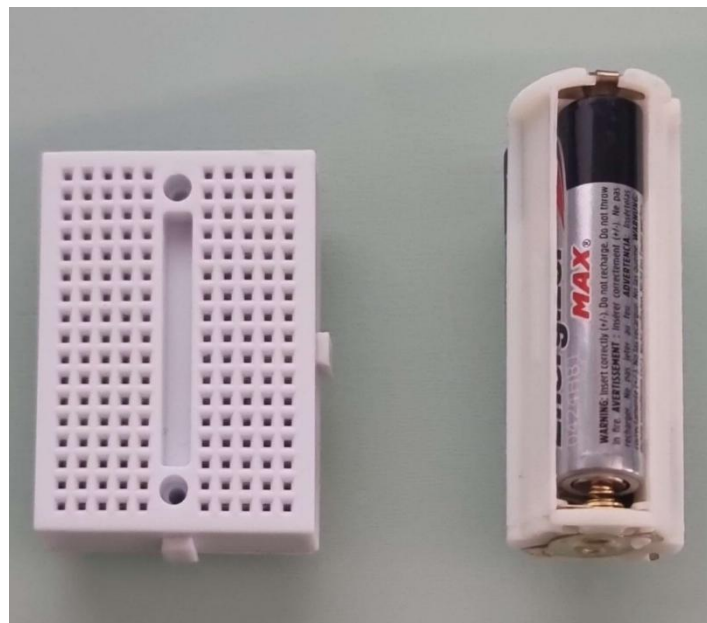
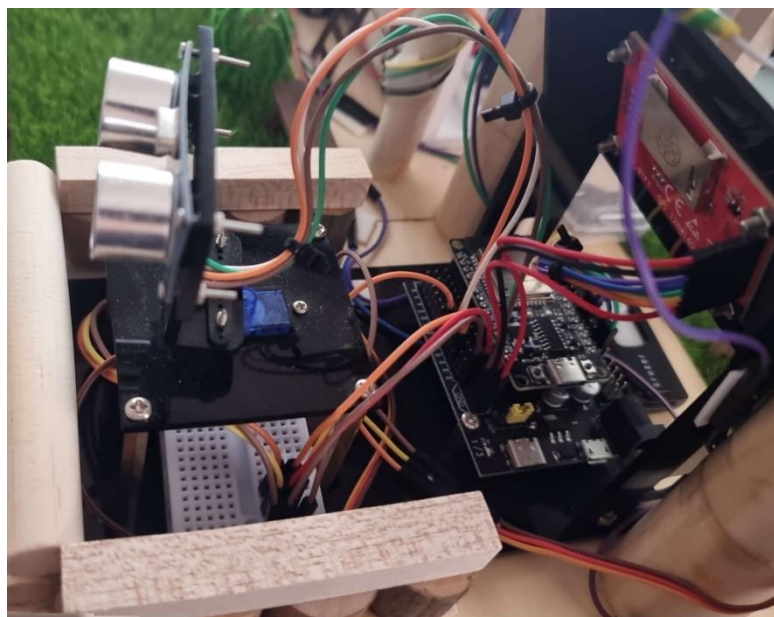
1. **זמינות אנרגיה:** הקבלים פועלים כ"מאגר חירום" המספק זרם מהיר למנועים בדיוק ברגעי השיא שזוהו בבדיקות.
2. **סינון רעשים:** המערך סופג הפרעות אלקטרומגנטיות המיוצרות על ידי המנועים, מה שמונע "לכלוך" חשמלי על קווי הנתונים של החיישן האולטרסוני.

דגשי ביצוע ותחזוקה

- **גישור נקודות מגע:** בשל המבנה המצומצם של מטריצת המיני, בוצעו גישורים על המטריצה בין השורות ליצירת פסי צבירה רחבים ורגליים משותפות לחיבורים.
- **שימור אנרגיה:** לצורך שמירה על חיי הסוללות, הוטמע נוהל ניתוק פיזי של קו הפלוס (VCC) בלבד. ניתוק זה מפסיק את פריקת הסוללות ואת זרם הזליגה של הקבלים, בעוד שהאדמה המשותפת נותרת מחוברת ליציבות מקסימלית בעת הפעלה מחדש.

תוצאה סופית : לאחר יישום השינויים, המערכת פועלת ברציפות ללא נפילות מתח, והתופעה שבה המסך נהיה שחור בוטלה לחלוטין.

נראה כעת חיבורים ותמונות כתייעוד למהלך העבודה :



ניתן לראות את אופן החיבורים הוספתי 4 קבלים בחיבור מקביל עם גישור בין הרגליים של המטריצה כתוצאה מכך שלא היו מספיק רגליים בטור אחד לכן חיברתי עם חוטים מוליכים לרגליים אחרות על גבי המטריצה. בית הסוללות שוכב בצורה נוחה ויציבה על יד המערכת של הרדאר מבלי להפריע לחיישן האולטרסוני ולמערכת עצמה לעבוד. זהו שלחב חשוב מאוד במהלך הפרויקט משום שמונע רעשים ונפילות מתח שהיו על המסך ושומר על חיי הבקר באופן כולל.

קוד סופי עבור המערכת בקרה וניטור סביבתי (SLAVE)

```

1 // ===== ספרייה =====
2 #include <TFT_eSPI.h> // ספרייה לשליטה במסך ה-TFT
3 #include <SPI.h> // למסך ESP32-בין ה-SPI תקשורת
4 #include <Wire.h> // לחיישנים I2C תקשורת
5 #include <WiFi.h> // ESP-NOW נדרש להפעלת
6 #include <esp_now.h> // ESP32 תקשורת אלחוטית בין שני
7
8 // ===== הגדרות חיישנים =====
9 // LM75 של חיישן הטמפרטורה I2C-כתובת ה
10 #define LM75_ADDR 0x48
11 // I2C פינים של תקשורת
12 #define SDA_PIN 32
13 #define SCL_PIN 33
14 // ===== פני חומרה =====
15 #define LIGHT_PIN 34 // פין חיישן האור
16 #define RELAY_PIN 13 // ממסר הפנס
17 #define RELAY_GUN_PIN 17 // ממסר מנגנון הירי
18 #define LED_RADAR_PIN 16 // לד שמראה זיהוי רדאר
19 #define TOUCH_IRQ 21 // פין אינטרפט של מסך המגע
20 // ===== אובייקט מסך =====
21 TFT_eSPI tft = TFT_eSPI(); // יצירת אובייקט לשליטה במסך
22
23 // מבנה הודעה (כמו במערכת הראשונה)
24 typedef struct {
25     uint8_t state; // משתנה שמכיל את מצב הרדאר או המרחק שנשלח מהמערכת הראשונה
26 } RadarMessage; // ESP-NOW הגדרת מבנה הנתונים שנשלח בתקשורת
27
28 // ===== משתני מצב המערכת =====
29 bool firstRadarIgnored = false; // מתעלם מהזיהוי הראשון כדי למנוע רעש בעת ההפעלה
30 int lastRadarDistance = -1; // המרחק האחרון שהתקבל מהרדאר // -1;
31 bool objectDetected = false; // מציין האם כרגע זוהה אובייקט
32 bool objectEverDetected = false; // מציין האם זוהה אובייקט לפחות פעם אחת
33 bool radarScreenJustOpened = false; // מציין שהמסך של הרדאר נפתח כרגע
34
35 // ===== בדיקת ניתוק רדאר =====
36 // זמן קבלת ההודעה האחרונה מהרדאר
37 unsigned long lastRadarMessageTime = 0;
38
39 // אם לא מתקבלת הודעה במשך זמן זה
40 // המערכת מניחה שהרדאר מנותק
41 const unsigned long RADAR_TIMEOUT = 2000;
42
43 // ===== משתני טמפרטורה =====
44 // שמירת הטמפרטורה האחרונה שהוצגה במסך
45 float lastTemp = -1000;
46 // זמן הקריאה האחרון מהחיישן
47 unsigned long lastRead = 0;
48
49 // ===== משתני תאורה =====
50 // ערך התאורה האחרון שנמדד
51 int lastLight = -1;
52 // זמן הקריאה האחרון מחיישן האור
53 unsigned long lastLightRead = 0;
54
55 struct Button { // מבנה נתונים שמייצג כפתור במסך
56     int x, y, w, h; // מיקום וגודל הכפתור על המסך (רוחב וגובה X,Y)
57     String label; // הטקסט שיופיע על הכפתור
58     uint16_t fillColor; // צבע הרקע של הכפתור
59     uint16_t textColor; // צבע הטקסט של הכפתור
60 };
61
62 // ===== מערכת הירי =====
63 // זמן הירי האחרון
64 unsigned long lastFireTime = 0;
65 // זמן שטיפנה מינימלי לפני ירי נוסף
66 const unsigned long RECHARGE_TIME = 50000;
67

```

שוורת 1-6: הספרייה הדרושה להפעלת רכיבי המערכת. הספרייה TFT_eSPI מאפשרת שליטה במסך ה-TFT והצגת מידע גרפי וטקסטואלי. הספרייה SPI מספקת פרוטוקול תקשורת מהיר המשמש בין היתר לתקשורת עם המסך. הספרייה Wire מאפשרת שימוש בפרוטוקול I2C לחיבור חיישנים למיקרובקר. בנוסף, הספרייה WiFi מאפשרת למודול ESP32 לעבוד עם רשת אלחוטית, והספרייה esp_now מאפשרת תקשורת אלחוטית ישירה בין מודולי ESP32 ללא צורך בראוטר.

שוורת 8-13: בחלק זה מוגדרים פרמטרים הקשורים לחיישן הטמפרטורה LM75 ולתקשורת I2C. הקבוע LM75_ADDR מייצג את כתובת החיישן על קו התקשורת I2C ומאפשר למיקרובקר לזהות אותו בין רכיבים נוספים. בנוסף מוגדרים הפינים SDA_PIN ו-SCL_PIN, המשמשים להעברת הנתונים ואת השעון בתקשורת I2C בין הבקר לחיישן.

שוורת 14-19: הגדרת הפינים בבקר המחוברים לרכיבי החומרה של המערכת. הפין LIGHT_PIN משמש לקריאת נתונים מחיישן אור. הפין RELAY_PIN מחובר לממסר המאפשר שליטה על התקן חיצוני. הפין RELAY_GUN_PIN משמש להפעלת ממסר נוסף הקשור למנגנון הירי. הפין LED_RADAR_PIN מחובר לנורת חיווי המופעלת כאשר הרדאר מזהה אובייקט. הפין TOUCH_IRQ משמש לקליטת אות מהמסך בעת זיהוי מגע.

שוורת 21: נוצר אובייקט בשם tft מתוך הספרייה TFT_eSPI. אובייקט זה מייצג את מסך ה-TFT במערכת ומאפשר להשתמש בפונקציות של הספרייה לצורך הצגת טקסט, ציור גרפיקה ועדכון נתונים בזמן אמת על המסך.

שוורת 23-26: קטע קוד זה מגדיר מבנה נתונים בשם RadarMessage, המשמש להעברת מידע בין יחידות ESP32 באמצעות פרוטוקול ESP-NOW. המבנה כולל משתנה אחד בשם state מסוג uint8_t, אשר מייצג את מצב הרדאר, למשל האם זוהתה תנועה או לא. שימוש במבנה נתונים מאפשר להעביר מידע בצורה מסודרת בין רכיבי המערכת.

שוורת 28-33: בחלק זה מוגדרים משתנים גלובליים המשמשים לניהול מצב מערכת הרדאר בזמן ריצה. המשתנה firstRadarIgnored מאפשר להתעלם מהקריאה הראשונה של החיישן כדי למנוע שגיאות מדידה. המשתנה lastRadarDistance שומר את המרחק האחרון שנמדד. המשתנה objectDetected מציין האם כרגע זוהה אובייקט, ואילו objectEverDetected מציין האם זוהה אובייקט לפחות פעם אחת מאז הפעלת המערכת. המשתנה radarScreenJustOpened משמש לזיהוי מצב שבו מסך הרדאר נפתח זה עתה, לצורך ביצוע פעולות אתחול או עדכון תצוגה.

שוורת 36-41: קטע קוד זה משמש לניטור תקשורת הרדאר עם המערכת. המשתנה lastRadarMessageTime שומר את הזמן שבו התקבלה ההודעה האחרונה מהרדאר. הקבוע RADAR_TIMEOUT מגדיר את פרק הזמן המרבי (במילישניות) שבו המערכת יכולה להמתין להודעה חדשה. אם לא מתקבלת הודעה בתוך זמן זה, המערכת מניחה כי חיישן הרדאר אינו פעיל או שהתקשורת נותקה.

שוורת 43-47: משתנים אלו משמשים לניהול קריאת הטמפרטורה מהחיישן. המשתנה lastTemp שומר את ערך הטמפרטורה האחרון שנמדד, כאשר ערך התחלתי נמוך במיוחד נבחר כדי לציין כי טרם התקבלה קריאה תקפה. המשתנה lastRead שומר את הזמן שבו בוצעה הקריאה האחרונה מהחיישן, ובכך מאפשר לשלוט בתדירות הקריאות ולמנוע קריאה רציפה ומהירה מדי מהחיישן.

שוורת 50-53: משתנים אלו משמשים לניהול קריאות מחיישן האור. המשתנה lastLight שומר את ערך עוצמת האור האחרון שנמדד, כאשר ערך התחלתי שלילי מציין כי טרם התקבלה קריאה. המשתנה lastLightRead שומר את הזמן שבו בוצעה הקריאה האחרונה מחיישן האור, ומאפשר שליטה על תדירות הדגימה של החיישן.

שוורת 55-60: מבנה הנתונים Button משמש לייצוג כפתורים גרפיים בממשק המשתמש על גבי המסך. המבנה כולל את מיקום הכפתור על המסך (x,y), את רוחבו וגובהו (w,h), (לואת הטקסט המוצג עליו (label). בנוסף מוגדרים צבע הרקע של הכפתור (fillColor) וצבע הטקסט (textColor) שימוש במבנה זה מאפשר ליצור ולנהל כפתורים בצורה מסודרת ואחידה בממשק הגרפי.

שוורת 62-66: קטע זה מנהל את זמני הפעולה של מערכת הירי. המשתנה lastFireTime שומר את הזמן שבו בוצעה פעולת הירי האחרונה. הקבוע RECHARGE_TIME מגדיר את פרק הזמן הנדרש לטעינה מחדש של המערכת לפני שניתן לבצע ירי נוסף. מנגנון זה מונע הפעלה רציפה של המערכת ומבטיח פעולה מבוקרת ובטוחה.

```

68 // ===== מעקב אחרי אובייקט =====
69 // הזמן האחרון שבו זוהה אובייקט
70 unsigned long lastObjectTime = 0;
71 // זמן שהלד נשאר דולק לאחר שהאובייקט נעלם
72 const int HOLD_TIME = 500;
73 // מצב הלד
74 bool ledIsOn = false;
75
76 // ===== מערכת הכיוון =====
77 // זמן התחלת הזיהוי
78 unsigned long detectionStartTime = 0;
79 // זמן המתנה עד שהסרנו מגיע למיקום
80 const int AIM_DELAY = 600;
81
82 // כפתורים - שמירה על כל המידות והקואורדינטות שלך
83 Button btnTemp = {0,0,200,60,"Temperature",TFT_RED,TFT_WHITE}; // טקסט וצבעים
84 Button btnLight = {0,0,200,60,"Light",TFT_GREEN,TFT_WHITE}; // הגדרת כפתור תאורה עם צבע ירוק וטקסט לבן
85 Button btnBack = {tft.width()-110, 193, 100, 40, "Back", TFT_DARKGREY, TFT_WHITE}; // כפתור חזרה שממוקם בצד ימין למטה של המסך
86 Button btnRadar = {0,0,200,60,"Radar",TFT_YELLOW,TFT_WHITE}; // כפתור מעבר למסך הרדאר עם צבע צהוב
87
88 enum Screen { MAIN, TEMP, LIGHT, RADAR }; // הגדרת סוג מסך אפשרי במערכת (מסך ראשי, טמפרטורה, תאורה ורדאר)
89 Screen currentScreen = MAIN; // משתנה שמכיל את המסך הנוכחי שמוצג על המסך
90
91 // ===== קריאת LM75 =====
92 float readLM75() { // פונקציה לקריאת הטמפרטורה מחיישן I2C
93     Wire.beginTransmission(LM75_ADDR); // שלול I2C-התחלת תקשורת עם החיישן לפי כתובת ה
94     Wire.write(0x00); // בחירת רגיסטר הטמפרטורה של החיישן
95     Wire.endTransmission(); // סיום שליחת הפקודה לחיישן
96     Wire.requestFrom(LM75_ADDR, 2); // בקשה לקבלת 2 בתים מחיישן (MSB ו LSB)
97     if (Wire.available() == 2) { // בדיקה שאכן התקבלו שני בתים של נתונים
98         int msb = Wire.read(); // קריאת הבית המשמעותי הגבוה (Most Significant Byte)
99         int lsb = Wire.read(); // קריאת הבית המשמעותי הנמוך (Least Significant Byte)
100         int raw = (msb << 8) | lsb; // חיבור שני הבתים למספר אחד שלם
101         return (raw >> 5) * 0.125; // יית הנתון הגולמי לטמפרטורה במעלות צלזיוס
102     }
103     return lastTemp; // הקריאה נכשלה - מחזירים את הטמפרטורה האחרונה שנמדדה
104 }
105
106 // פונקציה שמציירת כפתור על המסך
107 void drawButton(const Button &btn, bool pressed=false){ // מקבלת את נתוני הכפתור והאם הוא לחוץ
108     uint16_t color = pressed ? TFT_ORANGE : btn.fillColor; // אם הכפתור לחוץ הצבע יהיה כתום אחרת צבע הכפתור הרגיל
109     tft.fillRoundRect(btn.x, btn.y, btn.w, btn.h, 12, color); // מצייר מלבן עם פינות מעוגלות במיקום וגודל הכפתור
110     tft.drawRoundRect(btn.x, btn.y, btn.w, btn.h, 12, TFT_WHITE); // מצייר מסגרת לבנה סביב הכפתור
111     tft.setTextColor(btn.textColor, color); // מגדיר צבע טקסט וצבע רקע לטקסט
112     tft.setTextSize(3); // קובע את גודל הפונט
113     int txtW = btn.label.length() * 18; // מחשב רוחב משוער של הטקסט לפי מספר האותיות
114     int txtH = 24; // גובה משוער של הטקסט בפיקסלים
115     tft.setCursor(btn.x + (btn.w - txtW)/2, btn.y + (btn.h - txtH)/2); // מחשב מיקום כך שהטקסט יהיה באמצע הכפתור
116     tft.print(btn.label); // מציג את הטקסט של הכפתור
117 }
118
119 // פונקציה שמציירת טקסט על המסך
120 void drawText(const char* text, int x, int y, uint16_t color, int size){ // מקבלת טקסט מיקום צבע וגודל
121     tft.setTextColor(color, TFT_BLACK); // מגדיר צבע טקסט ורקע
122     tft.setTextSize(size); // קובע גודל פונט
123     tft.setCursor(x, y); // קובע את מיקום הטקסט במסך
124     tft.print(text); // מדפיס את הטקסט
125 }
126
127 // ===== מסך ראשי =====
128 void drawMainScreen(){ // פונקציה שמציירת את המסך הראשי של המערכת
129     tft.fillScreen(TFT_BLACK); // מנקה את המסך וצובע אותו בשחור
130     btnTemp.x = (tft.width() - btnTemp.w)/2; // מחשב את מיקום ה X-מחשב את מיקום ה
131     btnTemp.y = (tft.height() - btnTemp.h)/2 - 10; // מחשב את מיקום ה Y-מחשב את מיקום ה
132     drawButton(btnTemp); // מצייר את כפתור הטמפרטורה במסך
133     btnLight.x = btnTemp.x; // מגדיר שכפתור התאורה יהיה באותו קו
134     btnLight.y = btnTemp.y - btnLight.h - 5; // מחשב את כפתור התאורה מעל כפתור הטמפרטורה עם רווח קטן
135     drawButton(btnLight); // מצייר את כפתור התאורה
136     // כפתור Radar
137     btnRadar.x = (tft.width() - btnRadar.w)/2; // מחשב את כפתור הרדאר במרכז המסך בציר X
138     btnRadar.y = btnTemp.y + btnTemp.h + 5; // מחשב את כפתור הרדאר מתחת לכפתור הטמפרטורה עם רווח קטן
139     drawButton(btnRadar); // מצייר את כפתור הרדאר
140 }
141

```

שורות 68-74: ניהול את מצב זיהוי האובייקטים במערכת הרדאר. המשתנה lastObjectTime שומר את הזמן שבו זוהה האובייקט האחרון על ידי החיישן. הקבוע OBJECT_HOLD_TIME מגדיר את משך הזמן שבו המערכת ממשיכה להתייחס לאובייקט כקיים גם לאחר שהחיישן הפסיק לזהות אותו, ובכך מונע שינויי מצב מהירים הנגרמים מרעשי מדידה. המשתנה ledIsOn משמש למעקב אחר מצב נורת החיווי, ומציין האם נורת ה-LED דולקת או כבויה.

שורות 76-80: משתנים אלו משמשים לניהול תהליך הכיוון של המערכת לפני ביצוע פעולה. המשתנה detectionStartTime שומר את הזמן שבו התחיל זיהוי האובייקט. הקבוע AIM_DELAY מגדיר פרק זמן חשיה המאפשר למערכת להתייבב ולכוון את עצמה לעבר המטרה לפני הפעלת מנגנון הירי.

שורות 88-99: סוגי המסכים האפשריים בממשק המשתמש באמצעות מבנה enum, ההגדרה כוללת ארבעה מסכים עיקריים: המסך הראשי (MAIN), מסך הטמפרטורה (TEMP), מסך חיישן האור (LIGHT) ומסך הרדאר (RADAR). המשתנה currentScreen שומר את המסך הפעיל כרגע ומאפשר לתוכנה לדעת איזה ממשק להציג למשתמש בכל רגע נתון.

שורות 91-104: פונקציה זו אחראית לקריאת הטמפרטורה מחיישן LM75 באמצעות פרוטוקול התקשורת I2C. בתחילת הפעולה המיקר בקר פותח תקשורת עם החיישן ושולח בקשה לקרוא את רגיסטר הטמפרטורה. לאחר מכן המערכת מבקשת שני בתים של נתונים מהחיישן.

הבייט הראשון (MSB) והבייט השני (LSB) מחוברים יחד לערך מספרי אחד המייצג את קריאת החיישן. לאחר עיבוד הנתונים מתבצעת המרה לערך טמפרטורה במעלות צלזיוס בהתאם למבנה הנתונים של החיישן. במידה ולא התקבלה קריאה תקינה מהחיישן, הפונקציה מחזירה את ערך הטמפרטורה האחרון שנמדד.

שורות 106-117: פונקציה זו אחראית לציור כפתור גרפי על מסך ה-TFT כחלק מממשק המשתמש של המערכת. הפונקציה מקבלת מבנה נתונים מסוג Button המכיל את מאפייני הכפתור כגון מיקום, גודל, טקסט וצבעים. בנוסף ניתן לציין האם הכפתור במצב לחוץ. במקרה כזה צבע הכפתור משתנה לכתום כדי לספק משוב חזותי למשתמש. הפונקציה מציירת מלבן מעוגל המייצג את גוף הכפתור, מוסיפה מסגרת לבנה, ומציגה את הטקסט במרכז הכפתור על ידי חישוב מיקום הטקסט ביחס למידות הכפתור. ניתן ליצור ממשק משתמש אחיד וברור על המסך.

שורות 119-125: פונקציה זו מציגה טקסט על מסך ה-TFT. היא מקבלת את הטקסט, מיקום ההצגה על המסך (x, y), צבע הטקסט וגודל הגופן. הפונקציה מגדירה את צבע וגודל הטקסט, קובעת את מיקום ההדפסה באמצעות setCursor ולאחר מכן מציגה את הטקסט על המסך בעזרת print.

שורות 127-140: פונקציה זו מציירת את המסך הראשי של המערכת על מסך ה-TFT. בתחילה המסך נצבע בשחור באמצעות fillScreen כדי לנקות את התצוגה. לאחר מכן מחושב מיקום הכפתורים כך שיופיעו במרכז המסך. כפתור הטמפרטורה ממוקם ראשון, מתחתיו כפתור האור ולבסוף כפתור הרדאר. לאחר חישוב המיקום, הפונקציה drawButton מציירת כל כפתור על המסך וכך נוצר ממשק ראשי שמאפשר למשתמש לבחור בין פונקציות המערכת.


```

142 // ===== מסך טמפרטורה =====
143 void drawTempScreen(){ // פונקציה שמציירת את מסך הטמפרטורה
144     tft.fillScreen(TFT_BLACK); // מנקה את המסך וצובע אותו בשחור
145     drawText("Temperature is:", 10, 50, TFT_YELLOW, 2); // מציג כותרת שמראה שמדובר במסך טמפרטורה
146     drawButton(btnBack); // מצייר את כפתור החזרה למסך הראשי
147     lastTemp = -1000; // מאפס את ערך הטמפרטורה האחרון כדי לכפות עדכון חדש
148 }
149
150 // ===== עדכון טמפרטורה =====
151 void updateTemperature() { // פונקציה שמעדכנת את הטמפרטורה במסך
152     if (millis() - lastRead < 300) return; // לא נבצע קריאה חדשה אם עברו פחות מ-300 מילישניות
153     lastRead = millis(); // שמירת זמן הקריאה האחרון
154     float t = readLM75(); // קריאת הטמפרטורה מחיישן LM75
155     if (abs(t - lastTemp) >= 0.1) { // עדכון רק אם הטמפרטורה השתנתה לפחות ב-0.1 מעלות
156         lastTemp = t; // שמירת הטמפרטורה החדשה
157         tft.fillRect(10, 75, 200, 40, TFT_BLACK); // ניקוי האזור במסך שבו מוצגת הטמפרטורה
158         tft.setTextColor(TFT_WHITE, TFT_BLACK); // צבע טקסט לבן עם רקע שחור
159         tft.setTextSize(3); // גודל הטקסט
160         tft.setCursor(10, 75); // מיקום הטקסט במסך
161         tft.print(t, 1); // הדפסת ערך הטמפרטורה עם ספרה אחת אחרי הנקודה
162         tft.print(" C"); // הדפסת יחידת המידה צלזיוס
163     }
164 }
165
166 // ===== מסך תאורה =====
167 void drawLightScreen() { // פונקציה שמציירת את מסך התאורה
168     tft.fillScreen(TFT_BLACK); // ניקוי המסך
169     drawText("Light is:", 10, 50, TFT_YELLOW, 2); // הצגת כותרת למסך התאורה
170     drawButton(btnBack); // ציור כפתור חזרה למסך הראשי
171     lastLight = -1; // איפוס ערך התאורה האחרון כדי לכפות עדכון חדש
172 }
173
174 // ===== עדכון תאורה =====
175 void updateLight(){ // פונקציה שמעדכנת את ערך התאורה ומפעילה את הפנס לפי הצורך
176     if (millis() - lastLightRead < 300) return; // לא נבצע קריאה חדשה אם עברו פחות מ-300 מילישניות
177     lastLightRead = millis(); // שמירת הזמן הנוכחי כזמן הקריאה האחרון
178     int lightVal = analogRead(LIGHT_PIN); // קריאת ערך התאורה מחיישן האור
179     // לוגיקת הפנס: כאשר יש הרבה אור (ערך גבוה) הפנס נדלק
180     if (lightVal > 3000) {
181         digitalWrite(RELAY_PIN, LOW); // הדלקת הפנס באמצעות הממסר
182     } else {
183         digitalWrite(RELAY_PIN, HIGH); // כיבוי הפנס
184     }
185     if (lightVal != lastLight) { // בדיקה אם ערך התאורה השתנה מאז הקריאה הקודמת
186         lastLight = lightVal; // שמירת ערך התאורה החדש
187         // התנאי הזה מוודא שהמספרים יוצגו רק במסך התאורה
188         if (currentScreen == LIGHT) {
189             tft.fillRect(10, 75, 200, 40, TFT_BLACK); // ניקוי האזור שבו מוצג ערך התאורה
190             tft.setTextColor(TFT_WHITE, TFT_BLACK); // קביעת צבע הטקסט ללבן עם רקע שחור
191             tft.setTextSize(3); // קביעת גודל הטקסט
192             tft.setCursor(10, 75); // מיקום הדפסת המספר במסך
193             tft.print(lightVal); // הדפסת ערך התאורה
194         }
195     }
196 }
197
198 // ===== מסך רדאר =====
199 void drawRadarScreen(){ // פונקציה שמציירת את מסך הרדאר
200     tft.fillScreen(TFT_BLACK); // ניקוי המסך וצבע רקע שחור
201     drawText("Radar Status:", 10, 20, TFT_YELLOW, 2); // הצגת כותרת למסך הרדאר
202     // כפתור חזרה למסך הראשי
203     drawButton(btnBack); // ציור כפתור חזרה
204     radarScreenJustOpened = true; // סימון שהמסך נפתח כדי לעדכן את התצוגה בהמשך
205 }
206

```

שורות 142-148: פונקציה זו מציגה את מסך הטמפרטורה במערכת. בתחילה המסך מתנקה ונצבע בשחור, לאחר מכן מוצגת כותרת למסך באמצעות drawText. בנוסף מצויר כפתור חזרה (Back) שמאפשר לחזור למסך הראשי. לבסוף מאופס המשתנה lastTemp כדי לאלץ עדכון חדש של ערך הטמפרטורה.

שורות 150-164: פונקציה זו מעדכנת את ערך הטמפרטורה המוצג במסך. בתחילה נבדק אם עבר מספיק זמן מאז הקריאה האחרונה, כדי למנוע קריאות מהירות מדי מהחיישן. לאחר מכן מתבצעת קריאה חדשה מחיישן LM75. אם ערך הטמפרטורה השתנה באופן משמעותי, המסך מתעדכן: האזור שבו מוצגת הטמפרטורה נמחק, ולאחר מכן מודפס הערך החדש עם יחידות המידה במעלות צלזיוס.

שורות 166-172: פונקציה זו מציגה את מסך חיישן האור. תחילה המסך מתנקה ונצבע בשחור, לאחר מכן מוצגת כותרת למסך באמצעות drawText. בנוסף מצויר כפתור חזרה למסך הראשי. המשתנה lastLight מאופס כדי לאפשר עדכון חדש של ערך התאורה בקריאה הבאה מהחיישן.

שורות 174-196: פונקציה זו מעדכנת את ערך התאורה הנמדד מחיישן האור. בתחילה נבדק אם עברו לפחות 300 מילישניות מאז הקריאה הקודמת, כדי למנוע דגימה מהירה מדי של החיישן. לאחר מכן מתבצעת קריאה אנלוגית מהפין המחובר לחיישן האור. אם ערך התאורה גבוה מסף שנקבע, הממסר מופעל ומכבה את הפנס; אחרת הממסר מכובה והפנס נדלק. כאשר ערך התאורה משתנה, הערך החדש נשמר במשתנה lastLight. אם מסך התאורה פתוח באותו רגע, התצוגה מתעדכנת והערך החדש מוצג על המסך.

שורות 198-205: פונקציה זו מציגה את מסך הרדאר במערכת. בתחילה המסך מתנקה ונצבע בשחור באמצעות fillScreen. לאחר מכן מוצגת כותרת למסך בעזרת הפונקציה drawText המציגה את מצב הרדאר. בנוסף מצויר כפתור חזרה (Back) שמאפשר למשתמש לחזור למסך הראשי. לבסוף המשתנה radarScreenJustOpened מוגדר כ-true כדי לציין שהמסך נפתח זה עתה ולאפשר ביצוע פעולות אתחול או עדכון נתונים בפעם הראשונה שהמסך מוצג.

```

207 void onDataRecv(const esp_now_recv_info_t *info, const uint8_t *incomingData, int len) {
208     // פונקציה שנקציה שנקראת כאשר מתקבלת הודעה מ ESP-NOW
209     שמירת הזמן שבו התקבלה ההודעה האחרונה מהרדאר
210     lastRadarMessageTime = millis(); // יצירת משתנה למבנה הנתונים של הודעת הרדאר
211     RadarMessage msg; // העתקת הנתונים שהתקבלו לתוך המשתנה msg
212     memcpy(&msg, incomingData, sizeof(msg)); // שמירת המרחק או מצב הרדאר שהתקבל
213     lastRadarDistance = msg.state; // בדיקה אם המרחק נמצא בתחום שמוגדר כזיהוי אובייקט
214     bool rawDetection = (lastRadarDistance < 100 && lastRadarDistance > 2);
215     if (rawDetection) { // אם זוהה אובייקט
216         // אם זו תחילתו של זיהוי חדש נשמור את זמן ההתחלה
217         if (detectionStartTime == 0) {
218             detectionStartTime = millis(); // שמירת הזמן שבו התחיל הזיהוי
219         }
220         lastObjectTime = millis(); // שמירת הזמן האחרון שבו נראה האובייקט
221     } else { // אם אין זיהוי אובייקט
222         detectionStartTime = 0; // איפוס זמן תחילת הזיהוי
223     }
224     // --- בדיקה האם הסרוו הספיק להגיע למטרה (זמן כיוון) ---
225     bool isAimingComplete = (detectionStartTime != 0 && (millis() - detectionStartTime >= AIM_DELAY)); // בדיקה אם עבר זמן הכיוון
226     if (isAimingComplete && firstRadarIgnored) { // אם הכיוון הושלם וגם כבר התעלמנו מהזיהוי הראשון
227         if (!ledIsOn) { // אם הLED עדיין כבוי
228             digitalWrite(LED_RADAR_PIN, HIGH); // הדלקת הLED שמראה שיש נעילה על מטרה
229             ledIsOn = true; // עדכון מצב הLED
230             Serial.println("TARGET LOCKED - SERVO REACHED DESTINATION"); // הדפסת הודעה למוניטור
231         }
232         // --- בדיקה אם הקבל כבר נטען מספיק זמן לירי ---
233         bool isCharged = (millis() - lastFireTime >= RECHARGE_TIME); // בדיקה אם עבר זמן הטעינה
234         if (isCharged) { // אם הקבל טעון
235             Serial.println("FIRING!"); // הדפסת הודעת ירי
236             digitalWrite(RELAY_GUN_PIN, LOW); // הפעלת הממסר שמבצע את הירי
237             delay(150); // זמן קצר להפעלת הירי
238             digitalWrite(RELAY_GUN_PIN, HIGH); // החזרת הממסר למצב מנוחה
239             lastFireTime = millis(); // שמירת הזמן שבו בוצע הירי האחרון
240         }
241     } else { // אם אין זיהוי יציב או שהסרוו עדיין לא הגיע למטרה
242         // כיבוי הLED אם האובייקט נעלם למשך זמן מסוים
243         if (millis() - lastObjectTime > HOLD_TIME && ledIsOn) {
244             digitalWrite(LED_RADAR_PIN, LOW); // כיבוי הLED
245             ledIsOn = false; // עדכון מצב הLED
246         }
247     }
248     // --- טיפול בזיהוי הראשון לאחר הפעלת המערכת ---
249     if (rawDetection && !firstRadarIgnored) { // אם זו הפעם הראשונה שמתקבל זיהוי
250         firstRadarIgnored = true; // מסמנים שהתעלמנו מהזיהוי הראשון
251         detectionStartTime = 0; // איפוס זמן הזיהוי כדי שלא יתבצע ירי מיד
252     }
253     if (rawDetection) objectEverDetected = true; // סימון שבמהלך הפעולה לפחות פעם אחת זוהה אובייקט
254 }
255
256 void updateRadarScreen() { // פונקציה שמעדכנת את מסך הרדאר לפי מצב הזיהוי
257     static bool lastDisplayedState = false; // משתנה סטטי ששומר את מצב הזיהוי האחרון שהוצג במסך
258     bool objectCurrentlyDetected = (lastRadarDistance < 100 && lastRadarDistance > 2); // בדיקה אם כרגע מזוהה אובייקט לפי המרחק
259     if (radarScreenJustOpened) { // אם מסך הרדאר נפתח עכשיו
260         lastDisplayedState = !objectCurrentlyDetected; // סינוי הערך כדי להכריח עדכון ראשון של התצוגה
261         radarScreenJustOpened = false; // סימון שהמסך כבר לא חדש
262     }
263     if (objectCurrentlyDetected != lastDisplayedState) { // אם מצב הזיהוי השתנה מאז הפעם האחרונה שהוצג
264         tft.fillRect(10, 50, 220, 40, TFT_BLACK); // ניקוי האזור שבו מוצג מצב הרדאר
265         tft.setTextSize(2); // הגדלת גודל הטקסט
266         tft.setCursor(10, 50); // מיקום הטקסט במסך
267         if (objectCurrentlyDetected) { // אם זוהה אובייקט
268             tft.setTextColor(TFT_RED, TFT_BLACK); // צבע טקסט אדום עם רקע שחור
269             tft.print("Object detected"); // הדפסת הודעה שמראה שיש אובייקט
270         } else { // אם אין אובייקט
271             tft.setTextColor(TFT_WHITE, TFT_BLACK); // צבע טקסט לבן
272             tft.print("No object"); // הדפסת הודעה שאין אובייקט
273         }
274         lastDisplayedState = objectCurrentlyDetected; // שמירת המצב שהוצג כדי למנוע עדכון מיותר
275     }
276 }

```

שורות
:207-230
פונקציה זו מופעלת כאשר מתקבלת הודעה מחיישן הרדאר באמצעות פרוטוקול התקשורת ESP-NOW.

בתחילה נשמר זמן קבלת ההודעה מהרדאר והנתונים מועתקים למבנה RadarMessage שממנו מתקבל ערך המרחק. המערכת בודקת אם המרחק מצביע על זיהוי אובייקט ושומרת את זמן הזיהוי. אם עבר זמן החשייה (AIM_DELAY) נדלקת נורת החיישן ומודפסת הודעה שהמטרה ננעלה.

שורות 231-239: בודק אם עבר זמן הטעינה מחדש (RECHARGE_TIME) מאז הירי האחרון. אם המערכת מוכנה לירי, מודפסת הודעה למסך הסריאלי, הממסר מופעל לזמן קצר כדי לבצע ירי, ולאחר מכן זמן הירי האחרון מתעדכן. **שורות 240-246:** אחראי לכיבוי נורת החיישן של הרדאר כאשר לא זוהה אובייקט במשך זמן מסוים (HOLD_TIME) אם התנאי מתקיים והנורה דולקת, היא נכבית והמצב מתעדכן. **שורות 247-251:** גורם למערכת להתעלם מהזיהוי הראשון של הרדאר. פעולה זו נועדה למנוע תגובה שגויה בזמן האתחול, כאשר החיישן עדיין מתייצב. **שורה 252:** קטע זה גורם למערכת להתעלם מהזיהוי הראשון של הרדאר. פעולה זו נועדה למנוע תגובה שגויה בזמן האתחול, כאשר החיישן עדיין מתייצב.

שורות 255-275: פונקציה זו אחראית לעדכן את המידע המוצג במסך הרדאר בהתאם לנתונים המתקבלים מהחיישן. בתחילה מוגדר משתנה סטטי (lastDisplayedState) ששומר את מצב הזיהוי האחרון שהוצג על המסך, כדי לאפשר השוואה למצב החדש ולמנוע עדכון מיותר של התצוגה. לאחר מכן מתבצעת בדיקה האם הרדאר מזוהה אובייקט לפי ערך המרחק שנמדד (lastRadarDistance). אם המרחק נמצא בתחום שהוגדר, המערכת מתייחסת לכך כזיהוי אובייקט. כאשר מסך הרדאר נפתח בפעם הראשונה (radarScreenJustOpened) מאפסת את מצב התצוגה כדי להבטיח שהמידע יוצג מחדש באופן תקין.

בהמשך נבדק האם מצב הזיהוי הנוכחי שונה מהמצב שהוצג קודם. אם כן, האזור במסך שבו מוצג המידע מתנקה, ולאחר מכן מודפס הטקסט המתאים: אם זוהה אובייקט מוצגת הודעה "Object detected" בצבע אדום, ואם לא זוהה אובייקט מוצגת הודעה "No object" בצבע לבן. לבסוף נשמר המצב החדש במשתנה lastDisplayedState כדי לשמש להשוואה בעדכון הבא.


```

277 // ===== setup =====
278 void setup() { // הרכיבים את כל הרכיבים
279   Serial.begin(115200); // לצורך הדפסות ודיבוג
280   lastFireTime = millis(); // טעינת הקבל מהרגע שהמערכת נדלקת
281   Serial.println("Receiver starting..."); // התחלת הודעה למוניטור הסריאלי שהמערכת מתחילה לפעול
282   Wire.begin(SDA_PIN, SCL_PIN); // התחלת תקשורת I2C
283   tft.init(); // תחילת התחבורה מסך ה-TFT
284   tft.setRotation(1); // קביעת כיוון התצוגה של המסך
285   WiFi.mode(WIFI_STA); // בלבד WiFi כתחנת ESP32-הגדרת ה
286   if (esp_now_init() != ESP_OK) { // ניסיון לאתחול את מערכת התקשורת ESP-NOW
287     Serial.println("Error initializing ESP-NOW"); // הדפסת הודעת שגיאה אם האתחול נכשל
288     return; // לא הצליח לפעול ESP-NOW עצירת הפונקציה אם
289   }
290   esp_now_register_recv_cb(onDataRecv); // שתופעל כאשר מתקבלת הודעה callback רישום פונקציית
291   pinMode(TOUCH_IRQ, INPUT); // הגדרת פין האינטרפט של מסך המגע כקלט
292   lastRadarDistance = 999; // ערך גבוה שמייצג מצב התחלתי של "אין אובייקט"
293   drawMainScreen(); // ציור המסך הראשי של המערכת
294
295   pinMode(RELAY_PIN, OUTPUT); // הגדרת פין הממסר של הפנס כיציאה
296   digitalWrite(RELAY_PIN, HIGH); // כיבוי הפנס בהתחלה
297
298   pinMode(RELAY_GUN_PIN, OUTPUT); // הגדרת פין הממסר של מנגנון הירי כיציאה
299   digitalWrite(RELAY_GUN_PIN, HIGH); // מצב נצירה - התותח כבוי
300
301   pinMode(LED_RADAR_PIN, OUTPUT); // הגדרת פין הLED שמראה זיהוי אובייקט
302   digitalWrite(LED_RADAR_PIN, LOW); // הLED כבוי בתחילת העבודה
303 }
304
305 // ===== loop =====
306 void loop() { // הפונקציה הראשית שרצה שוב ושוב כל עוד המערכת פועלת
307   // בדיקה אם הרדאר מנותק (לא התקבלה הודעה זמן מסוים)
308   if (millis() - lastRadarMessageTime > RADAR_TIMEOUT) { // אם עבר יותר מדי זמן מאז ההודעה האחרונה
309     digitalWrite(LED_RADAR_PIN, LOW); // כיבוי הLED שמראה זיהוי
310     ledIsOn = false; // עדכון מצב הLED במערכת
311     lastRadarDistance = 999; // ערך גבוה שמייצג שאין אובייקט בסווח
312     detectionStartTime = 0; // איפוס זמן תחילת הזיהוי
313   }
314   updateLight(); // עדכון חיישן האור והפעלת הפנס לפי הצורך
315   uint16_t tx, ty; // משתנים שישמרו את מיקום הלחיצה במסך המגע
316
317   // עדכון המסך לפי המסך הפעיל כרגע
318   if (currentScreen == TEMP) { // אם המסך הנוכחי הוא מסך הטמפרטורה
319     updateTemperature(); // עדכון הטמפרטורה המוצגת במסך
320   }
321   else if (currentScreen == LIGHT) { // אם המסך הנוכחי הוא מסך התאורה
322     updateLight(); // עדכון ערך התאורה במסך
323   }
324   else if (currentScreen == RADAR) { // אם המסך הנוכחי הוא מסך הרדאר
325     updateRadarScreen(); // עדכון מצב זיהוי האובייקט במסך הרדאר
326   }
327
328   // בדיקה אם יש לחיצה במסך המגע
329   if (digitalRead(TOUCH_IRQ) == LOW) { // אם התקבלה הפרעת לחיצה מהמסך
330     if (tft.getTouch(&tx, &ty, 600)) { // קריאת מיקום הלחיצה במסך
331       אם אנחנו במסך הראשי
332       if (currentScreen == MAIN) {
333         if (tx >= btnTemp.x && tx <= btnTemp.x + btnTemp.w &&
334             ty >= btnTemp.y && ty <= btnTemp.y + btnTemp.h) {
335           drawButton(btnTemp, true); // ציור הכפתור במצב לחוץ
336           delay(100); // השהיה קצרה לאפקט לחיצה
337
338           currentScreen = TEMP; // מעבר למסך הטמפרטורה
339           drawTempScreen(); // ציור מסך הטמפרטורה
340           while (tft.getTouch(&tx, &ty, 600)); // המתנה עד שהמשתמש משחרר את המסך
341         }
342         else if (tx >= btnRadar.x && tx <= btnRadar.x + btnRadar.w &&
343             ty >= btnRadar.y && ty <= btnRadar.y + btnRadar.h) {
344           drawButton(btnRadar, true); // ציור הכפתור במצב לחוץ
345           delay(100); // השהיה קצרה
346           currentScreen = RADAR; // מעבר למסך הרדאר
347           objectDetected = false; // איפוס מצב זיהוי
348           objectEverDetected = false; // איפוס היסטוריית זיהוי
349           drawRadarScreen(); // ציור מסך הרדאר
350           while (tft.getTouch(&tx, &ty, 600)); // המתנה לשחרור הלחיצה
351         }
352       }
353     }
354   }

```

שורות 278-303: פונקציה זו מתבצעת פעם אחת עם הפעלת המערכת ומטרתה לאתחול את כל רכיבי החומרה והתוכנה הדרושים לפעולת המערכת. בתחילה מופעלת התקשורת הסריאלית במהירות של 115200 לצורך הדפסת הודעות בדיקה וניטור פעולת המערכת. לאחר מכן נשמר זמן הירי האחרון, ומודפסת הודעה המציינת שהמערכת מתחילה לפעול. בהמשך מתבצע אתחול של תקשורת I2C באמצעות הפינים שהוגדרו (SDA_PIN, SCL_PIN) לצורך תקשורת עם חיישנים המחוברים למערכת.

לאחר מכן מתבצע אתחול של מסך ה-TFT ונקבעת זווית התצוגה שלו. המערכת מוגדרת לעבוד במצב **WiFi Station** דבר המאפשר שימוש בפרוטוקול **ESP-NOW** לתקשורת אלחוטית עם מודול **ESP32** נוסף. לאחר מכן מתבצע אתחול של פרוטוקול **ESP-NOW** ובמידה והאתחול נכשל מודפסת הודעת שגיאה והפונקציה מופסקת. לאחר אתחול התקשורת נרשמת פונקציית **onDataRecv** כפונקציית **callback**, כלומר פונקציה שתופעל אוטומטית כאשר מתקבלת הודעה מהרדאר. בהמשך מוגדר פין המסך למגע (**TOUCH_IRQ**) כקלט, ומוגדר ערך התחלתי גדול למרחק הרדאר. לאחר מכן מצויר המסך הראשי של המערכת. לבסוף מוגדרים פני הפלט של הממסרים ונורת החיווי. פין הממסר של הפנס ופין מנגנון הירי מוגדרים כפלט ומכובים בתחילת העבודה, וכן פין נורת החיווי של הרדאר מוגדר כפלט ומכובה. פעולה זו מבטיחה שהמערכת תתחיל את פעולתה במצב בטוח ויציב.

שורות 305-326: פונקציית **loop** היא הלולאה הראשית של התוכנית והיא פועלת ברציפות כל עוד המערכת פעילה. בתחילה נבדק האם עבר זמן רב מאז התקבלה הודעה מהרדאר. אם כן, המערכת מניחה שהרדאר אינו מחובר, מכבה את נורת החיווי ומאפסת את נתוני הזיהוי. לאחר מכן מתבצע עדכון של חיישן האור. בהמשך הפונקציה בודקת איזה מסך מוצג כעת במערכת, ומעדכנת את הנתונים המתאימים: אם מוצג מסך הטמפרטורה מתבצע עדכון של הטמפרטורה, אם מוצג מסך התאורה מתבצע עדכון של ערך האור, ואם מוצג מסך הרדאר מתבצע עדכון של מצב זיהוי האובייקטים.

שורות 328-351: בודק האם המשתמש נגע במסך המגע. כאשר מתקבלת נגיעה, הפונקציה **getTouch** קוראת את מיקום הלחיצה ושומרת את הקואורדינטות במשתנים **tx** ו**ty**. לאחר מכן המערכת בודקת האם הלחיצה נמצאת באזור של אחד הכפתורים במסך הראשי. אם נלחץ כפתור הטמפרטורה, המערכת מציגה את הכפתור במצב לחוץ, עוברת למסך הטמפרטורה ומציירת אותו. אם נלחץ כפתור הרדאר, המערכת עוברת למסך הרדאר, מאפסת את משתני הזיהוי ומציגה את מסך הרדאר. לבסוף המערכת ממתינה לשחרור הלחיצה כדי למנוע זיהוי כפול של אותה נגיעה.

```

352 else if (tx >= btnLight.x && tx <= btnLight.x + btnLight.w && // בדיקה אם נלחץ כפתור התאורה
353         | | | ty >= btnLight.y && ty <= btnLight.y + btnLight.h) {
354     drawButton(btnLight, true); // ציור הכפתור במצב לחץ
355     delay(100); // השהיה קצרה
356     currentScreen = LIGHT; // מעבר למסך התאורה
357     drawLightScreen(); // ציור מסך התאורה
358     while (tft.getTouch(&tx, &ty, 600)); // המתנה לשחרור הלחיצה
359 }
360 } else { // אם אנחנו במסך משני (לא הראשי)
361     if (tx >= btnBack.x && tx <= btnBack.x + btnBack.w && // בדיקה אם נלחץ כפתור החזרה
362         | | | ty >= btnBack.y && ty <= btnBack.y + btnBack.h) {
363         drawButton(btnBack, true); // ציור כפתור החזרה במצב לחץ
364         delay(100); // השהיה קצרה
365         currentScreen = MAIN; // חזרה למסך הראשי
366         drawMainScreen(); // ציור המסך הראשי מחדש
367         while (tft.getTouch(&tx, &ty, 600)); // המתנה לשחרור הלחיצה
368     }
369 }
370 delay(50); // השהיה קטנה כדי למנוע קריאות כפולות של לחיצה
371 }
372 }
373 }
374 }

```

שורות 352–371: קטע קוד זה מטפל בלחיצה על כפתור התאורה וכפתור החזרה. אם המשתמש לחץ באזור של כפתור התאורה, הכפתור מוצג במצב לחץ, המערכת עוברת למסך התאורה ומציירת אותו על המסך. אם המשתמש נמצא במסך אחר ולוחץ על כפתור החזרה, הכפתור מוצג במצב לחץ והמערכת חוזרת למסך הראשי ומציירת אותו מחדש. לאחר כל לחיצה המערכת ממתינה לשחרור הנגיעה במסך כדי למנוע זיהוי כפול של אותה לחיצה. בנוסף קיימת השהיה קצרה בסוף הלולאה כדי למנוע קריאות מהירות מדי של מסך המגע.

זהו הקוד עבור מערכת הבקרה והניטור שפועלת בתור SLAVE ומקבלת ערכים ממערכת הרדאר, הנוסף מפעילה את הלד, התותח והפרוז'קטור כפי שהראתי בכל אחד מהתיעודים.

ייתכן כי יבוצעו שינויים קטנים בקוד לקראת מועד הבחינה, כגון התאמות בערכי החיישנים או בטווחי הסיבוב של המנועים.

➤ לחצו כאן לחזרה לתוכן העניינים

רפלקציה על התהליך ועל התוצר

במהלך ביצוע הפרויקט פיתחתי מערכת הגנה אקטיבית וניטור סביבתי בשם **SantiBot**, המשלבת מספר תתי-מערכות הפועלות יחד בזמן אמת. המערכת כוללת מערכת מכ"ם המבוססת על חיישן אולטרסאונד ומנוע סרור לסריקת שטח של 180 מעלות, מערכת תגובה המדמה תותח אלקטרומגנטי, וכן מערכת ניטור סביבתי הכוללת חיישני תאורה וטמפרטורה וממשק משתמש במסך מגע.

בתחילת העבודה היו מספר נושאים שלא היו ברורים לחלוטין. בין היתר היה צורך להבין כיצד לשלב בין מספר רכיבים שונים במערכת אחת – חיישנים, מנועים, מסכים ומערכות תקשורת. בנוסף, היה צורך ללמוד כיצד לנהל תקשורת אלחוטית בין שני בקרי ESP32 באמצעות פרוטוקול ESP-NOW וכיצד להציג נתונים בזמן אמת במסכי TFT תוך שמירה על ביצועי מערכת יציבים.

במהלך הפרויקט נתקלתי במספר קשיים טכניים. אחד האתגרים המרכזיים היה יצירת סנכרון מדויק בין מערכת הזיהוי לבין מערכת התגובה. היה צורך לוודא שהרדאר מזהה את האובייקט בצורה יציבה לפני שהתותח מגיב אליו, על מנת למנוע הפעלות שגויות. בנוסף, התמודדות עם תנועת מנועי הסרור תוך ביצוע מדידות מרחק מדויקות דרשה התאמות בקוד ובתזמון הפעולות. קושי נוסף היה בשילוב מספר חיישנים במערכת אחת תוך שמירה על קריאה יציבה של הנתונים. את הקשיים פתרתי באמצעות תהליך של ניסוי וטעייה, בדיקות חוזרות ושיפור הדרגתי של הקוד. נעשה שימוש בהדפסות ל-Serial Monitor לצורך איתור תקלות, וכן בוצעו התאמות בתזמון הפעולות ובמבנה התוכנה. בנוסף, נעשתה הפרדה בין מודולים שונים בקוד על מנת לשפר את יציבות המערכת ואת הקריאות של התוכנית.

במהלך העבודה על הפרויקט למדתי רבות על שילוב מערכות אלקטרוניות מורכבות. רכשתי ידע מעשי בעבודה עם מיקרו-בקרים מסוג ESP32 בתקשורת אלחוטית בין בקרים, בקריאת נתונים מחיישנים ובשליטה במנועים וממסרים. בנוסף, למדתי כיצד ליצור ממשק משתמש גרפי במסך TFT המאפשר הצגת מידע בצורה ברורה למפעיל.

מעבר לכך, במהלך הפרויקט רכשתי מיומנויות נוספות שלא היו חלק מרכזי בתוכנית הלימודים. לצורך הצגת הפרויקט והנגשתו יצרתי אתר אינטרנט ייעודי לפרויקט. את יכולת בניית האתרים למדתי באופן עצמאי באמצעות קורס מקוון, בעזרת למידה לעומק של השפות HTML, CSS ו-JavaScript כחלק מקישור הסרטונים לאתר. דבר שאפשר לי להציג את הפרויקט בצורה מקצועית יותר ולהמחיש את תהליך הפיתוח, הרכיבים והמערכת כולה.

בנוסף, במהלך בניית המערכת פיתחתי מיומנויות הלחמה וחיווט ברמה מקצועית יותר. עבודות ההלחמה בוצעו בעיקר במערכת הרובה האלקטרומגנטית ובחיבורים החשמליים הנלווים אליו, וכן בחיווט של רכיבי המערכת השונים. העבודה כללה חיבור יציב של חוטים, חיבורי מתח ובקרה בין הבקרים, הממסרים והרכיבים האלקטרוניים. מיומנויות אלו אפשרו לי להרכיב מערכת פיזית יציבה ואמינה, ולהפוך רעיון תיאורטי וחזון הנדסי למערכת מתפקדת במציאות.

המיומנויות המרכזיות שרכשתי במהלך הפרויקט כוללות תכנון מערכת הנדסית מלאה, כתיבת קוד בשפת Arduino עבור מערכות משובצות מחשב, כתיבת קוד בשפות HTML, CSS, JavaScript כחלק מתהליך הקמת האתר פתרון תקלות טכניות, וכן שילוב בין תחומי האלקטרוניקה, התוכנה והמכניקה. בנוסף פיתחתי חשיבה מערכתית המאפשרת להבין כיצד רכיבים שונים משפיעים זה על זה בתוך מערכת מורכבת.

ניהול הפרויקט באופן עצמאי אפשר לי ללמוד לעומק כל רכיב במערכת ולהתמודד עם בעיות טכניות באופן שיטתי. מצד אחד הדבר דרש השקעת זמן רבה יותר, אך מצד שני הוא תרם להבנה עמוקה יותר של אופן פעולת המערכת ושל עקרונות התכנון ההנדסי.

הפרויקט חיבר בין ידע תיאורטי שנלמד במהלך הלימודים לבין יישום מעשי בעולם האמיתי. הוא משלב נושאים מתחומי המיקרו-בקרים, מערכות חיישנים, תקשורת אלחוטית ומערכות בקרה.

בנוסף, ניתן לקשר את הטכנולוגיות שבהן השתמשתי לתחומים רבים כגון מערכות אבטחה, רובוטיקה, מערכות צבאיות ומערכות ניטור סביבתי.

נושא הפרויקט מתקשר גם לחיי היום-יום, שכן מערכות זיהוי וניטור אוטומטיות משמשות כיום במגוון רחב של יישומים – החל ממערכות אבטחה ועד מערכות ניטור תעשייתיות, צבאיות ומערכות רובוטיות.

לסיכום, העבודה על הפרויקט תרמה רבות לפיתוח הידע והיכולות ההנדסיות שלי. הפרויקט דרש תכנון, יצירתיות ופתרון בעיות בזמן אמת, והעניק ניסיון מעשי חשוב בתכנון ובפיתוח מערכות אלקטרוניות מורכבות. מעבר לכך, הפרויקט חיזק את היכולת שלי ללמוד תחומים חדשים באופן עצמאי, לפתח מיומנויות טכניות מתקדמות ולתרגם רעיון הנדסי למערכת מתפקדת במציאות.

➤ לחצו כאן לחזרה לתוכן העניינים

ביבליוגרפיה:

במהלך הפרויקט שלי נעזרתי במספר מקורות באינטרנט כדי לרכז ולסכם את המידע בצורה הכי פרקטית ומקצועית שאפשר. נעזרתי האתרים שיצא לי ללמוד מהם ולרכז מידע מדויק שעזר לי לקדם את הספר פרויקט.

האתרים שבעיקר השתמשתי בהם הם:

- האתר לאלקטרוניקה ומחשבים אתר גדול עם סיכומים רחבים בכל התחומים. דרך אתר זה למדתי עוד בלימודי במהלך התיכון והוא עזר לי המון כדי לחזור ואף לקדם את הידע שלי בכל תחום עולמות האלקטרוניקה, התכנות והמחשבים, קישור לאתר: <https://www.elec4u.co.il>
- אתר נוסף שעזר לי לגבי הפרויקט שלי גם במהלך הפרויקט בסוף לימודי בתיכון במקצוע אלקטרוניקה ומחשבים וגם בלימודי בתואר הטכנולוגי במהלך פרויקט הגמר זהו האתר של פורט, יש לו חומרי לימוד, סיכומים, דוגמאות ועוד הרבה דברים שימושיים שיכולים לעזור להבנת הנושאים בכל עולמות האלקטרוניקה, תכנות ומחשבים. קישור לאתר שלהם: <https://www.arikporat.com>
- ספר טוב מאוד שממנו לקחתי חלק מהתמונות ויש בוא Data-sheets בנושא ה-I2C ודברים טובים ושימושיים להבנת הפרוטוקול תקשורת I2C. קישור לספר זה שנעזרתי בו הוא: <https://www.st.com/resource/en/datasheet/vl53l8cx.pdf>
- מקום נוסף שהשתמשתי בוא על מנת לסכם מידע ולקחת חלק מהתמונות בנוגע למנוע SERVO זהו הספר עם הקישור הבא: <https://www.arikporat.com/wp-content/uploads/2024/03/%D7%9E%D7%A0%D7%95%D7%A2-SERVO.pdf>
- נעזרתי בדף הבא על מנת לסכם לגבי החיישן אור שהוא חלק מן המודול חיישן LDR שבו אני משתמש בפרויקט שלי, קישור לאתר הוא: <https://gabbyshimoni.wixsite.com/arduino-programming/ldr>
- כדי להעלות את האתר שלי השתמשתי בשני תוכנות הראשונה GitHub שבה עשיתי העליתי את התיקיה שבה נמצאים הקודם CSS+HTML בנוסף לכך שם נמצאים התמונות שקישרתי לאתר לסרטונים הראשיים בעמוד הבית. ובנוסף הקובץ PDF של הספר שלי, על מנת שניתן יהיה לפתוח אותו באתר אז Netlify צריך לזהות את מיקום קובץ ה-PDF ולכן הכנסתי אותו לתיקיה הראשית שממנה אני מעלה את כל המסמכים שלה ל-GitHub וב-Netlify אני למעשה מקשר את התיקיה שמאוכסנת ב-GitHub ולאחר מכן יוצר את הדומיין של האתר (הקישור) שדרכו למעשה אני יכול להיכנס אל הדף האינטרנט שלי.
- מאתר הבא שאציג נעזרתי בו כדי לסכם את החומר בנוגע למודול ממסר כפול 5V. קישור לאתר: <https://www.handasiya.co.il/aboutus>
- באתר הבא נעזרתי בהצגת תמונת החיבור של מיקרו בקר ESP32 יחד עם הממספר הכפול בספר פרויקט שלי, קישור לאתר: <https://randomnerdtutorials.com/micropython-relay-web-server-esp32-esp8266/>
- אתר נוסף שנעזרתי בו על מנת לסכם מידע על פרוטוקול תקשורת ESP_NOW, קישור לאתר הוא: <https://randomnerdtutorials.com>
- אתר של המנחה שלי: <https://elctronstudy.wordpress.com> שהייתי נעזר בקבלת ידע והשוואה על סיכומים שעשיתי ובדיקה האם ישנם דברים חשובים נוספים שאוכל לצרף לספר שלי

➡ לחצו כאן לחזרה לתוכן העניינים

קישור לאתר שלי

[/https://raybrook-radar-system.netlify.app](https://raybrook-radar-system.netlify.app)

באתר חשוב לדעת שעל מנת לצאת מהסרטון צריך ללחוץ מחוץ למסגר של הסרטון ימינה/שמאלה/למעלה/למטה לא משנה אבל כדי שהוא יפסיק להתנגן פשוט צריך ללחוץ מחוץ למסגרת. בנוסף יש את הכפתור "המבורגר" בצד ימין למעלה אפשר לעבור בין דפדפות של הבדיקות, הספר פרויקט שמצורף בקובץ PDF וגם את הסרטונים הראשיים של התותח האלקטרומגנטי ושל המערכת רדאר בתחילת האתר. הרעיון של האתר היה לצורך הנוחות שלי וההתקדמות שלי בפרויקט, לקחתי קורסים בחופשת הקיץ ולמדתי את השילובים של **HTML + CSS** + **JavaScript** למדתי להבין על מה כל חלק אחראי וכיצד לצרף סרטונים לאתר ולעביר חלוניות והכרתי את עולם התוכנות כמו **GitHub, Visual Studio Code** ועוד. האתר הוא אחלה רעיון להתפתח ואף לראות את ההתקדמות בפרויקט ולדעת את השלבים האחרונים בעשיית הדברים. מקווה שהחווייה בשימוש באתר תהיה מהנה ונוחה, תודה לכולם

➤ לחצו כאן לחזרה לתוכן העניינים